

Robot R10 (BMW-03-10-R1)

Depuración, correcciones de la revisión Gestamp y puntos abiertos

Gestamp, línea BMW G65, celda 03, estación 03-10-R1 · KUKA KR C4, KSS 8.3.29, robot 658424 · Ethos Automation México

Respaldo de partida: 658424 (v431_03_10_r1) del 2026-10-02 (commit 22a98cc). Estado de este reporte: commit 5a7547f, generado 2026-10-05 13:52 (hora del centro de México). Puntos abiertos: 86 de 101.

1. Resumen

Se depuró el programa del R10 sin cambiar lo que hace el robot. Se tradujeron los comentarios al inglés, cada acción de I/O y cada espera nombra su señal con dirección y nombre, se corrigieron los comentarios que contradecían al código, cada módulo lleva el encabezado Gestamp, y se eliminó el código basura: once módulos que nada llama (21 archivos), código comentado y 856 declaraciones y puntos sin uso. El R10 hace la misma operación que el R20 (pedestal CenterLine, tres tuercas en lugar de cinco); los hallazgos conservan la numeración del R20.

La prueba de que el comportamiento no cambió es mecánica: `tools/check_cleanup.py` quita lo que el compilador KRL ignora y compara línea por línea contra el respaldo original; además un revisor independiente comparó cada comentario contra el código.

Los problemas de comportamiento encontrados en la depuración no se corrigieron en ese paso: son los puntos abiertos de este documento (101 en total, 2 críticos y 25 altos). El programador los cierra modificando el programa; cada respaldo diario se audita contra el criterio de cierre de cada punto.

Gestamp revisó el programa el 2026-10-02 y entregó 22 puntos para esta estación (BMW-03-10R1). Están en la lista como G01-G22, con su texto original. 9 se corrigieron en la oficina en la versión que se entrega al programador (G04, G11, G12, G13, G14, G15, G16, G17, G20): falta cargarla y probarla en celda. El resto necesita trabajo en celda (seguridad, cargas, nombre del robot) o una decisión de Gestamp (sección 3).

La lista de Calvin (Ethos, 2026-10-03) se verificó punto por punto para el 10R1: 40 puntos de esta lista citan su número (columna Origen, "Calvin #n"). Esta lista es la única que se usa.

Críticos

- **F01 — Entrada al pedestal sin confirmar gun abierto, clamp abierto y QFP regresado**
- **F11 — PRELOAD en el submit mueve el pedestal en cualquier modo, sin dueño de las salidas**

2. Qué se hizo

2.1 Regla de trabajo

- Cambia lo que lee una persona, nunca lo que hace el robot: ninguna instrucción ejecutable cambió de texto, de orden ni de mayúsculas.
- Los encabezados de inline form con datos del smartPAD (%P) y las líneas dentro del form no se tocaron, salvo refrescar el nombre de herramienta en el texto visible y la corrección registrada de RejectGE4.
- Cada archivo conserva sus fines de línea (el respaldo mezcla CRLF y LF) y queda en ASCII.
- Cada eliminación ejecutable (módulo o declaración) está en el allowlist con su razón, y el verificador prueba que nada la referencia.

2.2 Formato de los comentarios

Cada acción de I/O y cada espera nombra la señal con dirección y etiqueta (docs/IO_MAP.md). Tres prefijos: NOTE (explicación), CHECK (confirmar en celda, cita el punto abierto) y WARNING (trampa, p. ej. no reabrir un inline form).

Antes	Después
<pre>;QUITAR AIRE LOWER PIN \$OUT[495]=FALSE \$OUT[496]=FALSE</pre>	<pre>;LOWER PIN BOTH VALVES OFF (OLD COMMENT: REMOVE AIR FROM THE LOWER PIN) - ; SIGNALS \$OUT[495] dopw1_AdvancePin OFF, \$OUT[496] dopw1_ReturnPin OFF \$OUT[495]=FALSE \$OUT[496]=FALSE</pre>

2.3 Módulos eliminados

Ninguno es llamado por nada; el verificador prueba que ninguna línea viva los referencia. La razón completa (en inglés) está en tools/cleanup_allowlist.json.

Archivo	Razón
Centerline/CENTERLINE_WELD1.src	Primera versión de la soldadura; usaba el bloque Bosch Unit 1 de AutomationCore, que no está mapeado.
Centerline/trigger1.src	Primer paso de la precarga de tuerca antigua (hoy PRELOAD); nada lo llama (Calvin #31, Gestamp 5).
Centerline/trigger1.dat	Datos de trigger1, sin datos globales.
Centerline/trigger2.src	Segundo paso de la precarga antigua (hoy PRELOAD); nada lo llama (Calvin #31, Gestamp 5).
Centerline/trigger2.dat	Datos de trigger2, sin datos globales.
Centerline/trigger3.src	Tercer paso de la precarga antigua (hoy PRELOAD); nada lo llama (Calvin #31, Gestamp 5).
Centerline/trigger3.dat	Datos de trigger3, sin datos globales.
CK1STXXX.src	Plantilla de revisión de tuerca de otra línea ("Line - Volvo", "Robot XXXXX"); nada la llama (Gestamp 5).
CK1STXXX.dat	Datos de CK1STXXX, sin datos globales.
POUNCE.src	Plantilla de otra línea ("Robot XXXXX"); el pounce de AutomationCore es XPOUNCE global.
POUNCE.dat	Datos de POUNCE, sin datos globales.
Utilities/capchange.src	Cambiador de caps de la pistola servo de una aplicación anterior; nada lo llama y no enlazaba.
Utilities/capchange.dat	Datos de capchange, solo locales.
Utilities/electrodechange.src	Cambio de electrodo viejo, reemplazado por GunElectrodeChange (el que llama cell.src); no enlazaba.
Utilities/ElectrodeChange.dat	Datos de electrodechange, sin datos globales.
Utilities/redrabbit.src	Red rabbit de una celda de birlos (TOOL 2, grippers 2-4, sensores di777..782); bloqueado con WAIT FOR FALSE después del INI; nada lo llama; no enlazaba (Gestamp 4).
Utilities/RedRabbit.dat	Datos de redrabbit, sin datos globales.

Archivo	Razón
StyleApps/Options/style1app2opt1AutoRR.src	Copia red rabbit de la revisión de tuerca; nada la llama: Style1Opt10AutoRR llama la de producción (Gestamp 5, 8).
StyleApps/Options/style1app2opt1AutoRR.dat	Datos de style1app2opt1AutoRR, solo puntos locales.
StyleApps/Options/style1app2opt2AutoRR.src	Copia red rabbit de la revisión de cámara; nada la llama (Gestamp 5, 8).
StyleApps/Options/style1app2opt2AutoRR.dat	Datos de style1app2opt2AutoRR, solo puntos locales.

2.4 Declaraciones y datos sin uso eliminados

Declaraciones que nada referencia: puntos y datos de movimientos que ya no existen, datos de inline forms NutWeld viejos y variables globales sin uso. Cada línea está en tools/cleanup_allowlist.d/ con su razón.

Archivo	Líneas	Qué se eliminó
Program/StyleApps/Options/style1app1opt1.dat	90	25 E6POS, 25 FDAT, 17 PDAT, 16 LDAT, 5 NUTWELDDAT, 1 TM_SUGG_T, 1 TQM_TQDAT_T
Program/StyleApps/Options/style1app1opt2.dat	124	35 E6POS, 35 FDAT, 28 PDAT, 23 LDAT, 3 NUTWELDDAT
Program/StyleApps/Options/style1app1opt3.dat	79	26 PDAT, 19 E6POS, 19 FDAT, 13 LDAT, 2 NUTWELDDAT
Program/StyleApps/Options/style1app2opt1.dat	143	43 E6POS, 43 FDAT, 33 PDAT, 23 LDAT, 1 NUTWELDDAT
Program/StyleApps/Options/style1app2opt2.dat	51	16 E6POS, 16 FDAT, 14 PDAT, 4 LDAT, 1 INT
Program/StyleDrops/Options/style1drop1opt1.dat	16	6 PDAT, 5 E6POS, 5 FDAT
Program/StyleDrops/Options/style1drop1opt2AutoRR.dat	80	26 E6POS, 26 FDAT, 25 PDAT, 3 LDAT
Program/StylePicks/Options/style1pick1opt1.dat	47	17 E6POS, 17 FDAT, 11 PDAT, 2 LDAT
Program/StylePicks/Options/style1pick1opt1AutoRR.dat	56	20 E6POS, 20 FDAT, 13 PDAT, 3 LDAT
Program/StylePicks/Options/style1pick1opt2.dat	43	16 E6POS, 16 FDAT, 9 PDAT, 2 LDAT
Program/Styles/style_1.dat	7	6 PDAT, 1 SIGNAL
Program/Utilities/RejectGE4.dat	32	11 E6POS, 11 FDAT, 10 PDAT
Program/masref_user.dat	67	22 E6POS, 22 FDAT, 12 PDAT, 11 LDAT
System/\$config.dat	21	17 BOOL, 3 INT, 1 SIGNAL
TP/AutomationCore/automationcoreroutines.dat	19	14 SIGNAL, 4 BOOL, 1 INT
Total	875	

2.5 Trampas corregidas o señaladas

- RejectGE4: el inline form del reset de \$OUT[106] traía oculto 5:TRUE y encerraba los resets de banderas; confirmarlo convertía el reset en set y borraba los resets. Quedó 5:FALSE y los resets fuera del fold, sin mover código (igual que en el R20).
- Forms GripperTech de picks y drop a banda: parámetros ocultos setgrripper=4;setstate=2. Reabrirlos habría regenerado la llamada para el gripper 4 (\$OUT[251]/[252]): el gripper 1 no se mueve y el check cae en timeout. Corregido en la versión de oficina (F31, G10).
- Drop red rabbit: suelta con el gripper 3, que en grp_data.dat tiene el mismo I/O que el gripper 1; funciona solo por ese mapeo duplicado (F19, F31). Quedó WARNING en el código.
- WARNING sobre los folds que no deben reabrirse: parámetros AutomationCore ocultos que no coinciden con la llamada (F21). Los TRIGGER escritos a mano dentro de inline forms (F32) se sacaron en la versión de oficina.
- CloseAndCheckAllClamps / OpenAndCheckAllClamps: se quitaron los folds comentados de los grippers 2-4 (Calvin #23, F45).

2.6 Comentarios que contradecían el código

Se reescribieron para describir lo que hace el código; cada uno lleva un CHECK para confirmarlo en celda (punto F27).

Programa	Decía	El código hace
centerline_weld	UPPER PIN EXTEND	retrae el upper pin (\$OUT[499] ON)
centerline_weld	SCHEDULE 7	selecciona el programa Bosch 2
centerline_weld	LOWER PIN RETURN (fin de soldadura)	avanza el lower pin (\$OUT[495] ON)
PRELOAD	CL_LowerPinRetreated	avanza el lower pin
PRELOAD	QFP ADV (en la espera de QFP regresado)	espera \$IN[482] QFP regresado
programas del pedestal	comentarios en español	traducidos y con la señal nombrada

2.7 Herramientas que quedan en el repositorio

- tools/check_cleanup.py — prueba que la depuración cambió comentarios y no comportamiento.
- tools/build_archive.py — arma el zip de respaldo a cargar sobre el respaldo original o sobre el último respaldo auditado (hoy bmw_03_10_r1 del 2026-10-05); con cualquier otro se niega si el robot cambió.
- tools/check_equivalence.py — separa los cambios reales de lógica de los renombres en los cambios de código posteriores a la depuración (sección 3).
- tools/audit_backup.py — audita cada respaldo diario contra el último estado auditado.
- tools/make_report.py — genera este PDF.

3. Revisión de Gestamp y correcciones de oficina

Documento de Gestamp "Robot Program Review - Findings" del 2026-10-02, sección BMW-03-10R1 (22 puntos). Cada punto quedó en la lista de puntos abiertos con el prefijo G y su texto original; los que coinciden con un hallazgo de Ethos remiten a él (columna Ver).

ID	Texto de Gestamp	Estado	Ver
G01	Robot name needs to change	Abierto	C02
G02	Safety Tool is not equal to Default TCP	Abierto	F42
G03	Tool 1 is used without Load data determination	Abierto	C08, F39, C09
G04	Program Red Rabbit() is blocked, might be an unnecessary copy	Corregido en oficina - probar en celda	F19
G05	Unused programs	Requiere decisión	G04, C28
G06	Electrode change confirmation only 0.1 s, no handshake	Requiere decisión	F43, F12
G07	Multiple options in Style_1 call the same sub-program	Requiere decisión	F02
G08	Separate Auto-Red Rabbit program that doesn't match production	Requiere decisión	F19, G21
G09	Part control to PLC is set/reset instead of reflecting the real part controls	Requiere decisión	F12, C01, C16
G10	Gripper commands not used at all times	Abierto	F31, F33, C22, F40
G11	Use of Goto labels	Corregido en oficina - probar en celda	F30, F07
G12	Use of I/O numbers instead of signal names	Corregido en oficina - probar en celda	F17, C23, F29
G13	Poor programming (10 lines for nut present)	Corregido en oficina - probar en celda	F28, F03, F31
G14	Gripper opened before the part control check	Corregido en oficina - probar en celda	
G15	Names, variables and comments not in English	Corregido en oficina - probar en celda	C26, F36
G16	NO variables for e.g. stroke limits	Corregido en oficina - probar en celda	F16, F44, F41
G17	Nut_weld_Retry logic useless	Corregido en oficina - probar en celda	F24
G18	NO check for Weld OK, just weld finished	Requiere decisión	F08
G19	Next_Nut_Ready logic can fail at block selection, reset, reboot	Abierto	F38, F14
G20	Change of xP0 deletes Request_next_nut inside inline forms	Corregido en oficina - probar en celda	F32, F14
G21	Style1app2Opt1 will not work for RedRabbit	Requiere decisión	F19, G08
G22	Use of AutomationCore is wrong in multiple places	Requiere decisión	F12, C01, F46, F21, F20, C11, C12, C18, C19, C14, F35

3.1 Qué se corrigió en la oficina

A diferencia de la depuración, aquí sí cambia el código. Cada cambio está en tools/code_changes.json con los puntos que atiende; resumen:

Punto	Cambio
G04 Programa RedRabbit bloqueado: borrado	Hecho en la depuración (commit 0e1ea54): redrabbit.src y RedRabbit.dat borrados, con la razón en tools/cleanup_allowlist; tools/check_cleanup.py prueba que nada los referencia. Falta que desaparezcan del controlador al cargar la versión de oficina: un restore no borra archivos, hay que borrarlos a mano.
G11 Sin GOTO ni etiquetas	Todos los GOTO y etiquetas se quitaron. Dry cycle = IF NOT di004UseDryCycle; tuerca precargada = IF NOT NUT_READY; chequeo de gun cerrado = LOOP/EXIT (después del HALT los timers se rearrancan, F07); Style1Opt1 = un IF/ELSE por ciclo con la estación leída una vez en nPickStation (F30). tools/check_equivalence.py muestra que, fuera de esos puntos, la lógica es la misma que en la base fb8a4c8. Falta probar en celda.

Punto	Cambio
G12 Nombres de señal en lugar de números de E/S	Los bits del pedestal recibieron nombre AutomationCore en automationcoreroutines.dat (do498UpperPinExtend, do499UpperPinRetract, do501PartClampClose, do502PartClampOpen, do503QFPAdvance, do504QFPReturn, do505QFPNutBlowOff, di466PartClampOpen, di467PartClampClosed, di470QFPAdvanced), los tres bits del faro en \$config.dat (CL_BeaconBit3001/3008/3024) y el código usa nombres (también los NutWeld dopw1_/dipw1_). Las direcciones no cambiaron. Los inline forms estándar de KUKA (WAIT FOR, OUT) siguen generando \$IN[n]/\$OUT[n] con el nombre en el form: así es el estándar y no se reescriben a mano.
G13 Código simplificado (nut present en una línea, duplicados fuera)	Cada revisión de tuerca es "WAIT SEC 0.2 / PARTPRESENTn = di227NutPresent1" dentro de IF NOT di004UseDryCycle (lee con el robot parado y escribe TRUE o FALSE; F28, F03); se quitaron los GRPg_Check redundantes (picks, drop al conveyor, RejectGE4), las líneas dobles de upper pin y la etiqueta WELD_END sin uso.
G14 Revisar gripper vacío antes del comando de gripper	La espera de gripper vacío (\$IN[253] di253PartPresent1 y \$IN[254] di254PartPresent2 OFF, inline form WAIT FOR) se movió antes del primer form Gripper SET (GRPg_SetStateAndCheck(1, 1, ...)) en los tres picks.
G15 Nombres, variables y comentarios en inglés	Hecho: comentarios, títulos de fold y &COMMENT en inglés (depuración, C26); en oficina NUT_STAR → NUT_START, CameraJudment → CameraJudgment, PartRejeted → PartRejected, di227SensorCamera → di227NutPresent1 (F36) y la carpeta Styles/options renombrada Styles/Options. Quedan "Hidraulyc_Valve" en las declaraciones del paquete NutWeld (archivo del proveedor, ningún programa del integrador lo usa) y el texto 'do106PartRejeted' de los inline forms OUT 106 de RejectGE4 (se corrige solo si se reabre el form).
G16 Constantes para límites de carrera, presiones y tiempos	Fold CENTERLINE CONTROL de \$config.dat con CL_GUN_WELD_MIN/MAX = 69000/73000 (la ventana que el código siempre revisó), CL_GUN_OPEN_MIN, CL_GUN_CLOSE_TIMEOUT, CL_GUN_IN_WINDOW_TIME, CL_GUN_PRESS_BASE/WELD/REST (405/1820/39425, los valores en uso; el orden de bytes sigue por confirmar, F44) y CL_WELD_PROGRAM; CENTERLINE_WELD y CENTERLINE_HOME los usan. Mismos valores. La verificación del pin superior sigue en F16; los tiempos de PRELOAD y el límite de agua en F41.
G17 Lógica Nut_weld_Retry inútil	Hecho en oficina (opción A de F24, como en R20): se borraron las ramas, las etiquetas y la variable. Sin cambio de comportamiento.
G20 Request_next_nut y NUT_START fuera de los inline forms	Cada TRIGGER quedó justo antes de su fold de movimiento (posición del form SYN OUT de KUKA: mismo movimiento, mismo punto de disparo; se conservan DISTANCE = 0 y 1); los resets del red rabbit van después del fold DropOff[2] Reset. La robustez ante selección de bloque, reset y reinicio sigue en F14 y F38 (G19).
Módulo borrado	Razón
Program/Centerline/centerline_loop.src	Lazo de prueba manual de las válvulas del pedestal, sin salida ni interlocks; nada lo llama (G05).

3.2 Cómo se verificó

- La depuración sigue probada: tools/check_cleanup.py --head fb8a4c8 (commit de la depuración) pasa.
- tools/check_equivalence.py compara cada programa contra la depuración traduciendo cada nombre de señal a su dirección y cada constante a su valor: los renombres desaparecen y solo queda la lógica que cambió. Ocho módulos salen equivalentes (solo renombres o sangría); en los demás, cada diferencia es una de las listadas arriba. Un cambio no listado en tools/code_changes.json lo hace fallar. La carpeta Styles/options pasó a Styles/Options (dos archivos movidos, mismo contenido).
- tools/audit_backup.py sobre el zip armado contra el commit de la depuración: 0 FAIL y 0 WARN (los INFO son esperas sin timeout que ya existían, F15). Reporte en audits/2026-10-04_oficina-gestamp.
- Revisión independiente de los comentarios contra el código: corrigió el texto del drop red rabbit (el gripper 3 sí abre el gripper 1), el de las tareas de fondo de sps.sub y el de las banderas de precarga.
- Lo que ninguna verificación de escritorio sustituye: la prueba en celda que pide cada punto "Corregido en oficina - probar en celda".

3.3 Lo que debe probar el programador en celda

G04 Programa RedRabbit bloqueado: borrado

- El programa carga sin errores de compilación (captura del navegador del smartPAD sin módulos en rojo). Un ciclo red rabbit (option 12) corre igual que antes de la carga (sus defectos siguen en F19). Anotar fecha.

G11 Sin GOTO ni etiquetas

- Prueba en celda: un ciclo sin dry cycle de estación 1 y uno de estación 2 (T1 y automático), un rechazo por nut check (option 1), un ciclo red rabbit (option 12), un pick en dry cycle (sin espera de pieza; el ciclo se sigue colgando en la primera weld app por F04, igual que antes) y el timeout de gun cerrado en T1 sin tuerca (HALT, START: vuelve a dar timeout si sigue sin cerrar). Anotar fecha y resultado de cada uno.

G12 Nombres de señal en lugar de números de E/S

- El programa carga sin errores de compilación (captura del navegador del smartPAD sin módulos en rojo). Un ciclo completo en automático sale bien. Anotar fecha.

G13 Código simplificado (nut present en una línea, duplicados fuera)

- Prueba en automático con option 1: 5 ciclos con pieza buena sin rechazo por tuercas; un ciclo con una tuerca faltante (o una pieza red rabbit) termina en RejectGE4; PARTPRESENT1..3 leídos en el display de variables. En cada pick, drop y reject el gripper confirma su estado con GRPg_SetStateAndCheck, sin el GRPg_Check extra. Anotar fecha y resultado.

G14 Revisar gripper vacío antes del comando de gripper

- Prueba en T1: con una pieza en el gripper al iniciar el pick, el robot se queda en la espera sin mover el gripper. Anotar fecha.

G15 Nombres, variables y comentarios en inglés

- El programa carga sin errores de compilación (captura del navegador del smartPAD sin módulos en rojo). Un ciclo completo en automático sale bien. Anotar fecha.

G16 Constantes para límites de carrera, presiones y tiempos

- Un ciclo de soldadura completo en automático; valores leídos en el display de variables iguales a los del fold CENTERLINE CONTROL. Anotar fecha.

G17 Lógica Nut_weld_Retry inútil

- Un ciclo completo en automático con las 3 tuercas, sin errores. Anotar fecha.

G20 Request_next_nut y NUT_START fuera de los inline forms

- Prueba: abrir el inline form PTP P12 de style1pick1opt1 y PTP P0 de style1app1opt1 y confirmarlos con Cmd OK sin cambiar nada (sin Touch Up, que reenseña el punto); el TRIGGER sigue antes de cada fold. Después, un ciclo completo en automático: NUT_READY y NEXT_NUT_READY (display de variables) pasan a FALSE y vuelven a TRUE una vez por tuerca (3 preloads por pieza). Anotar fecha.

3.4 Lo que queda fuera de la oficina

ID	Punto	Responsable	Estado
G01	Cambiar el nombre del robot	Programador robot (en celda) / Gestamp confirma el formato	Abierto
G02	Herramienta de seguridad igual al TCP de la herramienta 1	Programador robot / Seguridad	Abierto
G03	Determinar los datos de carga (herramienta 1 con y sin pieza)	Programador robot / Puesta en marcha	Abierto
G05	Programas sin uso	Programador robot / Gestamp controles	Requiere decisión
G06	Cambio de electrodo: confirmación de 0.1 s y sin handshake	Programador robot / Gestamp controles	Requiere decisión
G07	Options 1, 2, 7 y 8 de Style_1 llaman el mismo programa	Gestamp controles / Programador robot	Requiere decisión
G08	El ciclo red rabbit no sigue el camino de producción	Gestamp controles / Programador robot	Requiere decisión
G09	Pieza en gripper al PLC fijada por programa, no por los sensores	Programador robot / Gestamp controles	Requiere decisión
G10	Comandos de gripper GripperTech en todos los movimientos de gripper	Programador robot / Puesta en marcha	Abierto

ID	Punto	Responsable	Estado
G18	Sin chequeo de soldadura buena	Programador robot / Puesta en marcha	Requiere decisión
G19	Handshake de la tuerca frágil ante selección de bloque, reset y reinicio	Programador robot	Abierto
G21	El nut check no funciona para red rabbit	Gestamp controles / Programador robot	Requiere decisión
G22	Uso incorrecto de AutomationCore	Gestamp controles / Programador robot	Requiere decisión

4. Cómo vamos a trabajar

Día 0

- Cargar en el robot la versión de oficina por WorkVisual con el paquete de make_package.py (solo los programas que cambian, armado sobre el último respaldo auditado) y borrar en el mismo deploy los archivos de la lista de LEEME.txt y la carpeta vieja Styles/optiones: si quedan, no compila. Nunca hacer deploy de un proyecto WorkVisual guardado de antes (trae el programa viejo: así se perdió la depuración el 2026-10-05).
- Hacer las pruebas en celda de los puntos "Corregido en oficina - probar en celda" (sección 3.3).
- Correr un ciclo en T1 con override reducido y uno en automático.
- Sacar un respaldo completo (Archive → All): es el punto de partida de las auditorías.

Cada día

- El programador modifica el programa para cerrar puntos. Respetar docs/CONVENTIONS.md: comentarios en inglés con la señal nombrada, sin código comentado, sin reabrir inline forms con líneas escritas a mano.
- En el Excel de puntos abiertos, por cada punto trabajado: Estado = "Corregido - por auditar", qué cambió, en qué módulos, y la evidencia que pide el criterio de cierre (prueba en celda con fecha y resultado). No marca nada como Cerrado: eso lo hace la auditoría.
- Al final del día saca un respaldo completo y lo envía junto con el Excel.
- Edgar sube ambos aquí. La auditoría compara el respaldo contra el último auditado, revisa cada cambio de código, verifica cada punto marcado contra su criterio de cierre, corre las verificaciones mecánicas y regresa el Excel actualizado con el resultado y un reporte de auditoría.

Estados

Estado	Quién lo pone	Significa
Abierto	Auditoría	Pendiente de trabajar.
Requiere decisión	Auditoría	Falta información o decisión (Gestamp, seguridad, Ethos) antes de programar.
Corregido en oficina - probar en celda	Auditoría	Ethos lo corrigió en la versión entregada; el programador lo prueba en celda y lo pasa a "Corregido - por auditar" con la evidencia.
En proceso	Programador	Trabajo empezado, aún no listo para auditar.
Corregido - por auditar	Programador	Cambio hecho y probado; pide auditoría.
Cerrado - auditado	Auditoría	Cumple todos los criterios de cierre en el respaldo.
Reabierto	Auditoría	La auditoría no encontró el cambio o no cumple; ver nota.
Aceptado sin cambio	Ethos / Gestamp	Se decidió no corregir; queda la razón y quién lo aceptó.
Cerrado - depuración / No aplica / Se cierra con otro punto	Auditoría	Resuelto por la depuración, cumple, o duplica otro punto.

4.1 Auditorías

Cada respaldo auditado y cada versión entregada, del más viejo al más nuevo. El detalle técnico (en inglés) está en audits/<fecha>/AUDIT.md.

Fecha	Respaldo	Resultado
2026-10-04	658424_R10 (versión de oficina, tools/build_archive.py)	12 módulos con cambios de lógica, 8 equivalentes tras renombres, 1 borrado (centerline_loop.src), 2 movidos (Styles/optiones → Styles/Options), 104 idénticos (tools/check_equivalence.py contra la base fb8a4c8) Alta de los 22 puntos de Gestamp, F01-F47 y C01-C28. Corregidos en oficina, por probar en celda: G11-G17 y G20 (y F24, F28, F30, F32, F36), más G04 (borrado en la depuración, falta borrarlo en el controlador); cerrados por la depuración: F45, C03, C24, C26; con avance de oficina y todavía abiertos: G05, G10, F03, F07, F16, F17, F29, F31. Base: commit de depuración fb8a4c8; cambios listados en tools/code_changes.json. La lista de Calvin (Gstamp_Robot_Open_Issues_2026-10-03) se verificó punto por punto: cada fila de 10R1 o All tiene su punto (referencias en origen_ref).

Fecha	Respaldo	Resultado
2026-10-05	bmw_03_10_r1.zip (am.ini 2026-10-05 09:43, robot BMW_03_10_R1, serie 658424)	62 archivos cambiados, 7 agregados (GUN_OPEN_CHECK, proyecto WorkVisual BMW-03-10R1_v8), 39 KRL con cambios de código (-230/+1323 líneas) contra el estado auditado (b5a8803) La versión de oficina NO se cargó: 29 archivos son el original byte por byte, 16 se editaron encima del original y siguen en el controlador los 22 archivos borrados por la depuración. El programador trabajó sobre el original: quitó los GOTO (G11), reinició los timers del gun (F07), agregó GUN_OPEN_CHECK e interrupts de vigilancia del gun (F01 parcial), flujo de agua real (F09), evaluación por option en sensor y cámara (F02, F19 parcial), cambió el nombre del robot (G01) y activó AutomationCore_BKG editando el proveedor (F12, nuevo F49). Nuevo F48: interlock de drop-off movido después de la aproximación. Sin Excel del programador: ningún punto se marcó como corregido. Siguiendo paso: integrar sus cambios en la versión depurada y cargar esa. Reporte en audits/2026-10-05_09-43-37.
2026-10-05	658424_R10_2026-10-05_0819.zip (versión integrada, armada sobre bmw_03_10_r1.zip)	Contra la depuración fb8a4c8: 26 KRL con cambios de código, 2 módulos nuevos (GUN_OPEN_CHECK.src/.dat), 22 archivos fuera; contra el repositorio: idéntico (308 entradas). audit_backup: 0 FAIL, 0 WARN. Integración de los cambios del programador del 2026-10-05 en la versión depurada: GUN_OPEN_CHECK e interrupts del gun (F01 parcial), agua real (F09 parcial), evaluación por option en sensor y cámara (F02 parcial, F19 parcial), AutomationCore_BKG activo con ediciones del proveedor (F12/F49 pendientes de decisión), nombre BMW_03_10_R1 y mapeo TBEN conservados. F48: el zip regresa el interlock de drop-off antes del primer movimiento y conserva el AtDrop3 del programador (la pieza cae mejor en el conveyor; validado por Edgar con el programador). Ningún punto se cierra: todos piden prueba en celda con evidencia y respaldo. Reporte en audits/2026-10-05_integracion. El zip 658424_R10_2026-10-05_0809 (con AtDrop3 original) queda reemplazado por el 0819; no cargar el 0809.
2026-10-05	bmw_03_10_r1.zip (am.ini 2026-10-05 14:46)	Contra el estado auditado: 19 archivos originales, 17 editados encima del original, 22 borrados aún en el controlador; contra el respaldo de las 09:43: apps de soldadura partidas en A (estación 1, igual) y B (estación 2, puntos de soldadura retocados 1-5 mm), punto de cámara P3 re-enseñado (83 mm, ~16°), AC_ApplicationCheck(1) después de P22, PTP P1 quitado del drop. La versión integrada (zip 0819) no se cargó. Revisión independiente (2 agentes) de GUN_OPEN_CHECK/interrupts y del background de AutomationCore: integración fiel; huecos de vigilancia del gun (F01); con el background activo el Request to Enter deja de detener el robot si \$OUT[930] quedó en ON (F35 sube a Alto), el background pisa las habilitaciones del Bosch, el cambio de electrodo corre sin probar (F43, F49). Nuevo F50. Sin Excel del programador. Reporte en audits/2026-10-05_14-46-44.
2026-10-05	R10_paquete_2026-10-05_1351.zip (versión integrada sobre el respaldo de las 14:46; zip completo 658424_R10_2026-10-05_1351)	Contra el respaldo de las 14:46: 57 programas a reemplazar, 2 a agregar (Styles/Options), 24 a borrar. Contra la depuración fb8a4c8: 38 KRL con cambios de código; audit_backup 0 FAIL, 0 WARN; zip idéntico al repositorio. Integrados: apps A/B por estación (F51) y punto de cámara. No integrados: permiso de aplicación después de P22 (F50) y P1 quitado del drop (preguntar al programador). Correcciones de la revisión independiente: interrupts antes de cada GUN_OPEN_CHECK y mensaje en GUN_OPEN_LOST (F01), aplicación 1 liberada por TRIGGER al salir de la cámara (F48, F20), background de AutomationCore apagado con sus salidas en OFF al arrancar (F49). Un revisor independiente revisó la integración antes de la entrega: sin defectos bloqueantes. Se entrega como paquete para WorkVisual (tools/make_package.py), no como respaldo completo. Reporte en audits/2026-10-05_integracion_2.

5. Puntos abiertos — resumen

Fondo rojo: puntos críticos. G01–G22: puntos de la revisión de Gestamp (fondo azul en el ID). Fnn/Cnn: revisión de Ethos; los que vienen de la lista de Calvin dicen "Calvin #n".

ID	Sev.	Punto	Responsable	Estado
G01	Medio	Cambiar el nombre del robot (Calvin Std 1)	Programador robot (en celda) / Gestamp confirma el formato	Abierto
G02	Alto	Herramienta de seguridad igual al TCP de la herramienta 1 (Calvin #13)	Programador robot / Seguridad	Abierto
G03	Alto	Determinar los datos de carga (herramienta 1 con y sin pieza) (Calvin #16)	Programador robot / Puesta en marcha	Abierto
G04	Bajo	Programa RedRabbit bloqueado: borrado (Calvin Std 9)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
G05	Bajo	Programas sin uso (Calvin #31)	Programador robot / Gestamp controles	Requiere decisión
G06	Medio	Cambio de electrodo: confirmación de 0.1 s y sin handshake (Calvin #29)	Programador robot / Gestamp controles	Requiere decisión
G07	Alto	Options 1, 2, 7 y 8 de Style_1 llaman el mismo programa (Calvin Std 7)	Gestamp controles / Programador robot	Requiere decisión
G08	Alto	El ciclo red rabbit no sigue el camino de producción (Calvin #26)	Gestamp controles / Programador robot	Requiere decisión
G09	Medio	Pieza en gripper al PLC fijada por programa, no por los sensores (Calvin #8)	Programador robot / Gestamp controles	Requiere decisión
G10	Medio	Comandos de gripper GripperTech en todos los movimientos de gripper (Calvin #25)	Programador robot / Puesta en marcha	Abierto
G11	Medio	Sin GOTO ni etiquetas (Calvin Std 2)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
G12	Medio	Nombres de señal en lugar de números de E/S (Calvin Std 3)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
G13	Medio	Código simplificado (nut present en una línea, duplicados fuera) (Calvin Std 6)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
G14	Medio	Revisar gripper vacío antes del comando de gripper (Calvin #25)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
G15	Bajo	Nombres, variables y comentarios en inglés (Calvin Std 4)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
G16	Medio	Constantes para límites de carrera, presiones y tiempos (Calvin Std 5)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
G17	Bajo	Lógica Nut_weld_Retry inútil (Calvin #27)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
G18	Alto	Sin chequeo de soldadura buena (Calvin #2)	Programador robot / Puesta en marcha	Requiere decisión
G19	Alto	Handshake de la tuerca frágil ante selección de bloque, reset y reinicio (Calvin #11)	Programador robot	Abierto
G20	Alto	Request_next_nut y NUT_START fuera de los inline forms (Calvin #28)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
G21	Alto	El nut check no funciona para red rabbit (Calvin #26)	Gestamp controles / Programador robot	Requiere decisión
G22	Alto	Uso incorrecto de AutomationCore (Calvin Std 8)	Gestamp controles / Programador robot	Requiere decisión
F01	Crítico	Entrada al pedestal sin confirmar gun abierto, clamp abierto y QFP regresado	Programador robot	Abierto
F02	Alto	Options 2, 7 y 8 no evalúan el sensor de tuercas: solo decide la cámara	Programador robot + Gestamp controles	Requiere decisión
F03	Alto	PARTPRESENT1..3 y bscrapGE4 no se resetean al inicio del ciclo (Calvin #8)	Programador robot	Abierto
F04	Alto	Dry cycle se cuelga en la primera weld app: PRELOAD borra NUT_READY y nunca lo pone	Programador robot	Abierto

ID	Sev.	Punto	Responsable	Estado
F05	Alto	Dry cycle releído en cada paso: un cambio a mitad de ciclo deja pasar pieza incompleta	Programador robot	Abierto
F06	Alto	Ciclos vacíos o silenciosos: pick ambiguo, style u option desconocidos	Programador robot	Abierto
F07	Alto	Chequeo de gun cerrado: HALT sin mensaje como manejo de falla	Programador robot	Abierto
F08	Alto	Resultado de soldadura Bosch no evaluado: weld complete se toma como soldadura buena (Calvin #2)	Programador robot + Gestamp controles	Requiere decisión
F09	Alto	Resultado del flujo de agua descartado; \$OUT[475] forzado en ON	Programador robot + Puesta en marcha	Abierto
F10	Alto	Sin estado seguro del pedestal al cancelar o resetear el programa	Programador robot	Abierto
F11	Crítico	PRELOAD en el submit mueve el pedestal en cualquier modo, sin dueño de las salidas	Seguridad + Programador robot	Requiere decisión
F12	Alto	Background de AutomationCore y NutWeld deshabilitado: los bits de estado al PLC no cambian	Gestamp controles + Programador robot	Requiere decisión
F13	Medio	PRELOAD sin timeouts por paso, sin falla y banderas de tuerca lista sin confirmar alimentación (Calvin #6)	Programador robot	Abierto
F14	Medio	NUT_READY no se consume al soldar; la siguiente tuerca depende solo de TRIGGERS (Calvin #11)	Programador robot	Abierto
F15	Medio	Unos 35 WAIT FOR sobre entradas de proceso sin timeout ni mensaje (Calvin #1)	Programador robot	Abierto
F16	Medio	Posición del pin superior nunca verificada	Programador robot / Puesta en marcha	Abierto
F17	Medio	Declaraciones traslapadas; nombres dobles en los bits del pedestal	Programador robot	Abierto
F18	Medio	KRC_IO.xml mapea \$OUT[820..979] y \$IN[820..930] sobre los mismos bytes del adapter PLC	Gestamp controles / Programador robot	Requiere decisión
F19	Alto	Red rabbit: el ciclo no puede dar resultado y suelta la pieza por un mapeo duplicado (Calvin #26)	Programador robot / Gestamp controles	Requiere decisión
F20	Medio	Reject sin permiso del PLC para la posición de rechazo; Application 1 sin liberar	Programador robot / Gestamp controles	Requiere decisión
F21	Medio	Parámetros ocultos de inline forms AutomationCore no coinciden con la llamada	Programador robot	Abierto
F22	Medio	Movimientos del pedestal mezclan modo base y external TCP	Ethos (Edgar) / Programador robot	Requiere decisión
F23	Bajo	Posición del LPT armada con 3 bytes en otro orden que en R20	Puesta en marcha / Programador robot	Abierto
F24	Bajo	Camino de retry de soldadura muerto: NUT_WELD_RETRY nunca es TRUE (Calvin #27)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
F25	Bajo	Feed de respaldo en CENTERLINE_WELD: pulso de feed de duración cero (Calvin #19)	Programador robot	Abierto
F26	Bajo	Timers \$TIMER[10..15] de PRELOAD y CENTERLINE_WELD sin constantes con nombre	Programador robot	Abierto
F27	Bajo	Confirmar en celda funciones de comentarios corregidos: blow-off, pines, programa Bosch	Puesta en marcha / Gestamp controles	Abierto
F28	Bajo	Sensor de tuerca leído al llegar al punto; el dwell iba después de la lectura	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
F29	Bajo	Faro KL50L2: lógica muerta y un bit sin mapear	Programador robot / Puesta en marcha	Abierto
F30	Bajo	Style1Opt1 sin GOTO: un camino IF/ELSE por ciclo	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
F31	Medio	Forms GripperTech: parámetros ocultos de otro gripper; drop red rabbit con el gripper 3 (Calvin #25)	Programador robot / Puesta en marcha	Abierto
F32	Medio	TRIGGERS y resets escritos a mano dentro de folds de inline form (Calvin #28)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda

ID	Sev.	Punto	Responsable	Estado
F33	Medio	GripperConfig.xml del HMI desincronizado con grp_data.dat	Programador robot	Abierto
F34	Bajo	CENTERLINE_HOME: polaridad de \$OUT[473] y falta el reset del Bosch	Programador robot / Puesta en marcha	Requiere decisión
F35	Medio	El mastering reference deja \$OUT[930] en ON para siempre	Programador robot	Abierto
F36	Bajo	\$IN[227] declarado con dos nombres (NUT_PRESENT1, di227SensorCamera)	Programador robot (prueba en celda)	Corregido en oficina - probar en celda
F37	-	Presión de intensificado no confirmada antes de weld start: no aplica en R10 (Calvin #1)	Programador robot	No aplica
F38	Alto	Estado de la tuerca no se reinicializa al arrancar, en reset ni después de un reinicio (Calvin #11)	Programador robot	Abierto
F39	Medio	Supervisión de par en cero; chequeo de datos de carga solo avisa; carga tecleada (Calvin #16)	Programador robot / Puesta en marcha	Abierto
F40	Medio	Drops y reject abren el gripper sin confirmar pieza; el drop red rabbit se mueve antes de gripper vacío (Calvin #25)	Programador robot	Abierto
F41	Bajo	Tiempos de PRELOAD, límite de flujo de agua y esperas fijas sin constantes (Calvin Std 5)	Programador robot	Abierto
F42	Bajo	Envoltentes de trabajo por software apagadas (Calvin #30)	Programador robot / Seguridad / Gestamp	Requiere decisión
F43	Medio	Cambio de electrodo: reset de stepper por pulso de 0.1 s y handshake que no cierra (Calvin #29)	Programador robot / Gestamp controles	Requiere decisión
F44	Alto	Palabra de presión del gun escrita en dos órdenes de bytes (405/1820 contra 39425)	Puesta en marcha / Programador robot	Abierto
F45	Bajo	Rutinas de clamps: grippers 2-4 deshabilitados, solo existe el gripper 1 (Calvin #23)	Programador robot	Cerrado - depuración
F46	Medio	EndOfCycle del proveedor editado: Work Complete nunca se manda al PLC	Gestamp controles / Ethos (Edgar)	Requiere decisión
F47	Medio	Camino del brake test enseñado con tool 3 / base 1 de otro robot	Programador robot / Puesta en marcha	Abierto
F48	Alto	Drop al conveyor: interlock de drop-off movido después de la aproximación (corregido); AtDrop3 re-enseñado (se conserva)	Programador robot / Gestamp controles	Corregido en oficina - probar en celda
F49	Alto	AutomationCore_BKG activado y editado sin decisión registrada	Gestamp controles + Programador robot	Requiere decisión
F50	Alto	Permiso de aplicación 1 (nut check / cámara) revisado después de llegar a P22	Programador robot	Abierto
F51	Bajo	Apps de soldadura duplicadas por estación de pick (A/B): cada cambio va en las dos	Programador robot	Sin acción - documentado
C01	-	AutomationCore_Bkg deshabilitado: el estatus al PLC no se actualiza	Gestamp controles / Ethos (Edgar)	Se cierra con otro punto
C02	-	Designación BMW-03-10-R1 en los encabezados; confirmar formato con Gestamp	Ethos (Edgar) / Gestamp controles	Requiere decisión
C03	-	Encabezado estándar Gestamp en cada módulo del integrador	Programador robot	Cerrado - depuración
C04	-	Movimientos de pedestal interpolados sobre el dado fijo (external TCP)	Ethos (Edgar) / Programador robot	Requiere decisión
C05	-	TCP remoto del pedestal (BASE_DATA[3]) sin medir	Puesta en marcha / Programador robot	Abierto
C06	-	Herramientas y bases con nombre: nombres de otro proyecto	Programador robot	Abierto
C07	-	Un user frame por utillaje: picks, nut check, reject y drops en WORLD	Gestamp controles / Ethos (Edgar)	Requiere decisión
C08	-	Datos de carga: LOAD_DATA[1] tecleado; sin carga con/sin pieza	Programador robot / Puesta en marcha	Abierto
C09	-	Detección de colisión apagada en todo el programa; la reacción es un HALT	Programador robot / Puesta en marcha	Abierto
C10	-	Falta el programa de mastering ZERO_G1	Programador robot	Abierto

ID	Sev.	Punto	Responsable	Estado
C11	-	Zonas sin documentar: ninguna zona pedida, falta la forma NA-FM-71-138.1	Ethos (Edgar) / Gestamp controles	Requiere decisión
C12	-	Pounce común: el ciclo no pasa por pounce y do007RobotAtPounce no se calcula	Gestamp controles / Ethos (Edgar)	Requiere decisión
C13	-	Manejo de zonas coherente	Programador robot	No aplica
C14	-	Brake test: el 'due' no se reporta y el camino usa tool/base de otro robot	Programador robot	Abierto
C15	-	Dry cycle seleccionable y sin colgar el robot	Programador robot	Se cierra con otro punto
C16	-	Pieza presente / pieza en gripper	Programador robot	Se cierra con otro punto
C17	-	Verificación de tuercas y red rabbit	Programador robot	Se cierra con otro punto
C18	-	Handshakes AutomationCore de pick, application y drop	Programador robot	Se cierra con otro punto
C19	-	Request to enter: doCriticalWZ nunca se escribe; el PLC ve entrada permitida	Programador robot / Seguridad	Requiere decisión
C20	-	Conteo de soldaduras al final del ciclo: #CHECK apagado y WeldNoRequired distinto de 3	Programador robot	Abierto
C21	-	Fallas por mensajes AutomationCore: ningún HALT sin mensaje ni WAIT FOR FALSE	Programador robot	Abierto
C22	-	Grippers por inline forms GripperTech	Programador robot / Puesta en marcha	Abierto
C23	-	Nombres AutomationCore para la E/S, sin direcciones literales	Programador robot	Se cierra con otro punto
C24	-	Los comentarios nombran la señal con su dirección	Programador robot	Cerrado - depuración
C25	-	Inline forms consistentes con el código	Programador robot	Se cierra con otro punto
C26	-	Solo inglés en comentarios, títulos de fold y &COMMENT	Programador robot	Cerrado - depuración
C27	-	Proceso controlado por robot no compartido; regla de dos procesos	Ethos (Edgar)	No aplica
C28	-	Ediciones del integrador en archivos del proveedor: listarlas en la entrega	Ethos (Edgar) / Programador robot	Abierto

6. Puntos abiertos — detalle

G01 — Cambiar el nombre del robot

Gestamp: "Robot name needs to change" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 1

Severidad: Medio · **Tipo:** Configuración en celda · **Responsable:** Programador robot (en celda) / Gestamp confirma el formato · **Estado:** Abierto

Módulos: am.ini, C/KRC/Roboter/Rdc/RobotData.xml, \$ROBNAME (smartHMI)

Qué está mal. El nombre del robot en el controlador sigue siendo V431_03_10_R1 (am.ini: RobName; RobotData.xml: RobotName); también da nombre a los respaldos (v431_03_10_r1.zip). Gestamp identifica esta estación como BMW-03-10R1.

Qué hacer. Pedir a Gestamp el texto exacto del nombre (p. ej. BMW_03_10_R1; el guion puede no ser aceptado). Cambiarlo en el smartHMI (Puesta en servicio → Datos del robot → Nombre del robot) en modo experto. El encabezado de los programas ya dice BMW-03-10-R1 (depuración, C02).

Criterio de cierre (se verifica en el respaldo):

- Formato confirmado por Gestamp por escrito (correo adjunto a la lista).
- En el respaldo, am.ini (RobName) y C/KRC/Roboter/Rdc/RobotData.xml (RobotName) traen el nombre nuevo; el nombre del archivo de respaldo también.
- Si el formato confirmado difiere de BMW-03-10-R1, la línea "; Robot 03-10-R1 (...)" de los encabezados se ajusta igual (C02).

Evidencia del programador. Correo de Gestamp con el formato; captura de la pantalla Datos del robot con el nombre nuevo.

Depende de / se cierra con: C02

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 1 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - avance parcial: El nombre del robot ya es BMW_03_10_R1 (am.ini y RobotData.xml). Falta el formato confirmado por Gestamp por escrito (criterio 1); si difiere, se cambia otra vez.
- 2026-10-05 — Integrado: El nombre BMW_03_10_R1 se conserva en la versión integrada. Falta el formato confirmado por Gestamp.

G02 — Herramienta de seguridad igual al TCP de la herramienta 1

Gestamp: "Safety Tool is not equal to Default TCP" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 2

Severidad: Alto · **Tipo:** Configuración de seguridad · **Responsable:** Programador robot / Seguridad · **Estado:** Abierto

Módulos: C/KRC/Roboter/Config/User/Common/SafetyConfigData.xml, \$config.dat (TOOL_DATA[1])

Qué está mal. No se puede verificar desde el respaldo: la geometría de la herramienta de seguridad vive en el controlador de seguridad, no en texto. La bitácora de seguridad muestra tres cambios a la herramienta 1 el 2026-08-30. TOOL_DATA[1] (TOOL 1, EOAT 1) = {X 0, Y 0, Z 380.79, A -90, B -90, C -90}.

Qué hacer. En el smartHMI comparar el TCP de la herramienta de seguridad 1 con TOOL_DATA[1] y corregirlo (y las esferas de la herramienta si aplica). Es un cambio de configuración de seguridad: nueva suma de verificación y prueba de aceptación (checklist de SafeOperation).

Criterio de cierre (se verifica en el respaldo):

- Captura de la herramienta de seguridad 1 con el TCP igual a TOOL_DATA[1] del respaldo.
- Checksum nuevo anotado y checklist de aceptación firmado por quien tenga la autorización.

Evidencia del programador. Capturas antes/después, checksum y checklist.

Depende de / se cierra con: F42

Nota. Conviene hacerlo en la misma sesión de seguridad que F42 (espacios de SafeOperation). Si G03 o C06 cambian TOOL_DATA[1], repetir la comparación.

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 2 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G03 — Determinar los datos de carga (herramienta 1 con y sin pieza)

Gestamp: "Tool 1 is used without Load data determination" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 3

Severidad: Alto · **Tipo:** Prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto

Módulos: \$config.dat (LOAD_DATA[1]), KRC/STEU/Mada/\$custom.dat (\$LDC_CONFIG[1])

Qué está mal. LOAD_DATA[1] = {M 210, CM X 270 Y 0 Z 240, J 105/105/105}: valores redondos tecleados, no determinados. El chequeo de datos de carga de la herramienta 1 solo avisa (\$LDC_CONFIG[1] = #WARNONLY). Con datos de carga falsos la detección de colisión y la dinámica no son confiables (C08, C09, F39).

Qué hacer. Correr KUKA Load Data Determination para la herramienta 1 (EOAT 1) sin pieza y con la pieza GE4; declarar la carga con pieza (segunda herramienta o carga adicional, según el estándar Gestamp) y usarla en los movimientos con pieza.

Criterio de cierre (se verifica en el respaldo):

- LOAD_DATA[1] en \$config.dat ya no es {M 210, CM {X 270, Y 0, Z 240}, J {105, 105, 105}}: trae masa, centro de masa e inercias determinados.
- Existe la carga con pieza (si difiere 5 % o más) y los movimientos entre el cierre del gripper en el pick y su apertura en drop, drop red rabbit o reject la usan (C08).
- Protocolo de Load Data Determination (captura o archivo) con fecha.

Evidencia del programador. Captura del resultado de Load Data Determination sin y con pieza, con fecha.

Depende de / se cierra con: C08, F39, C09

Nota. Calvin describe el R10 como robot de pistola; es un robot de manejo (EOAT 1 + pieza). Los hechos se sostienen.

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 3 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G04 — Programa RedRabbit bloqueado: borrado

Gestamp: "Program Red Rabbit() is blocked, might be an unnecessary copy" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 4

Severidad: Bajo · **Tipo:** Código · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/Utilities/redrabbit.src, KRC/R1/Program/Utilities/RedRabbit.dat

Qué está mal. redrabbit.src (copia de una celda de birlos) se bloqueaba con WAIT FOR FALSE justo después del INI, nadie lo llamaba y no podía enlazar. El ciclo red rabbit real es Style1Opt10AutoRR (option 12), que tiene sus propios problemas (F19).

Qué hacer. Hecho en la depuración (commit 0e1ea54): redrabbit.src y RedRabbit.dat borrados, con la razón en tools/cleanup_allowlist; tools/check_cleanup.py prueba que nada los referenciaba. Falta que desaparezcan del controlador al cargar la versión de oficina: un restore no borra archivos, hay que borrarlos a mano.

Criterio de cierre (se verifica en el respaldo):

- En el respaldo no están KRC/R1/Program/Utilities/redrabbit.src ni RedRabbit.dat (borrados también en el controlador).
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.

- El programa carga sin errores de compilación (captura del navegador del smartPAD sin módulos en rojo). Un ciclo red rabbit (option 12) corre igual que antes de la carga (sus defectos siguen en F19). Anotar fecha.

Evidencia del programador. Captura del navegador sin redrabbit y fecha del ciclo red rabbit.

Depende de / se cierra con: F19

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 4 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

G05 — Programas sin uso

Gestamp: "Unused programs" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 5

Severidad: Bajo · **Tipo:** Código + configuración · **Responsable:** Programador robot / Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Centerline/centerline_loop.src, KRC/R1/safetest.src, KRC/R1/Program/MoveToPurge.src, KRC/R1/TP/GLUETECH, 11 módulos borrados por la depuración (tools/cleanup_allowlist)

Qué está mal. Había 11 módulos que nada llamaba: CENTERLINE_WELD1, trigger1-3, CK1STXXX, POUNCE, capchange, electrodechange, redrabbit y las dos copias AutoRR del nut check y de la cámara; además CENTERLINE_LOOP (lazo manual de válvulas del pedestal sin salida ni interlocks). Quedan safetest.src (lazo PTP sin fin a un punto, prueba manual que nada llama) y el paquete GlueTech (TP/GLUETECH y Program/MoveToPurge.src), instalado en un robot de soldadura de tuercas y sin uso.

Qué hacer. Hecho: la depuración borró los 11 módulos (razón y prueba en tools/cleanup_allowlist) y la oficina borró CENTERLINE_LOOP.src (tools/code_changes.json). Falta: (1) decidir con Gestamp si safetest se queda como prueba manual documentada o se borra; (2) desinstalar GlueTech desde el smartHMI (es un paquete instalado: no se borra a mano); (3) borrar en el controlador los módulos borrados al cargar la versión de oficina.

Criterio de cierre (se verifica en el respaldo):

- En el respaldo no están los 11 módulos borrados por la depuración ni KRC/R1/Program/Centerline/centerline_loop.src (borrados también en el controlador: un restore no borra archivos).
- safetest: decisión de Gestamp anotada en la lista; si se borra, no está en el respaldo; si se queda, su encabezado dice para qué prueba es y quién la usa.
- GlueTech desinstalado: no aparece en la lista de paquetes del smartHMI, y ni KRC/R1/TP/GLUETECH ni Program/MoveToPurge.src están en el respaldo.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- El programa carga sin errores de compilación (captura del navegador del smartPAD sin módulos en rojo). Un ciclo completo en automático sale bien. Anotar fecha.

Evidencia del programador. Decisión sobre safetest; captura de la lista de paquetes instalados después de desinstalar GlueTech.

Depende de / se cierra con: G04, C28

Nota. Parte de oficina hecha (12 módulos borrados en el repositorio); falta la decisión de safetest, la desinstalación de GlueTech y el borrado en el controlador.

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 5 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - pendiente: Los 22 archivos borrados en la depuración siguen en el controlador; dos copias AutoRR fueron editadas (sin uso).

G06 — Cambio de electrodo: confirmación de 0.1 s y sin handshake

Gestamp: "Electrode change confirmation only 0.1 s, no handshake" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 6

Severidad: Medio · **Tipo:** Decisión + código · **Responsable:** Programador robot / Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Utilities/gunelectrodechange.src, KRC/R1/cell.src

Qué está mal. En gunelectrodechange.src el reset del stepper es PULSE(dopw1_StepperReset,TRUE,0.1) (\$OUT[485]) sin confirmar \$IN[487] dipw1_EndOfStepper, y el handshake con el PLC no cierra (F43). Además la rutina nunca corre: cell.src la llama con \$OUT[111..113], que solo escribe AutomationCore_Bkg, deshabilitado (F12).

Qué hacer. Ver F43: definir con Gestamp cómo se pide el cambio de electrodo (bit del PLC o contador), confirmar el reset del stepper y cerrar el handshake por niveles.

Criterio de cierre (se verifica en el respaldo):

- Se cumplen los criterios de F43.

Evidencia del programador. Ver F43.

Depende de / se cierra con: F43, F12

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 6 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G07 — Options 1, 2, 7 y 8 de Style_1 llaman el mismo programa

Gestamp: "Multiple options in Style_1 call the same sub-program" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 7

Severidad: Alto · **Tipo:** Decisión Gestamp + código · **Responsable:** Gestamp controles / Programador robot · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Styles/style_1.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src

Qué está mal. Style_1 manda las options 1, 2, 7 y 8 a Style1Opt1. El nut check (style1app2opt1) solo decide la option 1; con 2, 7 u 8 decide solo la cámara y, si no contesta ni OK ni NG, bscrapGE4 conserva su valor anterior (F02).

Qué hacer. Gestamp define qué es cada option. Cada option distinta tiene su propio programa o su propia evaluación; si son iguales a la 1, las evaluaciones las incluyen.

Criterio de cierre (se verifica en el respaldo):

- Definición de options por escrito de Gestamp.
- Se cumplen los criterios de F02.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Definición de Gestamp y prueba por option.

Depende de / se cierra con: F02

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 7 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G08 — El ciclo red rabbit no sigue el camino de producción

Gestamp: "Separate Auto-Red Rabbit program that doesn't match production" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 8

Severidad: Alto · **Tipo:** Decisión + código · **Responsable:** Gestamp controles / Programador robot · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src

Qué está mal. El red rabbit (option 12, Style1Opt10AutoRR) tiene su propio pick y su propio drop. El drop cierra el gripper 1 con salidas escritas a mano, suelta con el form del gripper 3 (que comparte la E/S del gripper 1), se mueve antes de confirmar gripper vacío, y el ciclo no puede dar resultado al PLC (F19).

Qué hacer. Definir con Gestamp el procedimiento red rabbit: número de option, qué pieza (qué tuercas faltan), qué se espera de cada chequeo y dónde termina la pieza. Reusar el pick y los chequeos de producción y cambiar solo lo necesario.

Criterio de cierre (se verifica en el respaldo):

- Procedimiento red rabbit de Gestamp por escrito.
- Se cumplen los criterios de F19.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Procedimiento y prueba red rabbit con resultado esperado.

Depende de / se cierra con: F19, G21

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 8 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G09 — Pieza en gripper al PLC fijada por programa, no por los sensores

Gestamp: "Part control to PLC is set/reset instead of reflecting the real part controls" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 9

Severidad: Medio · **Tipo:** Decisión + código · **Responsable:** Programador robot / Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/System/sps.sub

Qué está mal. \$OUT[18] do018Part1InGripper lo pone el inline form OUT 18 de los picks (fuera de dry cycle) y lo quitan el drop al conveyor, el drop red rabbit y RejectGE4; si la pieza se cae o alguien la quita a mano, el PLC sigue viendo pieza. PARTPRESENT1..3 no son señales al PLC: son las banderas internas del nut check (su problema es F03).

Qué hacer. Que \$OUT[18] siga a los sensores \$IN[253] di253PartPresent1 y \$IN[254] di254PartPresent2 en todo momento (una línea en sps.sub, o donde decida Gestamp) y quitar los OUT 18 de picks, drops y reject.

Criterio de cierre (se verifica en el respaldo):

- \$OUT[18] se escribe en un solo lugar, a partir de di253/di254; los inline forms OUT 18 de picks, drops y RejectGE4 ya no están.
- Prueba: quitar la pieza a mano del gripper en T1 → \$OUT[18] OFF de inmediato; con la pieza tomada → ON. Anotar fecha.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.

- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Dónde se escribe y prueba con fecha.

Depende de / se cierra con: F12, C01, C16

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 9 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G10 — Comandos de gripper GripperTech en todos los movimientos de gripper

Gestamp: "Gripper commands not used at all times" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 10

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/TP/GripperSpotTech/grp_data.dat, C/KRC/User/TP/Gripper_SpotTech/GripperConfig.xml

Qué está mal. El drop red rabbit cierra el gripper 1 con salidas escritas a mano (\$OUT[249] OFF, \$OUT[250] ON, sin check) y suelta con el form del gripper 3. Los forms SET de los tres picks y del drop al conveyor traían parámetros ocultos de otro gripper (setgrripper=4;setstate=2) y cada SET con check iba seguido de un GRPg_Check redundante (F31). GripperConfig.xml está desincronizado con grp_data.dat (F33).

Qué hacer. Hecho en oficina: parámetros ocultos de los forms SET de los tres picks y del drop al conveyor corregidos a setgrripper=1 con el estado de la llamada; GRPg_Check redundantes quitados (picks, drop al conveyor, RejectGE4). Falta: drop red rabbit con forms GripperTech del gripper 1 (F31, F19) y re-sincronizar GripperConfig.xml (F33).

Criterio de cierre (se verifica en el respaldo):

- Se cumplen los criterios de F31 y F33.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Prueba en el smartPAD: abrir y confirmar (Cmd OK) un form Gripper SET de un pick y el del drop al conveyor sin cambiar nada; en el respaldo siguiente la llamada sigue siendo GRPg_SetStateAndCheck(1, ...) con el mismo estado. Un ciclo completo en automático de estación 1 y de estación 2. Anotar fecha.

Evidencia del programador. Ver F31 y F33; fecha de la prueba Cmd OK y del ciclo.

Depende de / se cierra con: F31, F33, C22, F40

Nota. Parte de oficina hecha (parámetros ocultos y GRPg_Check redundantes, tools/code_changes.json); falta el drop red rabbit y F33.

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 10 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G11 — Sin GOTO ni etiquetas

Gestamp: "Use of Goto labels" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 11
Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/Styles/Options/style1opt1.src

Qué está mal. Había GOTO en 9 módulos: saltos de dry cycle (picks, weld apps, nut check), retry muerto, salto de tuerca precargada y chequeo de gun cerrado en CENTERLINE_WELD, y la estructura de Style1Opt1.

Qué hacer. Hecho en oficina: todos los GOTO y etiquetas se quitaron. Dry cycle = IF NOT di004UseDryCycle; tuerca precargada = IF NOT NUT_READY; chequeo de gun cerrado = LOOP/EXIT (después del HALT los timers se rearrancan, F07); Style1Opt1 = un IF/ELSE por ciclo con la estación leída una vez en nPickStation (F30). tools/check_equivalence.py muestra que, fuera de esos puntos, la lógica es la misma que en la base fb8a4c8. Falta probar en celda.

Criterio de cierre (se verifica en el respaldo):

- En el respaldo ningún módulo del integrador (KRC/R1/Program, KRC/R1/cell.src, sps.sub) tiene GOTO ni etiquetas.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Prueba en celda: un ciclo sin dry cycle de estación 1 y uno de estación 2 (T1 y automático), un rechazo por nut check (option 1), un ciclo red rabbit (option 12), un pick en dry cycle (sin espera de pieza; el ciclo se sigue colgando en la primera weld app por F04, igual que antes) y el timeout de gun cerrado en T1 sin tuerca (HALT, START: vuelve a dar timeout si sigue sin cerrar). Anotar fecha y resultado de cada uno.

Evidencia del programador. Fecha y resultado de cada prueba.

Depende de / se cierra con: F30, F07

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 11 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Auditoría - avance en el original: El programador quitó los GOTO por su cuenta en el programa original (apps, CENTERLINE_WELD, picks, nut check, cámara, Style1Opt1). La lógica es equivalente a la versión de oficina; hay que integrarla en la versión depurada, no cargar la suya.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

G12 — Nombres de señal en lugar de números de E/S

Gestamp: "Use of I/O numbers instead of signal names" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 12

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/System/sps.sub, KRC/R1/TP/AutomationCore/automationcoreroutines.dat, KRC/R1/System/\$config.dat

Qué está mal. PRELOAD, CENTERLINE_WELD, CENTERLINE_HOME y sps.sub usaban \$IN[n]/\$OUT[n] literales para el pedestal, el agua y el faro KL50L2.

Qué hacer. Hecho en oficina: los bits del pedestal recibieron nombre AutomationCore en automationcoreroutines.dat (do498UpperPinExtend, do499UpperPinRetract, do501PartClampClose, do502PartClampOpen, do503QFPAdvance, do504QFPReturn, do505QFPNutBlowOff, di466PartClampOpen, di467PartClampClosed, di470QFPAdvanced), los tres bits del faro en \$config.dat (CL_BeaconBit3001/3008/3024) y el código usa nombres (también los NutWeld dopw1_/dipw1_). Las direcciones no cambiaron. Los inline forms estándar de KUKA (WAIT FOR, OUT) siguen generando \$IN[n]/\$OUT[n] con el nombre en el form: así es el estándar y no se reescriben a mano.

Criterio de cierre (se verifica en el respaldo):

- En el respaldo ninguna línea ejecutable del integrador fuera de un inline form KUKA usa \$IN[n]/\$OUT[n] literal (la auditoría lo revisa).
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- El programa carga sin errores de compilación (captura del navegador del smartPAD sin módulos en rojo). Un ciclo completo en automático sale bien. Anotar fecha.

Evidencia del programador. Captura del navegador y fecha del ciclo de prueba.

Depende de / se cierra con: F17, C23, F29

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 12 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

G13 — Código simplificado (nut present en una línea, duplicados fuera)

Gestamp: "Poor programming (10 lines for nut present)" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 13

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/Utilities/RejectGE4.src

Qué está mal. Cada revisión de tuerca de style1app2opt1 eran unas 10 líneas (GOTO de dry cycle, IF que solo ponía TRUE, dwell después de leer). Había código repetido: un GRPg_Check del mismo estado después de cada GRPg_SetStateAndCheck y las líneas de upper pin extend dobles en la alimentación local de CENTERLINE_WELD.

Qué hacer. Hecho en oficina: cada revisión de tuerca es "WAIT SEC 0.2 / PARTPRESENTn = di227NutPresent1" dentro de IF NOT di004UseDryCycle (lee con el robot parado y escribe TRUE o FALSE; F28, F03); se quitaron los GRPg_Check redundantes (picks, drop al conveyor, RejectGE4), las líneas dobles de upper pin y la etiqueta WELD_END sin uso.

Criterio de cierre (se verifica en el respaldo):

- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Prueba en automático con option 1: 5 ciclos con pieza buena sin rechazo por tuercas; un ciclo con una tuerca faltante (o una pieza red rabbit) termina en RejectGE4; PARTPRESENT1..3 leídos en el display de

variables. En cada pick, drop y reject el gripper confirma su estado con GRPg_SetStateAndCheck, sin el GRPg_Check extra. Anotar fecha y resultado.

Evidencia del programador. Fecha, número de ciclos y resultado.

Depende de / se cierra con: F28, F03, F31

Nota. Con options 2, 7 y 8 el sensor sigue sin decidir (F02): la prueba de tuerca faltante se hace con option 1.

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 13 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

G14 — Revisar gripper vacío antes del comando de gripper

Gestamp: "Gripper opened before the part control check" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 14

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src

Qué está mal. Los tres picks abrían el gripper y después revisaban que no hubiera pieza.

Qué hacer. Hecho en oficina: la espera de gripper vacío (\$IN[253] di253PartPresent1 y \$IN[254] di254PartPresent2 OFF, inline form WAIT FOR) se movió antes del primer form Gripper SET (GRPg_SetStateAndCheck(1, 1, ...)) en los tres picks.

Criterio de cierre (se verifica en el respaldo):

- En los tres picks la espera de gripper vacío está antes del primer comando de gripper.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Prueba en T1: con una pieza en el gripper al iniciar el pick, el robot se queda en la espera sin mover el gripper. Anotar fecha.

Evidencia del programador. Fecha y resultado.

Nota. Los drops y el reject (abrir sin confirmar pieza) son F40.

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 14 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

G15 — Nombres, variables y comentarios en inglés

Gestamp: "Names, variables and comments not in English" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 15

Severidad: Bajo · **Tipo:** Código · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/TP/AutomationCore/automationcoreroutines.dat, KRC/R1/System/\$config.dat, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/Request_next_nut.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src

Qué está mal. Comentarios en español y nombres mal escritos: NUT_STAR, di100/di101CameraJudgment, do106/di106PartRejeted, la carpeta Program/Styles/options; \$IN[227] se llamaba di227SensorCamera aunque es el sensor de tuercas (F36).

Qué hacer. Hecho: comentarios, títulos de fold y &COMMENT en inglés (depuración, C26); en oficina NUT_STAR → NUT_START, CameraJudgment → CameraJudgment, PartRejeted → PartRejected, di227SensorCamera → di227NutPresent1 (F36) y la carpeta Styles/options renombrada Styles/Options. Quedan "Hidraulyc_Valve" en las declaraciones del paquete NutWeld (archivo del proveedor, ningún programa del integrador lo usa) y el texto 'do106PartRejeted' de los inline forms OUT 106 de RejectGE4 (se corrige solo si se reabre el form).

Criterio de cierre (se verifica en el respaldo):

- La auditoría no encuentra comentarios en español ni los nombres viejos (NUT_STAR, Judgment, Rejeted, di227SensorCamera) en código ejecutable ni en declaraciones del respaldo.
- En el controlador ya no existe la carpeta KRC/R1/Program/Styles/options: style1opt1 y Style1Opt10AutoRR están solo en Styles/Options (la carpeta vieja se borra a mano; si queda, hay dos módulos con el mismo nombre).
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- El programa carga sin errores de compilación (captura del navegador del smartPAD sin módulos en rojo). Un ciclo completo en automático sale bien. Anotar fecha.

Evidencia del programador. Captura del navegador sin errores y fecha del ciclo.

Depende de / se cierra con: C26, F36

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 15 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

G16 — Constantes para límites de carrera, presiones y tiempos

Gestamp: "NO variables for e.g. stroke limits" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 16

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/System/\$config.dat, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src

Qué está mal. CENTERLINE_WELD tenía escritos la ventana del LPT con gun cerrado (69000..73000), el límite de gun abierto (120000), los tiempos del chequeo de gun cerrado (1000 y 300 ms), las palabras de presión (405, 1820, 39425) y el programa Bosch (2). CL_GUN_WELD_MIN/MAX estaban declaradas

65000..76000 y sin uso.

Qué hacer. Hecho en oficina: fold CENTERLINE CONTROL de \$config.dat con CL_GUN_WELD_MIN/MAX = 69000/73000 (la ventana que el código siempre revisó), CL_GUN_OPEN_MIN, CL_GUN_CLOSE_TIMEOUT, CL_GUN_IN_WINDOW_TIME, CL_GUN_PRESS_BASE/WELD/REST (405/1820/39425, los valores en uso; el orden de bytes sigue por confirmar, F44) y CL_WELD_PROGRAM; CENTERLINE_WELD y CENTERLINE_HOME los usan. Mismos valores. La verificación del pin superior sigue en F16; los tiempos de PRELOAD y el límite de agua en F41.

Criterio de cierre (se verifica en el respaldo):

- No quedan esos números en líneas ejecutables de CENTERLINE_WELD / CENTERLINE_HOME.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Un ciclo de soldadura completo en automático; valores leídos en el display de variables iguales a los del fold CENTERLINE CONTROL. Anotar fecha.

Evidencia del programador. Fecha del ciclo de prueba.

Depende de / se cierra con: F16, F44, F41

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 16 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

G17 — Lógica Nut_weld_Retry inútil

Gestamp: "Nut_weld_Retry logic useless" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 17

Severidad: Bajo · **Tipo:** Código · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/System/\$config.dat

Qué está mal. NUT_WELD_RETRY nunca se ponía en TRUE: las tres ramas GOTO RETRY_NUT1 no hacían nada (F24).

Qué hacer. Hecho en oficina (opción A de F24, como en R20): se borraron las ramas, las etiquetas y la variable. Sin cambio de comportamiento.

Criterio de cierre (se verifica en el respaldo):

- NUT_WELD_RETRY y RETRY_NUT1 no aparecen en el respaldo.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Un ciclo completo en automático con las 3 tuercas, sin errores. Anotar fecha.

Evidencia del programador. Ciclo completo sin errores, con fecha.

Depende de / se cierra con: F24

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 17 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

G18 — Sin chequeo de soldadura buena

Gestamp: "NO check for Weld OK, just weld finished" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 18

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Styles/Options/style1opt1.src

Qué está mal. CENTERLINE_WELD toma weld complete (\$IN[483] dipw1_WeldComplete) como soldadura buena; la falla Bosch \$IN[485] no se lee y sus dos nombres se contradicen (dipw1_Fault en NutWeld, di485NoFaults en AutomationCore) (F08).

Qué hacer. Confirmar en celda la señal de soldadura OK / falla del timer Bosch y su polaridad; después de weld complete revisarla y, si es NOK, mensaje y la pieza a RejectGE4. Usar la señal NutWeld del bus (\$IN[485]); el patrón del CENTERLINE_WELD1 borrado usaba señales Bosch de AutomationCore que no están en el bus de este robot.

Criterio de cierre (se verifica en el respaldo):

- Se cumplen los criterios de F08.

Evidencia del programador. Ver F08.

Depende de / se cierra con: F08

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 18 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G19 — Handshake de la tuerca frágil ante selección de bloque, reset y reinicio

Gestamp: "Next_Nut_Ready logic can fail at block selection, reset, reboot" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 19

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/Request_next_nut.src, KRC/R1/System/\$config.dat, KRC/R1/cell.src, KRC/R1/System/sps.sub, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src

Qué está mal. El handshake de la tuerca vive en globales persistentes de \$config.dat (NUT_READY declarado TRUE, NUT_START, NEXT_NUT_READY, NUT_PRELOAD_STEP, STEP*DONE...). Nada las inicializa al arrancar el programa, en reset ni al reiniciar el controlador (F38). Una selección de bloque sobre un fold de movimiento salta el TRIGGER que pide la tuerca, y NUT_READY no se consume al soldar (F14): la tuerca 1 puede pasar con un TRUE viejo, y con la petición de la tuerca 2 o 3 saltada la app espera NEXT_NUT_READY para siempre.

Qué hacer. Ver F38 (inicialización del estado) y F14 (consumo de NUT_READY y petición en el flujo).

Criterio de cierre (se verifica en el respaldo):

- Se cumplen los criterios de F38 y F14.

Evidencia del programador. Ver F38 y F14.

Depende de / se cierra con: F38, F14

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 19 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G20 — Request_next_nut y NUT_START fuera de los inline forms

Gestamp: "Change of xP0 deletes Request_next_nut inside inline forms" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 20

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src

Qué está mal. Los TRIGGER de la siguiente tuerca (NUT_STAR en P12/P10 de los picks, Request_next_nut() en el fold P0 de las apps 1 y 2) y los resets del red rabbit estaban escritos dentro de folds de inline form: un Touch Up o reabrir el form los borraba (F32).

Qué hacer. Hecho en oficina: cada TRIGGER quedó justo antes de su fold de movimiento (posición del form SYN OUT de KUKA: mismo movimiento, mismo punto de disparo; se conservan DISTANCE = 0 y 1); los resets del red rabbit van después del fold DropOff[2] Reset. La robustez ante selección de bloque, reset y reinicio sigue en F14 y F38 (G19).

Criterio de cierre (se verifica en el respaldo):

- Ningún fold de inline form contiene líneas escritas a mano (la auditoría lo revisa).
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Prueba: abrir el inline form PTP P12 de style1pick1opt1 y PTP P0 de style1app1opt1 y confirmarlos con Cmd OK sin cambiar nada (sin Touch Up, que reenseña el punto); el TRIGGER sigue antes de cada fold. Después, un ciclo completo en automático: NUT_READY y NEXT_NUT_READY (display de variables) pasan a FALSE y vuelven a TRUE una vez por tuerca (3 preloads por pieza). Anotar fecha.

Evidencia del programador. Fecha y resultado.

Depende de / se cierra con: F32, F14

Nota. Diferencia con la versión anterior: antes el TRIGGER estaba dentro del fold y una selección de bloque sobre ese movimiento lo ejecutaba; ahora está una línea arriba y hay que seleccionar la línea del TRIGGER. En operación normal, paso a paso en T1 y Touch Up no cambia nada. La solución de fondo es F14.

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 20 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

G21 — El nut check no funciona para red rabbit

Gestamp: "Style1app2Opt1 will not work for RedRabbit" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 21

Severidad: Alto · **Tipo:** Decisión + código · **Responsable:** Gestamp controles / Programador robot · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src

Qué está mal. El ciclo red rabbit corre como option 12 y style1app2opt1 solo tiene CASE 1 y CASE 10: con option 12 no se pone ni \$OUT[141] do141RedRabbitFailed ni \$OUT[142] do142RedRabbitPassed; la cámara solo escribe bscrapGE4 (F19).

Qué hacer. Con la definición de la pieza red rabbit (G08), evaluar el resultado esperado: passed si el chequeo detecta la falta, failed si no; un solo resultado combinado de sensor y cámara para la option que

use el red rabbit.

Criterio de cierre (se verifica en el respaldo):

- Se cumplen los criterios de F19 para el nut check y la cámara.

Evidencia del programador. Ver F19.

Depende de / se cierra con: F19, G08

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 21 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

G22 — Uso incorrecto de AutomationCore

Gestamp: "Use of AutomationCore is wrong in multiple places" — Gestamp - Robot Program Review - Findings (2026-10-02), BMW-03-10R1, punto 22

Severidad: Alto · **Tipo:** Decisión Gestamp + código · **Responsable:** Gestamp controles / Programador robot · **Estado:** Requiere decisión

Módulos: KRC/R1/System/sps.sub, KRC/R1/TP/AutomationCore/automationcoreroutines.src, inline forms AutomationCore, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/cell.src, KRC/R1/Program/masref_user.src

Qué está mal. Gestamp no dice en qué lugares. Lo que encontramos: AutomationCore_Bkg deshabilitado (F12, C01), EndOfCycle del proveedor editado sin Work Complete (F46), parámetros ocultos de forms que no coinciden con la llamada (F21), reject sin drop-off ni liberación de application 1 (F20), ninguna zona pedida (C11), pounce no usado (C12), handshakes (C18), request to enter (C19), brake test (C14) y do930 que queda en ON (F35).

Qué hacer. Pedir a Gestamp la lista concreta y atenderla junto con los puntos relacionados.

Criterio de cierre (se verifica en el respaldo):

- Lista de Gestamp recibida y anotada aquí.
- Cada lugar de la lista corregido y auditado.

Evidencia del programador. Lista de Gestamp.

Depende de / se cierra con: F12, C01, F46, F21, F20, C11, C12, C18, C19, C14, F35

Historial de auditoría:

- 2026-10-04 — Alta (Gestamp): Punto 22 de la revisión de Gestamp del 2026-10-02 (BMW-03-10R1).

CRÍTICO — F01 — Entrada al pedestal sin confirmar gun abierto, clamp abierto y QFP regresado

Severidad: Crítico · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/centerline_weld.src

Qué está mal. Antes del LIN de aproximación a la pose de soldadura (2 m/s) las weld apps solo esperan la bandera del preload: NUT_READY en style1app1opt1, NEXT_NUT_READY en style1app1opt2 y opt3. PRELOAD pone esas banderas después de mandar gun open, upper pin retract y clamp open sin leer retroalimentación, y en el paso 20 da por regresado el QFP por tiempo (\$TIMER[12], 800 ms) sin leer \$IN[482] dipw1_SpearHome. CENTERLINE_WELD revisa clamp abierto (\$IN[466]) y QFP regresado (\$IN[482]) con el robot ya en la pose, y gun abierto (CL_GunStrokePositionLPT > CL_GUN_OPEN_MIN) solo al final. Si el gun o el clamp quedaron cerrados (válvula pegada, falta de aire, una corrida cancelada de CENTERLINE_LOOP), el robot mete pieza y EOAT contra el electrodo, el clamp o el slide.

Qué hacer. En cada weld app, después de la espera de NUT_READY / NEXT_NUT_READY e inmediatamente antes del LIN de aproximación, una espera supervisada (timeout, mensaje que nombre la condición que falta, \$OUT[4] do004ProcessFault) de gun abierto, clamp abierto y QFP regresado y no avanzado. En PRELOAD, el paso 20 no termina sin \$IN[482] dipw1_SpearHome ON. Las esperas que ya existen dentro de CENTERLINE_WELD se quedan.

Criterio de cierre (se verifica en el respaldo):

- En style1app1opt1, opt2 y opt3, inmediatamente antes del LIN de aproximación (P20, P6 y P13 respectivamente), después de la espera de NUT_READY / NEXT_NUT_READY y fuera de cualquier fold de inline form, existe una espera supervisada (timeout + mensaje + \$OUT[4] do004ProcessFault) de: CL_GunStrokePositionLPT > CL_GUN_OPEN_MIN, \$IN[466] di466PartClampOpen ON, \$IN[482] dipw1_SpearHome ON y \$IN[470] di470QFPAdvanced OFF. Corre también en dry cycle, porque el LIN de aproximación se ejecuta en los dos casos.
- La espera detiene el advance run (WAIT FOR, o WAIT SEC 0 antes de una evaluación con IF o lazo): la condición sobre CL_GunStrokePositionLPT, que es una variable y no una entrada, no se evalúa por adelantado.
- En PRELOAD, paso 20: STEP20DONE (y por tanto NUT_READY / NEXT_NUT_READY = TRUE) exige \$IN[482] dipw1_SpearHome ON además de \$TIMER[12].
- Prueba en T1 con override reducido, una por condición: clamp no abierto, QFP no regresado y gun no abierto (sensor desconectado o válvula sin aire). En cada caso el robot se detiene antes del LIN de aproximación con el mensaje que nombra la condición y \$OUT[4] ON, y continúa solo al corregirla. Anotar fecha, override y resultado de cada caso.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F01) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Módulos cambiados; timeout elegido (ms) y texto del mensaje; por cada condición probada: cómo se provocó, fecha, override, punto donde se detuvo el robot y mensaje mostrado.

Nota. No quitar las esperas de \$IN[466] y \$IN[482] dentro de CENTERLINE_WELD. Si F04 condiciona la espera de NUT_READY a no-dry, no la mueve después del LIN de aproximación. No protege contra un segundo preload encolado (F11). Si F23 cambia la composición del LPT, el límite de gun abierto se revisa con ella. Avance 2026-10-05: el gun abierto se confirma antes de entrar (GUN_OPEN_CHECK) y se vigila con interrupts 20-22 mientras el robot entra y sale; el aire (\$IN[3132]) está desactivado hasta probar el presostato.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - avance parcial: Nuevo GUN_OPEN_CHECK antes de entrar y al terminar la soldadura (gun en 137000-142000, submit corriendo, agua; aire desactivado hasta probar \$IN[3132]) e interrupts 20-22 con BRAKE F mientras el robot entra y sale. Falta: clamp abierto (\$IN[466]) y QFP regresado (\$IN[482]) antes de entrar, y la prueba en celda. Los límites 137000/142000 están dos veces (interrupts y GUN_OPEN_CHECK.dat): unificar con las constantes de G16.
- 2026-10-05 — Integrado - avance parcial: GUN_OPEN_CHECK e interrupts 20-22 del programador integrados en la versión depurada (rango 137000-142000 en GUN_OPEN_CHECK.dat; CL_GUN_OPEN_MIN eliminado). Falta clamp abierto \$IN[466] y QFP regresado \$IN[482] antes de entrar, y la prueba en celda.
- 2026-10-05 — Revisión independiente: GUN_OPEN_CHECK e interrupts 20-22: KRL válido e integración fiel. Huecos: los interrupts solo disparan en el cambio FALSE→TRUE; al salir se reactivan después de las esperas de clamp/QFP y de la presión de reposo, y al entrar el submit solo se revisa al inicio de GUN_OPEN_CHECK. Corrección: INTERRUPT ON 20-22 antes de cada GUN_OPEN_CHECK. Además: tras reconocer GUN_OPEN_LOST el robot puede quedar parado sin mensaje; la vigilancia se apaga un punto antes de salir; una selección de paso salta el interlock.
- 2026-10-05 — Integrado - correcciones de la revisión: Versión integrada sobre el respaldo de las 14:46: INTERRUPT ON 20-22 antes de cada GUN_OPEN_CHECK (entrada y después de soldar) y mensaje de estado en GUN_OPEN_LOST después del reconocimiento. Sigue pendiente clamp abierto / QFP regresado antes de entrar.

F02 — Options 2, 7 y 8 no evalúan el sensor de tuercas: solo decide la cámara

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot + Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/Styles/style_1.src, KRC/R1/Program/Styles/Options/style1opt1.src

Qué está mal. Style_1 manda las options 1, 2, 7 y 8 a Style1Opt1. El nut check (style1app2opt1) decide solo CASE 1 (escribe bscrapGE4) y CASE 10 (salidas red rabbit), sin CASE 2/7/8 ni DEFAULT: con 2, 7 u 8 el resultado del sensor no se usa y bscrapGE4 conserva su último valor. La cámara (style1app2opt2) no tiene switch de option: pone FALSE con OK y no-NG, TRUE con no-OK y NG, y cualquier otra combinación lo deja igual. nOption es una máscara de bits (GetOptions). Si el PLC manda option 2, falta la tuerca 3, el sensor la ve y la cámara no (o no contesta ni OK ni NG), la pieza va al conveyor bueno.

Qué hacer. Pedir por escrito al programador del PLC qué options manda producción. Evaluar el sensor en todas las options de producción (CASE explícitos o la regla que confirme el PLC); poner bscrapGE4 = TRUE al inicio de cada inspección y FALSE solo en la rama de pase explícito; una option desconocida da falla con mensaje y \$OUT[4] do004ProcessFault.

Criterio de cierre (se verifica en el respaldo):

- Respuesta escrita del programador PLC (Gestamp) con la lista de options que manda producción, adjunta a la lista.
- En style1app2opt1 toda option de producción que llega a Style1Opt1 (1, 2, 7, 8 o las confirmadas) entra en la evaluación del sensor; ninguna sale sin escribir bscrapGE4.
- bscrapGE4 = TRUE se asigna antes de leer \$IN[227] di227NutPresent1 (nut check) y antes de pedir la cámara con \$OUT[80] do080RepositionTooling1; bscrapGE4 = FALSE solo aparece en la rama de pase. En la cámara, una combinación de \$IN[100]/\$IN[101] distinta de OK ON y NG OFF deja la pieza en scrap.
- Ninguna option pasa en silencio: una option que no es de producción ni red rabbit da mensaje y \$OUT[4] do004ProcessFault y bscrapGE4 queda en TRUE.
- Prueba en celda con option 2 (y 7/8 si producción las usa): (1) pieza con una tuerca faltante termina en RejectGE4 aunque la cámara dé OK (cámara simulada desde el PLC); (2) pieza con las 3 tuercas termina en Style1Drop1Opt1; (3) cámara NG termina en RejectGE4. Anotar fecha, option, caso y destino de la pieza.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:/;NOTE:/;WARNING: que citan (F02) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Nombre de quien respondió por el PLC, fecha y lista de options; criterio implementado; resultado de los tres casos de prueba con option y fecha.

Nota. Option 12 (red rabbit) es F19. bscrapGE4 lo tocan también F03 (TRUE al inicio del ciclo), F05 (dry cycle) y F08 (falla de soldadura): revisarlos juntos. En style1app2opt2 no abrir el form Application[1] Reset: su AC_CmdParam=2 oculto no coincide con la llamada (F21). C17 depende de este punto.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - corregido sin marcar: Sensor y cámara evalúan nOption 1, 2, 7 y 8; la cámara sin resultado válido da scrap. Falta la prueba en celda (una tuerca faltante con option 2) y la confirmación del PLC de qué options manda.
- 2026-10-05 — Integrado - avance parcial: Sensor y cámara deciden para options 1, 2, 7 y 8; la cámara sin resultado válido da scrap. Faltan: respuesta escrita del PLC con las options de producción, bscrapGE4 = TRUE antes de leer, falla con mensaje para una option desconocida y la prueba en celda.

F03 — PARTPRESENT1..3 y bscrapGE4 no se resetean al inicio del ciclo

Origen: Revisión Ethos (2026-10-04)

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/System/\$config.dat

Qué está mal. PARTPRESENT1..3 son globales persistentes de \$config.dat. Se borran al final de Style1Opt1, en RejectGE4 y en el drop red rabbit, nunca al inicio del ciclo. bscrapGE4 solo lo escriben el sensor con option 1, la cámara y RejectGE4 (FALSE). Si un ciclo pasa el nut check, se detiene en el handshake de la cámara y se cancela, el siguiente arranca con los valores viejos.

Qué hacer. Al inicio de cada ciclo, antes del pick, borrar PARTPRESENT1..3 y poner bscrapGE4 = TRUE. Asignar cada bandera desde el sensor en los dos sentidos (hecho en oficina, G13).

Criterio de cierre (se verifica en el respaldo):

- En Style1Opt1, antes de cualquier pick, PARTPRESENT1..3 = FALSE y bscrapGE4 = TRUE.
- En Style1Opt10AutoRR, antes del pick, PARTPRESENT1..3 = FALSE (el nut check es compartido con el red rabbit).
- En style1app2opt1 cada uno de los 3 checks asigna PARTPRESENTn = di227NutPresent1 (TRUE o FALSE) (hecho en oficina, G13).
- Prueba en celda: cancelar un ciclo después del nut check (p. ej. en la espera de \$IN[80] di080ToolRepositioned1), reiniciar y verificar en la pantalla de variables que PARTPRESENT1..3 = FALSE y bscrapGE4 = TRUE antes de llegar al nut check. Anotar fecha y valores leídos.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F03) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Dónde se hace el reset; captura de la pantalla de variables al reiniciar, con fecha.

Depende de / se cierra con: F02

Nota. Hecho en oficina: el nut check asigna PARTPRESENTn en los dos sentidos y lee con el robot parado (G13, F28). Falta el reset al inicio del ciclo. bscrapGE4 = TRUE al inicio depende de F02: con options 2/7/8 el sensor no decide y la cámara puede dejarlo sin tocar. C17 depende de este punto. Calvin #8 llama "part present al PLC" a PARTPRESENT1..3; son banderas internas (lo del PLC es G09).

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - sin cambio: PARTPRESENTn sigue poniéndose solo en TRUE y se lee antes del WAIT SEC 0.2; se limpia al final de Style1Opt1, no al inicio.

F04 — Dry cycle se cuelga en la primera weld app: PRELOAD borra NUT_READY y nunca lo pone

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/Request_next_nut.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/System/\$config.dat

Qué está mal. Los picks ponen NUT_START por TRIGGER en P12 / P10 aunque esté activo el dry cycle. PRELOAD acepta el arranque (NUT_READY = FALSE, NUTCYCLE_ACTIVE = TRUE), pero sus pasos solo corren con \$IN[4] di004UseDryCycle OFF, así que no vuelve a poner NUT_READY. La tuerca 1 espera NUT_READY antes de su rama de dry cycle; las tuercas 2 y 3 esperan NEXT_NUT_READY, que tampoco pone nadie. Con dry cycle el robot se queda esperando para siempre en la primera weld app, sin mensaje y con el robot in cycle.

Qué hacer. Que el dry cycle no pase por esperas que nadie va a cumplir ni deje un preload a medias: no iniciar preload en dry cycle y dejar las banderas en TRUE, o condicionar las esperas a no-dry sin cambiarlas de lugar (siguen antes del LIN de aproximación, F01) y evitar que PRELOAD quede con NUTCYCLE_ACTIVE en TRUE. Verificar el cuelgue en el robot antes del cambio.

Criterio de cierre (se verifica en el respaldo):

- En style1app1opt1..3, en producción la espera de NUT_READY / NEXT_NUT_READY sigue antes del LIN de aproximación (P20, P6, P13), junto con la espera de F01. Con dry cycle no se ejecuta ninguna espera que dependa de PRELOAD.
- En PRELOAD, con dry cycle activo un NUT_START no deja NUTCYCLE_ACTIVE = TRUE con NUT_READY = FALSE; al terminar un dry cycle no queda un preload iniciado ni un NUT_START pendiente que alimente una tuerca al quitar el dry cycle.
- Prueba en celda: con dry cycle desde el inicio, ciclo completo desde la estación 1 y desde la estación 2 hasta el final del ciclo; después, sin dry cycle, un ciclo productivo con una sola tuerca alimentada para cada soldadura (sin doble tuerca ni espera infinita). Anotar fecha y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F04) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Resultado antes del cambio (¿se colgó en la primera weld app?: sí/no); opción implementada; resultado de los dos dry cycles y del ciclo productivo posterior, con fecha.

Nota. Si ya existe la bandera de dry cycle latcheada de F05, usarla aquí. El arranque de PRELOAD es el mismo código que el gate de F11: coordinar. Al cerrarse este punto y F05 se cierra C15.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F05 — Dry cycle releído en cada paso: un cambio a mitad de ciclo deja pasar pieza incompleta

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/System/\$config.dat

Qué está mal. \$IN[4] di004UseDryCycle se lee por separado en los picks, en cada weld app, en los tres bloques del nut check y en su decisión, en PRELOAD y en sps.sub; nunca se latchea. La cámara no tiene prueba de dry cycle. En dry cycle el sensor no toca bscrapGE4. Si se sueldan las tuercas 1 y 2 y el dry cycle se activa antes de la pose de la tuerca 3 (con su preload terminado), la tuerca 3 se salta y, si la cámara dice OK, una pieza con 2 tuercas va al drop bueno.

Qué hacer. Leer \$IN[4] una sola vez al inicio del ciclo en una bandera declarada y usar solo esa bandera en todo el ciclo (picks, weld apps, nut check, cámara y PRELOAD). En dry cycle exigir gripper vacío o mandar la pieza a rechazo.

Criterio de cierre (se verifica en el respaldo):

- Existe una sola bandera de dry cycle declarada con comentario (p. ej. en \$config.dat), asignada desde \$IN[4] di004UseDryCycle una sola vez al inicio del ciclo, antes del pick, y nada la cambia hasta el final del ciclo.
- En los picks, style1app1opt1..3, style1app2opt1, style1app2opt2 y PRELOAD no queda ninguna lectura ejecutable de di004UseDryCycle; solo de la bandera (la de sps.sub para el agua se resuelve en F09).
- Con la bandera en TRUE se cumple, y está comentado en el código, uno de: (a) el pick verifica gripper vacío después de cerrar (\$IN[253] y \$IN[254] OFF) y da falla con mensaje si hay pieza; o (b) la pieza

termina en RejectGE4 y nunca en Style1Drop1Opt1.

- Prueba en celda en producción (option 1), con la pieza de prueba segregada: activar \$IN[4] desde el PLC (1) durante un preload y (2) con el preload de la tuerca 3 terminado y el robot yendo a su pose. En los dos casos el ciclo en curso termina como productivo (3 soldaduras e inspecciones), la celda no se cuelga y el ciclo siguiente corre en dry cycle. Anotar fecha, momento del cambio y destino de la pieza.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F05) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Nombre de la bandera y módulo donde se asigna; opción elegida, (a) o (b); resultado de los dos casos con fecha.

Nota. Que PRELOAD use la bandera latcheada evita además el cuelgue de F04 cuando el dry cycle cambia durante un preload. En style1pick1opt2 no abrir los forms PickUp[2]: su AC_CmdParam=1 oculto no coincide con la llamada (F21).

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F06 — Ciclos vacíos o silenciosos: pick ambiguo, style u option desconocidos

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto
Módulos: KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/style_1.src, KRC/R1/cell.src

Qué está mal. Style1Opt1 hace pick en la estación 1 solo con di065 AND NOT di066 y en la estación 2 con lo inverso; con las dos señales o ninguna en ON no corre nada (nPickStation = 0), se borran las banderas y el ciclo termina. cell.src tiene un DEFAULT vacío para styles distintos de 1. Style_1 contesta una option desconocida con WAIT FOR FALSE: espera infinita sin mensaje ni falla al PLC.

Qué hacer. Sustituir los defaults silenciosos por falla (mensaje AutomationCore + \$OUT[4] do004ProcessFault) y esperar con mensaje a que exista exactamente un permiso de pick antes de elegir estación.

Criterio de cierre (se verifica en el respaldo):

- En Style1Opt1, mientras \$IN[65] di065PickupMachine1 y \$IN[66] di066PickupMachine2 no tengan exactamente una en ON, el robot espera con un mensaje que nombra ambas señales (con ambas en ON además \$OUT[4] do004ProcessFault); no existe camino por Style1Opt1 que termine sin pick.
- En cell.src, el DEFAULT del SWITCH nStyle muestra mensaje y pone \$OUT[4] do004ProcessFault, y un style no ejecutado no llega a EndOfCycle.
- En Style_1 no queda WAIT FOR FALSE; el DEFAULT muestra mensaje con el número de option y pone \$OUT[4] do004ProcessFault.
- En los dos DEFAULT la falla se sostiene hasta que el operador o el PLC la reconoce (diálogo o mensaje con acuse); no es un pulso que MaintainSystem borra en la siguiente vuelta de CELL.
- Prueba en celda con el PLC: (1) style distinto de 1, (2) option no válida, (3) \$IN[65] y \$IN[66] ambas ON, (4) ambas OFF y después una sola en ON. En cada caso hay mensaje en el smartPAD; en (1) a (3) además \$OUT[4] ON hasta el acuse; en (4) el ciclo sigue normal al quedar una sola en ON. Anotar fecha y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F06) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Texto de cada mensaje y cómo se reconoce; resultado de las cuatro pruebas con fecha.

Depende de / se cierra con: F02

Nota. La lista de options válidas en Style_1 sale de la respuesta del PLC de F02 (hoy 1, 2, 7, 8 y 12). Hoy EndOfCycle no manda Work Complete (F46): un ciclo vacío no se distingue de uno bueno en el PLC. Contribuye a C21.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F07 — Chequeo de gun cerrado: HALT sin mensaje como manejo de falla

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Centerline/centerline_weld.src

Qué está mal. CENTERLINE_WELD arranca \$TIMER[14] (CL_GUN_CLOSE_TIMEOUT, 1000 ms) y \$TIMER[15] (CL_GUN_IN_WINDOW_TIME, 300 ms) y confirma el gun cerrado con CL_GunStrokePositionLPT en CL_GUN_WELD_MIN..MAX (69000..73000). Al vencer el timeout pone NUT_READY = FALSE y ejecuta HALT sin mensaje que diga la falla. En el respaldo recibido, además, los timers no se reanuncian después del resume y el lazo podía girar para siempre.

Qué hacer. Cambiar el HALT por un diálogo que nombre la falla (sin tuerca / doble tuerca / gun no cerró) con Retry, y Abort que abra gun y clamp y mande la pieza a rechazo.

Criterio de cierre (se verifica en el respaldo):

- En CENTERLINE_WELD no queda HALT en el chequeo de gun cerrado.
- Al vencer el timeout se muestra un diálogo AutomationCore (Set_KrDIg o equivalente) con \$OUT[4] do004ProcessFault ON, que nombra la falla (según CL_GunStrokePositionLPT respecto a la ventana) y ofrece Retry y Abort.
- Retry vuelve a armar el timeout y el tiempo en ventana: un segundo timeout es posible (hecho en oficina para el HALT actual, G11).
- Abort abre el gun (\$OUT[472] dopw1_GunWork OFF, \$OUT[471] dopw1_GunHome ON) y el clamp (\$OUT[501] do501PartClampClose OFF, \$OUT[502] do502PartClampOpen ON), no da weld start (\$OUT[483] dopw1_WeldInit) para esa tuerca, y la pieza termina en RejectGE4.
- Prueba en T1, sin pieza, sin nadie en el pedestal y con el timer Bosch sin corriente de soldadura: cerrar el gun sin tuerca en el pin: aparece el diálogo; Retry sin corregir hace que el diálogo vuelva a salir; Abort abre gun y clamp. Anotar fecha, valor de CL_GunStrokePositionLPT leído y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F07) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Texto del diálogo y teclas; valor de CL_GunStrokePositionLPT en la falla; resultado de Retry y de Abort; cómo se marca el rechazo; fecha.

Nota. Hecho en oficina (G11): el chequeo es LOOP/EXIT y después del HALT los dos timers se reanuncian, así que un segundo timeout es posible. Falta el diálogo con Retry/Abort en lugar del HALT. Si el rechazo se marca en bscrapGE4, revisar la misma trampa que en F08 (F02/F03). Las salidas de Abort pueden compartir la rutina de estado seguro de F10. Contribuye a C21.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - avance en el original: En el original del programador los dos timers se reinician después del HALT (igual que la versión de oficina). Sigue sin mensaje y sin do004ProcessFault: el criterio no se cumple completo.

F08 — Resultado de soldadura Bosch no evaluado: weld complete se toma como soldadura buena

Origen: Revisión Ethos (2026-10-04)

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot + Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Styles/Options/style1opt1.src

Qué está mal. CENTERLINE_WELD pone \$OUT[483] dopw1_WeldInit, espera solo \$IN[483] dipw1_WeldComplete y sigue. \$IN[485] no se lee en ningún código activo y sus dos nombres se contradicen (dipw1_Fault en NutWeld, di485NoFaults en AutomationCore). La espera de feed complete está comentada. Los chequeos posteriores revisan presencia de tuerca, no calidad de soldadura.

Qué hacer. Confirmar la polaridad de \$IN[485] con la lista de I/O del timer Bosch. Después, esperar weld complete O falla con timeout; en falla o timeout: liberar presión, abrir el gun, marcar la pieza como scrap y mostrar mensaje.

Criterio de cierre (se verifica en el respaldo):

- Polaridad de \$IN[485] confirmada con la lista de I/O de Bosch (documento y página) o con prueba en celda (valor leído con el timer en falla y sin falla), adjunta a la lista.
- En CENTERLINE_WELD, después de \$OUT[483] dopw1_WeldInit ON, la espera es \$IN[483] dipw1_WeldComplete O falla (\$IN[485] con la polaridad confirmada) con timeout; el código usa el nombre de \$IN[485] que coincide con esa polaridad.
- En falla o timeout: presión base (CL_GUN_PRESS_BASE, con el orden de bytes que confirme F44), intensify liberado (\$OUT[473] dopw1_Nut1Intensify_Home en reposo, \$OUT[494] dopw1_BlowOff OFF), \$OUT[483] dopw1_WeldInit OFF, gun abierto, mensaje con \$OUT[4] do004ProcessFault, y la pieza termina en RejectGE4 aunque después pasen el nut check y la cámara.
- Prueba en celda: provocar una falla del timer Bosch durante una soldadura (o simularla según el manual Bosch): el robot no sigue como soldadura buena, el gun abre, aparece el mensaje y la pieza va a RejectGE4. Anotar fecha, cómo se provocó la falla y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F08) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Fuente de la polaridad de \$IN[485]; timeout elegido; cómo se marca el scrap; resultado de la prueba de falla con fecha.

Depende de / se cierra con: F44

Nota. Si F02/F03 ponen bScrapGE4 = TRUE al inicio de cada inspección y FALSE en el pase, una falla de soldadura guardada solo en bScrapGE4 se borraría: usar una bandera propia. \$OUT[484] dopw1_FaultReset nunca se pulsa: definir cómo se recupera el timer después de una falla. El CENTERLINE_WELD1 borrado (patrón que cita Calvin) usaba señales Bosch de AutomationCore que no están en el bus de este robot: no copiar sus direcciones. La espera de \$IN[479] sí está activa en R10 (F37), sin timeout (F15).

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F09 — Resultado del flujo de agua descartado; \$OUT[475] forzado en ON

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot + Puesta en marcha · **Estado:** Abierto

Módulos: KRC/R1/System/sps.sub, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/Utilities/gunelectrodechange.src, KRC/R1/System/\$config.dat

Qué está mal. sps.sub calcula SV0500_FLOW y CL_WaterOK = (SV0500_FLOW > 1230), pero pone la BOOL dipw1_WaterOk (una señal NutWeld convertida en variable por el integrador) en TRUE en las dos ramas: el resultado del flujo se tira. Nada en la ruta de soldadura lee ninguna de las dos, ni la temperatura

del transformador \$IN[475] dipw1_XtfmrTempOk. El límite es > 1230 mientras el comentario viejo decía "1230 = water OK". sps.sub pone \$OUT[475] dopw1_StartWater en TRUE en cada ciclo fuera de dry cycle, nunca lo apaga, ignora \$IN[13] di013WaterEnable y sobrescribe el water-off de CENTERLINE_HOME y GunElectrodeChange.

Qué hacer. Escribir el resultado real del flujo (FALSE con flujo bajo); exigir agua OK y temperatura de transformador OK antes de \$OUT[483] dopw1_WeldInit, con espera supervisada; manejar \$OUT[475] desde \$IN[13] di013WaterEnable, encendiendo y apagando; confirmar el límite de flujo.

Criterio de cierre (se verifica en el respaldo):

- En sps.sub, con SV0500_FLOW bajo el límite la bandera de agua usada por la soldadura queda en FALSE (ya no hay TRUE en las dos ramas); una sola bandera (CL_WaterOK o dipw1_WaterOk) y la otra se quita o se deja documentada.
- En CENTERLINE_WELD, antes de \$OUT[483] dopw1_WeldInit ON, existe una espera supervisada (timeout + mensaje + \$OUT[4] do004ProcessFault) de agua OK y \$IN[475] dipw1_XtfmrTempOk ON; sin ellas no hay weld start.
- En sps.sub, \$OUT[475] dopw1_StartWater se escribe en ambos sentidos según \$IN[13] di013WaterEnable; una sola lógica decide \$OUT[475] (el water-off de CENTERLINE_HOME y de gunelectrodechange se respeta o se quita, y el comentario dice quién manda).
- El límite de SV0500_FLOW (valor y operador) coincide con un valor confirmado por escrito (Gestamp o CenterLine) y el ;CHECK: del límite en sps.sub se elimina.
- Prueba en celda: (1) con \$IN[13] OFF, \$OUT[475] pasa a OFF; (2) con el agua cerrada el robot no da weld start y muestra mensaje; (3) con agua y transformador normales la soldadura arranca. Anotar fecha y SV0500_FLOW leído con agua normal y con agua cerrada.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;NOTE:;WARNING: que citan (F09) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Documento con el límite de flujo; SV0500_FLOW con agua normal y cerrada; valor de \$IN[475] en condición normal; resultado de las tres pruebas con fecha.

Nota. Antes de cargar, confirmar con el PLC que \$IN[13] di013WaterEnable llega en ON en producción. Si se reactiva AutomationCore_Bkg (F12), su escritura de \$OUT[475] se quita o pasa a ser la única. IO_MAP lista \$IN[475] como mapeado y sin leer.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - corregido sin marcar: sps.sub calcula dipw1_WaterOk del sensor Parker (flujo en 0.1 l/min, mínimo 150 con histéresis 10) y \$OUT[475] sigue a \$IN[13] di013WaterEnable. GUN_OPEN_CHECK espera agua antes de entrar. Falta: confirmar el mínimo (el comentario cita una medición del 20R1) y la prueba en celda sin agua.
- 2026-10-05 — Integrado - avance parcial: El agua se calcula del SV0500 (mínimo 150 = 15 l/min, histéresis 10) y GUN_OPEN_CHECK la exige antes de entrar y al salir; \$OUT[475] sigue a \$IN[13]. Faltan: espera de agua y transformador antes del weld start en CENTERLINE_WELD, límite confirmado por escrito y la prueba en celda.
- 2026-10-05 — Evidencia parcial: Respaldo 14:46: nWaterFlow = 233 (23.3 l/min) con agua normal en este robot, arriba del mínimo de 15 l/min. Falta la prueba con agua cerrada y la espera antes del weld start.

F10 — Sin estado seguro del pedestal al cancelar o resetear el programa

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/System/sps.sub, KRC/R1/cell.src, KRC/R1/Program/Centerline/centerline_weld.src

Qué está mal. El único hook de reset de programa (interrupt 91 de sps.sub, RESET_OUT, con \$PRO_STATE1 = #P_FREE) borra los bits de PGNO. Nada regresa a estado seguro clamp, gun, palabra de presión, intensify, blow-off, weld start o contactor enable al cancelar. Si la soldadura se cuelga y el operador cancela y reinicia, la pieza queda sujeta en el gun cerrado y el diálogo HomeCheck de AutomationCore puede mover PTP HOME en T1 jalando el gripper contra ella. Si el timer Bosch se resetea con WeldInit todavía en alto, puede arrancar una soldadura mientras alguien manipula la pieza.

Qué hacer. Una rutina que lleve el pedestal a estado seguro al cancelar o resetear el programa (weld start y contactor off, presión base, intensify y blow-off off, gun y clamp abiertos) y bloquear el movimiento a HOME mientras el clamp o el gun estén cerrados. Abrir gun y clamp sin enabling switch se decide con la revisión de seguridad de F11.

Criterio de cierre (se verifica en el respaldo):

- Existe una rutina de estado seguro del pedestal que se ejecuta cuando el programa se cancela (\$PRO_STATE1 = #P_FREE) y cuando se resetea (\$PRO_STATE1 = #P_RESET).
- La rutina deja: \$OUT[483] dopw1_WeldInit OFF, \$OUT[481] dopw1_WeldContactEnable OFF, \$OUT[482] dopw1_WeldOnExternal OFF, CL_GunPressureCmd = presión base (valor según F44), \$OUT[473] dopw1_Nut1Intensify_Home en su reposo, \$OUT[494] dopw1_BlowOff OFF, \$OUT[476] dopw1_StartFeed OFF.
- La rutina deja además gun abierto y clamp abierto, salvo que la revisión de seguridad de F11 decida por escrito que en cancel/reset no se mueven actuadores sin enabling switch; en ese caso no los toca, el comentario cita esa decisión y la protección es el bloqueo de HOME.
- En cell.src ningún camino llega a InitializeSystem(), HOME() o HomeCheck() mientras el gun no esté abierto (CL_GunStrokePositionLPT > CL_GUN_OPEN_MIN) o el clamp no esté abierto (\$IN[466] di466PartClampOpen ON): espera supervisada con mensaje y \$OUT[4] do004ProcessFault, sin movimiento.
- Prueba en T1, sin pieza, sin nadie en el pedestal y con el timer Bosch sin corriente: detener CENTERLINE_WELD con el clamp cerrado y cancelar el programa; leer cada salida de la lista. Repetir con reset. Después, con el clamp detectado como no abierto, seleccionar CELL: el robot no se mueve a HOME y muestra mensaje. Anotar fecha y valores leídos.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F10) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Dónde se dispara la rutina; decisión de F11 sobre gun y clamp; valores de I/O leídos; resultado de la prueba de bloqueo de HOME; fecha.

Depende de / se cierra con: F11, F44

Nota. Las salidas de soldadura y el bloqueo de HOME no dependen de F11 y se pueden hacer ya. Preferir los folds USER de sps.sub y no tocar los folds AUTOEXT. Si se reactiva AutomationCore_Bkg (F12) sin quitar sus escrituras de weld enable, anula este estado seguro.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

CRÍTICO — F11 — PRELOAD en el submit mueve el pedestal en cualquier modo, sin dueño de las salidas

Severidad: Crítico · **Tipo:** Código + prueba en celda · **Responsable:** Seguridad + Programador robot · **Estado:** Requiere decisión

Módulos: KRC/R1/System/sps.sub, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/Request_next_nut.src, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/System/\$config.dat

Qué está mal. sps.sub llama PRELOAD en cada ciclo del submit, en todos los modos. Una vez que NUT_START se pone (por TRIGGER de movimiento, que también dispara al ir paso a paso en T1), PRELOAD escribe por su cuenta feed, QFP, gun, pins, clamp y blow-off; CENTERLINE_WELD y CENTERLINE_HOME escriben las mismas salidas y ninguno revisa al otro. NUT_START solo se consume sin preload activo: una solicitud durante un preload arranca un segundo preload justo después. En T1, un operador que pasa P12 paso a paso y camina al pedestal ve moverse feed, gun, clamp, slide y pins durante unos 4 s sin enabling switch.

Qué hacer. Primero la revisión de seguridad: ¿se corta el aire del pedestal al abrir la puerta? Después: un solo dueño del pedestal; PRELOAD solo corre con automático externo, \$MOVE_ENABLE y una liberación del lado del robot; los programas manuales esperan o abortan PRELOAD antes de escribir válvulas; una solicitud durante un preload activo no deja pasar al robot con una bandera de un solo ciclo.

Criterio de cierre (se verifica en el respaldo):

- Revisión de seguridad documentada (Gestamp / Seguridad) que responde si el aire del pedestal se corta con la puerta abierta, con referencia al plano o a una prueba física con fecha, adjunta a la lista.
- En PRELOAD (o en su llamada en sps.sub), el arranque y los pasos están condicionados a \$MODE_OP == #EX, \$MOVE_ENABLE TRUE y una bandera de liberación declarada que pone y quita el programa del robot; con la condición en FALSE, PRELOAD no escribe ninguna salida del pedestal.
- Si la condición cae con un preload a medias, el comportamiento está escrito en un comentario de PRELOAD y un preload interrumpido no puede terminar en NUT_READY / NEXT_NUT_READY = TRUE sin haber completado sus pasos.
- PRELOAD no arranca mientras CENTERLINE_WELD o CENTERLINE_HOME están corriendo, y esos programas no escriben válvulas del pedestal mientras NUTCYCLE_ACTIVE es TRUE.
- Un NUT_START recibido con un preload activo no produce una bandera de un solo ciclo de submit que el robot pueda tomar como buena.
- En T1/T2 el ciclo no se queda esperando NUT_READY sin mensaje.
- Prueba en celda, sin nadie en el pedestal: (1) en T1, correr el pick paso a paso hasta pasar P12 y soltar el enabling switch: el pedestal no se mueve; (2) con CELL cancelado, poner NUT_START = TRUE desde la pantalla de variables: el pedestal no se mueve; (3) en automático externo, un ciclo productivo completo alimenta y suelda las 3 tuercas. Anotar fecha y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:/;NOTE:/;WARNING: que citan (F11) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Documento de la revisión de seguridad; nombre de la bandera de liberación y dónde se pone y se quita; comportamiento elegido; resultado de las tres pruebas con fecha.

Nota. El gate toca el mismo arranque de PRELOAD que F04: coordinar. Relacionado con F10, F12 (doCriticalWZ), F13, F14 y F38.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F12 — Background de AutomationCore y NutWeld deshabilitado: los bits de estado al PLC no cambian

Severidad: Alto · **Tipo:** Decisión Gestamp · **Responsable:** Gestamp controles + Programador robot · **Estado:** Requiere decisión

Módulos: KRC/R1/System/sps.sub, KRC/R1/TP/AutomationCore/automationcoreroutines.src, KRC/R1/TP/NutWeld/nutweldroutines.src, KRC/R1/System/\$config.dat, KRC/R1/Program/Utilities/pouncetohome.src, KRC/R1/Program/Utilities/gunelectrodechange.src, KRC/R1/cell.src

Qué está mal. En sps.sub las llamadas NutWeld_BKG () y AutomationCore_BKG () estaban comentadas (los folds quedaron vacíos). Por eso nunca se escriben: brake test pendiente (\$OUT[21] do021BrakeTestReqd), mastering pendiente (\$OUT[22]), alarma de batería (\$OUT[6]), robot en pounce

(\$OUT[7] do007RobotAtPounce, lo usa HomeCheck), cambio de electrodo pendiente (\$OUT[111..113], que cell.src revisa: GunElectrodeChange nunca corre, F43), weld mode, bits de nodos de bus, speed not 100 %, doCriticalWZ (\$OUT[145]) y nivel de tolva. La edición de AutomationCore en bas.src sigue deteniendo cada PTP/LIN cuando el PLC pide entrar (\$IN[6] di006RequestToEnter), pero doCriticalWZ queda en FALSE y el PLC ve entrada permitida mientras PRELOAD o CENTERLINE_WELD siguen moviendo el pedestal.

Qué hacer. Decidir con Gestamp si el background task es obligatorio. Si se reactiva: antes quitar sus escrituras de weld enable que pelean con CENTERLINE_WELD y su water-on (\$OUT[475]), y cerrar F35 (\$OUT[930] en ON bloquearía toda request to enter) y F17 (NutWeld_BKG). Si no: reproducir en sps.sub los bits requeridos.

Criterio de cierre (se verifica en el respaldo):

- Decisión escrita de Gestamp (nombre, fecha): reactivar AutomationCore_BKG y/o NutWeld_BKG, o reproducir los bits en sps.sub.
- Si se reactiva AutomationCore_BKG: F35 está cerrado antes; en automationcoreroutines.src ya no hay escrituras de weld enable por dry cycle ni de \$OUT[475] que contradigan F09; cada cambio en el archivo del proveedor queda en la lista de C28.
- Si se reactiva NutWeld_BKG: F17 está cerrado antes (nombres de pistola 2/3 traslapados con CL_GunPressureCmd).
- Si se reproducen en sps.sub: se escriben en cada ciclo \$OUT[21], \$OUT[22], \$OUT[6], \$OUT[7], \$OUT[46], los bits de nodo de bus que Gestamp pida, \$OUT[146], \$OUT[145] doCriticalWZ, el nivel de tolva y la petición de cambio de electrodo que se acuerde en F43, con la lógica aprobada por escrito.
- En cualquier caso, \$OUT[145] doCriticalWZ está en TRUE (entrada no permitida) mientras CENTERLINE_WELD corre o PRELOAD está activo.
- Prueba en celda: verificar desde el PLC cada bit que se pueda provocar (p. ej. override < 100 % en automático externo; una solicitud de entrada durante CENTERLINE_WELD no recibe permiso); los demás se revisan en código. Anotar fecha y resultado por bit.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F12) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Documento o correo de Gestamp con la decisión; lista de cambios en archivos del proveedor; tabla de bits verificados con resultado y fecha.

Depende de / se cierra con: F17, F35

Nota. Ethos (Edgar) lleva la pregunta a Gestamp. Al cerrarse se cierra C01; aporta a C12, C14 y C19.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - cambio sin decisión: El programador activó AutomationCore_BKG en sps.sub y editó la rutina del proveedor (sin soldadores 2-3, Request to Enter sin WAIT). La decisión de Gestamp no está registrada. Ver F49.
- 2026-10-05 — Integrado - pendiente decisión: AutomationCore_BKG queda activo en la versión integrada, como lo dejó el programador. Falta la decisión de Gestamp (F49).
- 2026-10-05 — Integrado: Background de AutomationCore apagado otra vez en la versión integrada (F49).

F13 — PRELOAD sin timeouts por paso, sin falla y banderas de tuerca lista sin confirmar alimentación

Origen: Revisión Ethos (2026-10-04)

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/System/\$config.dat, KRC/R1/Program/StyleApps/Options/style1applopt1.src, KRC/R1/Program/StyleApps/Options/style1applopt2.src, KRC/R1/Program/StyleApps/Options/style1applopt3.src

Qué está mal. En PRELOAD los pasos 0 y 10 esperan los switches del QFP (\$IN[482] dipw1_SpearHome, \$IN[470] di470QFPAdvanced) sin límite de tiempo y sin mensaje; el paso 20 corre solo con timers.

NUT_PRELOAD_FAULT nunca se pone TRUE. NUT_READY / NEXT_NUT_READY se ponen por tiempos fijos (2000 ms de feed, 500 ms tras el avance, 800 ms de blow-off, 800 ms tras mandar regresar el QFP) sin leer \$IN[481] dipw1_FeedComplt (en el bus) ni \$IN[480] dipw1_HopperLowLevel. Con la tolva vacía se declara tuerca lista sin tuerca; si un switch no llega, el robot espera sin mensaje.

Qué hacer. Agregar a cada condición que PRELOAD espera un timeout que ponga NUT_PRELOAD_FAULT = TRUE y detenga la secuencia sin poner las banderas de tuerca lista; exigir feed complete y nivel de tolva OK; las weld apps esperan tuerca lista O falla y, con falla, dan mensaje y \$OUT[4] sin entrar al pedestal. PRELOAD corre en el submit: timeouts con timers revisados en cada ciclo, sin bloquear.

Criterio de cierre (se verifica en el respaldo):

- En PRELOAD toda condición que la secuencia espera (QFP regresado/avanzado en los pasos 0 y 10, QFP regresado al final del paso 20, \$IN[481] dipw1_FeedComplt) tiene timeout; al vencer: NUT_PRELOAD_FAULT = TRUE, queda registrado el paso y la señal, y la secuencia no pone NUT_READY ni NEXT_NUT_READY.
- PRELOAD no contiene WAIT FOR ni WAIT SEC (corre en el submit).
- Existe al menos una asignación NUT_PRELOAD_FAULT = TRUE en código activo, y la falla solo se borra en un punto definido (reset del operador o rutina de reset), dicho en el comentario.
- Fuera de dry cycle, NUT_READY / NEXT_NUT_READY = TRUE solo con \$IN[481] visto ON durante la alimentación, \$IN[480] en nivel OK (polaridad confirmada) y QFP regresado confirmado al final del paso 20.
- En style1applopt1..3 la espera previa al LIN de aproximación termina con la bandera de tuerca lista o con NUT_PRELOAD_FAULT; con falla hay mensaje, \$OUT[4] do004ProcessFault ON y el robot no ejecuta el LIN de aproximación.
- Prueba en celda: (a) tolva vacía o alimentador apagado: no hay tuerca lista, aparece el mensaje y el robot no entra al pedestal; (b) aire del QFP cerrado: timeout del paso con mensaje. Anotar fecha, timeout y texto del mensaje.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F13) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Timeout por condición (ms); polaridad de \$IN[480] y \$IN[481] leída en el smartPAD; resultado de las pruebas (a) y (b) con fecha.

Depende de / se cierra con: F01, F11

Nota. Calvin lo califica Alto (#6). Se queda en Medio: con una tuerca faltante el gun cierra fuera de la ventana 69000..73000 y el chequeo de gun cerrado detiene el ciclo (F07); la celda para, la pieza no sale mala. Timers nuevos: ver F26.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F14 — NUT_READY no se consume al soldar; la siguiente tuerca depende solo de TRIGGERS

Origen: Revisión Ethos (2026-10-04)

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/Request_next_nut.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/System/\$config.dat

Qué está mal. NUT_READY significa 'tuerca cargada' pero solo se borra cuando PRELOAD acepta una petición nueva o en el timeout de gun cerrado, nunca cuando la soldadura consume la tuerca. La siguiente tuerca se pide solo por TRIGGER de movimiento (picks en P12/P10, final de las apps 1 y 2). Un ciclo cancelado puede dejar una tuerca en el pin y el siguiente pick alimenta otra (doble tuerca); un TRIGGER del pick saltado deja pasar la tuerca 1 con un TRUE viejo y sin tuerca: solo la ventana del LPT (F07) evita soldar sin tuerca. El pick red rabbit también arranca un preload aunque no sigue ninguna soldadura.

Qué hacer. Borrar NUT_READY en el flujo de soldadura en cuanto la tuerca se consumió; confirmar el pin vacío antes de alimentar; pedir la siguiente tuerca con una instrucción en el flujo (no solo en un TRIGGER), en main run; una petición repetida no debe provocar una segunda alimentación.

Criterio de cierre (se verifica en el respaldo):

- En CENTERLINE_WELD después de \$IN[483] dipw1_WeldComplete, o inmediatamente después de CENTERLINE_WELD() en style1app1opt1..3, existe NUT_READY = FALSE (y la bandera que usen las tuercas 2 y 3) antes de salir del pedestal.
- La petición de la tuerca 1 (picks) y de las tuercas 2 y 3 (final de style1app1opt1 y opt2) existe como instrucción en el flujo, fuera de TRIGGER, precedida de un paro de advance run; en las apps va después del movimiento que saca pieza y EOAT del alcance del QFP y de los pines.
- En PRELOAD una petición que llega con preload activo o con tuerca cargada sin consumir no arranca otra alimentación.
- Antes de alimentar se confirma pin vacío: por sensor si existe (nombre y dirección), o por estado si se confirma en sitio que no hay sensor; el comentario del paso dice cuál.
- El pick red rabbit no pide preload, o la lista anota por qué sí.
- Prueba en celda: (a) cancelar el ciclo con tuerca precargada y reiniciar desde home: no hay doble tuerca; (b) cancelar después de soldar la tuerca 1 y reiniciar con selección de paso en style1app1opt2: el robot no entra a soldar sin preload nuevo. Anotar fecha y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:/;NOTE:/;WARNING: que citan (F14) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Cómo se confirma el pin vacío; resultado de las pruebas (a) y (b) con fecha.

Depende de / se cierra con: F11, F25, F32, F38

Nota. Trampa de advance run: una asignación o llamada escrita después de un movimiento corre varios movimientos antes de que el robot llegue; sin paro de advance run PRELOAD movería QFP y pines con la pieza todavía en el pedestal. IO_MAP marca \$IN[468] y \$IN[469] como switches sin identificar: confirmar en sitio si alguno es el sensor de pin. Los TRIGGER ya no están dentro de inline forms (G20/F32).

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F15 — Unos 35 WAIT FOR sobre entradas de proceso sin timeout ni mensaje

Origen: Revisión Ethos (2026-10-04)

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/cell.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/Utilities/gunelectrodechange.src

Qué está mal. Las esperas de Bosch ready/complete, presión de intensificado (\$IN[479]), ventanas del LPT, gun open, clamp abierto/cerrado, QFP avanzado/regresado, NUT_READY / NEXT_NUT_READY, el handshake de la cámara (\$IN[80]) y las de gripper vacío / pieza presente son WAIT FOR sin supervisión. Si una señal no llega, el robot queda parado 'en ciclo' sin mensaje que diga qué espera y sin falla al PLC. No se usa el idioma de AutomationCore (SetWaitingMessage).

Qué hacer. Una rutina de espera supervisada con timeout, \$OUT[4] do004ProcessFault y mensaje que nombre la señal con dirección, con reintento; usarla en todas las esperas de dispositivo y proceso. En el pedestal, el abort deja weld start apagado y gun y clamp abiertos (coordinar con F10). Las esperas de permiso del PLC llevan al menos mensaje de espera. Entregar la tabla de esperas con su timeout.

Criterio de cierre (se verifica en el respaldo):

- En los módulos listados no queda espera sin timeout sobre \$IN[466], \$IN[467], \$IN[470], \$IN[479] dipw1_IntensfPresOk, \$IN[482], \$IN[483], \$IN[484], CL_GunStrokePositionLPT, NUT_READY, NEXT_NUT_READY, \$IN[80] di080ToolRepositioned1 ni \$IN[253]/\$IN[254].
- Al vencer un timeout: \$OUT[4] do004ProcessFault ON, mensaje que nombra la señal con dirección, y el programa no continúa sin la condición (reintento o abort explícito).
- En CENTERLINE_WELD y CENTERLINE_HOME el abort de una espera vencida deja el pedestal abierto: primero \$OUT[483] dopw1_WeldInit OFF, luego gun abierto y clamp abierto.
- En cell.src la espera de \$IN[8] di008ReturnFromRepair tiene al menos mensaje de espera que nombra la señal.
- La lista trae una tabla módulo / señal / timeout (s) / reacción que coincide con el código del respaldo.
- Prueba en T1: provocar una espera vencida por grupo: pedestal (clamp sin aire), intensificado (\$IN[479]), cámara (\$IN[80] sin respuesta), Bosch (ready ausente) y gripper (pieza presente); anotar mensaje, \$OUT[4] y fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F15) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Tabla de esperas con timeout y reacción; resultado de las pruebas con texto del mensaje y fecha.

Depende de / se cierra con: F01, F07, F10

Nota. Los forms GripperTech (GRPg_SetStateAndCheck) ya revisan el estado del gripper con su estrategia de error (3 s): no cuentan aquí. SetWaitingMessage no tiene timeout y, si el operador borra el mensaje, la espera termina como 'simulada': no usarlo tal cual en esperas del pedestal. Las esperas de PRELOAD son F13; el WAIT FOR FALSE de Style_1 es F06. C21 se cierra con F06, F07 y F15.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F16 — Posición del pin superior nunca verificada

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto

Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/System/\$config.dat, KRC/R1/System/sps.sub

Qué está mal. sps.sub arma CL_WeldPinPosition (\$IN[591..606] CL_PIN_BYTE0..1) y ningún programa lo lee. CL_PIN_WELD_MIN/MAX (10000..13500) están declaradas y sin uso. Al final de la soldadura el pin superior se manda regresar 0.2 s después de ver el gun abierto, sin confirmación, y la app retira la pieza.

Qué hacer. En CENTERLINE_WELD, confirmar con espera supervisada el pin superior en posición de soldadura (CL_PIN_WELD_MIN..MAX) antes de \$OUT[483] dopw1_WeldInit, y retraído antes de terminar. Declarar la ventana de retraído con valores medidos, sin traslape con la de soldadura.

Criterio de cierre (se verifica en el respaldo):

- En CENTERLINE_WELD, antes de dopw1_WeldInit = TRUE, hay espera supervisada (timeout + mensaje + \$OUT[4] do004ProcessFault) de CL_WeldPinPosition dentro de CL_PIN_WELD_MIN..CL_PIN_WELD_MAX.
- Después del comando de retraer (\$OUT[499] do499UpperPinRetract ON) y antes del END hay espera supervisada de CL_WeldPinPosition dentro de una ventana de retraído declarada como constantes en \$config.dat.
- Las ventanas coinciden con los valores medidos anotados en la lista (la tabla del encabezado da retraído 27252, extendido 6602, soldadura 11674) y no se traslapan; la tabla del encabezado coincide con las constantes.
- Las ventanas del gun usan CL_GUN_WELD_MIN/MAX y CL_GUN_OPEN_MIN (hecho en oficina, G16).
- Prueba en celda en T1: provocar que el pin superior no llegue a la ventana de soldadura: no hay weld start y aparece el mensaje; anotar fecha y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:/;NOTE:/;WARNING: que citan (F16) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Valores de CL_WeldPinPosition leídos con el pin retraído, extendido y en posición de soldadura, y límites elegidos. Resultado de la prueba con fecha.

Depende de / se cierra con: F27

Nota. Hecho en oficina (G16): la ventana del gun son constantes: CL_GUN_WELD_MIN/MAX = 69000..73000 (la que el código siempre revisó; estaban declaradas 65000..76000 sin uso) y CL_GUN_OPEN_MIN. Falta la verificación del pin superior. La asignación de válvulas de pines está por confirmar (F27).

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F17 — Declaraciones traslapadas; nombres dobles en los bits del pedestal

Severidad: Medio · **Tipo:** Código · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/System/\$config.dat, KRC/R1/TP/AutomationCore/automationcoreroutines.dat, KRC/R1/TP/NutWeld/nutweldroutines.dat

Qué está mal. CL_GunPressureCmd (\$OUT[506..521]) se traslapa con los nombres AutomationCore do512..do521 y NutWeld dopw2_*; CL_LPT_BYTE0..3 (\$IN[559..590]) con dipw3_* y di561..di590; CL_PIN_BYTE0/1 (\$IN[591..606]) con aiNutSensorPW3. Las pistolas 2 y 3 no existen en este robot. \$IN[466], \$IN[467] y \$IN[470] siguen con un segundo nombre Spare (di466Spare...). \$OUT[477]/\$OUT[478] tienen nombres invertidos entre NutWeld (LevelOk/LowLevel) y AutomationCore (LowLevel/LevelOk). \$OUT[500] (do500Reserved) no tiene función conocida.

Qué hacer. Marcar como retirados los nombres de pistola 2/3 que se traslapan y los Spare sobrantes, documentar el nombre correcto de \$OUT[477]/\$OUT[478] e identificar \$OUT[500]. Hacerlo antes de reactivar cualquier background task (F12) y actualizar el proyecto WorkVisual.

Criterio de cierre (se verifica en el respaldo):

- \$IN[466], \$IN[467], \$IN[470], \$OUT[498], \$OUT[499] y \$OUT[501..505] tienen nombre AutomationCore y el código del pedestal no usa direcciones literales (hecho en oficina, G12).

- Las declaraciones de pistola 2/3 que se traslapan con CL_GunPressureCmd, CL_LPT_BYTE0..3 y CL_PIN_BYTE0/1 y los nombres Spare de \$IN[466/467/470] llevan comentario de retirados, y ningún programa del integrador los lee ni escribe.
- \$OUT[477]/\$OUT[478]: el comentario de la declaración dice cuál nombre es el correcto (confirmado con F29).
- \$OUT[500] recibe nombre después de confirmar en celda qué mueve, o deja de usarse.
- Ninguna dirección cambió; cada edición en archivos del proveedor (automationcoreroutines.dat, nutweldroutines.dat) aparece en la lista de C28.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F17) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Qué mueve \$OUT[500] (confirmado en celda) o constancia de que ya no se usa; fecha en que se actualizó el proyecto WorkVisual desde el controlador.

Depende de / se cierra con: F29

Nota. Hecho en oficina (G12): nombres AutomationCore para \$IN[466], \$IN[467], \$IN[470], \$OUT[498], \$OUT[499] y \$OUT[501..505], y el código del pedestal usa solo nombres. Falta lo de los criterios 2 a 4. No 'corregir' las palabras de presión aquí: eso es F44. C23 se cierra con este punto.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F18 — KRC_IO.xml mapea \$OUT[820..979] y \$IN[820..930] sobre los mismos bytes del adapter PLC

Severidad: Medio · **Tipo:** WorkVisual / configuración · **Responsable:** Gestamp controles / Programador robot · **Estado:** Requiere decisión

Módulos: C/KRC/Roboter/Config/User/Common/KRC_IO.xml, KRC/R1/Program/masref_user.src, KRC/R1/Program/Utilities/gunelectrodechange.src, KRC/R1/TP/AutomationCore/automationcoreroutines.dat, KRC/R1/TP/BrakeTest/braketeststart.src, KRC/R1/TP/BrakeTest/braketestback.src

Qué está mal. Igual que en R20: \$OUT[820..979] está mapeado a los mismos 20 bytes del EIP-ADAPTER que \$OUT[1..160], y \$IN[820..930] a los mismos que \$IN[1..111] (más \$IN[973..979]). Así do930MasterRefInProcess cae en el bit del PLC de \$OUT[111] do111Gun1ElectrodeChange y di823ToolIDBit0 es físicamente \$IN[4] di004UseDryCycle. Como en R10 \$OUT[930] se queda en ON después del primer mastering reference (F35), el PLC ve el bit de cambio de electrodo de la pistola 1 encendido.

Qué hacer. Revisar en WorkVisual el tamaño de assembly del adapter y acordar con controles Gestamp el mapa de tags. Ningún par de rangos KRC debe compartir bytes del adapter; do930/do931 y cualquier otra señal 8xx/9xx que se use van en bits propios acordados, o se dejan de usar. El cambio se hace en WorkVisual y se despliega; KRC_IO.xml no se edita a mano.

Criterio de cierre (se verifica en el respaldo):

- En KRC_IO.xml no hay dos entradas <Out> ni dos <In> que cubran los mismos bits del mismo segmento EIP-ADAPTER.
- do930MasterRefInProcess, do931BrakeTestInProcess, di823ToolIDBit0 y di825ToolIDBit1 están en bits propios del mapa acordado, o ningún código activo los usa.
- El tamaño de assembly acordado con Gestamp está en la lista (captura de WorkVisual).
- Prueba fuera de ciclo: correr el mastering reference y confirmar en el PLC que el tag de \$OUT[111] no cambia; cambiar \$IN[4] desde el PLC y confirmar que di823ToolIDBit0 no lo sigue. Anotar fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F18) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Mapa de tags robot-PLC acordado con Gestamp, adjunto; fecha del deploy y resultado de la prueba.

Depende de / se cierra con: F35

Nota. KRC_IO.xml lo genera WorkVisual: cambiarlo en el proyecto, desplegar y después cargar el proyecto actualizado. Si se reactiva AutomationCore_Bkg (F12), su request to enter lee do930..do932: este punto se cierra antes.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F19 — Red rabbit: el ciclo no puede dar resultado y suelta la pieza por un mapeo duplicado

Origen: Revisión Ethos (2026-10-04)

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src,
KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src,
KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src,
KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/TP/GripperSpotTech/grp_data.dat

Qué está mal. Style_1 corre el red rabbit como option 12 (Style1Opt10AutoRR), que llama el nut check y la cámara de producción. El nut check pone las salidas red rabbit solo para CASE 10: con option 12 nunca se pone \$OUT[141] do141RedRabbitFailed ni \$OUT[142] do142RedRabbitPassed; la cámara solo escribe bscrapGE4. El drop red rabbit (style1drop1opt2AutoRR) primero cierra el gripper 1 con salidas escritas a mano (\$OUT[249] OFF, \$OUT[250] ON) y después "abre" con GRPg_SetStateAndCheck(3, 1, ...). El gripper 3 está inactivo en grp_data.dat pero configurado en la misma E/S que el gripper 1 y GRPg_SetState no mira la bandera de activo: el gripper 1 sí abre y el check del gripper 1 OPEN pasa. La liberación funciona solo por ese mapeo duplicado. Después el robot se mueve (LIN P23) antes de esperar gripper vacío, y las salidas se borran en el drop, antes de que el PLC las lea; MaintainSystem las borra otra vez al inicio del siguiente ciclo.

Qué hacer. Decidir con Gestamp el procedimiento red rabbit (número de option, qué es un pase: todas las tuercas faltantes vistas por sensor Y cámara), evaluarlo en un solo lugar con un veredicto exclusivo que se mantenga hasta que el PLC lo lea, soltar con el gripper 1 y esperar gripper vacío antes de moverse.

Criterio de cierre (se verifica en el respaldo):

- Procedimiento red rabbit de Gestamp por escrito (option, pieza, resultado esperado de cada chequeo), adjunto.
- Para la option red rabbit el resultado sale de una sola evaluación que combina sensor y cámara y asigna las dos salidas en el mismo punto: exactamente una de \$OUT[141] / \$OUT[142] en TRUE y la otra en FALSE; resultado contradictorio, parcial o cámara sin respuesta dan Failed.
- El veredicto se mantiene hasta que el PLC lo lee (con el handshake de fin de ciclo que decida F46); ya no se borra en style1drop1opt2AutoRR antes del fin del ciclo.
- style1drop1opt2AutoRR suelta con un form GripperTech del gripper 1 (GRPg_SetStateAndCheck(1, 1, ...)), sin salidas \$OUT[249]/\$OUT[250] escritas a mano ni forms del gripper 3, y espera gripper vacío (\$IN[253] y \$IN[254] OFF) antes de LIN P23.
- Prueba en celda: correr el red rabbit con la pieza red rabbit y otra vez con una pieza buena de 3 tuercas; anotar qué ve el PLC en \$OUT[141]/\$OUT[142] al final del ciclo (esperado: Passed con la red rabbit, Failed con la buena) y que la pieza se suelta y el robot no se mueve antes de gripper vacío. Fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F19) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Descripción de la pieza red rabbit y regla de veredicto; lectura del PLC en las dos pruebas, con fecha.

Depende de / se cierra con: F46, F31, F40, F03

Nota. Gestamp (puntos 8 y 21) describe lo mismo; G08 y G21 se cierran con este punto. Mientras no se cambie, no reconfigurar el gripper 3 en GripperTech ni re-sincronizar GripperConfig.xml quitándole esa E/S (F33): el drop red rabbit dejaría de soltar. Hecho en oficina (G20): los resets del red rabbit ya están fuera del fold DropOff[2] Reset.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - avance parcial: Con option 12 el sensor y la cámara ya ponen \$OUT[141]/\$OUT[142]. No es un veredicto exclusivo (los dos chequeos pueden poner Failed y Passed a la vez) y el drop red rabbit los sigue borrando antes del fin de ciclo.
- 2026-10-05 — Integrado - avance parcial: Option 12 ya pone \$OUT[141]/\$OUT[142] en sensor y cámara. Falta el veredicto único y que el drop no lo borre antes de que el PLC lo lea.

F20 — Reject sin permiso del PLC para la posición de rechazo; Application 1 sin liberar

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src

Qué está mal. RejectGE4 abre el gripper sobre la posición de rechazo (LIN P23) sin AC_DropOffCheck ni petición de zona. Cuando el nut check decide scrap, Style1Opt1 va a RejectGE4 y se salta el AC_Application(1,True) del final de style1app2opt2, así que Application 1 queda ocupada hasta MaintainSystem. La trampa del inline form OUT 106 (dato oculto 5:TRUE) ya la corrigió la depuración.

Qué hacer. Antes de entrar a la posición de rechazo, pedir permiso al PLC con el mecanismo AutomationCore que se acuerde con controles Gestamp y liberarlo al salir. En el camino de rechazo por nut check, liberar Application 1 igual que en el camino bueno.

Criterio de cierre (se verifica en el respaldo):

- En Style1Opt1, en el camino de rechazo por nut check se ejecuta AC_Application(1,TRUE) (o lo que pone \$OUT[70] do070ApplicationClear1 ON) antes de que termine el ciclo; todo camino que pasó por AC_ApplicationCheck(1) tiene su liberación.
- En RejectGE4, antes de LIN P23, hay petición de permiso con el número acordado (AC_DropOffCheck(n) o zona) con espera y mensaje, y su liberación después de salir de la posición (P24 o después).
- Si se agregan inline forms AutomationCore, su AC_CmdParam oculto corresponde a la llamada según la convención de F21.
- Prueba en celda: con el permiso de rechazo apagado en el PLC el robot se detiene antes de P23 con mensaje y continúa al darlo; después de un rechazo por nut check el PLC ve \$OUT[70] ON al terminar el ciclo. Anotar fecha y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F20) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Número de drop-off o zona y señal del PLC acordados; resultado de la prueba con fecha.

Depende de / se cierra con: F21

Nota. La liberación de Application 1 se puede hacer ya; el interlock espera la definición del PLC. No reabrir los forms OUT 106 de RejectGE4. C18 se cierra con F20, F21 y F46.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Integrado - avance: RejectGE4 libera la aplicación 1 por TRIGGER en P23, también después de un scrap del sensor (antes nunca se liberaba en esa ruta). El lugar de rechazo sigue sin interlock con el PLC.

F21 — Parámetros ocultos de inline forms AutomationCore no coinciden con la llamada

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src

Qué está mal. style1pick1opt2 tiene AC_CmdParam=1 en sus dos folds de pick pero llama AC_pickUPCheck(2) / AC_pickUP(2,True); style1app2opt2 tiene AC_CmdParam=2 y llama AC_Application(1,True); style1drop1opt1 tiene AC_CmdParam=2 y llama DropOff 1, mientras el drop red rabbit tiene AC_CmdParam=2 y llama DropOff 2: no hay un desfase constante. Abrir y confirmar uno de estos forms regenera la llamada desde el parámetro oculto y puede cambiar el número de pickup, application o drop-off. La depuración no tocó esas líneas y puso un ;WARNING: sobre cada fold.

Qué hacer. Establecer en el smartPAD qué escribe el form AutomationCore en AC_CmdParam para cada tipo y número (insertando el form en un módulo de prueba) y, con esa convención, corregir el parámetro oculto de cada fold. Corregir el parámetro, nunca la llamada.

Criterio de cierre (se verifica en el respaldo):

- La convención queda anotada en la lista para cada tipo de form usado (PickUp, Application, DropOff) con foto del form insertado en un módulo de prueba y el AC_CmdParam que genera. El módulo de prueba no queda en el controlador.
- Todos los folds ;Params llfProvider=ac_zoneilf' cumplen la convención: style1pick1opt2 (2 folds), style1app2opt2 (1), style1drop1opt1 (2), style1drop1opt2AutoRR (2), y los de style1pick1opt1, style1pick1opt1AutoRR y style1app2opt1 si la convención los contradice.
- Las llamadas AC_ son las mismas del respaldo anterior; solo cambió la línea ;Params.
- Prueba en el smartPAD: abrir y confirmar (Cmd OK) cada form corregido; en el respaldo siguiente el cuerpo de cada fold es la misma llamada de antes. Anotar fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F21) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Foto del form de prueba por tipo y convención; lista de folds corregidos con valor anterior y nuevo; resultado de la prueba Cmd OK con fecha.

Depende de / se cierra con: F32

Nota. Cambiar la llamada para que coincida con el parámetro cambiaría el pickup o drop-off real: no hacerlo. La verificación de la depuración protege las líneas ;Params: en este punto su cambio es el esperado y se declara en tools/code_changes.json.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F22 — Movimientos del pedestal mezclan modo base y external TCP

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Ethos (Edgar) / Programador robot · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.dat, KRC/R1/Program/StyleApps/Options/style1app1opt2.dat, KRC/R1/Program/StyleApps/Options/style1app1opt3.dat, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/System/\$config.dat

Qué está mal. De los 22 movimientos enseñados en base 3 "NUT WELDER", 11 corren en modo base y 11 en external TCP; las tres presentaciones de soldadura (P19 en la app 1, P16 en las apps 2 y 3) están en modo base. Gestamp pide movimientos de pedestal interpolados alrededor del dado fijo (C04). Ningún movimiento de producción usa la herramienta 2 en R10 (el único era redrabbt.src, borrado).

Qué hacer. Decidir una regla de frame para el pedestal y, con BASE_DATA[3] medido (C05), re-enseñar los movimientos del pedestal de forma consistente.

Criterio de cierre (se verifica en el respaldo):

- La regla de frame del pedestal está anotada en la lista (qué movimientos de base 3 van en external TCP y cuáles en modo base; quién decidió y fecha).
- Cada FDAT con BASE_NO 3 usado por un movimiento tiene el IPO_FRAME que pide la regla.
- Si la regla usa external TCP, BASE_DATA[3] es el valor medido en sitio (C05) y todos los puntos de base 3 se re-enseñaron o verificaron en T1 después del cambio (lista de puntos).
- Prueba en T1 a velocidad reducida con pieza: las tres presentaciones llegan a su pose sin contacto; en el primer ciclo con soldadura CL_GunStrokePositionLPT con el gun cerrado queda en CL_GUN_WELD_MIN..MAX en cada tuerca. Anotar lectura por tuerca y fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F22) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Regla de frame decidida; valor de BASE_DATA[3] y método; puntos re-enseñados o verificados; lecturas del LPT por tuerca.

Depende de / se cierra con: C05, F32

Nota. Cambiar BASE_DATA[3] mueve físicamente los 22 puntos de base 3: no correr en automático sin verificarlos. Los TRIGGER ya no están dentro de folds (F32), así que un Touch Up no los borra. C04 se cierra con este punto y C05.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F23 — Posición del LPT armada con 3 bytes en otro orden que en R20

Severidad: Bajo · **Tipo:** Confirmar en sitio · **Responsable:** Puesta en marcha / Programador robot · **Estado:** Abierto

Módulos: KRC/R1/System/sps.sub, KRC/R1/System/\$config.dat, KRC/R1/Program/Centerline/centerline_weld.src

Qué está mal. R10 arma CL_GunStrokePositionLPT = BYTE1*65536 + BYTE2*256 + BYTE3 (sps.sub): 24 bits, CL_LPT_BYTE0 sin usar, así que el desbordamiento de 32 bits de R20 no existe aquí. R20 arma el valor con los cuatro bytes en el orden BYTE1, BYTE0, BYTE3, BYTE2 con el mismo mapeo de bus. Una de las dos no es el formato del BTL6.

Qué hacer. Confirmar el formato de salida del BTL6 de este pedestal (manual Balluff o configuración del transductor). Con el gun abierto el valor debe leer unos 139800 (tabla del encabezado de CENTERLINE_WELD) y con el gun sobre la tuerca unos 70900.

Criterio de cierre (se verifica en el respaldo):

- Formato del BTL6 confirmado (documento y página, o configuración leída del transductor) y anotado en la lista.
- La fórmula de sps.sub corresponde a ese formato; si cambia, la lista trae CL_GunStrokePositionLPT leído antes y después con gun abierto y cerrado sobre tuerca, y las ventanas (CL_GUN_WELD_MIN/MAX, CL_GUN_OPEN_MIN) siguen siendo válidas.
- Si se pasa a componer 32 bits, el cálculo no puede desbordar INT (protección como en el F23 de R20).
- Comentarios según docs/CONVENTIONS.md; el ;CHECK: (F23) de sps.sub y la nota de \$config.dat se quitan o se actualizan.

Evidencia del programador. Documento del formato; lecturas de CL_GunStrokePositionLPT con gun abierto y cerrado sobre tuerca.

Nota. Si el formato confirmado es el de R20, el punto se abre también allá. No tocar las ventanas sin medir.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F24 — Camino de retry de soldadura muerto: NUT_WELD_RETRY nunca es TRUE

Origen: Revisión Ethos (2026-10-04)

Severidad: Bajo · **Tipo:** Código · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/System/\$config.dat

Qué está mal. NUT_WELD_RETRY nunca se ponía TRUE, así que las tres ramas GOTO RETRY_NUT1 de style1app1opt1..3 nunca corrían.

Qué hacer. Decisión A (como en R20): borrar las ramas, las etiquetas y la variable. Hecho en oficina (G17).

Criterio de cierre (se verifica en el respaldo):

- Decisión anotada: A (borrar), Ethos, 2026-10-04, por el punto 17 de Gestamp.
- En KRC/R1 no aparece NUT_WELD_RETRY ni RETRY_NUT1.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Un ciclo completo en automático con las 3 tuercas sin errores. Anotar fecha.

Evidencia del programador. Ciclo completo sin errores, con fecha.

Nota. Sin cambio de comportamiento: las ramas nunca corrían. Si algún día se diseña un retry, el disparo natural es el Retry del diálogo de F07 y necesita F14.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

F25 — Feed de respaldo en CENTERLINE_WELD: pulso de feed de duración cero

Origen: Revisión Ethos (2026-10-04)

Severidad: Bajo · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/System/\$config.dat

Qué está mal. Si NUT_READY está en FALSE al entrar, CENTERLINE_WELD alimenta su propia tuerca con la pieza ya en la pose: avanza el QFP, pone \$OUT[476] dopw1_StartFeed ON e inmediatamente OFF (la espera de \$IN[481] dipw1_FeedComplt está comentada), espera WAIT SEC 1.5 y regresa el QFP. Ese pulso puede no llegar nunca al bus, y el programa sigue igual con o sin tuerca.

Qué hacer. Quitar el camino de alimentación local, o cambiarlo por una petición de preload con espera de NUT_READY. Entrar con NUT_READY en FALSE tiene que terminar en una falla visible o en una espera con timeout, nunca en un cierre de gun sin tuerca.

Criterio de cierre (se verifica en el respaldo):

- En centerline_weld.src no queda ninguna escritura a ON de \$OUT[476] dopw1_StartFeed ni de \$OUT[503] do503QFPAdvance: la alimentación queda solo en la secuencia de preload.
- Si NUT_READY es FALSE al entrar, CENTERLINE_WELD no cierra clamp ni gun sin un preload terminado: (a) se detiene con mensaje y \$OUT[4] do004ProcessFault ON; o (b) pide un preload (solo si NUTCYCLE_ACTIVE es FALSE) y espera NUT_READY con timeout, y al vencer da mensaje y \$OUT[4].
- Prueba en T1 sin pieza, con el robot fuera del pedestal y nadie en él: NUT_READY = FALSE por corrección de variable y CENTERLINE_WELD solo: con (a) se detiene con mensaje; con (b) pide o espera el preload y, con el preload impedido, da timeout con mensaje sin cerrar clamp ni gun. Anotar opción, fecha y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F25) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Opción elegida, (a) o (b); resultado de la prueba en T1; fecha.

Depende de / se cierra con: F11, F14

Nota. No se acepta 'arreglarlo' alargando el pulso. Calvin (#19) propone reactivar la espera de \$IN[481] dipw1_FeedComplt con timeout en el camino local; Ethos (Edgar, 2026-10-04) decidió para R10 lo mismo que para R20: se mantiene el criterio de este punto, quitar el camino local o convertirlo en petición de preload. Si se elige (b), confirmar antes en celda que un preload (QFP, pines, blow-off) puede correr con la pieza en la pose de soldadura; si no puede, la opción es (a). La etiqueta NUT_READY: ya no existe: G11 cambió el salto por IF NOT NUT_READY. F01, F11, F13 y F14 tocan el mismo módulo y las mismas variables: coordinar.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-04 — Decisión: Se mantiene el criterio de F25 frente a la propuesta de Calvin #19, igual que en R20 (decisión de Edgar, 2026-10-04).

F26 — Timers \$TIMER[10..15] de PRELOAD y CENTERLINE_WELD sin constantes con nombre

Severidad: Bajo · **Tipo:** Código · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/System/\$config.dat

Qué está mal. PRELOAD usa \$TIMER[10..13] y CENTERLINE_WELD \$TIMER[14..15], con el índice escrito como número. No chocan con código activo del proveedor. La depuración dejó comentarios de propiedad, pero el código sigue con índices literales. Al terminar un preload se detienen \$TIMER[10..12] pero no \$TIMER[13].

Qué hacer. Declarar en \$config.dat un nombre por cada índice de timer usado, con comentario de programa dueño y uso; usar esos nombres en PRELOAD y CENTERLINE_WELD. Detener \$TIMER[13] al terminar el preload.

Criterio de cierre (se verifica en el respaldo):

- \$config.dat declara un nombre para cada índice de \$TIMER que usan PRELOAD y CENTERLINE_WELD, con comentario de dueño y uso; cada nombre declarado se usa.
- En PRELOAD.src y centerline_weld.src no queda ningún \$TIMER[n] ni \$TIMER_STOP[n] con índice numérico.
- Fuera de PRELOAD y CENTERLINE_WELD ningún módulo que se ejecute usa esos índices; tampoco son timers del proveedor (torque monitoring 7, 8 y 18; NutWeld 41 y 42; AutomationCore 55).
- Al completar un preload, PRELOAD detiene \$TIMER[13] igual que los de feed, advance y return.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F26) se quitan o se actualizan; el cambio está declarado en

tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Confirmación de que el proyecto WorkVisual se actualizó desde el controlador después del cambio en \$config.dat.

Nota. Si F07 o F13 cambian la forma de temporizar, ajustar la asignación. Antes de cualquier deploy de WorkVisual, cargar el proyecto desde el controlador.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F27 — Confirmar en celda funciones de comentarios corregidos: blow-off, pines, programa Bosch

Severidad: Bajo · **Tipo:** Confirmar en sitio · **Responsable:** Puesta en marcha / Gestamp controles · **Estado:** Abierto
Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/PRELOAD.src

Qué está mal. La depuración corrigió comentarios que contradecían al código: UPPER PIN EXTEND sobre el retract, LOWER PIN RETURN y CL_LowerPinRetreated sobre el advance, SCHEDULE 7 sobre el programa 2, QFP ADV sobre la verificación de QFP regresado, y los comentarios en español del pedestal. Ahora describen el código, pero nadie ha confirmado en la celda qué hace físicamente cada salida. Quedan cinco ;CHECK: (F27): tres en centerline_weld.src y dos en PRELOAD.src.

Qué hacer. Confirmar en la celda, con los planos del pedestal CenterLine o forzando salidas en T1 (sin pieza, robot fuera del pedestal, nadie en él): (1) qué acciona \$OUT[494] dopw1_BlowOff, que se conmuta durante el feed y durante el intensify; (2) la asignación de válvulas de los pines: superior \$OUT[498]/\$OUT[499], inferior \$OUT[495] dopw1_AdvancePin y \$OUT[496] dopw1_ReturnPin; (3) que el programa 2 (gdopw1_SpotNumber = CL_WELD_PROGRAM) es el schedule Bosch previsto.

Criterio de cierre (se verifica en el respaldo):

- En la lista, para cada uno de los tres puntos: resultado, cómo se confirmó, quién y fecha.
- Los cinco ;CHECK: que citan (F27) se quitaron o se cambiaron por un ;NOTE: con el hecho confirmado.
- Ningún punto se cierra con una contradicción pendiente: si la confirmación contradijo al código, el respaldo trae la corrección o la lista remite al punto nuevo.
- Comentarios según docs/CONVENTIONS.md; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Para cada punto: lo observado, hoja del plano o foto, y foto de la lista de programas del timer Bosch con el programa 2. Fecha y quién confirmó.

Nota. El programa Bosch lo confirma quien administra el timer (Gestamp). Se puede hacer en la misma visita que F34 y F44. Con lo confirmado se actualiza docs/IO_MAP.md.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F28 — Sensor de tuerca leído al llegar al punto; el dwell iba después de la lectura

Severidad: Bajo · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda
Módulos: KRC/R1/Program/StyleApps/Options/style1app2opt1.src

Qué está mal. Cada revisión de tuerca hacía LIN (paro exacto, P28, P32, P36), leía \$IN[227] y después WAIT SEC 0.2: el asentamiento quedaba después de la muestra.

Qué hacer. Hecho en oficina (G13): cada revisión es WAIT SEC 0.2 y luego PARTPRESENTn = di227NutPresent1; la lectura se hace con el robot parado y una tuerca faltante escribe FALSE (parte de F03).

Criterio de cierre (se verifica en el respaldo):

- En style1app2opt1, en los tres bloques NUT CHECK, PARTPRESENTn se asigna después de WAIT SEC 0.2.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Prueba en automático, override 100 %, fuera de dry cycle y con option 1: ciclos consecutivos con pieza buena sin ningún rechazo por la revisión de tuercas; un ciclo con pieza red rabbit (option 12) deja PARTPRESENT1..3 en FALSE en el display de variables antes del drop. Anotar fecha, número de ciclos y resultado.

Evidencia del programador. Fecha, override, número de ciclos y estado de PARTPRESENT1..3 en el ciclo red rabbit.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

F29 — Faro KL50L2: lógica muerta y un bit sin mapear

Severidad: Bajo · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto
Módulos: KRC/R1/System/sps.sub, KRC/R1/System/\$config.dat, C/KRC/Roboter/Config/User/Common/KRC_IO.xml

Qué está mal. sps.sub maneja el KL50L2 a partir de \$OUT[478]/\$OUT[477], que solo escribe NutWeld_Bkg, deshabilitado: el bloque nunca actúa. Rojo pone \$OUT[3008] y \$OUT[3024]; verde pone \$OUT[3001] y apaga \$OUT[3024]; \$OUT[3008] y \$OUT[3001] nunca se apagan. \$OUT[3001] no está mapeado (KRC_IO.xml mapea \$OUT[3000], [3008], [3016], [3024]) y R20 usa \$OUT[3000] y \$OUT[3016] para el mismo faro.

Qué hacer. Confirmar el mapa de bits del KL50L2; (a) manejar el faro directo desde \$IN[480] dipw1_HopperLowLevel con encendido y apagado explícitos, usando bits mapeados con nombre; o (b) quitar el bloque.

Criterio de cierre (se verifica en el respaldo):

- Mapa de bits del KL50L2 de este pedestal confirmado (qué bit es cada color) y anotado.
- Con (a): la polaridad de \$IN[480] confirmada en celda; el bloque lee \$IN[480], no \$OUT[477]/\$OUT[478], y en cada ciclo deja cada bit usado en un estado definido; solo usa bits mapeados en KRC_IO.xml. Con (b): el bloque ya no existe y ningún módulo escribe esos bits.
- Con (a): prueba en celda con la tolva en nivel bajo y en nivel OK; el faro cambia con el nivel. Anotar fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F29) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Opción elegida; estado de \$IN[480] con tolva baja y llena; estado del faro; fecha.

Nota. Hecho en oficina (G12): los tres bits tienen nombre (CL_BeaconBit3001, CL_BeaconBit3008, CL_BeaconBit3024); la lógica no cambió. Los nombres se ajustan cuando se confirme el mapa. La opción (b) deja el faro siempre apagado: avisar a Gestamp.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F30 — Style1Opt1 sin GOTO: un camino IF/ELSE por ciclo

Severidad: Bajo · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/Styles/Options/style1opt1.src

Qué está mal. Style1Opt1 usaba GOTO para terminar el ciclo después de un RejectGE4 y para no correr la estación 2 después de un ciclo de estación 1 (las tres etiquetas eran una sola salida). Gestamp no permite GOTO (G11).

Qué hacer. Hecho en oficina: la estación se lee una vez en nPickStation (1, 2 o 0) y el resto es un IF/ELSE: después de cada RejectGE4() no corre nada más; la estación 1 nunca corre la 2; todos los caminos terminan en el borrado de PARTPRESENT1..3. Con ninguna estación sigue siendo un ciclo vacío (F06).

Criterio de cierre (se verifica en el respaldo):

- En style1opt1.src no hay GOTO ni etiquetas y se conservan las tres propiedades: (1) después de RejectGE4() no corren ni la cámara ni el drop; (2) un ciclo de estación 1 no corre la estación 2; (3) todos los caminos pasan por el borrado de PARTPRESENT1..3.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Prueba en celda: un ciclo bueno de estación 1, uno de estación 2, un rechazo por nut check y uno por cámara (si se puede provocar). Anotar fecha y resultado.

Evidencia del programador. Fecha y resultado de cada prueba.

Nota. Si F03 o F06 cambian style1opt1.src, se vuelven a revisar las tres propiedades.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

F31 — Forms GripperTech: parámetros ocultos de otro gripper; drop red rabbit con el gripper 3

Origen: Revisión Ethos (2026-10-04)

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/TP/GripperSpotTech/grp_data.dat

Qué está mal. R10 usa forms GripperTech en picks, drops y reject; solo el gripper 1 está activo en grp_data.dat (\$OUT[249]/[250], \$IN[257]/[258]). En los tres picks y en el drop al conveyor los forms SET traían setgrripper=4;setstate=2 mientras el código llama GRPg_SetStateAndCheck(1, 1 o 2, ...): abrir y confirmar uno de esos forms regeneraba la llamada para el gripper 4 (\$OUT[251]/[252], \$IN[259]/[260], en el bus), el gripper 1 ya no se movía y el check daba timeout (3 s). Cada SET con check iba seguido de un GRPg_Check redundante. El drop red rabbit escribe \$OUT[249]/[250] a mano y suelta con el gripper 3, que comparte la E/S del gripper 1 (F19).

Qué hacer. Hecho en oficina (G10): parámetros ocultos de los forms SET de picks y drop al conveyor corregidos a setgrripper=1 con el estado de la llamada; GRPg_Check redundantes quitados. Falta: reemplazar las salidas escritas a mano y el form del gripper 3 del drop red rabbit por forms del gripper 1 (con F19) y probar en celda.

Criterio de cierre (se verifica en el respaldo):

- En los módulos listados cada form Gripper SET/CHECK tiene en su línea ;Params el mismo gripper y estado que su llamada (hecho en oficina para picks y drop al conveyor) y no queda GRPg_Check que repita el check del SET anterior.
- style1drop1opt2AutoRR no escribe \$OUT[249]/\$OUT[250] a mano y no usa forms del gripper 3; ningún módulo de producción usa los grippers 2-4.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Prueba en el smartPAD: abrir y confirmar (Cmd OK) un form SET de cada módulo; en el respaldo siguiente las llamadas no cambiaron. En T1 sin pieza: con un switch del gripper 1 desconectado el check da su error (estrategia 1) en pick, drop y reject. Anotar fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F31) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Lista de forms revisados; resultado de la prueba Cmd OK y de la prueba con switch desconectado; fecha.

Depende de / se cierra con: F33, F19

Nota. Parte de oficina hecha (tools/code_changes.json, G10, G13, G14). No guardar la configuración de GripperTech en el HMI mientras F33 esté abierto.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F32 — TRIGGERs y resets escritos a mano dentro de folds de inline form

Origen: Revisión Ethos (2026-10-04)

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src

Qué está mal. Los TRIGGER de NUT_STAR (picks, P12 / P10) y de Request_next_nut() (final de las apps 1 y 2, en el fold P0) estaban escritos a mano dentro de folds de inline form de movimiento; el drop red rabbit tenía los resets de banderas dentro del form DropOff[2] Reset. Un Touch Up o reabrir el form los borraba: la tuerca 1 se quedaba sin preload (NUT_READY con un TRUE viejo, F14) o la tuerca 2/3 esperaba NEXT_NUT_READY para siempre.

Qué hacer. Hecho en oficina (G20): cada TRIGGER quedó justo antes de su fold de movimiento (posición del form SYN OUT de KUKA: mismo movimiento, mismo punto de disparo); los resets del red rabbit van después del fold DropOff[2] Reset.

Criterio de cierre (se verifica en el respaldo):

- Los folds PTP P12 (style1pick1opt1, style1pick1opt1AutoRR), P10 (style1pick1opt2) y P0 (style1app1opt1 y opt2) contienen solo las líneas que genera el form; los TRIGGER están fuera, inmediatamente antes.
- El fold DropOff[2] Reset de style1drop1opt2AutoRR contiene solo su ;Params y AC_DropOff (2,True); los resets se ejecutan después.
- Los encabezados ;FOLD ... ;%{PE} de los folds tocados son idénticos a los del respaldo recibido: no se reabrió ningún form.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los

módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.

- Prueba en T1 o en automático con override reducido, fuera de dry cycle: en un ciclo completo NUT_READY y NEXT_NUT_READY pasan a FALSE y vuelven a TRUE una vez por tuerca (3 preloads por pieza). Un ciclo red rabbit deja \$OUT[141], \$OUT[142] y PARTPRESENT1..3 en FALSE. Anotar fecha.

Evidencia del programador. Resultado de la prueba de ciclo; fecha.

Nota. Gestamp (punto 20) describe exactamente esto para xP0. Una selección de bloque sobre el fold salta el TRIGGER (hay que seleccionar la línea del TRIGGER): F14, F38.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.
- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

F33 — GripperConfig.xml del HMI desincronizado con grp_data.dat

Severidad: Medio · **Tipo:** WorkVisual / configuración · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: C:/KRC/User/TP/Gripper_SpotTech/GripperConfig.xml, KRC/R1/TP/GripperSpotTech/grp_data.dat

Qué está mal. GripperConfig.xml (plug-in del HMI, idéntico al de R20) tiene todos los grippers inactivos con E/S dummy, mientras grp_data.dat tiene el gripper 1 activo en \$OUT[249..250] / \$IN[257..258] (y el gripper 3 inactivo en la misma E/S). Si alguien guarda la configuración de GripperTech en el smarthMI, grp_data.dat se regenera con la E/S dummy: cada GRPg_SetStateAndCheck(1, ...) mandaría y revisaría los bits equivocados.

Qué hacer. Mientras no se re-sincronice, nadie guarda la configuración de GripperTech en el HMI. Re-sincronizar el XML con grp_data.dat (gripper 1 activo, mismas salidas, entradas y estados) y comprobar que un guardado desde el HMI regenera un grp_data.dat equivalente.

Criterio de cierre (se verifica en el respaldo):

- Control en cada respaldo mientras el punto esté abierto: grp_data.dat conserva GRPg_Grp[1] con Act TRUE, salidas 249/250 y entradas 257/258 (nadie guardó desde el HMI).
- Para cerrar, el GripperConfig.xml del respaldo tiene el gripper 1 activo con salidas 249/250 y entradas 257/258 sin GhostMode, y los valores por estado coinciden con grp_data.dat.
- Guardado de prueba desde el HMI hecho entre dos respaldos: GRPg_Grp, GRPg_GrpPar y GRPg_State del gripper 1 no cambian, y el drop red rabbit sigue soltando (o F19/F31 ya lo pasaron al gripper 1).

Evidencia del programador. Cómo se re-sincronizó y fecha; foto de la configuración de GripperTech; en qué respaldo se hizo el guardado de prueba.

Depende de / se cierra con: F31

Nota. Editar grp_data.dat a mano no cierra el punto. Un deploy de WorkVisual también puede regenerarlo. Mientras el drop red rabbit use el gripper 3 (F19), la re-sincronización debe conservar su E/S o el drop dejará de soltar.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F34 — CENTERLINE_HOME: polaridad de \$OUT[473] y falta el reset del Bosch

Severidad: Bajo · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:**

Requiere decisión

Módulos: KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/Centerline/centerline_weld.src

Qué está mal. CENTERLINE_HOME pone \$OUT[473] dopw1_Nut1Intensify_Home en OFF, mientras CENTERLINE_WELD lo deja en ON en reposo. CENTERLINE_HOME no resetea weld start (\$OUT[483]), contactor enable (\$OUT[481]), weld on (\$OUT[482]) ni el número de programa (\$OUT[486..493]). Su water-off lo sobrescribe sps.sub (F09) y PRELOAD puede reescribir sus salidas (F11).

Qué hacer. Confirmar en la celda qué hace \$OUT[473] en ON y su estado de reposo, y dejar el mismo valor en CENTERLINE_HOME y CENTERLINE_WELD. Agregar a CENTERLINE_HOME el reset del Bosch (se puede hacer ya).

Criterio de cierre (se verifica en el respaldo):

- En la lista: función física de \$OUT[473] y su estado de reposo, cómo se confirmó y fecha.
- CENTERLINE_HOME y CENTERLINE_WELD escriben el mismo valor de reposo para \$OUT[473].
- CENTERLINE_HOME pone \$OUT[483] dopw1_WeldInit OFF, \$OUT[481] dopw1_WeldContactEnable OFF, \$OUT[482] dopw1_WeldOnExternal OFF y gdopw1_SpotNumber = 0.
- Prueba en T1 sin pieza, robot fuera del pedestal y nadie en él: poner \$OUT[481] y \$OUT[482] en ON y gdopw1_SpotNumber = 2 desde el display, correr CENTERLINE_HOME y verificar los resets y \$OUT[473] en el reposo confirmado. Anotar fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F34) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Función de \$OUT[473] observada, estado de reposo decidido y resultado de la prueba; fecha.

Nota. Para la prueba no forzar \$OUT[483]. Si F10 crea una rutina de estado seguro, CENTERLINE_HOME puede llamarla. Se puede confirmar en la misma visita que F27.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F35 — El mastering reference deja \$OUT[930] en ON para siempre

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/masref_user.src, KRC/R1/System/masref_main.src

Qué está mal. MASREFSTARTG1 pone do930MasterRefInProcess en TRUE y MASREFBACKG1 lo pone en TRUE otra vez al final, donde la línea vieja comentada ponía do828 en FALSE. Nada más escribe \$OUT[930]: después de la primera prueba de mastering reference queda en ON para siempre. La rutina de request to enter de AutomationCore (edición del integrador) niega la entrada mientras está en ON, así que si se reactiva el background (F12) la request to enter queda bloqueada. \$OUT[930] cae además en el bit del PLC de \$OUT[111] (F18). El camino cierra el gripper 1 antes de ir al switch de referencia.

Qué hacer. Poner do930MasterRefInProcess en FALSE al final de MASREFBACKG1, después del último movimiento de regreso.

Criterio de cierre (se verifica en el respaldo):

- En masref_user.src, MASREFBACKG1 termina con do930MasterRefInProcess = FALSE después del último movimiento; MASREFSTARTG1 es el único TRUE.
- Prueba en T1 (sin pieza en el gripper): correr el mastering reference; \$OUT[930] está en ON durante la prueba y en OFF al terminar; el PLC ve el tag de \$OUT[111] en OFF al final (F18). Anotar fecha.
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F35) se quitan o se actualizan; el cambio está declarado en

tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Fecha y estado de \$OUT[930] antes, durante y después de la prueba.

Depende de / se cierra con: F18

Nota. Requisito antes de reactivar AutomationCore_Bkg (F12). masref_user no es módulo del ciclo de producción.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Severidad subida a Alto: Con AutomationCore_BKG activo (F49), \$OUT[930] pegado en ON impide conceder el Request to Enter: \$OUT[145] doCriticalWZ queda en TRUE y AC_PointArrival ya no detiene el robot cuando el operador pide entrar. Corregir F35 antes de producción.
- 2026-10-05 — Severidad regresa a Medio: Con el background apagado en la versión integrada, \$OUT[930] pegado ya no impide el Request to Enter. Sigue pendiente apagar \$OUT[930] al final de la referencia de masterización.

F36 — \$IN[227] declarado con dos nombres (NUT_PRESENT1, di227SensorCamera)

Severidad: Bajo · **Tipo:** Código · **Responsable:** Programador robot (prueba en celda) · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/System/\$config.dat, KRC/R1/TP/AutomationCore/automationcoreroutines.dat, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src

Qué está mal. \$IN[227] estaba declarado como NUT_PRESENT1 en \$config.dat y como di227SensorCamera en automationcoreroutines.dat: es el sensor de tuercas, no la cámara.

Qué hacer. Hecho en oficina (G15): di227SensorCamera renombrado di227NutPresent1 y la declaración NUT_PRESENT1 de \$config.dat borrada; el nut check y los comentarios usan di227NutPresent1.

Criterio de cierre (se verifica en el respaldo):

- En todo KRC/ del respaldo, \$IN[227] tiene una sola declaración SIGNAL: di227NutPresent1.
- NUT_PRESENT1 y di227SensorCamera ya no aparecen en código ni declaraciones de KRC/.
- La edición en automationcoreroutines.dat aparece en la lista de ediciones del integrador en archivos del proveedor (C28).
- Respaldo completo (Archive → All) después de cargar en el robot la versión de oficina (zip de tools/build_archive.py): los módulos coinciden con el zip entregado (la auditoría lo compara), los módulos borrados o movidos ya no están en el controlador (un restore no borra archivos) y el proyecto WorkVisual se actualizó desde el controlador antes de cualquier deploy.
- El programa carga sin errores de compilación (captura del navegador del smartPAD sin módulos en rojo). Un ciclo con option 1 lee las 3 tuercas (PARTPRESENT1..3 en TRUE en el display de variables). Anotar fecha.

Evidencia del programador. Captura del navegador sin errores y fecha del ciclo.

Depende de / se cierra con: C28

Nota. Cambia \$config.dat: un deploy desde el proyecto WorkVisual viejo restauraría la declaración NUT_PRESENT1.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - no cargado: Respaldo bmw_03_10_r1 (2026-10-05): la versión de oficina no está cargada; el robot tiene el programa original (29 archivos idénticos al respaldo de partida, 16 editados encima del original). Sin prueba posible.

- 2026-10-05 — Integrado - por probar: Sigue en la versión integrada (658424_R10_2026-10-05_0819.zip, armada sobre el respaldo del programador). Cargarla y hacer la prueba de este punto.

F37 — Presión de intensificado no confirmada antes de weld start: no aplica en R10

Origen: Revisión Ethos (2026-10-04)

Severidad: - · **Tipo:** Sin acción · **Responsable:** Programador robot · **Estado:** No aplica

Módulos: KRC/R1/Program/Centerline/centerline_weld.src

Qué está mal. En R20 la espera de presión de intensificado está desactivada (Calvin #1). R10 conserva WAIT FOR dipw1_IntensfPresOk (\$IN[479]) en CENTERLINE_WELD antes de weld start, como dice Calvin.

Qué hacer. Nada en este punto. La espera no tiene timeout (F15) y la presión que recibe de verdad el regulador es F44.

Criterio de cierre (se verifica en el respaldo):

- Se reabre si la espera de \$IN[479] dipw1_IntensfPresOk de CENTERLINE_WELD se desactiva o se quita.

Depende de / se cierra con: F15, F44

Historial de auditoría:

- 2026-10-04 — Alta (Calvin): Revisión Ethos del respaldo 658424: el punto 1 de Calvin (20R1) no aplica en R10.

F38 — Estado de la tuerca no se reinicializa al arrancar, en reset ni después de un reinicio

Origen: Revisión Ethos (2026-10-04)

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/System/\$config.dat, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/cell.src, KRC/R1/System/sps.sub

Qué está mal. El handshake de la tuerca vive en globales persistentes de \$config.dat (NUT_READY declarado TRUE, NUT_START, NUTCYCLE_ACTIVE, NUT_PRELOAD_STEP, STEP0/10/20DONE, STEP10_ADV_STARTED, NEXT_NUT_READY / NEXT_NUT_REQUEST). Nada las inicializa al arrancar el programa, al cancelar o resetear, ni al arrancar el controlador; después de un reinicio a mitad de un preload, PRELOAD retoma el paso guardado; después de una selección de bloque los TRIGGER del handshake se saltan.

Qué hacer. Una rutina de inicialización llamada desde el USER INIT de sps.sub, desde cell.src antes del primer ciclo y desde la rutina de estado seguro de F10, que derive NUT_READY de la E/S real. Un preload interrumpido nunca termina en tuerca lista.

Criterio de cierre (se verifica en el respaldo):

- Existe una rutina de inicialización del estado de la tuerca, llamada desde el USER INIT de sps.sub y desde cell.src antes del primer ciclo (y desde la rutina de F10), que deja NUTCYCLE_ACTIVE = FALSE, NUT_PRELOAD_STEP = 0, STEP*DONE y STEP10_ADV_STARTED en FALSE, NUT_START y NEXT_NUT_REQUEST en FALSE, y NUT_READY / NEXT_NUT_READY a partir de la E/S real.
- La seguridad del ciclo ya no depende del valor inicial declarado en \$config.dat.
- Prueba en celda: (a) cortar el ciclo a mitad de un preload y reiniciar el controlador: PRELOAD no retoma el paso y las banderas reflejan el pin; (b) reset del programa con tuerca precargada y sin ella: el siguiente ciclo no suelda sin tuerca ni alimenta una segunda. Anotar fecha y resultado.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F38) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Dónde se llama la rutina y resultado de las dos pruebas.

Depende de / se cierra con: F14, F10, F11, F13

Nota. Gestamp (punto 19) pide lo mismo; G19 se cierra con este punto y F14. F14 cubre el consumo de NUT_READY; F10 el estado seguro de las salidas.

Historial de auditoría:

- 2026-10-04 — Alta (Calvin): Revisión Ethos del respaldo 658424; el punto sale de la lista de Calvin (Calvin #11).

F39 — Supervisión de par en cero; chequeo de datos de carga solo avisa; carga tecleada

Origen: Revisión Ethos (2026-10-04)

Severidad: Medio · **Tipo:** Configuración + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto

Módulos: KRC/STEU/Mada/\$custom.dat, KRC/R1/System/\$config.dat, KRC/R1/System/bas.src

Qué está mal. \$TORQMON_DEF[1..12] y \$TORQMON_COM_DEF[1..12] están en 0 en KRC/STEU/Mada/\$custom.dat (líneas 80-104). \$LDC_CONFIG[1] = {UNDERLOAD #WARNONLY, OVERLOAD #WARNONLY}: el chequeo de datos de carga solo avisa. LOAD_DATA[1] = M 210, CM {270, 0, 240}, J {105, 105, 105}: redondos, tecleados (Gestamp punto 3). Todos los movimientos de producción corren con TQ_STATE FALSE.

Qué hacer. Con los datos de carga determinados (G03/C08), definir \$TORQMON_DEF / \$TORQMON_COM_DEF según el manual de KUKA (o dejar por escrito por qué se quedan) y decidir la reacción del chequeo de datos de carga de la herramienta 1.

Criterio de cierre (se verifica en el respaldo):

- En el respaldo \$TORQMON_DEF y \$TORQMON_COM_DEF tienen valores definidos, o la lista trae la justificación escrita (referencia al manual) de dejarlos en 0.
- \$LDC_CONFIG[1] tiene la reacción acordada con Gestamp (p. ej. OVERLOAD #STOPROBOT), anotada.
- Prueba en celda después de G03: un ciclo completo en automático sin falsos paros por par ni por carga. Anotar fecha y valores.

Evidencia del programador. Valores puestos, referencia del manual y fecha de la prueba.

Depende de / se cierra con: G03, C08, C09

Nota. Es machine data: hacerlo en la misma visita que C09. Calvin describe R10 como robot de pistola; es de manejo: los hechos se sostienen.

Historial de auditoría:

- 2026-10-04 — Alta (Calvin): Revisión Ethos del respaldo 658424; el punto sale de la lista de Calvin (Calvin #16).

F40 — Drops y reject abren el gripper sin confirmar pieza; el drop red rabbit se mueve antes de gripper vacío

Origen: Revisión Ethos (2026-10-04)

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src

Qué está mal. G14 corrigió el orden al inicio de los picks. El drop al conveyor y RejectGE4 abren el gripper sin confirmar antes \$IN[253] di253PartPresent1 y \$IN[254] di254PartPresent2 ON: una pieza perdida en el camino se "entrega" y \$OUT[18] se apaga como si nada. El drop red rabbit se mueve (LIN P23) antes de esperar gripper vacío.

Qué hacer. Espera supervisada de pieza presente antes de abrir en los dos drops y en RejectGE4; espera de gripper vacío antes de alejarse en el drop red rabbit.

Criterio de cierre (se verifica en el respaldo):

- En style1drop1opt1, style1drop1opt2AutoRR y RejectGE4 hay una espera de \$IN[253] y \$IN[254] ON antes del form que abre el gripper (fuera de dry cycle, como las demás esperas de pieza).

- En style1drop1opt2AutoRR la espera de gripper vacío está antes de LIN P23.
- Prueba en T1: quitar la pieza del gripper antes de llegar al drop: el robot se detiene antes de abrir con mensaje. Anotar fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F40) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Fecha y resultado.

Depende de / se cierra con: G14, G10, F31, F19

Nota. Si F19 reescribe el drop red rabbit, el orden de este punto va incluido.

Historial de auditoría:

- 2026-10-04 — Alta (Calvin): Revisión Ethos del respaldo 658424; el punto sale de la lista de Calvin (Calvin #25).

F41 — Tiempos de PRELOAD, límite de flujo de agua y esperas fijas sin constantes

Origen: Revisión Ethos (2026-10-04)

Severidad: Bajo · **Tipo:** Código · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/System/sps.sub, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/System/\$config.dat

Qué está mal. G16 llevó a constantes la ventana del gun, las presiones y los tiempos del chequeo de gun cerrado. Quedan números: los tiempos de PRELOAD (2000, 500, 800 y 800 ms), el límite de flujo SV0500_FLOW > 1230 de sps.sub y los WAIT SEC fijos de CENTERLINE_HOME y CENTERLINE_WELD.

Qué hacer. Declarar constantes con nombre en \$config.dat (mismos valores) y usarlas en el código; registrar los nombres en tools/renames.json para que check_equivalence muestre que el valor no cambió.

Criterio de cierre (se verifica en el respaldo):

- En PRELOAD.src, sps.sub, CENTERLINE_HOME.src y centerline_weld.src no quedan esos números en líneas ejecutables: usan constantes de \$config.dat con los mismos valores.
- Un ciclo completo en automático sin cambio de tiempo de ciclo. Anotar fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F41) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Lista de constantes y fecha del ciclo.

Depende de / se cierra con: G16, F26, F09

Nota. Si F13 o F09 cambian esos valores, se declaran directamente con el valor nuevo.

Historial de auditoría:

- 2026-10-04 — Alta (Calvin): Revisión Ethos del respaldo 658424; el punto sale de la lista de Calvin (Calvin Std 5).

F42 — Envolventes de trabajo por software apagadas

Origen: Revisión Ethos (2026-10-04)

Severidad: Bajo · **Tipo:** Configuración / decisión · **Responsable:** Programador robot / Seguridad / Gestamp · **Estado:** Requiere decisión

Módulos: KRC/R1/Mada/\$machine.dat, KRC/STEU/Mada/\$custom.dat, C/KRC/Roboter/Config/User/Common/SafetyConfigData.xml

Qué está mal. \$AXWORKSPACE[1..8] y \$CYLWORKSPACE[1..8] están en #OFF (KRC/R1/Mada/\$machine.dat, líneas 378-385 y 403-410). \$WORKSPACE[1] y [2] (KRC/STEU/Mada/\$custom.dat) están en #OUTSIDE: solo señalan, no detienen el robot. SafeOperation

tiene un espacio de monitoreo "Nest Workspace" (última edición 2026-09-13). No está documentado qué cubren los espacios de seguridad.

Qué hacer. Decidir con Gestamp si hacen falta envolventes de software con paro además de SafeOperation; si no, dejar por escrito, con captura del smartHMI, qué cubren el Cell area y los espacios de seguridad.

Criterio de cierre (se verifica en el respaldo):

- Decisión anotada en la lista (quién y fecha).
- Si se activan: valores en el respaldo y prueba en T1 de que el robot se detiene al entrar a la zona. Si no: captura de la configuración de SafeOperation (Cell area y espacios) adjunta.
- \$WORKSPACE[1] y [2]: uso documentado (qué salida dan y quién la lee) o en #OFF.

Evidencia del programador. Decisión y capturas.

Depende de / se cierra con: G02

Nota. Conviene hacerlo en la misma sesión de seguridad que G02.

Historial de auditoría:

- 2026-10-04 — Alta (Calvin): Revisión Ethos del respaldo 658424; el punto sale de la lista de Calvin (Calvin #30).

F43 — Cambio de electrodo: reset de stepper por pulso de 0.1 s y handshake que no cierra

Origen: Revisión Ethos (2026-10-04)

Severidad: Medio · **Tipo:** Decisión + código · **Responsable:** Programador robot / Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Utilities/gunelectrodechange.src, KRC/R1/cell.src

Qué está mal. En gunelectrodechange.src el reset del stepper es PULSE(dopw1_StepperReset,TRUE,0.1) (\$OUT[485]) sin revisar \$IN[487] dipw1_EndOfStepper; \$OUT[92] do092RobotCapChg1Cmplt solo se apaga si \$OUT[111] ya está en OFF, y \$OUT[91] do091RobotReadyCapChange se apaga sin esperar a que el PLC baje \$IN[92]. Las esperas no tienen timeout. cell.src llama GunElectrodeChange cuando \$OUT[111..113] están en ON, pero esas salidas solo las escribe el background deshabilitado (F12): tal como está, el cambio de electrodo nunca corre. Calvin cita electrodechange.src, una copia vieja que nada llamaba (borrada en la depuración); la rutina viva tiene el mismo patrón.

Qué hacer. Decidir con Gestamp cómo se pide el cambio (bit del PLC o contador); confirmar el reset del stepper con \$IN[487]; cerrar el handshake por niveles; supervisar las esperas; quitar de la rutina lo de pistolas 2 y 3.

Criterio de cierre (se verifica en el respaldo):

- Forma de pedir el cambio de electrodo acordada con Gestamp por escrito, y cell.src llama la rutina con esa petición, fuera de ciclo.
- El reset del stepper se confirma con \$IN[487] (espera supervisada) en lugar de solo un pulso de 0.1 s.
- do092 y do091 se apagan siempre al terminar, después de que el PLC bajó su petición; ninguna salida del handshake queda en ON.
- Prueba en celda: petición, cambio, confirmación, reset de stepper confirmado y regreso a home. Anotar fecha.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F43) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Señal de petición acordada y resultado de la prueba.

Depende de / se cierra con: F12, F15, C20

Nota. Gestamp (punto 6) pide lo mismo; G06 se cierra con este punto.

Historial de auditoría:

- 2026-10-04 — Alta (Calvin): Revisión Ethos del respaldo 658424; el punto sale de la lista de Calvin (Calvin #29).
- 2026-10-05 — Auditoría - riesgo nuevo: Con AutomationCore_BKG activo, do111Gun1ElectrodeChange sigue a dipw1_EndofStepper: cell.src ya llama GunElectrodeChange cuando el stepper llega al final. El camino que F43 describe como nunca ejecutado ahora corre: probarlo antes de producción.
- 2026-10-05 — Integrado - riesgo: En la versión integrada GunElectrodeChange se ejecuta al final del stepper. Probar antes de producción.
- 2026-10-05 — Riesgo activo: Con el background activo, el cambio de electrodo corre al final del stepper y espera \$IN[92] del PLC en un diálogo; nunca se ha probado. Además \$OUT[111] comparte bit del adaptador con \$OUT[930] (F18).

F44 — Palabra de presión del gun escrita en dos órdenes de bytes (405/1820 contra 39425)

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Puesta en marcha / Programador robot · **Estado:** Abierto
Módulos: KRC/R1/System/\$config.dat, KRC/R1/Program/Centerline/centerline_weld.src, C/KRC/Roboter/Config/User/Common/KRC_IO.xml

Qué está mal. CL_GunPressureCmd (\$OUT[506..521]) está mapeado en KRC_IO.xml igual que en R20: el regulador recibe la palabra con los bytes intercambiados. En R20 todos los valores se escriben intercambiados (0.30 MPa = 1365 se escribe 21765). R10 escribe 405 (general, CL_GUN_PRESS_BASE) y 1820 (intensify, CL_GUN_PRESS_WELD, "0.40 MPa" en la tabla) sin intercambiar, y 39425 (= 0.09 MPa intercambiado, CL_GUN_PRESS_REST) al final de cada soldadura. Si el regulador de R10 no está configurado distinto, el comando general le llega como 38145 y el de intensify como 7175, fuera de 0..4095; si no intercambia, el que queda fuera de rango es 39425. Uno de los dos juegos está mal, y el gun reposa con 39425 entre soldaduras. \$IN[479] se cumple en producción, así que la presión de soldadura está al menos arriba del ajuste del presostato.

Qué hacer. Leer en celda el display del ITV (o su salida de monitor) en reposo, con el gun cerrado antes del intensify y durante el intensify; comparar con la configuración del regulador (orden de bytes, rango de 12 bits). Escribir los tres valores en el orden que espera el regulador, como constantes (ya existen, G16).

Criterio de cierre (se verifica en el respaldo):

- Lecturas del ITV en reposo, con gun cerrado antes del intensify y durante el intensify, con fecha, y la configuración del regulador (orden de bytes y rango) anotadas en la lista.
- CL_GUN_PRESS_BASE, CL_GUN_PRESS_WELD y CL_GUN_PRESS_REST en \$config.dat están en el mismo orden de bytes, el que confirma la lectura, y su comentario da la presión en MPa; la presión de soldadura sigue siendo la que Gestamp aprueba por escrito.
- La tabla de presiones del encabezado de CENTERLINE_WELD y los ;CHECK: (F44) se actualizan.
- Prueba en celda: un ciclo completo; el ITV muestra la presión esperada en cada fase y \$IN[479] se cumple en el intensify. Anotar fecha y lecturas.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F44) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Lecturas del ITV antes y después del cambio, configuración del regulador y aprobación de Gestamp de la presión de soldadura.

Depende de / se cierra con: G16

Nota. Cambiar los valores cambia la presión real de soldadura de un proceso que ya produce: coordinar con Gestamp (proceso) antes de cargar. F08 y F10 usan la presión base que salga de aquí. Se puede hacer en la misma visita que F27.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F45 — Rutinas de clamps: grippers 2-4 deshabilitados, solo existe el gripper 1

Origen: Revisión Ethos (2026-10-04)

Severidad: Bajo · **Tipo:** Sin acción · **Responsable:** Programador robot · **Estado:** Cerrado - depuración

Módulos: KRC/R1/Program/Utilities/closeandcheckallclamps.src, KRC/R1/Program/Utilities/openandcheckallclamps.src, KRC/R1/TP/GripperSpotTech/grp_data.dat

Qué está mal. CloseAndCheckAllClamps / OpenAndCheckAllClamps (solo las llama el mastering reference) tenían comentados el set y el check de los grippers 2-4. Los grippers 2-4 están inactivos en grp_data.dat: el 3 duplica la E/S del gripper 1, el 4 apunta a \$OUT[251]/[252], \$IN[259]/[260] (sin función en el EOAT de R10) y las entradas del 2 (\$IN[269..272]) no están en el bus. Reactivarlos no agregaría nada.

Qué hacer. Hecho en la depuración: se quitaron los folds deshabilitados (solo líneas de comentario) y se documentó que las rutinas manejan solo el gripper 1. Sin cambio de comportamiento.

Criterio de cierre (se verifica en el respaldo):

- Las rutinas siguen manejando solo el gripper 1 y su comentario lo dice (verificación de regresión en cada respaldo).
- Se reabre si el EOAT de R10 recibe un gripper 2-4 con función real.

Historial de auditoría:

- 2026-10-04 — Alta (Calvin): Revisión Ethos del respaldo 658424; el punto sale de la lista de Calvin (Calvin #23).

F46 — EndOfCycle del proveedor editado: Work Complete nunca se manda al PLC

Severidad: Medio · **Tipo:** Decisión Gestamp · **Responsable:** Gestamp controles / Ethos (Edgar) · **Estado:** Requiere decisión

Módulos: KRC/R1/TP/AutomationCore/automationcoreroutines.src, KRC/R1/cell.src

Qué está mal. La rutina del proveedor EndOfCycle (automationcoreroutines.src) se editó en este robot (;=====SL=====): do015WorkComplete = TRUE, la espera de di015WorkCompleteAckn y do005RobotInCycle = FALSE están comentados. Work Complete nunca va al PLC; Robot In Cycle baja hasta el inicio de la siguiente vuelta (MaintainSystem). R20 tiene la rutina estándar. La celda corre, así que el PLC no espera Work Complete de 10R1, pero es un handshake del estándar Gestamp (C18, C20).

Qué hacer. Decidir con Gestamp y el programador del PLC: restaurar la rutina del proveedor o documentar la desviación.

Criterio de cierre (se verifica en el respaldo):

- Decisión escrita de Gestamp (quién y fecha).
- Si se restaura: EndOfCycle es igual a la rutina estándar de AutomationCore (comparada con la de R20); el PLC maneja Work Complete. Prueba en celda: el PLC ve \$OUT[15] do015WorkComplete en cada ciclo, contesta \$IN[15] y \$OUT[5] do005RobotInCycle baja al terminar. Anotar fecha.
- Si se mantiene: la edición está en la lista de C28 y el comentario de cell.src cita la decisión.
- Comentarios según docs/CONVENTIONS.md; el ;WARNING: (F46) de cell.src se quita o se actualiza.

Evidencia del programador. Decisión de Gestamp y resultado de la prueba.

Depende de / se cierra con: C28

Nota. El veredicto red rabbit (F19) y el conteo de soldaduras (C20) dependen de este handshake.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F47 — Camino del brake test enseñado con tool 3 / base 1 de otro robot

Severidad: Medio · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto

Módulos: KRC/R1/TP/BrakeTest/braketeststart.src, KRC/R1/TP/BrakeTest/braketestback.src, KRC/R1/System/\$config.dat

Qué está mal. TP/BrakeTest/braketeststart.src mueve PTP a m y h (la misma pose, 2.6 m de alto) con Tool[3] y Base[1]; el texto del fold dice "Tool 3" y "3-30-R1 03-20B", copiado de otro robot. En R10 la base 1 es "10A" y la herramienta 3 "TOOL 3". Los puntos son cartesianos, así que el camino depende de lo que tengan la herramienta 3 y la base 1. braketestback.src regresa con la herramienta 1. Los archivos del proveedor también traen la edición de do931BrakeTestInProcess.

Qué hacer. Correr el brake test despacio en T1 y re-enseñar el camino con tool 1 / base 0 (de preferencia puntos de ejes).

Criterio de cierre (se verifica en el respaldo):

- Los folds de braketeststart.src ya no muestran Tool[3] ni Base[1]:3-30-R1: usan tool 1 / base 0 o puntos de ejes.
- Prueba en T1 a velocidad reducida: el brake test sale de HOME, llega a su pose sin interferencia y regresa a HOME. Anotar fecha y override.
- La edición en archivos del proveedor queda en la lista de C28.
- Comentarios del código cambiado según docs/CONVENTIONS.md (inglés, señal con dirección y nombre); los ;CHECK:;/NOTE:;/WARNING: que citan (F47) se quitan o se actualizan; el cambio está declarado en tools/code_changes.json y tools/check_equivalence.py pasa; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Puntos re-enseñados y fecha de la prueba.

Depende de / se cierra con: C28

Nota. Re-enseñar edita archivos del proveedor (TP/BrakeTest): listarlo en C28. C06 y C14 dependen de este punto.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

F48 — Drop al conveyor: interlock de drop-off movido después de la aproximación (corregido); AtDrop3 re-enseñado (se conserva)

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Gestamp controles · **Estado:** Corregido en oficina - probar en celda

Módulos: KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.dat

Qué está mal. AC_DropOffCheck(1) (espera \$IN[75] di075DropOffMachine1) estaba antes de cualquier movimiento; ahora va después de PTP P5 y PTP P1: el robot se acerca al conveyor sin permiso del PLC. AtDrop3 se re-enseñó con 26° más en B y 34° en C, sin explicación.

Qué hacer. Hecho en oficina: la versión integrada deja AC_DropOffCheck(1) antes del primer movimiento, como estaba. AtDrop3 se conserva como lo enseñó el programador el 2026-10-05: así la pieza cae mejor en el conveyor (validado por Edgar Montiel con el programador).

Criterio de cierre (se verifica en el respaldo):

- AC_DropOffCheck(1) antes del primer movimiento hacia el conveyor (versión integrada).
- Prueba en celda con la versión integrada: con \$IN[75] di075DropOffMachine1 OFF el robot espera antes de moverse hacia el conveyor (mensaje de espera); con \$IN[75] ON hace 5 ciclos dejando la pieza en AtDrop3 sin contacto ni marca. Anotar fecha y resultado.
- Respaldo completo con la versión integrada cargada: style1drop1opt1.src/.dat coinciden con el zip entregado (la auditoría lo compara).

Evidencia del programador. Resultado de la prueba del interlock y de los 5 ciclos de drop, con fecha.

Nota. Razón del re-enseñado de AtDrop3 (B 4.3° → 30.1°, C -0.3° → -34.8°): la pieza cae mejor en el conveyor. Validado por Edgar Montiel con el programador el 2026-10-05.

Historial de auditoría:

- 2026-10-05 — Alta (auditoría): Encontrado en el respaldo bmw_03_10_r1.
- 2026-10-05 — Corregido en oficina: Excluido de la integración por decisión de Edgar: el zip integrado regresa el interlock de drop-off antes del primer movimiento y AtDrop3 al punto original. Probar 5 ciclos de drop al conveyor.
- 2026-10-05 — Decisión: Edgar: se conserva el AtDrop3 del programador (la pieza cae mejor en el conveyor; validado con él). Solo el interlock de drop-off regresa a su lugar original.
- 2026-10-05 — Respaldo 14:46: El programador además quitó PTP P1: un LIN de 330 mm de P5 a AtDrop3 bajando 166 mm a unos 30°, con el permiso de drop-off en P5. En la versión integrada el robot espera el permiso en el punto de cámara P3 con la aplicación 1 ya liberada: mover AC_Application(1,TRUE) a después de salir de la cámara.
- 2026-10-05 — Integrado: P1 se conserva (preguntar al programador por qué lo quitó). La aplicación 1 ya no se libera en el punto de cámara: la libera un TRIGGER en el primer movimiento fuera de la cámara (P5 del drop, P23 del rechazo, P28 del red rabbit).

F49 — AutomationCore_BKG activado y editado sin decisión registrada

Severidad: Alto · **Tipo:** Decisión Gestamp + código · **Responsable:** Gestamp controles + Programador robot · **Estado:** Requiere decisión

Módulos: KRC/R1/System/sps.sub, KRC/R1/TP/AutomationCore/automationcoreroutines.src, KRC/R1/TP/AutomationCore/automationcoreroutines.dat

Qué está mal. sps.sub ahora llama AutomationCore_BKG (F12). Escribe cada ciclo los bits de estado al PLC (aire, agua, tolvas, nut ready/fault, batería, velocidad), auto-selecciona CELL en EXT, pone do004ProcessFault en cada reset de programa, activa el camino de cambio de electrodo (F43) y el Request to Enter (\$OUT[145] doCriticalWZ). La rutina del proveedor se editó: sin soldadores 2-3 y Request to Enter sin WAIT (antes congelaba el submit).

Qué hacer. Registrar la decisión de Gestamp de activar el background. Probar en celda cada salida nueva con el PLC (aire, agua, tolvas, nut ready/fault, request to enter) y el cambio de electrodo. Documentar las ediciones del proveedor (C28).

Criterio de cierre (se verifica en el respaldo):

- Decisión de Gestamp por escrito (activar AutomationCore_BKG; NutWeld_BKG sigue apagado o se decide aparte).
- Prueba en celda con el PLC: request to enter concedido solo sin gun en trabajo ni gripper/brake test/mastering en proceso; cada bit de estado leído en el PLC con su condición forzada; un cambio de electrodo completo (F43).
- Ediciones del proveedor listadas en la entrega (C28).

Evidencia del programador. Correo de Gestamp, tabla de señales probadas con fecha y resultado, prueba de cambio de electrodo.

Depende de / se cierra con: F12, F43, C28

Nota. La edición del Request to Enter es razonable: el WAIT original detenía todo el submit (posición del gun, preload, gripper) mientras la petición estaba activa.

Historial de auditoría:

- 2026-10-05 — Alta (auditoría): Encontrado en el respaldo bmw_03_10_r1.
- 2026-10-05 — Integrado - pendiente decisión: Activación y ediciones del proveedor integradas tal como estaban en el robot; comentarios ;CHECK: en sps.sub, cell.src y gunelectrodechange. Falta decisión de Gestamp y prueba con el PLC y del cambio de electrodo.
- 2026-10-05 — Revisión independiente - riesgos confirmados: (1) Request to Enter: con \$OUT[930] pegado (F35) el robot ya no se detiene al pedir entrar. (2) El background escribe \$OUT[481]/\$OUT[482]

(habilitaciones del Bosch) cada ciclo y pisa las de CENTERLINE_WELD; en dry cycle puede colgar WAIT FOR dipw1_Ready. (3) Cambio de electrodo activo sin probar (F43). (4) AC_MODE sin inicializar en el CWRITE del auto select. (5) do004ProcessFault en cada reset. (6) bACEntryGranted persistente. Corre así en el robot desde el 2026-10-05.

- 2026-10-05 — Integrado - background apagado: El background de AutomationCore se apaga otra vez hasta que Gestamp decida y se corrijan sus problemas; el submit pone \$OUT[145] y \$OUT[111] en OFF al arrancar. Las ediciones del proveedor quedan, inactivas (C28).

F50 — Permiso de aplicación 1 (nut check / cámara) revisado después de llegar a P22

Severidad: Alto · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/StyleApps/Options/style1app2opt1.src

Qué está mal. En el respaldo de las 14:46 el programador movió AC_ApplicationCheck(1) (espera \$IN[70]) a después de PTP P22. P22 es el punto de aproximación de la estación de inspección (unos 170 mm del punto de cámara P6 y 480 mm abajo de los puntos del nut check): el robot entra sin permiso del PLC mientras \$OUT[70]/\$OUT[24] siguen diciendo que está libre.

Qué hacer. Dejar AC_ApplicationCheck(1) antes del primer movimiento, como en la versión integrada. Si se movió por tiempo de ciclo, ver con el PLC si se puede pedir el permiso antes (al salir del pedestal).

Criterio de cierre (se verifica en el respaldo):

- AC_ApplicationCheck(1) antes de PTP P22 en el respaldo.
- Prueba en celda: con \$IN[70] OFF el robot espera antes de moverse a P22 (mensaje de espera); con \$IN[70] ON el ciclo sigue normal. Anotar fecha y resultado.

Evidencia del programador. Resultado de la prueba con fecha; razón del cambio si la hubo.

Historial de auditoría:

- 2026-10-05 — Alta (auditoría): Encontrado en el respaldo de las 14:46; confirmado por revisión independiente.
- 2026-10-05 — Integrado: No se integra: en la versión integrada AC_ApplicationCheck(1) queda antes de PTP P22.

F51 — Apps de soldadura duplicadas por estación de pick (A/B): cada cambio va en las dos

Severidad: Bajo · **Tipo:** Sin acción · **Responsable:** Programador robot · **Estado:** Sin acción - documentado

Módulos: KRC/R1/Program/StyleApps/Options/style1app1opt1A..3B, KRC/R1/Program/Styles/Options/style1opt1.src

Qué está mal. El programador partió las tres apps de soldadura por estación de pick: A (estación 1, posiciones sin cambio) y B (estación 2, posiciones de soldadura re-enseñadas 1-5 mm porque la pieza queda distinta en el gripper). El código de A y B es el mismo.

Qué hacer. Se integra tal cual. Cualquier cambio posterior a una app de soldadura se hace en A y en B.

Criterio de cierre (se verifica en el respaldo):

- Documentado; se revisa en cada auditoría que A y B sigan iguales en código.

Historial de auditoría:

- 2026-10-05 — Alta (auditoría): Integrado en la versión depurada.

C01 — AutomationCore_Bkg deshabilitado: el estatus al PLC no se actualiza

Severidad: - · **Tipo:** Decisión Gestamp · **Responsable:** Gestamp controles / Ethos (Edgar) · **Estado:** Se cierra con otro punto

Módulos: KRC/R1/System/sps.sub, KRC/R1/TP/AutomationCore/automationcoreroutines.src

Qué está mal. El estándar Gestamp pide que el background task de AutomationCore mantenga el estatus hacia el PLC. En sps.sub la llamada AutomationCore_BKG () estaba comentada (fold Automation Core PLC vacío), así que nunca se escriben, entre otros, \$OUT[21], \$OUT[22], \$OUT[6], \$OUT[7]

do007RobotAtPounce, \$OUT[111..113], los bits de bus OK, \$OUT[146] ni \$OUT[145] doCriticalWZ. Es el mismo problema que F12.

Qué hacer. Se atiende en F12.

Criterio de cierre (se verifica en el respaldo):

- Se cierra cuando se cierra F12.

Evidencia del programador. La evidencia se registra en F12.

Depende de / se cierra con: F12

Nota. Al cerrar F12 se actualiza también el ;NOTE: de sps.sub que cita (F12, C01).

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C02 — Designación BMW-03-10-R1 en los encabezados; confirmar formato con Gestamp

Severidad: - · **Tipo:** Información Gestamp · **Responsable:** Ethos (Edgar) / Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/cell.src, KRC/R1/System/sps.sub, KRC/R1/Program/Styles/style_1.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/Request_next_nut.src, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/HOME.src, KRC/R1/Program/masref_user.src, KRC/R1/Program/tm_useraction.src, KRC/R1/Program/Utilities/hometopounce.src, KRC/R1/Program/Utilities/pouncetohome.src, KRC/R1/Program/Utilities/hometorepair.src, KRC/R1/Program/Utilities/repairtohome.src, KRC/R1/Program/Utilities/gunelectrodechange.src, KRC/R1/Program/Utilities/closeandcheckallclamps.src, KRC/R1/Program/Utilities/openandcheckallclamps.src, KRC/R1/safetest.src

Qué está mal. Los 31 encabezados de los módulos del integrador dicen "; Line - BMW G65 (cell 03)" y "; Robot 03-10-R1 (BMW-03-10-R1[-PNW1|-MH1])". La revisión de Gestamp (2026-10-02) llama a esta estación BMW-03-10R1 (el encabezado de la hoja no se ve en la foto; el contenido coincide con este respaldo). El encabezado viejo de HOME decía "Line - Volvo".

Qué hacer. Confirmar con Gestamp el formato exacto (BMW-03-10R1 o BMW-03-10-R1), el nombre de línea y los códigos PNW1/MH1; si difiere, ajustar los encabezados.

Criterio de cierre (se verifica en el respaldo):

- Confirmación escrita de Gestamp del formato, nombre de línea y códigos de dispositivo.
- Los encabezados del respaldo usan ese formato; los módulos nuevos también.

Evidencia del programador. Correo o documento de Gestamp; si cambió, cuántos módulos se actualizaron.

Nota. Un nombre de módulo KRL no admite '-' (máximo 24 caracteres): la designación vive en el encabezado y en &COMMENT. El nombre del robot en el controlador es G01.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C03 — Encabezado estándar Gestamp en cada módulo del integrador

Severidad: - · **Tipo:** Sin acción · **Responsable:** Programador robot · **Estado:** Cerrado - depuración

Módulos: KRC/R1/cell.src, KRC/R1/System/sps.sub, KRC/R1/Program/Styles/style_1.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/Request_next_nut.src, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/HOME.src, KRC/R1/Program/masref_user.src, KRC/R1/Program/tm_useraction.src, KRC/R1/Program/Utilities/hometopounce.src, KRC/R1/Program/Utilities/pouncetohome.src, KRC/R1/Program/Utilities/hometorepair.src, KRC/R1/Program/Utilities/repairtohome.src, KRC/R1/Program/Utilities/gunelectrodechange.src, KRC/R1/Program/Utilities/closeandcheckallclamps.src, KRC/R1/Program/Utilities/openandcheckallclamps.src, KRC/R1/safetest.src

Qué está mal. Los módulos del integrador no traían el encabezado estándar Gestamp. La depuración lo agregó a los 31 módulos (después del fold INI, o tras el DEF si no hay INI); los .dat no llevan encabezado.

Qué hacer. Nada que corregir. Mantener el encabezado y ponerlo en todo módulo nuevo con el formato de docs/CONVENTIONS.md.

Criterio de cierre (se verifica en el respaldo):

- Ningún módulo que hoy tiene el bloque ';' Gestamp Standards' lo pierde en el respaldo.
- Todo .src nuevo del integrador trae el bloque completo (Gestamp Standards / Line / Robot 03-10-R1 (designación) / Application) y un &COMMENT con 03-10-R1.

Nota. Solo verificación de regresión en cada respaldo. El encabezado nunca va dentro de un fold de tech package ni de un inline form.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C04 — Movimientos de pedestal interpolados sobre el dado fijo (external TCP)

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Ethos (Edgar) / Programador robot · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src

Qué está mal. El estándar Gestamp (Pedestal Applications, Remote TCP) pide que los movimientos en el pedestal se interpolen alrededor del dado fijo (external TCP con la base 3 NUT WELDER). De los 22 movimientos en base 3, 11 van en external TCP y ninguna de las tres presentaciones de soldadura (P19 de la app 1, P16 de las apps 2 y 3) lo usa (F22).

Qué hacer. Decidir y dejar por escrito la convención de frames del pedestal (la del estándar o una excepción aprobada por Gestamp) y re-enseñar en T1 los movimientos de pedestal con esa convención.

Criterio de cierre (se verifica en el respaldo):

- Decisión escrita de la convención de frames del pedestal adjunta a la lista.
- Se cumplen los criterios de F22 y C05.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C04) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Decisión escrita; la evidencia de re-enseñanza se registra en F22.

Depende de / se cierra con: C05, F22

Nota. Cambiar solo la casilla extTCP del inline form sin re-enseñar el punto cambia la posición física. C05 y C04 van en el mismo respaldo.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C05 — TCP remoto del pedestal (BASE_DATA[3]) sin medir

Severidad: - · **Tipo:** Confirmar en sitio · **Responsable:** Puesta en marcha / Programador robot · **Estado:** Abierto

Módulos: KRC/R1/System/\$config.dat, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src

Qué está mal. El estándar pide el origen del TCP remoto a 1/4 de pulgada (6.35 mm) del electrodo, dado o pin estacionario en dirección axial, con +X y +Y a escuadra con el dado. BASE_DATA[3] NUT WELDER vale {X 21.84, Y 1866.09, Z 658.47, A 90, B 0, C 180}: no hay registro de que se haya medido con ese criterio.

Qué hacer. Verificar el mastering y medir in situ la herramienta fija del pedestal con el procedimiento KUKA de herramienta fija, con el origen y la orientación del estándar, y guardar el resultado en BASE_DATA[3]. En el mismo respaldo se re-enseñan (C04/F22) o verifican los 22 puntos en base 3.

Criterio de cierre (se verifica en el respaldo):

- Registro de medición adjunto: método KUKA de herramienta fija, referencia física, cómo se dio el desfase axial de 6.35 mm y la orientación X/Y, error mostrado, valores resultantes, fecha y verificación de mastering previa.
- BASE_DATA[3] del respaldo es el valor medido y BASE_NAME[3] sigue siendo NUT WELDER.
- Prueba en T1: con la pieza en una pose de soldadura, jog de orientación en external TCP: la pieza gira sobre el electrodo sin desplazarse; anotar resultado.
- En el mismo respaldo en que cambia BASE_DATA[3], cada punto en Base[3] (style1app1opt1..3, RejectGE4, style1app2opt2) está re-enseñado o la lista anota que se recorrió en T1 y quedó correcto.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C05) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Foto de la pantalla de resultado de la medición; verificación de mastering; resultado de la prueba de giro; lista de puntos en base 3 con lo hecho.

Nota. Cambiar BASE_DATA[3] mueve físicamente todos los puntos en base 3: no correr en automático hasta re-enseñarlos o verificarlos en T1.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C06 — Herramientas y bases con nombre: nombres de otro proyecto

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/System/\$config.dat, KRC/R1/TP/BrakeTest/braketeststart.src

Qué está mal. El estándar pide herramientas y bases con nombre y ninguna herramienta sin nombre en producción. Las herramientas se llaman "TOOL 1".."TOOL 3" (sin decir qué son) y las bases "10A", "20B" y "PED SEALER" vienen de otro proyecto. Los nombres viejos de herramienta en el texto de los folds ya los refrescó la depuración. El brake test usa la herramienta 3 y la base 1 con nombres de 03-30-R1 (F47).

Qué hacer. Dar nombre descriptivo a la herramienta 1 (EOAT 1) y a las bases usadas, y limpiar o renombrar las que no se usan; con F47 el brake test deja de usar tool 3 / base 1.

Criterio de cierre (se verifica en el respaldo):

- En \$config.dat TOOL_NAME y BASE_NAME de cada herramienta y base usada describen lo que son (p. ej. EOAT 1, NUT WELDER); las que no se usan quedan vacías o con su nombre real.
- Ningún movimiento de producción ni de servicio usa una herramienta o base sin nombre o con nombre de otro robot (F47 cerrado).
- Prueba: abrir un form de movimiento de cada módulo de producción y confirmar que muestra el nombre nuevo sin cambiar el punto (sin Touch Up). Anotar fecha.

- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C06) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Lista de nombres anterior y nuevo; fecha.

Depende de / se cierra con: F47

Nota. Renombrar una herramienta no mueve puntos; cambiar sus datos sí. No usar TOOL 2 para otra cosa sin revisar que ningún punto la use.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C07 — Un user frame por utillaje: picks, nut check, reject y drops en WORLD

Severidad: - · **Tipo:** Decisión Gestamp · **Responsable:** Gestamp controles / Ethos (Edgar) · **Estado:** Requiere decisión
Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/System/\$config.dat

Qué está mal. El estándar pide un user frame (base) por utillaje. Las estaciones de pick, el nut check, la cámara, el reject y los drops están enseñados casi todos en WORLD (la aproximación a la cámara P6 y la salida del reject P24 en Base[3]). BASE_DATA[1] "10A" y [2] "20B" tienen nombres de otro proyecto.

Qué hacer. Que Gestamp decida si exige bases por utillaje en esta estación o acepta WORLD por escrito. Si las exige: verificar el mastering, medir una base por utillaje, con nombre, y re-enseñar en T1 los puntos de cada utillaje en su base.

Criterio de cierre (se verifica en el respaldo):

- Decisión escrita de Gestamp adjunta: (a) acepta WORLD y el punto se cierra sin cambio de código; o (b) exige una base por utillaje.
- Si (b): cada utillaje tiene su BASE_DATA medida y su BASE_NAME; los movimientos de aproximación, trabajo y salida usan esa base; registro de medición y prueba en T1 y en automático con fecha.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C07) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Decisión de Gestamp; si se miden bases, registro y pruebas.

Nota. Cambiar la base de un punto sin re-enseñarlo cambia su posición física. Si se re-enseñan trayectorias, revisar después C08 y C09.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C08 — Datos de carga: LOAD_DATA[1] tecleado; sin carga con/sin pieza

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto
Módulos: KRC/R1/System/\$config.dat, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src

Qué está mal. El estándar pide un payload por escenario de carga, determinado (LoadDataDetermination): EOAT sin pieza y con pieza, salvo que difieran menos de 5 %. LOAD_DATA[1] = {M 210, CM X 270 Y 0 Z 240, J 105/105/105}, valores redondos tecleados (F39), y no hay datos de carga con pieza.

Qué hacer. Determinar con LoadDataDetermination la carga del EOAT 1 sin pieza y con la pieza GE4 (G03). Si difieren 5 % o más, dos juegos de datos y cada trayectoria usa el que corresponde.

Criterio de cierre (se verifica en el respaldo):

- LOAD_DATA[1] coincide con el resultado de LoadDataDetermination del EOAT 1 sin pieza.
- Si la carga con pieza difiere 5 % o más: otra herramienta con TOOL_DATA idéntico a TOOL_DATA[1], nombre y LOAD_DATA con pieza, usada por los movimientos con pieza (del cierre del gripper en el pick a su apertura en drop, drop red rabbit o reject); si difieren menos, la lista anota las dos masas.
- Ningún movimiento de producción usa una herramienta con LOAD_DATA por defecto o cero.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C08) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Resultados de LoadDataDetermination sin y con pieza, fecha y quién lo corrió.

Depende de / se cierra con: G03, F39

Nota. Si se edita el FDAT directo en el .dat, actualizar también el texto Tool[n]:<nombre> del fold. Los datos de carga correctos son requisito para C09.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C09 — Detección de colisión apagada en todo el programa; la reacción es un HALT

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto

Módulos: KRC/R1/System/\$config.dat, KRC/R1/TP/AutomationCore/automationglobalpos.dat, KRC/R1/Program/tm_useraction.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/HOME.src, KRC/R1/Program/Utilities/hometorepair.src, KRC/R1/Program/Utilities/repairtohome.src, KRC/R1/Program/masref_user.src

Qué está mal. El estándar pide la detección de colisión activa todo el tiempo; apagarla requiere aprobación escrita. \$TORQMON está en 0 (F39) y todos los movimientos de producción tienen TQ_STATE FALSE. tm_useraction, la reacción a una colisión, solo hace HALT (C21).

Qué hacer. Activar la detección y ajustarla en celda con el método del estándar, con los datos de carga ya correctos (C08). Si no se puede, conseguir la aprobación escrita de Gestamp para operar sin ella.

Criterio de cierre (se verifica en el respaldo):

- Opción A: detección activa (bTQM_KCPSTATUS = TRUE y TQ_STATE TRUE en los FDAT de los movimientos que corren en automático, incluidos FHOME y los de automationglobalpos.dat); excepciones anotadas con su razón; valores de sensibilidad anotados; ciclos en automático externo a 100 % sin disparo en falso, con número de ciclos y fecha.
- Opción B: aprobación escrita de Gestamp para operar sin detección de colisión, adjunta, y el ;NOTE: de tm_useraction la cita.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C09) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Opción A: foto de bTQM_ACTIVE en automático externo, tabla de valores y ciclos sin disparo en falso. Opción B: documento de Gestamp.

Depende de / se cierra con: C08, F39

Nota. TQ_STATE se puede cambiar en el FDAT del .dat sin reabrir el inline form. Los cambios en automationglobalpos.dat son ediciones de archivos del proveedor: listarlas en C28.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C10 — Falta el programa de mastering ZERO_G1

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/ZERO_G1.src (nuevo)

Qué está mal. El estándar pide un programa ZERO_G1: trayectoria segura de home a la posición de mastering, una pausa y regreso. En el respaldo no existe; solo está la prueba de mastering reference de KUKA (masref_main, masref_user).

Qué hacer. Crear ZERO_G1 con su encabezado: arranca en HOME, va por puntos libres de interferencia a la posición de mastering, se detiene para la revisión visual con mensaje y regresa a HOME.

Criterio de cierre (se verifica en el respaldo):

- Existe ZERO_G1.src (y su .dat) en KRC/R1/Program con el bloque ' Gestamp Standards' y &COMMENT, y está declarado en tools/code_changes.json (added_files).
- Arranca y termina en HOME; la pose de mastering es igual a \$MAMES[1..6] de \$machine.dat (A1 -20, A2 -120, A3 110, A4 0, A5 0, A6 0), con pausa explícita y mensaje que pide revisar las marcas; si no se alcanza en un paso, dos poses (A1-A3, luego A4-A6) con el visto bueno de Ingeniería Gestamp.
- Prueba en T1 con override reducido: HOME, pose de mastering y regreso sin contacto; foto de las marcas de cada eje; fecha.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C10) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Fotos de las marcas de mastering y fecha de la prueba.

Nota. La pausa de ZERO_G1 es una detención intencional de verificación, no manejo de falla (no cuenta contra C21).

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C11 — Zonas sin documentar: ninguna zona pedida, falta la forma NA-FM-71-138.1

Severidad: - · **Tipo:** Información Gestamp · **Responsable:** Ethos (Edgar) / Gestamp controles · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Utilities/hometorepair.src, KRC/R1/Program/Utilities/repairtohome.src, KRC/R1/Program/Utilities/hometopounce.src, KRC/R1/Program/Styles/Options/style1opt1.src

Qué está mal. El estándar pide que cada entrada y salida de zona lleve un comentario con este robot, la zona y el robot socio, y la forma NA-FM-71-138.1 (Robot Zone Assignment). En R10 no se pide ninguna zona en ninguna parte (producción, repair, pounce).

Qué hacer. Conseguir de Gestamp los robots de la celda 03 y las trayectorias que interfieren con R10, llenar NA-FM-71-138.1 y que Gestamp la apruebe. Después pedir y liberar en el programa cada zona de la forma, con el comentario completo.

Criterio de cierre (se verifica en el respaldo):

- NA-FM-71-138.1 llena y aprobada por Gestamp, adjunta (o constancia escrita de que R10 no comparte zonas).
- Toda zona de la forma tiene su petición y su liberación en el módulo correspondiente, con el comentario 'Zn ENTER/EXIT 03-10-R1 ... WITH <robot socio>' y el AC_CmdParam del form igual al número de zona.
- Prueba en celda por zona: R10 espera en la petición mientras la zona está ocupada y su salida de zona libre está OFF dentro y ON al salir; anotar fecha.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C11) se quita o se actualiza; ningún encabezado de inline form ni línea dentro

de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Forma aprobada; fecha y resultado por zona.

Nota. HOME debe quedar libre de toda interferencia (estándar). La coherencia pedir/liberar es C13.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C12 — Pounce común: el ciclo no pasa por pounce y do007RobotAtPounce no se calcula

Severidad: - · **Tipo:** Decisión Gestamp · **Responsable:** Gestamp controles / Ethos (Edgar) · **Estado:** Requiere decisión

Módulos: KRC/R1/Program/Utilities/hometopounce.src, KRC/R1/Program/Utilities/pouncetohome.src, KRC/R1/TP/AutomationCore/automationglobalpos.dat, KRC/R1/TP/AutomationCore/automationcoreroutines.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/cell.src

Qué está mal. El estándar pide un pounce común por el que pasan todos los programas de estilo. El ciclo de R10 no pasa por ningún pounce y \$OUT[7] do007RobotAtPounce solo lo escribe AutomationCore_Bkg, deshabilitado (F12), así que HomeCheck nunca ejecuta PounceToHome.

Qué hacer. Que Gestamp decida si exige el pounce común. Si sí: enseñar o verificar XPOUNCE, hacer que el ciclo pase por él y que do007 se calcule (vía F12). Si no: aprobación escrita y comentar que HomeToPounce y PounceToHome no se usan.

Criterio de cierre (se verifica en el respaldo):

- Decisión escrita de Gestamp adjunta.
- Si se exige: Style1Opt1 y Style1Opt10AutoRR pasan por el mismo pounce; XPOUNCE verificado en T1 con la herramienta 1; \$OUT[7] se escribe cíclicamente y HomeCheck ejecuta PounceToHome con el robot en el pounce; prueba con fecha.
- Si no se exige: el comentario de HomeToPounce y PounceToHome dice que no se usan por decisión de Gestamp.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C12) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Decisión de Gestamp; si aplica, fecha y resultado de la prueba de do007.

Depende de / se cierra con: F12

Nota. Re-enseñar XPOUNCE edita automationglobalpos.dat, archivo del proveedor: listarlo en C28.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C13 — Manejo de zonas coherente

Severidad: - · **Tipo:** Sin acción · **Responsable:** Programador robot · **Estado:** No aplica

Módulos: KRC/R1/Program/Utilities/hometorepair.src, KRC/R1/Program/Utilities/repairtohome.src, KRC/R1/Program/Utilities/hometopounce.src, KRC/R1/Program/Utilities/pouncetohome.src

Qué está mal. R10 no usa zonas: no hay nada que pedir y liberar de forma incoherente.

Qué hacer. Nada hoy. Si C11 o C12 agregan zonas, cada trayectoria pide antes de entrar y libera al salir exactamente las zonas que cruza.

Criterio de cierre (se verifica en el respaldo):

- Se reabre si C11 o C12 agregan zonas: entonces cada zona pedida se libera al salir de ella (no solo por MaintainSystem), con prueba en T1 de las salidas de zona libre.

Depende de / se cierra con: C11

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C14 — Brake test: el 'due' no se reporta y el camino usa tool/base de otro robot

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/TP/BrakeTest/braketeststart.src, KRC/R1/TP/BrakeTest/braketestback.src, KRC/R1/cell.src, KRC/R1/System/sps.sub, KRC/R1/TP/AutomationCore/automationcoreroutines.src

Qué está mal. El estándar pide reportar al PLC cuando el brake test está pendiente y ejecutarlo en home. El brake test corre a petición del PLC (\$IN[21] di021BrakeTestRequest) después de HomeCheck, pero \$OUT[21] do021BrakeTestReqd solo lo escribe AutomationCore_Bkg, deshabilitado (F12): el PLC nunca sabe que está pendiente. El camino de inicio está enseñado con tool 3 / base 1 de otro robot (F47), y los archivos del proveedor traen la edición de do931BrakeTestInProcess, que además cae en un bit del adapter compartido (F18).

Qué hacer. Calcular do021BrakeTestReqd cíclicamente (\$BRAKETEST_MONTIME, \$BRAKETEST_REQ_EX, \$BRAKETEST_REQ_INT) por F12 o en sps.sub; mantener la señal de brake test en proceso hasta que el robot regresó; acordar con Gestamp el bit (F18); re-enseñar el camino (F47).

Criterio de cierre (se verifica en el respaldo):

- \$OUT[21] do021BrakeTestReqd se escribe en cada ciclo del submit con la condición de brake test pendiente.
- La señal de brake test en proceso se apaga después del último movimiento de regreso, con el robot en HOME, y su bit está acordado con Gestamp (F18).
- Prueba en celda: con el brake test pendiente el PLC ve do021 ON; al pedir di021 el robot sale de HOME, hace el test y regresa a HOME; do021 baja al terminar. Anotar fecha.
- La edición en archivos del proveedor queda listada en C28.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C14) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Fecha y resultado de la prueba.

Depende de / se cierra con: F12, F18, F47

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C15 — Dry cycle seleccionable y sin colgar el robot

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Se cierra con otro punto

Módulos: KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/System/sps.sub

Qué está mal. El estándar pide dry cycle seleccionable desde el PLC sin que el robot se cuelgue. Hoy se cuelga en la primera weld app (F04), y como \$IN[4] se lee en cada paso, cambiarlo a medio ciclo puede soltar como buena una pieza sin soldar ni inspeccionar (F05).

Qué hacer. Se atiende en F04 y F05.

Criterio de cierre (se verifica en el respaldo):

- Se cierra cuando se cierran F04 y F05.

Evidencia del programador. La evidencia se registra en F04 y F05.

Depende de / se cierra con: F04, F05

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C16 — Pieza presente / pieza en gripper

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Se cierra con otro punto

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src

Qué está mal. Los picks esperan gripper vacío antes del primer comando de gripper (G14) y pieza presente después de cerrar; drops y reject esperan gripper vacío después de abrir. Pero \$OUT[18]do018Part1InGripper está latcheado por programa, no sigue a los sensores (Gestamp punto 9), y los drops y el reject no confirman la pieza antes de abrir (F40).

Qué hacer. Se atiende en G09 y F40.

Criterio de cierre (se verifica en el respaldo):

- Se cierra cuando se cierran G09 y F40.
- Se reabre si un cambio quita o salta alguna espera de pieza fuera de dry cycle.

Evidencia del programador. La evidencia se registra en G09 y F40.

Depende de / se cierra con: G09, F40

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C17 — Verificación de tuercas y red rabbit

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Se cierra con otro punto

Módulos: KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/System/\$config.dat

Qué está mal. El estándar pide verificar las tuercas y comprobar la detección con red rabbit. Las options 2, 7 y 8 no evalúan el sensor (F02); PARTPRESENT1..3 y bscrapGE4 no se resetean al inicio del ciclo (F03); el ciclo red rabbit no puede dar resultado (F19).

Qué hacer. Se atiende en F02, F03 y F19.

Criterio de cierre (se verifica en el respaldo):

- Se cierra cuando se cierran F02, F03 y F19.

Evidencia del programador. La evidencia se registra en F02, F03 y F19.

Depende de / se cierra con: F02, F03, F19

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C18 — Handshakes AutomationCore de pick, application y drop

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Se cierra con otro punto

Módulos: KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/TP/AutomationCore/automationcoreroutines.src

Qué está mal. Los handshakes de pick, application y drop están pareados, salvo: RejectGE4 abre el gripper sin AC_DropOffCheck y un rechazo desde el nut check no libera Application 1 (F20); los parámetros ocultos de varios forms no coinciden con la llamada (F21); y EndOfCycle no manda Work Complete (F46).

Qué hacer. Se atiende en F20, F21 y F46.

Criterio de cierre (se verifica en el respaldo):

- Se cierra cuando se cierran F20, F21 y F46.

Evidencia del programador. La evidencia se registra en F20, F21 y F46.

Depende de / se cierra con: F20, F21, F46

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C19 — Request to enter: doCriticalWZ nunca se escribe; el PLC ve entrada permitida

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Seguridad · **Estado:** Requiere decisión
Módulos: KRC/R1/System/sps.sub, KRC/R1/TP/AutomationCore/automationcoreroutines.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/centerline_weld.src

Qué está mal. El estándar pide respetar la petición de entrada del PLC. El robot se detiene en el siguiente punto cuando sube \$IN[6] di006RequestToEnter (TRIGGER AC_PointArrival en bas.src), pero \$OUT[145] doCriticalWZ solo lo escribe AutomationCore_Bkg, deshabilitado (F12): queda FALSE y el PLC ve entrada permitida mientras PRELOAD o CENTERLINE_WELD siguen accionando el pedestal (F11). Si se reactiva el background, \$OUT[930] en ON (F35) bloquearía toda entrada.

Qué hacer. Que doCriticalWZ se escriba cíclicamente y diga entrada no permitida mientras haya un proceso que no se detiene en un punto (preload, soldadura, brake test, mastering reference). Cómo depende de F12; qué pasa con el pedestal al abrir la puerta depende de la revisión de seguridad de F11.

Criterio de cierre (se verifica en el respaldo):

- \$OUT[145] doCriticalWZ se escribe en cada ciclo del submit.
- Su condición incluye PRELOAD con un paso activo y CENTERLINE_WELD en ejecución (o el estado real del pedestal), además de do930/do931/do932 ya corregidos (F35, F18).
- Prueba en celda: sin proceso de pedestal, \$IN[6] detiene el robot en el siguiente punto con doCriticalWZ OFF; con un preload y con una soldadura en curso, doCriticalWZ queda ON mientras dura el proceso. Anotar fecha.
- Resultado de la revisión de seguridad de F11 adjunto.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C19) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Fecha y resultado de las pruebas; documento de la revisión de seguridad.

Depende de / se cierra con: F12, F11, F35

Nota. AC_Request_to_enter contiene un WAIT FOR dentro del submit: evaluarlo antes de reactivar AutomationCore_Bkg. Cambiar automationcoreroutines.src es edición de archivo del proveedor: C28.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C20 — Conteo de soldaduras al final del ciclo: #CHECK apagado y WeldNoRequired distinto de 3

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto
Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/TP/AutomationCore/automationcoreroutines.src, KRC/R1/TP/AutomationCore/automationcoredata.dat

Qué está mal. El estándar pide verificar al final del ciclo que se hicieron todas las soldaduras. En EndOfCycle (automationcoreroutines.src, editado en este robot, F46) AC_Process_Count(#CHECK) está comentado. WeldNoRequired en automationcoredata.dat no vale 3 para las options de producción ([1,1]=1, [1,2]=2, [1,7]=6, [1,8]=4). AC_Process_Count(#Count1up) solo lo llama NutWeld_Weld del proveedor, que este programa no usa: CENTERLINE_WELD no cuenta.

Qué hacer. Contar una soldadura por cada soldadura buena en CENTERLINE_WELD (con el resultado de F08), poner WeldNoRequired = 3 en las options de producción y activar la verificación al final del ciclo, con \$OUT[47] do047QualityFault si no coincide.

Criterio de cierre (se verifica en el respaldo):

- CENTERLINE_WELD llama AC_Process_Count(#Count1up) una vez por tuerca y solo con soldadura buena (F08).
- WeldNoRequired[1,1], [1,2], [1,7] y [1,8] = 3 (más las options que confirme F02) y la del red rabbit = 0.
- AC_Process_Count(#CHECK) se ejecuta al final de cada ciclo de producción y el conteo arranca en 0 en cada ciclo, también después de un ciclo cancelado.
- Prueba en celda: ciclo normal con do047 OFF; ciclo con una soldadura faltante (método seguro anotado) con do047 ON; ciclo cancelado seguido de uno completo con do047 OFF; resultado en dry cycle definido con F05. Anotar fecha.
- Si se modifica EndOfCycle, la edición queda listada en C28 (y es coherente con lo que decida F46).
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C20) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Fecha y resultado de las pruebas con el estado de do047QualityFault.

Depende de / se cierra con: F46, F08, F02, F05

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C21 — Fallas por mensajes AutomationCore: ningún HALT sin mensaje ni WAIT FOR FALSE

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot · **Estado:** Abierto

Módulos: KRC/R1/Program/Styles/style_1.src, KRC/R1/cell.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/tm_useraction.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src

Qué está mal. El estándar pide que toda falla salga por mensajes AutomationCore con falla al PLC. Hoy: Style_1 contesta una option desconocida con WAIT FOR FALSE y hay ciclos vacíos sin falla (F06); CENTERLINE_WELD maneja el timeout de gun cerrado con HALT sin mensaje (F07); unos 35 WAIT FOR no tienen timeout ni mensaje (F15); y tm_useraction, la reacción a colisión, solo hace HALT.

Qué hacer. Cerrar F06, F07 y F15, y en tm_useraction dar mensaje y \$OUT[4] do004ProcessFault antes de detener.

Criterio de cierre (se verifica en el respaldo):

- F06, F07 y F15 cerrados.
- tm_useraction pone \$OUT[4] do004ProcessFault ON y muestra un mensaje de celda que nombra la colisión antes de detener el programa.
- En KRC/R1/Program, cell.src y sps.sub no queda WAIT FOR FALSE, y cada HALT que quede va precedido de mensaje y \$OUT[4] ON (salvo la pausa de ZERO_G1, C10).
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C21) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. Si la detección de colisión está activa (C09): fecha y resultado de la prueba de reacción.

Depende de / se cierra con: F06, F07, F15

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C22 — Grippers por inline forms GripperTech

Severidad: - · **Tipo:** Código + prueba en celda · **Responsable:** Programador robot / Puesta en marcha · **Estado:** Abierto
Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/TP/GripperSpotTech/grp_data.dat

Qué está mal. Cumple en parte: picks, drop al conveyor y RejectGE4 usan forms GripperTech. Los parámetros ocultos de esos forms eran de otro gripper (corregido en oficina, F31) y el drop red rabbit escribe el gripper 1 a mano y suelta con el gripper 3 (F19, F31). GripperConfig.xml está desincronizado (F33).

Qué hacer. Se atiende en F31, F33 y F19.

Criterio de cierre (se verifica en el respaldo):

- Se cumplen F31, F33 y F19: en los módulos listados toda acción de gripper es un form GripperTech del gripper 1 con parámetros iguales a la llamada.
- General: comentarios nuevos o cambiados según docs/CONVENTIONS.md (inglés, señal con dirección); el comentario que cite (C22) se quita o se actualiza; ningún encabezado de inline form ni línea dentro de un fold de form cambia sin estar declarado (F32, F21, F31).

Evidencia del programador. La evidencia se registra en F31, F33 y F19.

Depende de / se cierra con: F31, F33, F19

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C23 — Nombres AutomationCore para la E/S, sin direcciones literales

Severidad: - · **Tipo:** Código · **Responsable:** Programador robot · **Estado:** Se cierra con otro punto
Módulos: KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/System/sps.sub, KRC/R1/System/\$config.dat, KRC/R1/TP/AutomationCore/automationcoreroutines.dat

Qué está mal. El estándar pide nombres de señal estilo AutomationCore y no direcciones literales. Hecho en oficina (G12) fuera de los inline forms de KUKA; quedan los nombres traslapados de pistola 2/3 y los Spare sobrantes (F17), el mapa del faro (F29) y el nombre único de \$IN[227] (F36, hecho en oficina).

Qué hacer. Se atiende en F17, F29 y F36.

Criterio de cierre (se verifica en el respaldo):

- Se cierra cuando se cierran F17, F29 y F36.

Evidencia del programador. La evidencia se registra en F17, F29 y F36.

Depende de / se cierra con: F17, F29, F36

Nota. Los inline forms estándar de KUKA (WAIT FOR, OUT) generan \$IN[n]/\$OUT[n] con el nombre en el form: no se reescriben a mano.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C24 — Los comentarios nombran la señal con su dirección

Severidad: - · **Tipo:** Sin acción · **Responsable:** Programador robot · **Estado:** Cerrado - depuración

Módulos: KRC/R1/cell.src, KRC/R1/System/sps.sub, KRC/R1/Program/Styles/style_1.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/Request_next_nut.src, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/HOME.src, KRC/R1/Program/masref_user.src, KRC/R1/Program/tm_useraction.src, KRC/R1/Program/Utilities/hometopounce.src, KRC/R1/Program/Utilities/pouncetohome.src, KRC/R1/Program/Utilities/hometorepair.src, KRC/R1/Program/Utilities/repairtohome.src, KRC/R1/Program/Utilities/gunelectrodechange.src, KRC/R1/Program/Utilities/closeandcheckallclamps.src, KRC/R1/Program/Utilities/openandcheckallclamps.src, KRC/R1/safetest.src

Qué está mal. Los comentarios no nombraban la señal. La depuración comentó cada acción de E/S y cada espera con dirección y nombre (docs/IO_MAP.md), en inglés, y tools/check_cleanup.py verificó que cada nombre esté declarado en esa dirección.

Qué hacer. Nada. Mantener el formato en todo código nuevo o cambiado.

Criterio de cierre (se verifica en el respaldo):

- Toda acción de E/S o espera nueva o cambiada del respaldo tiene en las líneas de arriba un comentario con dirección y nombre declarado.
- Ningún comentario nuevo o cambiado escribe '\$IN/\$OUT[n] nombre' con un nombre no declarado en ese índice.

Nota. Solo verificación de regresión en cada respaldo.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C25 — Inline forms consistentes con el código

Severidad: - · **Tipo:** Código · **Responsable:** Programador robot · **Estado:** Se cierra con otro punto

Módulos: KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src

Qué está mal. El estándar pide que los inline forms coincidan con el código que generan. La depuración corrigió donde era seguro (form OUT 106 de RejectGE4, F20) y dejó ;WARNING: en el resto: parámetros ocultos AutomationCore (F21), parámetros ocultos GripperTech (F31, corregidos en oficina en picks y drop al conveyor) y líneas escritas a mano dentro de folds (F32, corregido en oficina).

Qué hacer. Se atiende en F21, F31 y F32.

Criterio de cierre (se verifica en el respaldo):

- Se cierra cuando se cierran F21, F31 y F32.

Evidencia del programador. La evidencia se registra en F21, F31 y F32.

Depende de / se cierra con: F21, F31, F32

Nota. El form OUT 106 de RejectGE4 ya corregido no debe volver a 5:TRUE.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C26 — Solo inglés en comentarios, títulos de fold y &COMMENT

Severidad: - · **Tipo:** Sin acción · **Responsable:** Programador robot · **Estado:** Cerrado - depuración

Módulos: KRC/R1/cell.src, KRC/R1/System/sps.sub, KRC/R1/Program/Styles/style_1.src, KRC/R1/Program/Styles/Options/style1opt1.src, KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1.src, KRC/R1/Program/StylePicks/Options/style1pick1opt2.src, KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src, KRC/R1/Program/StyleApps/Options/style1app1opt1.src, KRC/R1/Program/StyleApps/Options/style1app1opt2.src, KRC/R1/Program/StyleApps/Options/style1app1opt3.src, KRC/R1/Program/StyleApps/Options/style1app2opt1.src, KRC/R1/Program/StyleApps/Options/style1app2opt2.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src, KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src, KRC/R1/Program/Utilities/RejectGE4.src, KRC/R1/Program/Centerline/PRELOAD.src, KRC/R1/Program/Centerline/Request_next_nut.src, KRC/R1/Program/Centerline/centerline_weld.src, KRC/R1/Program/Centerline/CENTERLINE_HOME.src, KRC/R1/Program/HOME.src, KRC/R1/Program/masref_user.src, KRC/R1/Program/tm_useraction.src, KRC/R1/Program/Utilities/hometopounce.src, KRC/R1/Program/Utilities/pouncetohome.src, KRC/R1/Program/Utilities/hometorepair.src, KRC/R1/Program/Utilities/repairtohome.src, KRC/R1/Program/Utilities/gunelectrodechange.src, KRC/R1/Program/Utilities/closeandcheckallclamps.src, KRC/R1/Program/Utilities/openandcheckallclamps.src, KRC/R1/safetest.src

Qué está mal. Había comentarios fuera de inglés. La depuración dejó comentarios, títulos de fold y &COMMENT en inglés; los identificadores mal escritos se renombraron en oficina (G15).

Qué hacer. Nada. Todo comentario, título de fold, &COMMENT y mensaje nuevo va en inglés.

Criterio de cierre (se verifica en el respaldo):

- No aparece texto en español en comentarios, títulos de fold o &COMMENT nuevos o cambiados del respaldo.
- Si se renombra un identificador del integrador, se renombra en la declaración y en todas sus referencias en el mismo respaldo, y los módulos compilan.

Nota. Los renombres de oficina (NUT_START, CameraJudgment, PartRejected, di227NutPresent1, carpeta Styles/Options) se prueban en la celda con G15.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C27 — Proceso controlado por robot no compartido; regla de dos procesos

Severidad: - · **Tipo:** Sin acción · **Responsable:** Ethos (Edgar) · **Estado:** No aplica

Qué está mal. Cumple: R10 hace manejo de material (no cuenta como proceso) y un solo proceso, la soldadura de tuercas en el pedestal CenterLine, que controla solo este robot.

Qué hacer. Nada.

Criterio de cierre (se verifica en el respaldo):

- Se reabre solo si R10 pasa de dos procesos o si otro robot empieza a usar el pedestal CenterLine.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).

C28 — Ediciones del integrador en archivos del proveedor: listarlas en la entrega

Severidad: - · **Tipo:** Procedimiento · **Responsable:** Ethos (Edgar) / Programador robot · **Estado:** Abierto

Módulos: KRC/R1/System/bas.src, KRC/R1/TP/AutomationCore/automationcoreroutines.src, KRC/R1/TP/AutomationCore/automationcoreroutines.dat, KRC/R1/TP/GripperSpotTech/Grp_Func.src, KRC/R1/TP/GripperSpotTech/grp_user.src, KRC/R1/TP/BrakeTest/braketeststart.src, KRC/R1/TP/BrakeTest/braketestback.src, KRC/R1/TP/GLUETECH, KRC/R1/Program/MoveToPurge.src

Qué está mal. El estándar pide paquetes del proveedor sin modificar. Hay ediciones del integrador en bas.src (TRIGGER AC_PointArrival en cada juego de parámetros PTP/LIN), automationcoreroutines.src (EndOfCycle sin Work Complete, F46; agua y faro; speed not 100; Stop_at_End_of_Cycle en MaintainSystem; request to enter con do930..932), Grp_Func.src (do121GripperFault), grp_user.src (do932GripperInProgress) y BrakeTest (do931, camino con tool 3 / base 1, F47). GlueTech está instalado sin uso (G05). Una actualización de paquete las borraría sin aviso.

Qué hacer. Mantener una lista de entrega con cada edición en archivos del proveedor (archivo, rutina o fold, qué cambia y por qué), incluidas las de oficina y las que agreguen otros puntos, y entregarla con el respaldo final.

Criterio de cierre (se verifica en el respaldo):

- Documento de entrega adjunto con las ediciones de la sección 'Integrator edits found inside vendor files' de docs/FINDINGS.md más las nuevas, cada una con archivo, rutina o fold, cambio y razón.
- En cada respaldo, todo cambio en archivos KUKA de sistema o de tech package (KRC/R1/TP/**, KRC/R1/System/ salvo \$config.dat y sps.sub) o en machine data, respecto al último estado auditado, aparece en esa lista con fecha; un cambio sin listar reabre el punto.
- La versión final refleja lo decidido en F12, F46 y lo cambiado por C09, C12, C14, C19, C20, C22, F17, F36 y F47.

Evidencia del programador. Documento de entrega con fecha de la última revisión.

Depende de / se cierra con: F12, F46, C09, C12, C14, C19, C20

Nota. No es requisito deshacer las ediciones existentes, sino que queden listadas. Ediciones de oficina en archivos del proveedor: automationcoreroutines.dat (nombres do498/499/501..505, di466/467/470, di227NutPresent1, CameraJudgment, PartRejected). Agregarlas a la lista de entrega.

Historial de auditoría:

- 2026-10-04 — Alta (Ethos): Revisión Ethos del respaldo 658424 (v431_03_10_r1.zip del 2026-10-02).
- 2026-10-05 — Auditoría - ediciones nuevas: Ediciones nuevas en archivos del proveedor: automationcoreroutines.src (AutomationCore_Bkg, AC_Request_to_enter), automationcoreroutines.dat (bACEntryGranted), nutweldroutines.dat (nMaxGunNr=1, dipw1_AirOK en \$IN[3132]). Listarlas en la entrega.
- 2026-10-05 — Integrado - ediciones listadas: Ediciones del proveedor integradas y declaradas en tools/code_changes.json: automationcoreroutines.src/.dat, nutweldroutines.dat.

Anexo A. Comentarios CHECK y WARNING en el programa

Cada línea que pide confirmar algo en celda (CHECK) o advierte de una trampa (WARNING), con archivo y línea en el estado actual. El número entre paréntesis es el punto abierto.

Archivo:línea	Tipo	Comentario
Centerline/CENTERLINE_HOME.src:18	CHECK	\$OUT[473] dopw1_Nut1Intensify_Home IS SWITCHED OFF HERE BUT IS ON AT REST IN CENTERLINE_WELD (POLARITY TO CONFIRM); THE BOSCH HANDSHAKE (\$OUT[481..483]) AND THE WELD PROGRAM NUMBER ARE NOT RESET HERE (F34)
Centerline/CENTERLINE_HOME.src:23	WARNING	WHILE A PRELOAD IS ACTIVE (NUTCYCLE_ACTIVE), PRELOAD() IN THE SUBMIT INTERPRETER REWRITES THE SAME OUTPUTS EVERY CYCLE - THIS PROGRAM DOES NOT CHECK IT (F11)
Centerline/GUN_OPEN_CHECK.src:22	CHECK	CLAMP OPEN \$IN[466] AND QFP RETURNED \$IN[482] ARE NOT CHECKED BEFORE ENTERING (F01)
Centerline/PRELOAD.src:18	WARNING	DRIVES THE PEDESTAL VALVES ON ITS OWN, WITH NO ROBOT, MODE OR ENABLING-SWITCH INTERLOCK, AND SHARES THEM WITH CENTERLINE_WELD AND CENTERLINE_HOME WITHOUT OWNERSHIP (F11)
Centerline/PRELOAD.src:145	CHECK	OLD COMMENT SAID LOWER PIN RETREATED; THE CODE ADVANCES THE PIN (F27)
Centerline/PRELOAD.src:179	CHECK	OLD COMMENT SAID LOWER PIN RETREATED; THE CODE ADVANCES THE PIN (F27)
Centerline/centerline_weld.src:16	WARNING	PEDESTAL ENTRY IS ONLY PARTLY CONFIRMED. THE CALLER CHECKS GUN OPEN (GUN_OPEN_CHECK) AND MONITORS IT WITH INTERRUPTS 20-22; CLAMP OPEN AND QFP RETURNED ARE CHECKED BELOW ONLY AFTER THE ROBOT IS ALREADY AT THE WELD POSITION (F01)
Centerline/centerline_weld.src:42	CHECK	THIS TABLE IS NOT BYTE-SWAPPED. KRC_IO.xml SENDS THE WORD TO THE REGULATOR BYTE-SWAPPED, AS ON R20 (THERE 0.30 MPa IS WRITTEN 21765). THIS PROGRAM WRITES CL_GUN_PRESS_BASE = 405 AND CL_GUN_PRESS_WELD = 1820 (NOT SWAPPED) AND CL_GUN_PRESS_REST = 39425 AT THE END (SWAPPED 0.09 MPa) - CONFIRM WHICH IS RIGHT ON THE ITV (F44)
Centerline/centerline_weld.src:62	CHECK	OLD COMMENT SAID UPPER PIN EXTEND; THE CODE RETRACTS THE UPPER PIN HERE (F27)
Centerline/centerline_weld.src:99	CHECK	ON AND OFF BACK TO BACK IS A ZERO-LENGTH FEED PULSE; THE FEEDER MAY NEVER SEE IT (F25)
Centerline/centerline_weld.src:187	WARNING	HALT WITH NO MESSAGE IS STILL THE ONLY FAULT HANDLING - A DIALOG THAT NAMES THE FAULT (NO NUT / DOUBLE NUT / GUN NOT CLOSED) WITH RETRY AND ABORT IS STILL TO DO (F07)
Centerline/centerline_weld.src:208	CHECK	OLD COMMENT SAID SCHEDULE 7; THE CODE SELECTS PROGRAM 2 - CONFIRM ON THE BOSCH TIMER (F27)
Centerline/centerline_weld.src:278	CHECK	OLD COMMENT SAID LOWER PIN RETURN; THE CODE ADVANCES THE PIN HERE (F27)
Centerline/centerline_weld.src:294	CHECK	CL_GUN_PRESS_REST (39425) BELOW IS THE R20 BYTE-SWAPPED 0.09 MPa, NOT THE CL_GUN_PRESS_BASE USED ABOVE; NOT SWAPPED IT IS OUT OF THE ITV RANGE (0..4095). THE GUN RESTS AT THIS VALUE UNTIL THE NEXT WELD (F44)
Centerline/gun_open_check.dat:10	WARNING	INTERRUPTS 20/21 IN style1app1opt1A..3B REPEAT THIS RANGE AS LITERALS - CHANGE BOTH TOGETHER
Centerline/gun_open_check.dat:17	CHECK	DISABLED UNTIL TESTED: WITH THE AIR CLOSED \$IN[3132] MUST GO OFF, THEN SET TRUE (F01)
Centerline/gun_open_check.dat:21	CHECK	NORMAL FLOW 23-26 L/MIN WAS MEASURED ON 03-20-R1; CONFIRM THE MINIMUM ON THIS ROBOT (F09)
StyleApps/Options/style1app1opt1A.src:30	WARNING	BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) - NOT CLAMP OPEN OR QFP RETURNED (F01)
StyleApps/Options/style1app1opt1A.src:53	WARNING	THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN GUN_OPEN_CHECK.dat (F01)

Archivo:línea	Tipo	Comentario
StyleApps/Options/style1app1opt1B.src: 30	WARNING	BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) - NOT CLAMP OPEN OR QFP RETURNED (F01)
StyleApps/Options/style1app1opt1B.src: 53	WARNING	THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN GUN_OPEN_CHECK.dat (F01)
StyleApps/Options/style1app1opt2A.src: 30	WARNING	BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) - NOT CLAMP OPEN OR QFP RETURNED (F01)
StyleApps/Options/style1app1opt2A.src: 52	WARNING	THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN GUN_OPEN_CHECK.dat (F01)
StyleApps/Options/style1app1opt2B.src: 30	WARNING	BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) - NOT CLAMP OPEN OR QFP RETURNED (F01)
StyleApps/Options/style1app1opt2B.src: 52	WARNING	THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN GUN_OPEN_CHECK.dat (F01)
StyleApps/Options/style1app1opt3A.src: 30	WARNING	BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) - NOT CLAMP OPEN OR QFP RETURNED (F01)
StyleApps/Options/style1app1opt3A.src: 46	WARNING	THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN GUN_OPEN_CHECK.dat (F01)
StyleApps/Options/style1app1opt3B.src: 30	WARNING	BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) - NOT CLAMP OPEN OR QFP RETURNED (F01)
StyleApps/Options/style1app1opt3B.src: 46	WARNING	THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN GUN_OPEN_CHECK.dat (F01)
StyleApps/Options/style1app2opt1.src:1 34	CHECK	THE CAMERA CHECK ALSO SETS \$OUT[141]/\$OUT[142]: BOTH CAN BE ON TOGETHER, AND THE RED RABBIT DROP CLEARS THEM BEFORE THE END OF THE CYCLE (F19)
StyleDrops/Options/style1drop1opt1.src: 30	WARNING	HIDDEN AC_CmdParam=2 - CONFIRMING THIS FORM REGENERATES AC_DropOffCheck(2) (F21)
StyleDrops/Options/style1drop1opt1.src: 108	WARNING	HIDDEN AC_CmdParam=2 - CONFIRMING THIS FORM REGENERATES AC_DropOff(2,TRUE) (F21)
StyleDrops/Options/style1drop1opt2AutoRR.src:26	WARNING	THE RELEASE COMMAND USES GRIPPER 3. grp_data.dat MAPS GRIPPER 3 TO THE SAME I/O AS GRIPPER 1 (\$OUT[249]/[250], \$IN[257]/[258]) AND GripperTech DOES NOT CHECK THE ACTIVE FLAG, SO GRIPPER 1 OPENS. IT WORKS ONLY BECAUSE OF THAT DUPLICATE MAPPING (F19, F31)
StyleDrops/Options/style1drop1opt2AutoRR.src:124	WARNING	GRIPPER 3 SHARES GRIPPER 1'S I/O IN grp_data.dat; IF GRIPPER 3 IS EVER RECONFIGURED THIS RELEASE STOPS WORKING (F19, F31)
StylePicks/Options/style1pick1opt2.src:2 8	WARNING	THE HIDDEN FORM PARAMETER IS AC_CmdParam=1 - OPENING AND CONFIRMING THIS FORM REGENERATES AC_pickUPCheck(1) (PICKUP 1). DO NOT TOUCH UP THIS FOLD (F21)
StylePicks/Options/style1pick1opt2.src:1 33	WARNING	HIDDEN AC_CmdParam=1 - CONFIRMING THIS FORM REGENERATES AC_pickUP(1,TRUE) (F21)
Styles/Options/Style1Opt10AutoRR.src:1 1	WARNING	THE NUT CHECK AND THE CAMERA CHECK EACH SET \$OUT[141] do141RedRabbitFailed OR \$OUT[142] do142RedRabbitPassed FOR OPTION 12 - BOTH CAN BE ON TOGETHER, AND THE DROP CLEARS THEM BEFORE THE END OF THE CYCLE. THE DROP RELEASES THE PART WITH GRIPPER 3, WHICH SHARES GRIPPER 1'S I/O (F19, F31)
Utilities/gunelectrodechange.src:37	CHECK	NEVER RUN BEFORE - TEST THE WHOLE SEQUENCE AND THE PLC HANDSHAKE (F43, F49)
masref_user.src:139	CHECK	THE FLAG IS SET ON AGAIN AT THE END INSTEAD OF OFF (THE OLD do828 LINE WAS FALSE); NOTHING EVER SWITCHES \$OUT[930] OFF. AutomationCore REFUSES REQUEST TO ENTER WHILE IT IS ON (F35)
System/\$config.dat:783	CHECK	KRC_IO.xml SENDS THIS WORD BYTE-SWAPPED, AS ON R20 WHERE THE REGULATOR NEEDS SWAPPED VALUES (39425 = 0.09 MPa). 405 AND 1820 ARE NOT SWAPPED, 39425 IS - ONE SET IS WRONG (F44)

Archivo:línea	Tipo	Comentario
System/\$config.dat:846	CHECK	BASE AND WELD ARE NOT BYTE-SWAPPED, REST IS; THE BUS SWAPS THE WORD - CONFIRM ON THE REGULATOR WHICH IS RIGHT BEFORE CHANGING ANY OF THEM (F44)
System/sps.sub:120	CHECK	R20 BUILDS THE SAME VALUE FROM ALL FOUR BYTES IN ANOTHER ORDER - CONFIRM THE BTL6 OUTPUT FORMAT ON THIS PEDESTAL (F23)
System/sps.sub:135	CHECK	nWaterFlowMin 150 (15.0 L/MIN) TO BE CONFIRMED ON THIS ROBOT (F09)
System/sps.sub:159	CHECK	\$OUT[3001] IS NOT MAPPED; R20 USES \$OUT[3000] AND \$OUT[3016] FOR THE SAME BEACON (F29)
System/sps.sub:175	WARNING	RUNS IN EVERY OPERATING MODE (ALSO T1/T2, NO ENABLING SWITCH) AND WRITES THE SAME VALVES AS THE ROBOT PROGRAMS, WITH NO OWNERSHIP BETWEEN THEM (F11)
cell.src:69	WARNING	THE VENDOR ROUTINE EndOfCycle WAS EDITED ON THIS ROBOT: IT NO LONGER SETS \$OUT[15] do015WorkComplete NOR WAITS FOR \$IN[15] di015WorkCompleteAckn - THE PLC NEVER RECEIVES WORK COMPLETE FROM THIS ROBOT (F46)
cell.src:110	CHECK	GunElectrodeChange NOW RUNS AT THE STEPPER END - TEST IT BEFORE PRODUCTION (F43, F49)

Anexo B. Programas modificados, con los comentarios de la depuración

Listado completo de cada programa modificado en su estado actual. Sombreado amarillo: líneas que agregó o cambió la depuración (comentarios; el código no cambió). Sombreado azul: líneas de las correcciones de oficina (cambios de código y sus comentarios). Los encabezados de inline form se recortan después de ":%{" porque el resto son datos del smartPAD.

KRC/R1/Program/Centerline/CENTERLINE_HOME.src

```
1 &ACCESS RV0
2 &REL 1
3 &COMMENT 03-10-R1 PNW1 MANUAL HOME
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\Roboter\Template\vorgabe
6 DEF CENTERLINE_HOME()
7 ;*****
8 ; Gestamp Standards
9 ; Line - BMW G65 (cell 03)
10 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
11 ; Application - CENTERLINE PEDESTAL HOME
12 ;*****
13 ; MANUAL SERVICE PROGRAM - NOT CALLED BY THE CYCLE
14 ; HOMES THE CENTERLINE PEDESTAL, NO ROBOT MOTION: OUTPUTS OFF, LOWER PIN RETURN, QFP RETURN,
15 ; UPPER PIN RETRACT, GUN OPEN, PART CLAMP OPEN. TIMED STEPS - ONLY THE QFP RETURN IS
16 ; CONFIRMED BY A SWITCH. NO INTERLOCKS: RUN IT ONLY WITH ROBOT AND PEOPLE CLEAR OF THE PEDESTAL.
17 ;NOTE: PEDESTAL REFERENCE VALUES AND PRESSURE TABLE: SEE CENTERLINE_WELD (centerline_weld.src)
18 ;CHECK: $OUT[473] dopw1_Nut1Intensify_Home IS SWITCHED OFF HERE BUT IS ON AT REST IN
19 ; CENTERLINE_WELD (POLARITY TO CONFIRM); THE BOSCH HANDSHAKE ($OUT[481..483]) AND THE
20 ; WELD PROGRAM NUMBER ARE NOT RESET HERE (F34)
21 ;NOTE: $OUT[475] dopw1_StartWater IS SWITCHED BACK ON BY sps.sub IN THE NEXT SUBMIT CYCLE
22 ; OUTSIDE DRY CYCLE (F09)
23 ;WARNING: WHILE A PRELOAD IS ACTIVE (NUTCYCLE_ACTIVE), PRELOAD() IN THE SUBMIT INTERPRETER
24 ; REWRITES THE SAME OUTPUTS EVERY CYCLE - THIS PROGRAM DOES NOT CHECK IT (F11)
25 ;*****
26
27 ;GUN GENERAL PRESSURE - SIGNALS $OUT[506..521] CL_GunPressureCmd = CL_GUN_PRESS_BASE (F44)
28 CL_GunPressureCmd=CL_GUN_PRESS_BASE
29
30
31 ;OUTPUTS OFF - SIGNALS $OUT[473] dopw1_Nut1Intensify_Home OFF, $OUT[475] dopw1_StartWater OFF,
32 ; $OUT[476] dopw1_StartFeed OFF
33 dopw1_Nut1Intensify_Home=FALSE
34 dopw1_StartWater=FALSE
35 dopw1_StartFeed=FALSE
36 ;OUTPUTS OFF - SIGNALS $OUT[494] dopw1_BlowOff OFF, $OUT[505] do505QFPNutBlowOff OFF,
37 ; $OUT[500] do500Reserved (UNKNOWN - ONLY EVER SWITCHED OFF) OFF
38 dopw1_BlowOff=FALSE
39 do500Reserved=FALSE
40 do505QFPNutBlowOff=FALSE
41
42 ;LOWER PIN RETURN (DOWN) - SIGNALS $OUT[495] dopw1_AdvancePin OFF, WAIT 0.2 S,
43 ; $OUT[496] dopw1_ReturnPin ON, WAIT 1 S
44 dopw1_AdvancePin=FALSE
45 WAIT SEC 0.2
46 dopw1_ReturnPin=TRUE
47 WAIT SEC 1.0
```

48

```
49 ;QFP RETURN - SIGNALS $OUT[503] do503QFPAdvance OFF, WAIT 0.2 S,
50 ; $OUT[504] do504QFPReturn ON
51 do503QFPAdvance=FALSE
52 WAIT SEC 0.2
53 do504QFPReturn=TRUE
54 ;WAIT QFP RETURNED - SIGNAL $IN[482] dipw1_SpearHome ON, THEN 0.5 S
55 WAIT FOR dipw1_SpearHome
56 WAIT SEC 0.5
57
58 ;UPPER (WELD) PIN RETRACT - SIGNALS $OUT[498] do498UpperPinExtend OFF, WAIT 0.2 S,
59 ; $OUT[499] do499UpperPinRetract ON, WAIT 1 S
60 do498UpperPinExtend=FALSE
61 WAIT SEC 0.2
62 do499UpperPinRetract=TRUE
63 WAIT SEC 1.0
64
65 ;GUN OPEN - SIGNALS $OUT[472] dopw1_GunWork OFF, WAIT 0.2 S, $OUT[471] dopw1_GunHome ON,
66 ; WAIT 1.5 S (GUN STROKE LPT NOT CHECKED)
67 dopw1_GunWork=FALSE
68 WAIT SEC 0.2
69 dopw1_GunHome=TRUE
70 WAIT SEC 1.5
71
72 ;PART CLAMP OPEN - SIGNALS $OUT[501] do501PartClampClose OFF, WAIT 0.2 S,
73 ; $OUT[502] do502PartClampOpen ON, WAIT 1 S - $IN[466] di466PartClampOpen NOT CHECKED
74 do501PartClampClose=FALSE
75 WAIT SEC 0.2
76 do502PartClampOpen=TRUE
77 WAIT SEC 1.0
78
79 END
```

KRC/R1/Program/Centerline/GUN_OPEN_CHECK.src

```
1 &ACCESS RV01
2 &REL 2
3 &COMMENT 03-10-R1 PNW1 GUN OPEN CHECK
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\Roboter\Template\vorgabe
6 DEF GUN_OPEN_CHECK( )
7 ;*****
8 ; Gestamp Standards
9 ; Line - BMW G65 (cell 03)
10 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
11 ; Application - NUT WELDER GUN OPEN INTERLOCK BEFORE THE ROBOT MOVES INTO OR OUT OF THE PEDESTAL
12 ;*****
13 ; CALLED BY stylelapplopt1A..3B BEFORE ENTERING THE PEDESTAL AND AGAIN AFTER THE WELD, AND BY
14 ; CENTERLINE_WELD AFTER THE GUN OPENS. CHECKS IN THIS ORDER, EACH WITH A STATE MESSAGE AND A WAIT (THE ROBOT DOES NOT MOVE):
15 ; 1. SUBMIT RUNNING ($PRO_STATE0): CL_GunStrokePositionLPT IS ONLY UPDATED BY SPS.SUB
16 ; 2. AIR PRESSURE - SIGNAL $IN[3132] dipw1_AirOK ON (ONLY WITH bGunAirCheck TRUE)
17 ; 3. COOLING WATER - dipw1_WaterOk (SET BY SPS.SUB FROM THE SV0500 FLOW, ONLY WITH bGunWaterCheck)
18 ; 4. GUN OPEN - CL_GunStrokePositionLPT IN nGunOpenMin..nGunOpenMax, nGunOpenWaitMs WITHOUT MESSAGE
19 ; GUN_OPEN_LOST (BELOW) IS THE ROUTINE OF THE GUN MONITORING INTERRUPTS 20-22 OF THE WELD APPS
20 ; (stylelapplopt1A..3B); THE APPS SWITCH THE INTERRUPTS ON BEFORE CALLING THIS CHECK.
21 ; SETTINGS IN GUN_OPEN_CHECK.dat. WRITTEN BY THE PROGRAMMER ON 2026-10-05 (F01)
22 ;CHECK: CLAMP OPEN $IN[466] AND QFP RETURNED $IN[482] ARE NOT CHECKED BEFORE ENTERING (F01)
23 DECL KrlMsg_T Msg
```

```

24 DECL KrlMsg_T MsgW
25 DECL KrlMsgPar_T Par[3]
26 DECL KrlMsgOpt_T Opt
27 DECL INT nHandle, nWaitMs
28 DECL BOOL bRes
29
30 ;STOP THE ADVANCE RUN: THE GUN POSITION IS READ WITH THE ROBOT STOPPED, NOT AHEAD OF THE MOTION
31 WAIT SEC 0
32
33 ;FOLD MESSAGE SETUP
34 Msg.Modul[]="GunOpenCheck"
35 MsgW.Modul[]="GunOpenCheck"
36 Par[1].Par_Type=#EMPTY
37 Par[2].Par_Type=#EMPTY
38 Par[3].Par_Type=#EMPTY
39 Opt.VL_Stop=TRUE
40 Opt.Clear_P_Reset=TRUE
41 Opt.Clear_P_SAW=FALSE
42 Opt.Log_To_DB=TRUE
43 ;ENDFOLD
44
45 ;1. SUBMIT RUNNING - OTHERWISE CL_GunStrokePositionLPT IS FROZEN (NO TIMEOUT, STATE MESSAGE)
46 IF $PRO_STATE0<>#P_ACTIVE THEN
47     Msg.Nr=1
48     Msg.Msg_txt[]="Submit (SPS) not running - gun position not valid - robot waiting"
49     Par[1].Par_Type=#EMPTY
50     nHandle=Set_KrlMsg(#STATE, Msg, Par[], Opt)
51     WAIT FOR $PRO_STATE0==#P_ACTIVE
52     bRes=Clear_KrlMsg(nHandle)
53     ;LET THE SUBMIT REFRESH THE GUN POSITION
54     WAIT SEC 0.5
55 ENDIF
56
57 ;2. AIR PRESSURE - WITHOUT AIR THE GUN CANNOT BE HELD OPEN (ONLY WITH bGunAirCheck TRUE)
58 IF bGunAirCheck THEN
59     ;AIR PRESSURE SWITCH - SIGNAL $IN[3132] dipw1_AirOK ON, ELSE MESSAGE AND WAIT (NO TIMEOUT)
60     IF NOT dipw1_AirOK THEN
61         Msg.Nr=3
62         Msg.Msg_txt[]="Nut welder air pressure not OK (dipw1_AirOK) - robot waiting"
63         Par[1].Par_Type=#EMPTY
64         nHandle=Set_KrlMsg(#STATE, Msg, Par[], Opt)
65         ;WAIT AIR PRESSURE - SIGNAL $IN[3132] dipw1_AirOK ON (NO TIMEOUT, STATE MESSAGE)
66         WAIT FOR dipw1_AirOK
67         bRes=Clear_KrlMsg(nHandle)
68     ENDIF
69 ENDIF
70
71 ;3. COOLING WATER - dipw1_WaterOk FROM THE SV0500 FLOW (SPS.SUB); MESSAGE WITH THE FLOW, WAIT
72 IF bGunWaterCheck THEN
73     IF NOT dipw1_WaterOk THEN
74         MsgW.Nr=6
75         MsgW.Msg_txt[]="Nut welder cooling water flow too low (%1 x0.1 l/min) - robot waiting"
76         Par[1].Par_Type=#VALUE
77         Par[1].Par_Int=nWaterFlow
78         nHandle=Set_KrlMsg(#STATE, MsgW, Par[], Opt)
79         WAIT FOR dipw1_WaterOk
80         bRes=Clear_KrlMsg(nHandle)

```

```

81     Par[1].Par_Type=#EMPTY
82     ENDIF
83 ENDIF
84
85 ;4. GUN OPEN - UP TO nGunOpenWaitMs WITHOUT MESSAGE
86 nWaitMs=0
87 WHILE ((CL_GunStrokePositionLPT<nGunOpenMin) OR (CL_GunStrokePositionLPT>nGunOpenMax)) AND (nWaitMs<nGunOpenWaitMs)
88     WAIT SEC 0.05
89     nWaitMs=nWaitMs+50
90 ENDWHILE
91
92 ;GUN STILL NOT OPEN - STATE MESSAGE WITH THE POSITION, WAIT (NO TIMEOUT)
93 IF (CL_GunStrokePositionLPT<nGunOpenMin) OR (CL_GunStrokePositionLPT>nGunOpenMax) THEN
94     Msg.Nr=2
95     Msg.Msg_txt[]="Nut welder gun not open (position %1) - robot waiting"
96     Par[1].Par_Type=#VALUE
97     Par[1].Par_Int=CL_GunStrokePositionLPT
98     nHandle=Set_KrLMsg(#STATE, Msg, Par[], Opt)
99     WAIT FOR (CL_GunStrokePositionLPT>=nGunOpenMin) AND (CL_GunStrokePositionLPT<=nGunOpenMax)
100    bRes=Clear_KrLMsg(nHandle)
101 ENDIF
102
103 END
104
105 GLOBAL DEF GUN_OPEN_LOST( )
106 ;INTERRUPT ROUTINE OF THE GUN MONITORING (INTERRUPTS 20-22 IN style1appt1A..3B): THE GUN LEFT THE
107 ;OPEN RANGE OR THE SUBMIT STOPPED WHILE THE ROBOT MOVES INTO OR OUT OF THE PEDESTAL. BRAKE F STOPS
108 ;THE ROBOT ON ITS PATH; IT CONTINUES WHEN THE QUIT MESSAGE IS ACKNOWLEDGED AND THE GUN IS OPEN AGAIN
109 DECL KrLMsg_T Msg
110 DECL KrLMsgPar_T Par[3]
111 DECL KrLMsgOpt_T Opt
112 DECL INT nHandle
113 DECL BOOL bRes
114
115 BRAKE F
116
117 ;FOLD MESSAGE SETUP
118 Msg.Modul[]="GunOpenCheck"
119 Par[1].Par_Type=#EMPTY
120 Par[2].Par_Type=#EMPTY
121 Par[3].Par_Type=#EMPTY
122 Opt.VL_Stop=TRUE
123 Opt.Clear_P_Reset=TRUE
124 Opt.Clear_P_SAW=FALSE
125 Opt.Log_To_DB=TRUE
126 ;ENDFOLD
127
128 IF $PRO_STATE0<>#P_ACTIVE THEN
129     Msg.Nr=5
130     Msg.Msg_txt[]="Submit (SPS) stopped while robot in welder - robot stopped, acknowledge"
131 ELSE
132     Msg.Nr=4
133     Msg.Msg_txt[]="Gun not open while robot in welder (pos. %1) - robot stopped, acknowledge"
134     Par[1].Par_Type=#VALUE
135     Par[1].Par_Int=CL_GunStrokePositionLPT
136 ENDIF
137 nHandle=Set_KrLMsg(#QUIT, Msg, Par[], Opt)

```

```

138
139 ;CONTINUE ONLY WHEN THE MESSAGE IS ACKNOWLEDGED, THE SUBMIT RUNS AND THE GUN IS OPEN AGAIN:
140 ;FIRST WAIT FOR THE ACKNOWLEDGEMENT
141 WHILE Exists_KrMsg(nHandle)
142     WAIT SEC 0.1
143 ENDWHILE
144 ;STILL NOT READY AFTER THE ACKNOWLEDGEMENT - STATE MESSAGE WHILE THE ROBOT WAITS, SO IT NEVER
145 ;STANDS IN THE PEDESTAL WITHOUT A MESSAGE (INDEPENDENT REVIEW 2026-10-05, F01)
146 IF ($PRO_STATE0<=>#P_ACTIVE) OR (CL_GunStrokePositionLPT<nGunOpenMin) OR (CL_GunStrokePositionLPT>nGunOpenMax) THEN
147     Msg.Nr=7
148     Msg.Msg_txt[]="Robot in welder waiting for gun open and Submit running"
149     Par[1].Par_Type=#EMPTY
150     nHandle=Set_KrMsg(#STATE, Msg, Par[], Opt)
151     WHILE ($PRO_STATE0<=>#P_ACTIVE) OR (CL_GunStrokePositionLPT<nGunOpenMin) OR (CL_GunStrokePositionLPT>nGunOpenMax)
152         WAIT SEC 0.1
153     ENDWHILE
154     bRes=Clear_KrMsg(nHandle)
155 ENDIF
156
157 END
158

```

KRC/R1/Program/Centerline/PRELOAD.src

```

1 &ACCESS RVP1
2 &COMMENT 03-10-R1 PNW1 NUT PRELOAD
3 DEF PRELOAD ( )
4 ;*****
5 ; Gestamp Standards
6 ; Line - BMW G65 (cell 03)
7 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
8 ; Application - NUT PRELOAD AT CENTERLINE PEDESTAL
9 ;*****
10 ; CALLED BY sps.sub IN EVERY SUBMIT CYCLE: RUNS IN THE SUBMIT INTERPRETER IN EVERY
11 ; OPERATING MODE (T1, T2, AUT, EXT), ALSO WHEN NO ROBOT PROGRAM IS RUNNING.
12 ; STARTED BY NUT_START (TRIGGER ON THE MOVE TO P12 / P10 OF THE PICKS, AND Request_next_nut
13 ; AT THE END OF WELD APPS 1-2, stylelapplopt1A/B, 2A/B):
14 ; STEP 0 - QFP RETURN; WHEN RETURNED, NUT FEED FOR 2000 MS
15 ; STEP 10 - GUN OPEN, UPPER PIN RETRACT, CLAMP OPEN; QFP RETURN UNTIL SEEN RETURNED, THEN
16 ; QFP ADVANCE; WHEN ADVANCED, NUT BLOW-OFF ONTO THE LOWER PIN, LOWER PIN ADVANCE
17 ; STEP 20 - AFTER 800 MS BLOW-OFF: QFP RETURN, BLOW-OFF OFF; 800 MS LATER NUT_READY=TRUE
18 ;WARNING: DRIVES THE PEDESTAL VALVES ON ITS OWN, WITH NO ROBOT, MODE OR ENABLING-SWITCH
19 ; INTERLOCK, AND SHARES THEM WITH CENTERLINE_WELD AND CENTERLINE_HOME WITHOUT OWNERSHIP (F11)
20 ;NOTE: USES $TIMER[10] FEED 2000 MS, $TIMER[11] ADVANCE 500 MS, $TIMER[12] RETURN 800 MS,
21 ; $TIMER[13] BLOW-OFF 800 MS - KEEP THESE TIMERS FREE FOR THIS PROGRAM (F26)
22 ;NOTE: NO STEP TIMEOUTS; NUT_PRELOAD_FAULT IS NEVER SET TRUE; NUT_READY IS SET ON TIMERS,
23 ; WITHOUT FEED COMPLETE OR FINAL QFP POSITION CONFIRMATION (F13)
24 ;NOTE: THE STEPS RUN ONLY WITH $IN[4] di004UseDryCycle OFF. IN DRY CYCLE A START STILL
25 ; CLEARS NUT_READY AND NOTHING SETS IT WHILE DRY CYCLE IS ON - THE WELD APPS WAIT FOR IT (F04)
26 ;*****
27
28 ;START A PRELOAD WHEN NUT_START IS SET AND NONE IS ACTIVE: NUT_START CLEARED, NUT_READY=FALSE,
29 ;STEP 0, STEP0/10/20DONE AND THE FAULT FLAG CLEARED, $TIMER[10..13] STOPPED AND ZEROED
30 ;(STEP10_ADV_STARTED IS NOT CLEARED HERE - ONLY WHEN A PRELOAD COMPLETES)
31 ;NOTE: A NUT_START SET WHILE A PRELOAD IS ACTIVE STAYS SET AND STARTS ANOTHER PRELOAD (A NEW
32 ; NUT FEED) AS SOON AS THIS ONE ENDS; NUT_READY IS THEN TRUE FOR ONLY ONE SUBMIT CYCLE (F11)
33 IF NUT_START AND NOT NUTCYCLE_ACTIVE THEN
34

```

```

35 NUT_START=FALSE
36 NUTCYCLE_ACTIVE=TRUE
37 NUT_READY=FALSE
38 NUT_PRELOAD_STEP=0
39 STEP0DONE=FALSE
40 STEP10DONE=FALSE
41 STEP20DONE=FALSE
42 NUT_PRELOAD_FAULT=FALSE
43 $TIMER_STOP[10]=TRUE
44 $TIMER[10]=0
45 $TIMER_STOP[11]=TRUE
46 $TIMER[11]=0
47 $TIMER_STOP[12]=TRUE
48 $TIMER[12]=0
49 $TIMER_STOP[13]=TRUE
50 $TIMER[13]=0
51
52 ENDIF
53
54 ;RUN THE STEPS ONLY OUTSIDE DRY CYCLE - SIGNAL $IN[4] di004UseDryCycle OFF
55 IF NUTCYCLE_ACTIVE AND NOT di004UseDryCycle THEN
56
57 SWITCH NUT_PRELOAD_STEP
58
59 CASE 0
60     ;STEP 0 - FEED A NUT
61
62     ;QFP RETURN - SIGNALS $OUT[503] do503QFPAdvance OFF, $OUT[504] do504QFPReturn ON
63     do503QFPAdvance=FALSE
64     do504QFPReturn=TRUE
65
66     ;WHEN QFP RETURNED AND NOT ADVANCED - SIGNALS $IN[482] dipw1_SpearHome ON,
67     ; $IN[470] di470QFPAdvanced OFF
68     IF (dipw1_SpearHome==TRUE) AND (di470QFPAdvanced==FALSE) THEN
69         ;FEED NUT - SIGNALS $OUT[476] dopw1_StartFeed ON (OFF WHEN STEP 10 COMMANDS QFP ADVANCE),
70         ; $OUT[494] dopw1_BlowOff OFF
71         dopw1_StartFeed=TRUE
72         dopw1_BlowOff=FALSE
73
74         ;START $TIMER[10] ONCE
75         IF $TIMER_STOP[10] THEN
76             $TIMER[10]=0
77             $TIMER_STOP[10]=FALSE
78         ENDIF
79
80         ;AFTER 2000 MS OF FEED: STEP 0 DONE
81         IF $TIMER[10]>=2000 THEN
82             $TIMER_STOP[10]=TRUE
83
84             STEP0DONE=TRUE
85         ENDIF
86
87     ENDIF
88
89     IF STEP0DONE THEN
90         NUT_PRELOAD_STEP = 10
91     ENDIF

```

```

92
93 CASE 10
94     ;STEP 10 - QFP ADVANCE AND BLOW THE NUT ONTO THE LOWER PIN
95
96     IF STEP0DONE THEN
97
98         ;WRITTEN IN EVERY SUBMIT CYCLE OF STEP 10:
99         ;GUN OPEN - SIGNALS $OUT[472] dopw1_GunWork OFF, $OUT[471] dopw1_GunHome ON
100        dopw1_GunWork=FALSE
101        dopw1_GunHome=TRUE
102        ;UPPER PIN RETRACT - SIGNALS $OUT[498] do498UpperPinExtend OFF,
103        ; $OUT[499] do499UpperPinRetract ON
104        do498UpperPinExtend=FALSE
105        do499UpperPinRetract=TRUE
106        ;PART CLAMP OPEN - SIGNALS $OUT[501] do501PartClampClose OFF,
107        ; $OUT[502] do502PartClampOpen ON
108        do501PartClampClose=FALSE
109        do502PartClampOpen=TRUE
110
111        ;UNTIL THE QFP HAS BEEN SEEN RETURNED (STEP10_ADV_STARTED):
112        IF NOT STEP10_ADV_STARTED THEN
113            ;BLOW-OFF ON - SIGNAL $OUT[494] dopw1_BlowOff ON
114            dopw1_BlowOff=TRUE
115            ;QFP RETURN - SIGNALS $OUT[503] do503QFPAdvance OFF, $OUT[504] do504QFPReturn ON
116            do503QFPAdvance=FALSE
117            do504QFPReturn=TRUE
118
119            ;WHEN QFP RETURNED AND NOT ADVANCED - SIGNALS $IN[482] dipw1_SpearHome ON,
120            ; $IN[470] di470QFPAdvanced OFF
121            IF (dipw1_SpearHome==TRUE) AND (di470QFPAdvanced==FALSE) THEN
122                STEP10_ADV_STARTED=TRUE
123
124            ENDIF
125        ELSE
126
127            ;QFP ADVANCE AND FEED OFF - SIGNALS $OUT[504] do504QFPReturn OFF,
128            ; $OUT[503] do503QFPAdvance ON, $OUT[476] dopw1_StartFeed OFF
129            do504QFPReturn=FALSE
130            do503QFPAdvance=TRUE
131            dopw1_StartFeed=FALSE
132
133            ;WHEN QFP ADVANCED AND NOT RETURNED - SIGNALS $IN[470] di470QFPAdvanced ON,
134            ; $IN[482] dipw1_SpearHome OFF
135            IF (di470QFPAdvanced==TRUE) AND (dipw1_SpearHome==FALSE) THEN
136                ;NUT BLOW-OFF - SIGNALS $OUT[505] do505QFPNutBlowOff ON, $OUT[494] dopw1_BlowOff OFF
137                do505QFPNutBlowOff=TRUE
138                dopw1_BlowOff=FALSE
139                ;START $TIMER[13] (BLOW-OFF TIME, CHECKED IN STEP 20) ONCE
140                IF $TIMER_STOP[13] THEN
141                    $TIMER[13]=0
142                    $TIMER_STOP[13]=FALSE
143                ENDIF
144                ;LOWER PIN ADVANCE - SIGNALS $OUT[496] dopw1_ReturnPin OFF, $OUT[495] dopw1_AdvancePin ON
145                ;CHECK: OLD COMMENT SAID LOWER PIN RETREATED; THE CODE ADVANCES THE PIN (F27)
146                dopw1_ReturnPin=FALSE
147                dopw1_AdvancePin=TRUE
148

```

```

149             ;START $TIMER[11] ONCE
150             IF $TIMER_STOP[11] THEN
151                 $TIMER[11]=0
152                 $TIMER_STOP[11]=FALSE
153             ENDIF
154
155             ;500 MS AFTER THE QFP ADVANCED: STEP 10 DONE
156             IF $TIMER[11]>=500 THEN
157                 $TIMER_STOP[11]=TRUE
158
159                 STEP10DONE=TRUE
160             ENDIF
161         ENDIF
162     ENDIF
163 ENDIF
164
165 ;GO TO STEP 20 WITH $TIMER[12] STOPPED AND ZEROED
166 IF STEP0DONE AND STEP10DONE THEN
167     NUT_PRELOAD_STEP = 20
168     $TIMER_STOP[12]=TRUE
169     $TIMER[12]=0
170
171     ENDIF
172
173 CASE 20
174     ;STEP 20 - AFTER 800 MS OF NUT BLOW-OFF ($TIMER[13]): QFP RETURN
175
176     IF STEP0DONE AND STEP10DONE AND ($TIMER[13]>=800)THEN
177
178         ;LOWER PIN ADVANCE - SIGNALS $OUT[496] dopw1_ReturnPin OFF, $OUT[495] dopw1_AdvancePin ON
179         ;CHECK: OLD COMMENT SAID LOWER PIN RETREATED; THE CODE ADVANCES THE PIN (F27)
180         dopw1_ReturnPin=FALSE
181         dopw1_AdvancePin=TRUE
182
183         ;QFP RETURN - SIGNALS $OUT[503] do503QFPAdvance OFF, $OUT[504] do504QFPReturn ON
184         do503QFPAdvance=FALSE
185         do504QFPReturn=TRUE
186         ;NUT BLOW-OFF OFF - SIGNAL $OUT[505] do505QFPNutBlowOff OFF
187         do505QFPNutBlowOff=FALSE
188
189         ;START $TIMER[12] ONCE
190         IF $TIMER_STOP[12] THEN
191             $TIMER[12]=0
192             $TIMER_STOP[12]=FALSE
193         ENDIF
194
195         ;800 MS AFTER THE QFP RETURN COMMAND: STEP 20 DONE
196         ;NOTE: QFP RETURNED $IN[482] dipw1_SpearHome IS NOT CHECKED BEFORE NUT_READY (F01, F13)
197         IF $TIMER[12]>=800 THEN
198             $TIMER_STOP[12]=TRUE
199             STEP20DONE=TRUE
200         ENDIF
201     ENDIF
202
203     ;PRELOAD COMPLETE: STEP 0, NUT_READY=TRUE, NEXT_NUT_READY=TRUE IF REQUESTED, CYCLE INACTIVE,
204     ;$TIMER[10..12] STOPPED AND ZEROED ($TIMER[13] IS LEFT RUNNING UNTIL THE NEXT START)
205     ;NOTE: NOTHING CLEARS NUT_READY WHEN THE NUT IS WELDED (F14); NEXT_NUT_READY IS WAITED FOR BY

```

```

206      ; NUTS 2 AND 3 (stylelapplopt2A/B, 3A/B)
207      IF STEP0DONE AND STEP10DONE AND STEP20DONE THEN
208          NUT_PRELOAD_STEP =0
209          NUT_READY=TRUE
210
211          IF NEXT_NUT_REQUEST THEN
212              NEXT_NUT_READY=TRUE
213              NEXT_NUT_REQUEST=FALSE
214          ENDIF
215          NUTCYCLE_ACTIVE=FALSE
216          STEP10_ADV_STARTED=FALSE
217
218          $TIMER_STOP[10]=TRUE
219          $TIMER[10]=0
220          $TIMER_STOP[11]=TRUE
221          $TIMER[11]=0
222          $TIMER_STOP[12]=TRUE
223          $TIMER[12]=0
224
225      ENDIF
226
227  ENDSWITCH
228
229
230 ENDIF
231 END

```

KRC/R1/Program/Centerline/Request_next_nut.src

```

1 &ACCESS RV
2 &COMMENT 03-10-R1 PNW1 NEXT NUT REQUEST
3 DEF Request_next_nut ( )
4 ;*****
5 ; Gestamp Standards
6 ; Line - BMW G65 (cell 03)
7 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
8 ; Application - REQUEST THE NEXT NUT PRELOAD AT CENTERLINE PEDESTAL
9 ;*****
10 ; CALLED BY TRIGGER (PRIO = -1) ON THE LAST MOTION OF stylelapplopt1A/B (DISTANCE = 0) AND
11 ; stylelapplopt2A/B (DISTANCE = 1), AFTER THE PEDESTAL (ALSO IN DRY CYCLE). SETS NUT_START: THE
12 ; SUBMIT PROGRAM PRELOAD() (CALLED BY sps.sub) THEN FEEDS AND LOADS THE NEXT NUT AND SETS
13 ; NUT_READY AND NEXT_NUT_READY - ITS STEPS RUN ONLY OUTSIDE DRY CYCLE (F04).
14 ;NOTE: A TRIGGER SKIPPED BY A PROGRAM RESET OR A BLOCK SELECTION MEANS NO PRELOAD REQUEST (F14)
15 ;NOTE: NUTS 2 AND 3 (stylelapplopt2A/B, 3A/B) WAIT FOR NEXT_NUT_READY BEFORE THEY MOVE IN; IN DRY
16 ; CYCLE NOTHING SETS IT AND THE CYCLE HANGS THERE (F04)
17 ;*****
18
19 ;CLEAR NEXT_NUT_READY, SET NEXT_NUT_REQUEST, START PRELOAD (NUT_START=TRUE)
20     NEXT_NUT_READY=FALSE
21     NEXT_NUT_REQUEST=TRUE
22     NUT_START=TRUE
23
24 END

```

KRC/R1/Program/Centerline/centerline_weld.src

```

1 &ACCESS RV01
2 &REL 38
3 &COMMENT 03-10-R1 PNW1 NUT WELD

```

```

4 &PARAM TEMPLATE = C:\KRC\Roboter\Template\vorgabe
5 DEF CENTERLINE_WELD()
6 ;*****
7 ; Gestamp Standards
8 ; Line - BMW G65 (cell 03)
9 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
10 ; Application - NUT WELD AT CENTERLINE PEDESTAL (ALL 3 NUTS)
11 ;*****
12 ; PRODUCTION NUT WELD, CALLED BY stylelapplopt1A..3B OUTSIDE DRY CYCLE WITH THE PART ALREADY
13 ; AT THE WELD POSITION (NO ROBOT MOTION HERE): PEDESTAL START POSITION, UPPER PIN EXTEND,
14 ; PART CLAMP CLOSE, GUN CLOSE, GUN STROKE CHECK (LPT), INTENSIFY, BOSCH WELD; THEN GUN
15 ; OPEN, UPPER PIN RETRACT, PART CLAMP OPEN, LOWER PIN ADVANCE, QFP RETURN.
16 ;WARNING: PEDESTAL ENTRY IS ONLY PARTLY CONFIRMED. THE CALLER CHECKS GUN OPEN (GUN_OPEN_CHECK) AND
17 ; MONITORS IT WITH INTERRUPTS 20-22; CLAMP OPEN AND QFP RETURNED ARE CHECKED BELOW ONLY AFTER THE
18 ; ROBOT IS ALREADY AT THE WELD POSITION (F01)
19 ;NOTE: THE NUT IS NORMALLY PRELOADED BY PRELOAD() IN THE SUBMIT INTERPRETER (NUT_READY TRUE);
20 ; PRELOAD DRIVES THE SAME PEDESTAL VALVES, WITH NO OWNERSHIP BETWEEN THE TWO (F11)
21 ;NOTE: NO WATER FLOW CHECK BEFORE THE WELD (F09), NO FAULT CHECK AFTER THE WELD (F08),
22 ; NO TIMEOUT OR MESSAGE ON THE WAITS (F15), PIN POSITIONS NOT VERIFIED (F16)
23 ;NOTE: NUT_READY IS NOT CLEARED WHEN THE NUT IS WELDED (F14)
24 ;NOTE: NOTHING PUTS THE PEDESTAL IN A SAFE STATE IF THIS PROGRAM IS CANCELLED (F10)
25 ;NOTE: USES $TIMER[14] (GUN-CLOSED TIMEOUT) AND $TIMER[15] (TIME IN THE WINDOW) (F26)
26 ;NOTE: LIMITS, PRESSURE WORDS AND WELD PROGRAM ARE CONSTANTS IN $config.dat (FOLD CENTERLINE CONTROL)
27 ;*****
28
29 ;PEDESTAL REFERENCE VALUES - INFORMATION ONLY
30 ;UPPER PIN POSITION CL_WeldPinPosition ($IN[591..606]) - NO PROGRAM READS IT (F16):
31 ;UPPER PIN RETRACT = 27252 RANGE 26000 - 28000
32 ;UPPER PIN EXTENDED = 6602 RANGE 6500- 7000
33 ;UPPER PIN WELD POSITION = 11674 RANGE 10000 - 13500
34
35 ;GUN STROKE CL_GunStrokePositionLPT ($IN[559..590], LPT):
36 ;GUN RETRACT = 139815 RANGE 137000 - 142000 (CHECKED: GUN_OPEN_CHECK, nGunOpenMin..MAX)
37 ;GUN EXTENDED = 61450 RANGE 60000 - 63000
38 ;GUN WELD POSITION = 70915 RANGE 65000 - 76000 (CHECKED: CL_GUN_WELD_MIN..MAX = 69000..73000)
39
40 ;ITV PRESSURE SET-POINT CL_GunPressureCmd ($OUT[506..521]):
41 ;FORMULA CMD = (MPa / 0.9) * 4095
42 ;CHECK: THIS TABLE IS NOT BYTE-SWAPPED. KRC_IO.xml SENDS THE WORD TO THE REGULATOR BYTE-SWAPPED,
43 ;CHECK: AS ON R20 (THERE 0.30 MPa IS WRITTEN 21765). THIS PROGRAM WRITES CL_GUN_PRESS_BASE = 405 AND
44 ;CHECK: CL_GUN_PRESS_WELD = 1820 (NOT SWAPPED) AND CL_GUN_PRESS_REST = 39425 AT THE END (SWAPPED)
45 ;CHECK: 0.09 MPa) - CONFIRM WHICH IS RIGHT ON THE ITV (F44)
46 ;PRESSURE 455 0.10MPa 14.50 PSI
47 ;PRESSURE 910 0.20MPa 29.01 PSI
48 ;PRESSURE 1365 0.30MPa 43.51 PSI
49 ;PRESSURE 1820 0.40MPa 58.02 PSI (CL_GUN_PRESS_WELD)
50 ;PRESSURE 2275 0.50MPa 72.52 PSI
51 ;PRESSURE 2503 0.55MPa 79.77 PSI
52 ;PRESSURE 2730 0.60MPa 87.02 PSI
53 ;CL_GUN_PRESS_BASE = 405 = 0.089 MPa IF NOT SWAPPED
54
55 ;PEDESTAL START POSITION (NOT A FULL HOME: THE LOWER PIN IS ADVANCED)
56
57 ;GUN OPEN - SIGNALS $OUT[472] dopw1_GunWork OFF, $OUT[471] dopw1_GunHome ON
58 dopw1_GunWork=FALSE
59 dopw1_GunHome=TRUE
60

```

```

61 ;UPPER PIN RETRACT - SIGNALS $OUT[498] do498UpperPinExtend OFF, $OUT[499] do499UpperPinRetract ON
62 ;CHECK: OLD COMMENT SAID UPPER PIN EXTEND; THE CODE RETRACTS THE UPPER PIN HERE (F27)
63 do498UpperPinExtend=FALSE
64 do499UpperPinRetract=TRUE
65
66 ;PART CLAMP OPEN - SIGNALS $OUT[501] do501PartClampClose OFF, $OUT[502] do502PartClampOpen ON
67 do501PartClampClose=FALSE
68 do502PartClampOpen=TRUE
69 ;WAIT PART CLAMP OPEN - SIGNAL $IN[466] di466PartClampOpen ON
70 WAIT FOR di466PartClampOpen
71
72 ;LOWER PIN ADVANCE - SIGNALS $OUT[496] dopw1_ReturnPin OFF, $OUT[495] dopw1_AdvancePin ON
73 dopw1_ReturnPin=FALSE
74 dopw1_AdvancePin=TRUE
75
76 ;QFP RETURN - SIGNALS $OUT[503] do503QFPAdvance OFF, $OUT[504] do504QFPReturn ON
77 do503QFPAdvance=FALSE
78 do504QFPReturn=TRUE
79 ;WAIT QFP RETURNED - SIGNAL $IN[482] dipw1_SpearHome ON
80 WAIT FOR dipw1_SpearHome
81
82 ;LOCAL NUT FEED - ONLY WHEN NO NUT WAS PRELOADED BY PRELOAD() (NUT_READY FALSE). NORMALLY
83 ; SKIPPED: PRELOAD SETS NUT_READY FOR EVERY NUT BEFORE THE CALLER MOVES IN
84 IF NOT NUT_READY THEN
85 ;LOWER PIN ADVANCE AND BLOW-OFF - SIGNALS $OUT[496] dopw1_ReturnPin OFF,
86 ; $OUT[495] dopw1_AdvancePin ON, $OUT[494] dopw1_BlowOff ON
87 dopw1_ReturnPin=FALSE
88 dopw1_AdvancePin=TRUE
89 dopw1_BlowOff=TRUE
90
91 ;QFP ADVANCE - SIGNALS $OUT[504] do504QFPReturn OFF, $OUT[503] do503QFPAdvance ON
92 do504QFPReturn=FALSE
93 do503QFPAdvance=TRUE
94 ;WAIT QFP ADVANCED - SIGNAL $IN[470] di470QFPAdvanced ON
95 WAIT FOR di470QFPAdvanced
96
97 ;FEED NUT - SIGNAL $OUT[476] dopw1_StartFeed ON, THEN OFF, THEN WAIT 1.5 S
98 ;NOTE: FEED COMPLETE CHECK $IN[481] dipw1_FeedComplt IS DISABLED (F25)
99 ;CHECK: ON AND OFF BACK TO BACK IS A ZERO-LENGTH FEED PULSE; THE FEEDER MAY NEVER SEE IT (F25)
100 dopw1_StartFeed=TRUE
101 dopw1_StartFeed=FALSE
102 WAIT SEC 1.5
103 ;BLOW-OFF OFF - SIGNAL $OUT[494] dopw1_BlowOff OFF
104 dopw1_BlowOff=FALSE
105
106 ;QFP RETURN - SIGNALS $OUT[503] do503QFPAdvance OFF, $OUT[504] do504QFPReturn ON
107 do503QFPAdvance=FALSE
108 do504QFPReturn=TRUE
109 ;WAIT QFP RETURNED - SIGNAL $IN[482] dipw1_SpearHome ON
110 WAIT FOR dipw1_SpearHome
111 ENDIF
112
113 ;UPPER PIN EXTEND - SIGNALS $OUT[499] do499UpperPinRetract OFF, $OUT[498] do498UpperPinExtend ON
114 do499UpperPinRetract=FALSE
115 do498UpperPinExtend=TRUE
116
117 ;GUN GENERAL PRESSURE - SIGNALS $OUT[506..521] CL_GunPressureCmd = CL_GUN_PRESS_BASE (F44)

```

```

118 CL_GunPressureCmd=CL_GUN_PRESS_BASE
119
120 ;RESET BOSCH HANDSHAKE - SIGNALS $OUT[483] dopw1_WeldInit OFF, $OUT[481] dopw1_WeldContactEnable OFF,
121 ; $OUT[482] dopw1_WeldOnExternal OFF, $OUT[486..493] gdopw1_SpotNumber = 0
122 dopw1_WeldInit=FALSE
123 dopw1_WeldContactEnable=FALSE
124 dopw1_WeldOnExternal=FALSE
125 gdopw1_SpotNumber=0
126 ;RESET PEDESTAL OUTPUTS - SIGNALS $OUT[476] dopw1_StartFeed OFF, $OUT[494] dopw1_BlowOff OFF,
127 ; $OUT[474] dopw1_Nut1Intensify OFF (NOT ON THE BUS), $OUT[473] dopw1_Nut1Intensify_Home ON
128 dopw1_StartFeed=FALSE
129 dopw1_Nut1Intensify=FALSE
130 dopw1_Nut1Intensify_Home=TRUE
131 dopw1_BlowOff=FALSE
132 ;GUN GENERAL PRESSURE AGAIN - SIGNALS $OUT[506..521] CL_GunPressureCmd = CL_GUN_PRESS_BASE
133 CL_GunPressureCmd=CL_GUN_PRESS_BASE
134 ;ENABLE BOSCH WELD TIMER - SIGNALS $OUT[481] dopw1_WeldContactEnable ON,
135 ; $OUT[482] dopw1_WeldOnExternal ON
136 dopw1_WeldContactEnable=TRUE
137 dopw1_WeldOnExternal=TRUE
138
139 ;WAIT BOSCH READY - SIGNAL $IN[484] dipw1_Ready ON
140 WAIT FOR dipw1_Ready
141
142 ;PART CLAMP CLOSE - SIGNALS $OUT[502] do502PartClampOpen OFF, $OUT[501] do501PartClampClose ON
143 do502PartClampOpen=FALSE
144 do501PartClampClose=TRUE
145 ;WAIT PART CLAMP CLOSED - SIGNAL $IN[467] di467PartClampClosed ON
146 WAIT FOR di467PartClampClosed
147
148 ;GUN CLOSE - SIGNALS $OUT[471] dopw1_GunHome OFF, WAIT 0.2 S, $OUT[472] dopw1_GunWork ON
149 dopw1_GunHome=FALSE
150 WAIT SEC 0.2
151 dopw1_GunWork=TRUE
152
153 ;LOWER PIN BOTH VALVES OFF (OLD COMMENT: REMOVE AIR FROM THE LOWER PIN) -
154 ; SIGNALS $OUT[495] dopw1_AdvancePin OFF, $OUT[496] dopw1_ReturnPin OFF
155 dopw1_AdvancePin=FALSE
156 dopw1_ReturnPin=FALSE
157
158 ;CONFIRM GUN CLOSED: CL_GunStrokePositionLPT IN CL_GUN_WELD_MIN..CL_GUN_WELD_MAX WITHOUT A BREAK
159 ; FOR CL_GUN_IN_WINDOW_TIME MS ($TIMER[15]), WITHIN CL_GUN_CLOSE_TIMEOUT MS ($TIMER[14])
160 ;START BOTH TIMERS
161 $TIMER_STOP[14]=TRUE
162 $TIMER[14]=0
163 $TIMER_STOP[14]=FALSE
164 $TIMER_STOP[15]=TRUE
165 $TIMER[15]=0
166 $TIMER_STOP[15]=FALSE
167
168 LOOP
169 IF (CL_GunStrokePositionLPT>=CL_GUN_WELD_MIN) AND (CL_GunStrokePositionLPT<=CL_GUN_WELD_MAX) THEN
170 ;IN THE WINDOW LONG ENOUGH: GUN CLOSED ON THE NUT
171 IF $TIMER[15]>=CL_GUN_IN_WINDOW_TIME THEN
172 EXIT
173 ENDIF
174 ELSE

```

```

175 ;OUT OF THE WINDOW: RESTART THE TIME IN THE WINDOW
176 $TIMER_STOP[15]=TRUE
177 $TIMER[15]=0
178 $TIMER_STOP[15]=FALSE
179
180 ;TIMEOUT: NUT_READY=FALSE AND HALT
181 IF $TIMER[14]>=CL_GUN_CLOSE_TIMEOUT THEN
182     $TIMER_STOP[14]=TRUE
183     $TIMER[14]=0
184     $TIMER_STOP[15]=TRUE
185     $TIMER[15]=0
186     NUT_READY=FALSE
187 ;WARNING: HALT WITH NO MESSAGE IS STILL THE ONLY FAULT HANDLING - A DIALOG THAT NAMES THE
188 ; FAULT (NO NUT / DOUBLE NUT / GUN NOT CLOSED) WITH RETRY AND ABORT IS STILL TO DO (F07)
189 HALT
190 ;AFTER THE RESUME: RESTART BOTH TIMERS, SO THE CHECK TIMES OUT AGAIN IF THE GUN IS STILL
191 ; NOT CLOSED (F07)
192 $TIMER_STOP[14]=FALSE
193 $TIMER_STOP[15]=FALSE
194 ENDIF
195 ENDIF
196 ENDLOOP
197
198 ;GUN CLOSED: STOP AND CLEAR BOTH TIMERS
199 $TIMER_STOP[14]=TRUE
200 $TIMER[14]=0
201 $TIMER_STOP[15]=TRUE
202 $TIMER[15]=0
203
204 ;WELD ON EXTERNAL - SIGNAL $OUT[482] dopw1_WeldOnExternal ON (ALREADY ON SINCE THE BOSCH ENABLE)
205 dopw1_WeldOnExternal=TRUE
206
207 ;SELECT THE WELD PROGRAM - SIGNALS $OUT[486..493] gdopw1_SpotNumber = CL_WELD_PROGRAM (2)
208 ;CHECK: OLD COMMENT SAID SCHEDULE 7; THE CODE SELECTS PROGRAM 2 - CONFIRM ON THE BOSCH TIMER (F27)
209 gdopw1_SpotNumber=CL_WELD_PROGRAM
210
211 ;WAIT BOSCH READY - SIGNAL $IN[484] dipw1_Ready ON
212 WAIT FOR dipw1_Ready
213
214 ;INTENSIFY - GUN PRESSURE - SIGNALS $OUT[506..521] CL_GunPressureCmd = CL_GUN_PRESS_WELD (F44)
215 CL_GunPressureCmd=CL_GUN_PRESS_WELD
216
217 ;INTENSIFY - SIGNALS $OUT[494] dopw1_BlowOff ON, $OUT[473] dopw1_Nut1Intensify_Home OFF,
218 ; $OUT[474] dopw1_Nut1Intensify ON (NOT MAPPED TO THE BUS, NO EFFECT)
219 dopw1_BlowOff=TRUE
220 dopw1_Nut1Intensify_Home=FALSE
221 dopw1_Nut1Intensify=TRUE
222
223 ;WAIT INTENSIFY PRESSURE OK - SIGNAL $IN[479] dipw1_IntensfPresOk ON (NO TIMEOUT, F15)
224 WAIT FOR dipw1_IntensfPresOk
225
226 ;CONFIRM GUN POSITION BEFORE WELDING - WAIT CL_GunStrokePositionLPT IN CL_GUN_WELD_MIN..MAX
227 WAIT FOR (CL_GunStrokePositionLPT>=CL_GUN_WELD_MIN) AND (CL_GunStrokePositionLPT<=CL_GUN_WELD_MAX)
228
229 ;WELD - SIGNAL $OUT[483] dopw1_WeldInit ON, WAIT $IN[483] dipw1_WeldComplete ON
230 dopw1_WeldInit=TRUE
231

```

```

232 WAIT FOR dipw1_WeldComplete
233
234 ;WELD START OFF - SIGNAL $OUT[483] dopw1_WeldInit OFF, WAIT $IN[483] dipw1_WeldComplete OFF
235 dopw1_WeldInit=FALSE
236 WAIT FOR NOT dipw1_WeldComplete
237 ;NOTE: NO FAULT CHECK AFTER THE WELD - $IN[485] dipw1_Fault IS NOT READ BY ANY ACTIVE CODE
238 ; (ONLY BY THE UNUSED NUTWELD AND DISABLED AUTOMATIONCORE ROUTINES); WELD COMPLETE
239 ; IS TAKEN AS A GOOD WELD (F08)
240
241 ;RELEASE INTENSIFY - SIGNALS $OUT[474] dopw1_Nut1Intensify OFF,
242 ; $OUT[473] dopw1_Nut1Intensify_Home ON, $OUT[494] dopw1_BlowOff OFF
243 dopw1_Nut1Intensify=FALSE
244 dopw1_Nut1Intensify_Home=TRUE
245 dopw1_BlowOff=FALSE
246
247 ;GUN GENERAL PRESSURE - SIGNALS $OUT[506..521] CL_GunPressureCmd = CL_GUN_PRESS_BASE (F44)
248 CL_GunPressureCmd=CL_GUN_PRESS_BASE
249
250 ;CLEAR WELD PROGRAM NUMBER - SIGNALS $OUT[486..493] gdopw1_SpotNumber = 0
251 gdopw1_SpotNumber=0
252
253 ;STOP THE GUN-CLOSED TIMER $TIMER[14]
254 $TIMER_STOP[14]=TRUE
255 $TIMER[14]=0
256
257 ;GUN OPEN - SIGNALS $OUT[472] dopw1_GunWork OFF, WAIT 0.2 S, $OUT[471] dopw1_GunHome ON
258 dopw1_GunWork=FALSE
259 WAIT SEC 0.2
260 dopw1_GunHome=TRUE
261
262 ;GUN OPEN INTERLOCK BEFORE THE PART IS RELEASED AND THE ROBOT LEAVES - GUN_OPEN_CHECK: SUBMIT RUNNING,
263 ;WATER OK, CL_GunStrokePositionLPT IN 137000..142000 (MESSAGE AFTER 2 S, WAITS), THEN 0.2 S (F01)
264 GUN_OPEN_CHECK()
265 WAIT SEC 0.2
266
267 ;UPPER PIN RETRACT - SIGNALS $OUT[498] do498UpperPinExtend OFF, $OUT[499] do499UpperPinRetract ON
268 do498UpperPinExtend=FALSE
269 do499UpperPinRetract=TRUE
270
271 ;PART CLAMP OPEN - SIGNALS $OUT[501] do501PartClampClose OFF, $OUT[502] do502PartClampOpen ON
272 do501PartClampClose=FALSE
273 do502PartClampOpen=TRUE
274 ;WAIT PART CLAMP OPEN - SIGNAL $IN[466] di466PartClampOpen ON
275 WAIT FOR di466PartClampOpen
276
277 ;LOWER PIN ADVANCE - SIGNALS $OUT[496] dopw1_ReturnPin OFF, $OUT[495] dopw1_AdvancePin ON
278 ;CHECK: OLD COMMENT SAID LOWER PIN RETURN; THE CODE ADVANCES THE PIN HERE (F27)
279 dopw1_ReturnPin=FALSE
280 dopw1_AdvancePin=TRUE
281
282 ;QFP RETURN - SIGNALS $OUT[503] do503QFPAdvance OFF, $OUT[504] do504QFPReturn ON
283 do503QFPAdvance=FALSE
284 do504QFPReturn=TRUE
285 ;WAIT QFP RETURNED - SIGNAL $IN[482] dipw1_SpearHome ON
286 WAIT FOR dipw1_SpearHome
287
288 ;FINAL RESET - SIGNALS $OUT[476] dopw1_StartFeed OFF, $OUT[483] dopw1_WeldInit OFF,

```

```

289 ; $OUT[486..493] gdopw1_SpotNumber = 0
290 dopw1_StartFeed=FALSE
291 dopw1_WeldInit=FALSE
292 gdopw1_SpotNumber=0
293
294 ;CHECK: CL_GUN_PRESS_REST (39425) BELOW IS THE R20 BYTE-SWAPPED 0.09 MPa, NOT THE CL_GUN_PRESS_BASE
295 ;CHECK: USED ABOVE; NOT SWAPPED IT IS OUT OF THE ITV RANGE (0..4095). THE GUN RESTS AT THIS VALUE
296 ;CHECK: UNTIL THE NEXT WELD (F44)
297 ;INTENSIFY OFF - SIGNALS $OUT[474] dopw1_Nut1Intensify OFF,
298 ; $OUT[473] dopw1_Nut1Intensify_Home ON, $OUT[494] dopw1_BlowOff OFF
299 dopw1_Nut1Intensify=FALSE
300 dopw1_Nut1Intensify_Home=TRUE
301 dopw1_BlowOff=FALSE
302 ;REST PRESSURE - SIGNALS $OUT[506..521] CL_GunPressureCmd = CL_GUN_PRESS_REST (39425)
303 CL_GunPressureCmd=CL_GUN_PRESS_REST
304
305 ;BOSCH DISABLE - SIGNALS $OUT[481] dopw1_WeldContactEnable OFF, $OUT[482] dopw1_WeldOnExternal OFF
306 dopw1_WeldContactEnable=FALSE
307 dopw1_WeldOnExternal=FALSE
308
309 END

```

KRC/R1/Program/HOME.src

```

1 &ACCESS RV01
2 &REL 5
3 &COMMENT 03-10-R1 MOVE HOME
4 &PARAM EDITMASK = *
5 DEF HOME( )
6 ;FOLD INI
7 ;FOLD BASISTECH INI
8 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
9 INTERRUPT ON 3
10 BAS (#INITMOV,0 )
11 ;ENDFOLD (BASISTECH INI)
12 ;FOLD USER INI
13 ;Make your modifications here
14
15 ;ENDFOLD (USER INI)
16 ;ENDFOLD (INI)
17 ;*****
18 ; Gestamp Standards
19 ; Line - BMW G65 (cell 03)
20 ; Robot 03-10-R1 (BMW-03-10-R1)
21 ; Application - MOVE HOME
22 ;*****
23 ; DIRECT PTP MOVE TO HOME FROM THE CURRENT POSITION - NO VIA POINTS. P1 (home.dat) HAS THE
24 ; SAME AXIS VALUES AS XHOME ($config.dat); THE FORM IS A KSS 4.1.5 ONE (OLD BAS CALLS)
25 ;*****
26 ;NOTE: THE ONLY CALLER IS CELL, AFTER InitializeSystem, WHOSE HomeCheck HAS ALREADY BROUGHT
27 ;NOTE: THE ROBOT INTO THE HOME WINDOW ($IN_HOME)
28 ;FOLD PTP P1 Vel=100 % PDAT1 Tool[1]:TOOL 1 Base[0] ;%{...}
29 $BWDSTART=FALSE
30 PDAT_ACT=PPDAT1
31 BAS(#PTP_DAT)
32 FDAT_ACT=FP1
33 BAS(#FRAMES)
34 BAS(#VEL_PTP,100)

```

```

35 PTP XP1
36 ;ENDFOLD
37 END
38

```

KRC/R1/Program/StyleApps/Options/style1applopt1A.src

```

1 &ACCESS RV01
2 &REL 339
3 &COMMENT 03-10-R1 PNW1 NUT 1 ST1
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 &PARAM DISKPATH = KRC:\R1\Program\StyleApps\Options
7 DEF style1applopt1A( )
8     ;FOLD INI
9     ;FOLD BASISTECH INI
10     GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11     INTERRUPT ON 3
12     BAS (#INITMOV,0 )
13     ;ENDFOLD (BASISTECH INI)
14     ;FOLD NutWeld
15     ;ENDFOLD (NutWeld)
16     ;FOLD USER INI
17     ;Make your modifications here
18
19     ;ENDFOLD (USER INI)
20     ;ENDFOLD (INI)
21 ;*****
22 ; Gestamp Standards
23 ; Line - BMW G65 (cell 03)
24 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
25 ; Application - WELD NUT 1 (PN1) AT THE CENTERLINE PEDESTAL - PART FROM PICKUP 1 (A)
26 ;*****
27 ; CALLED BY Style10pt1. APPROACH THE PEDESTAL (BASE 3 "NUT WELDER"), WAIT FOR THE NUT PRELOAD
28 ; (NUT_READY), MOVE THE PART TO THE WELD POSITION P19, WELD WITH CENTERLINE_WELD (NOT IN DRY
29 ; CYCLE), LEAVE THE PEDESTAL.
30 ;WARNING: BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) -
31 ; NOT CLAMP OPEN OR QFP RETURNED (F01)
32 ;NOTE: THE PEDESTAL MOVES MIX BASE MODE AND EXTERNAL-TCP MODE (extTCP); THE WELD POSITION
33 ; P19 IS TAUGHT IN BASE MODE (F22)
34
35 ;APPROACH THE PEDESTAL - P0, P8
36 ;FOLD PTP P0 CONT Vel=100 % PDAT0 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
37 $BWDSTART=FALSE
38 PDAT_ACT=PPDAT0
39 FDAT_ACT=FP0
40 BAS(#PTP_PARAMS,100)
41 PTP XP0 C_DIS
42 ;ENDFOLD
43 ;FOLD LIN P8 CONT Vel=2 m/s CPDAT12 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
44 $BWDSTART=FALSE
45 LDAT_ACT=LCPDAT12
46 FDAT_ACT=FP8
47 BAS(#CP_PARAMS,2)
48 LIN XP8 C_DIS C_DIS
49 ;ENDFOLD
50 ;GUN MONITORING - INTERRUPTS 20-22 CALL GUN_OPEN_LOST (BRAKE F, QUIT MESSAGE) WHEN THE GUN LEAVES THE
51 ;OPEN RANGE (CL_GunStrokePositionLPT 137000..142000) OR THE SUBMIT STOPS WHILE THE ROBOT MOVES INTO

```

```

52 ;OR OUT OF THE PEDESTAL. EDGE TRIGGERED: GUN_OPEN_CHECK CONFIRMS THE GUN OPEN AFTER EVERY ON (F01)
53 ;WARNING: THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN
54 ; GUN_OPEN_CHECK.dat (F01)
55 INTERRUPT DECL 20 WHEN CL_GunStrokePositionLPT<137000 DO GUN_OPEN_LOST()
56 INTERRUPT DECL 21 WHEN CL_GunStrokePositionLPT>142000 DO GUN_OPEN_LOST()
57 INTERRUPT DECL 22 WHEN $PRO_STATE0<=>#P_ACTIVE DO GUN_OPEN_LOST()
58 ;WAIT NUT PRELOADED - NUT_READY (SET WHEN THE PRELOAD SEQUENCE ENDS: TIMED, NO NUT SENSOR, F13)
59 ;NOTE: NO TIMEOUT, NO MESSAGE (F15). IN DRY CYCLE NOTHING SETS IT: THE CYCLE HANGS HERE (F04)
60 ;NOTE: NUT_READY IS NOT CLEARED AFTER A WELD: IF THE PICK TRIGGER WAS SKIPPED, THE TRUE OF THE
61 ; LAST CYCLE LETS THE ROBOT IN WITH NO NUT FED (F14)
62 WAIT FOR NUT_READY== TRUE
63 ;GUN MONITORING ON FIRST (THE INTERRUPTS FIRE ONLY ON A CHANGE), THEN THE INTERLOCK: GUN OPEN,
64 ;SUBMIT RUNNING, WATER OK (GUN_OPEN_CHECK); THE MONITORING STAYS ON WHILE THE ROBOT MOVES IN (F01)
65 INTERRUPT ON 20
66 INTERRUPT ON 21
67 INTERRUPT ON 22
68 GUN_OPEN_CHECK()
69
70 ;MOVE IN - P20 (2 M/S), THEN THE WELD POSITION P19 OUTSIDE DRY CYCLE
71 ;FOLD LIN P20 CONT Vel=2 m/s CPDAT8 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
72 $BWDSTART=FALSE
73 LDAT_ACT=LCPDAT8
74 FDAT_ACT=FP20
75 BAS(#CP_PARAMS,2)
76 LIN XP20 C_DIS C_DIS
77 ;ENDFOLD
78
79 ;OUTSIDE DRY CYCLE ONLY (SIGNAL $IN[4] di004UseDryCycle OFF): WELD POSITION P19 AND CENTERLINE_WELD()
80 ;NOTE: $IN[4] IS RE-READ AT EVERY STEP; SWITCHED ON MID-CYCLE, THIS NUT IS NOT WELDED AND
81 ; THE PART CAN STILL REACH THE GOOD DROP (F05)
82 IF NOT di004UseDryCycle THEN
83 ;FOLD LIN P19 Vel=2 m/s CPDAT18 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
84 $BWDSTART=FALSE
85 LDAT_ACT=LCPDAT18
86 FDAT_ACT=FP19
87 BAS(#CP_PARAMS,2)
88 LIN XP19
89 ;ENDFOLD
90 ;ROBOT AT THE WELD POSITION (WAIT SEC 0 STOPS THE ADVANCE RUN): GUN MONITORING OFF, THE GUN
91 ;CLOSES ON PURPOSE IN CENTERLINE_WELD
92 WAIT SEC 0
93 INTERRUPT OFF 20
94 INTERRUPT OFF 21
95 INTERRUPT OFF 22
96 ;NUT WELD AT THE PEDESTAL (CENTERLINE_WELD: CLAMP, GUN, INTENSIFY, BOSCH WELD, GUN OPEN)
97 CENTERLINE_WELD()
98 ;GUN MONITORING ON WHILE LEAVING, THEN THE GUN OPEN IS CONFIRMED AGAIN (GUN_OPEN_CHECK): A GUN THAT
99 ;LEFT THE OPEN RANGE AFTER THE CHECK INSIDE CENTERLINE_WELD WOULD NOT TRIGGER THE INTERRUPTS (F01)
100 INTERRUPT ON 20
101 INTERRUPT ON 21
102 INTERRUPT ON 22
103 GUN_OPEN_CHECK()
104 ENDIF
105 ;LEAVE THE PEDESTAL - P20, P8, P0 (THE DRY CYCLE CONTINUES HERE)
106
107 ;FOLD LIN P20 CONT Vel=2 m/s CPDAT16 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
108 $BWDSTART=FALSE

```

```

109 LDAT_ACT=LCPDAT16
110 FDAT_ACT=FP20
111 BAS(#CP_PARAMS,2)
112 LIN XP20 C_DIS C_DIS
113 ;ENDFOLD
114 ;ROBOT OUT OF THE PEDESTAL (WAIT SEC 0 STOPS THE ADVANCE RUN, NO APPROXIMATION HERE): GUN
115 ;MONITORING OFF
116 WAIT SEC 0
117 INTERRUPT OFF 20
118 INTERRUPT OFF 21
119 INTERRUPT OFF 22
120
121 ;FOLD LIN P8 CONT Vel=2 m/s CPDAT17 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
122 $BWDSTART=FALSE
123 LDAT_ACT=LCPDAT17
124 FDAT_ACT=FP8
125 BAS(#CP_PARAMS,2)
126 LIN XP8 C_DIS C_DIS
127 ;ENDFOLD
128 ;NEXT NUT - TRIGGER ON THE MOTION TO P0 (DISTANCE = 0: FIRES WHEN THE MOTION STARTS, PRIO -1):
129 ;Request_next_nut() CLEARS NEXT_NUT_READY AND STARTS THE PRELOAD OF NUT 2 (NUT_START=TRUE)
130 ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP OF P0 CANNOT DELETE IT; IT ACTS ON
131 ; THE NEXT MOTION. A BLOCK SELECTION ONTO THE MOTION FOLD SKIPS IT (SELECT THE TRIGGER LINE),
132 ; AND SO DOES A PROGRAM RESET: NUT 2 THEN WAITS FOR EVER FOR NEXT_NUT_READY (F14, F38)
133 TRIGGER WHEN DISTANCE = 0 DELAY = 0 DO Request_next_nut() PRIO= -1
134 ;FOLD PTP P0 Vel=100 % PDAT18 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
135 $BWDSTART=FALSE
136 PDAT_ACT=PPDAT18
137 FDAT_ACT=FP0
138 BAS(#PTP_PARAMS,100)
139 PTP XP0
140 ;ENDFOLD
141 END

```

KRC/R1/Program/StyleApps/Options/style1applopt1B.src

```

1 &ACCESS RV01
2 &REL 339
3 &COMMENT 03-10-R1 PNW1 NUT 1 ST2
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 &PARAM DISKPATH = KRC:\R1\Program\StyleApps\Options
7 DEF style1applopt1B( )
8   ;FOLD INI
9   ;FOLD BASISTECH INI
10   GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11   INTERRUPT ON 3
12   BAS (#INITMOV,0 )
13   ;ENDFOLD (BASISTECH INI)
14   ;FOLD NutWeld
15   ;ENDFOLD (NutWeld)
16   ;FOLD USER INI
17   ;Make your modifications here
18
19   ;ENDFOLD (USER INI)
20   ;ENDFOLD (INI)
21 ;*****
22 ; Gestamp Standards

```

```

23 ; Line - BMW G65 (cell 03)
24 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
25 ; Application - WELD NUT 1 (PN1) AT THE CENTERLINE PEDESTAL - PART FROM PICKUP 2 (B)
26 ;*****
27 ; CALLED BY Style10pt1. APPROACH THE PEDESTAL (BASE 3 "NUT WELDER"), WAIT FOR THE NUT PRELOAD
28 ; (NUT_READY), MOVE THE PART TO THE WELD POSITION P19, WELD WITH CENTERLINE_WELD (NOT IN DRY
29 ; CYCLE), LEAVE THE PEDESTAL.
30 ;WARNING: BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) -
31 ; NOT CLAMP OPEN OR QFP RETURNED (F01)
32 ;NOTE: THE PEDESTAL MOVES MIX BASE MODE AND EXTERNAL-TCP MODE (extTCP); THE WELD POSITION
33 ; P19 IS TAUGHT IN BASE MODE (F22)
34
35 ;APPROACH THE PEDESTAL - P0, P8
36 ;FOLD PTP P0 CONT Vel=100 % PDAT0 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
37 $BWDSTART=FALSE
38 PDAT_ACT=PPDAT0
39 FDAT_ACT=FP0
40 BAS(#PTP_PARAMS,100)
41 PTP XP0 C_DIS
42 ;ENDFOLD
43 ;FOLD LIN P8 CONT Vel=2 m/s CPDAT12 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
44 $BWDSTART=FALSE
45 LDAT_ACT=LCPDAT12
46 FDAT_ACT=FP8
47 BAS(#CP_PARAMS,2)
48 LIN XP8 C_DIS C_DIS
49 ;ENDFOLD
50 ;GUN MONITORING - INTERRUPTS 20-22 CALL GUN_OPEN_LOST (BRAKE F, QUIT MESSAGE) WHEN THE GUN LEAVES THE
51 ;OPEN RANGE (CL_GunStrokePositionLPT 137000..142000) OR THE SUBMIT STOPS WHILE THE ROBOT MOVES INTO
52 ;OR OUT OF THE PEDESTAL. EDGE TRIGGERED: GUN_OPEN_CHECK CONFIRMS THE GUN OPEN AFTER EVERY ON (F01)
53 ;WARNING: THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN
54 ; GUN_OPEN_CHECK.dat (F01)
55 INTERRUPT DECL 20 WHEN CL_GunStrokePositionLPT<137000 DO GUN_OPEN_LOST()
56 INTERRUPT DECL 21 WHEN CL_GunStrokePositionLPT>142000 DO GUN_OPEN_LOST()
57 INTERRUPT DECL 22 WHEN $PRO_STATE0<>#P_ACTIVE DO GUN_OPEN_LOST()
58 ;WAIT NUT PRELOADED - NUT_READY (SET WHEN THE PRELOAD SEQUENCE ENDS: TIMED, NO NUT SENSOR, F13)
59 ;NOTE: NO TIMEOUT, NO MESSAGE (F15). IN DRY CYCLE NOTHING SETS IT: THE CYCLE HANGS HERE (F04)
60 ;NOTE: NUT_READY IS NOT CLEARED AFTER A WELD: IF THE PICK TRIGGER WAS SKIPPED, THE TRUE OF THE
61 ; LAST CYCLE LETS THE ROBOT IN WITH NO NUT FED (F14)
62 WAIT FOR NUT_READY== TRUE
63 ;GUN MONITORING ON FIRST (THE INTERRUPTS FIRE ONLY ON A CHANGE), THEN THE INTERLOCK: GUN OPEN,
64 ;SUBMIT RUNNING, WATER OK (GUN_OPEN_CHECK); THE MONITORING STAYS ON WHILE THE ROBOT MOVES IN (F01)
65 INTERRUPT ON 20
66 INTERRUPT ON 21
67 INTERRUPT ON 22
68 GUN_OPEN_CHECK()
69
70 ;MOVE IN - P20 (2 M/S), THEN THE WELD POSITION P19 OUTSIDE DRY CYCLE
71 ;FOLD LIN P20 CONT Vel=2 m/s CPDAT8 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
72 $BWDSTART=FALSE
73 LDAT_ACT=LCPDAT8
74 FDAT_ACT=FP20
75 BAS(#CP_PARAMS,2)
76 LIN XP20 C_DIS C_DIS
77 ;ENDFOLD
78
79 ;OUTSIDE DRY CYCLE ONLY (SIGNAL $IN[4] di004UseDryCycle OFF): WELD POSITION P19 AND CENTERLINE_WELD()

```

```

80 ;NOTE: $IN[4] IS RE-READ AT EVERY STEP; SWITCHED ON MID-CYCLE, THIS NUT IS NOT WELDED AND
81 ; THE PART CAN STILL REACH THE GOOD DROP (F05)
82 IF NOT di004UseDryCycle THEN
83 ;FOLD LIN P19 Vel=2 m/s CPDAT18 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
84 $BWDSTART=FALSE
85 LDAT_ACT=LCPDAT18
86 FDAT_ACT=FP19
87 BAS(#CP_PARAMS,2)
88 LIN XP19
89 ;ENDFOLD
90 ;ROBOT AT THE WELD POSITION (WAIT SEC 0 STOPS THE ADVANCE RUN): GUN MONITORING OFF, THE GUN
91 ;CLOSES ON PURPOSE IN CENTERLINE_WELD
92 WAIT SEC 0
93 INTERRUPT OFF 20
94 INTERRUPT OFF 21
95 INTERRUPT OFF 22
96 ;NUT WELD AT THE PEDESTAL (CENTERLINE_WELD: CLAMP, GUN, INTENSIFY, BOSCH WELD, GUN OPEN)
97 CENTERLINE_WELD()
98 ;GUN MONITORING ON WHILE LEAVING, THEN THE GUN OPEN IS CONFIRMED AGAIN (GUN_OPEN_CHECK): A GUN THAT
99 ;LEFT THE OPEN RANGE AFTER THE CHECK INSIDE CENTERLINE_WELD WOULD NOT TRIGGER THE INTERRUPTS (F01)
100 INTERRUPT ON 20
101 INTERRUPT ON 21
102 INTERRUPT ON 22
103 GUN_OPEN_CHECK()
104 ENDIF
105 ;LEAVE THE PEDESTAL - P20, P8, P0 (THE DRY CYCLE CONTINUES HERE)
106
107 ;FOLD LIN P20 CONT Vel=2 m/s CPDAT16 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
108 $BWDSTART=FALSE
109 LDAT_ACT=LCPDAT16
110 FDAT_ACT=FP20
111 BAS(#CP_PARAMS,2)
112 LIN XP20 C_DIS C_DIS
113 ;ENDFOLD
114 ;ROBOT OUT OF THE PEDESTAL (WAIT SEC 0 STOPS THE ADVANCE RUN, NO APPROXIMATION HERE): GUN
115 ;MONITORING OFF
116 WAIT SEC 0
117 INTERRUPT OFF 20
118 INTERRUPT OFF 21
119 INTERRUPT OFF 22
120
121 ;FOLD LIN P8 CONT Vel=2 m/s CPDAT17 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
122 $BWDSTART=FALSE
123 LDAT_ACT=LCPDAT17
124 FDAT_ACT=FP8
125 BAS(#CP_PARAMS,2)
126 LIN XP8 C_DIS C_DIS
127 ;ENDFOLD
128 ;NEXT NUT - TRIGGER ON THE MOTION TO P0 (DISTANCE = 0: FIRES WHEN THE MOTION STARTS, PRIO -1):
129 ;Request_next_nut() CLEARS NEXT_NUT_READY AND STARTS THE PRELOAD OF NUT 2 (NUT_START=TRUE)
130 ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP OF P0 CANNOT DELETE IT; IT ACTS ON
131 ; THE NEXT MOTION. A BLOCK SELECTION ONTO THE MOTION FOLD SKIPS IT (SELECT THE TRIGGER LINE),
132 ; AND SO DOES A PROGRAM RESET: NUT 2 THEN WAITS FOR EVER FOR NEXT_NUT_READY (F14, F38)
133 TRIGGER WHEN DISTANCE = 0 DELAY = 0 DO Request_next_nut() PRIO= -1
134 ;FOLD PTP P0 Vel=100 % PDAT18 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
135 $BWDSTART=FALSE
136 PDAT_ACT=PPDAT18

```

```

137 FDAT_ACT=FP0
138 BAS(#PTP_PARAMS,100)
139 PTP XP0
140 ;ENDFOLD
141 END

```

KRC/R1/Program/StyleApps/Options/style1app1opt2A.src

```

1 &ACCESS RV01
2 &REL 246
3 &COMMENT 03-10-R1 PNW1 NUT 2 ST1
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 &PARAM DISKPATH = KRC:\R1\Program\StyleApps\Options
7 DEF style1app1opt2A( )
8     ;FOLD INI
9     ;FOLD BASISTECH INI
10     GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11     INTERRUPT ON 3
12     BAS (#INITMOV,0 )
13     ;ENDFOLD (BASISTECH INI)
14     ;FOLD NutWeld
15     ;ENDFOLD (NutWeld)
16     ;FOLD USER INI
17     ;Make your modifications here
18
19     ;ENDFOLD (USER INI)
20     ;ENDFOLD (INI)
21     ;*****
22     ; Gestamp Standards
23     ; Line - BMW G65 (cell 03)
24     ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
25     ; Application - WELD NUT 2 (PN2) AT THE CENTERLINE PEDESTAL - PART FROM PICKUP 1 (A)
26     ;*****
27     ; CALLED BY Style1opt1. APPROACH THE PEDESTAL (BASE 3 "NUT WELDER"), WAIT FOR THE NUT PRELOAD
28     ; (NEXT_NUT_READY), MOVE THE PART TO THE WELD POSITION P16, WELD WITH CENTERLINE_WELD (NOT IN DRY
29     ; CYCLE), LEAVE THE PEDESTAL.
30     ;WARNING: BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) -
31     ; NOT CLAMP OPEN OR QFP RETURNED (F01)
32     ;NOTE: THE PEDESTAL MOVES MIX BASE MODE AND EXTERNAL-TCP MODE (extTCP); THE WELD POSITION
33     ; P16 IS TAUGHT IN BASE MODE (F22)
34     ;APPROACH THE PEDESTAL - P0, P5
35     ;FOLD PTP P0 CONT Vel=100 % PDAT0 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
36     $BWDSTART=FALSE
37     PDAT_ACT=PPDAT0
38     FDAT_ACT=FP0
39     BAS(#PTP_PARAMS,100)
40     PTP XP0 C_DIS
41     ;ENDFOLD
42     ;FOLD LIN P5 Vel=2 m/s CPDAT11 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
43     $BWDSTART=FALSE
44     LDAT_ACT=LCPDAT11
45     FDAT_ACT=FP5
46     BAS(#CP_PARAMS,2)
47     LIN XP5
48     ;ENDFOLD
49     ;GUN MONITORING - INTERRUPTS 20-22 CALL GUN_OPEN_LOST (BRAKE F, QUIT MESSAGE) WHEN THE GUN LEAVES THE
50     ;OPEN RANGE (CL_GunStrokePositionLPT 137000..142000) OR THE SUBMIT STOPS WHILE THE ROBOT MOVES INTO

```

```

51 ;OR OUT OF THE PEDESTAL. EDGE TRIGGERED: GUN_OPEN_CHECK CONFIRMS THE GUN OPEN AFTER EVERY ON (F01)
52 ;WARNING: THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN
53 ; GUN_OPEN_CHECK.dat (F01)
54 INTERRUPT DECL 20 WHEN CL_GunStrokePositionLPT<137000 DO GUN_OPEN_LOST()
55 INTERRUPT DECL 21 WHEN CL_GunStrokePositionLPT>142000 DO GUN_OPEN_LOST()
56 INTERRUPT DECL 22 WHEN $PRO_STATE0<>#P_ACTIVE DO GUN_OPEN_LOST()
57 ;WAIT NUT PRELOADED - NEXT_NUT_READY (SET WHEN THE PRELOAD SEQUENCE ENDS: TIMED, NO NUT SENSOR, F13)
58 ;NOTE: NO TIMEOUT, NO MESSAGE (F15). IN DRY CYCLE NOTHING SETS IT: THE CYCLE HANGS HERE (F04)
59 WAIT FOR NEXT_NUT_READY==TRUE
60 ;GUN MONITORING ON FIRST (THE INTERRUPTS FIRE ONLY ON A CHANGE), THEN THE INTERLOCK: GUN OPEN,
61 ;SUBMIT RUNNING, WATER OK (GUN_OPEN_CHECK); THE MONITORING STAYS ON WHILE THE ROBOT MOVES IN (F01)
62 INTERRUPT ON 20
63 INTERRUPT ON 21
64 INTERRUPT ON 22
65 GUN_OPEN_CHECK()
66 ;MOVE IN - P6 (2 M/S), THEN THE WELD POSITION P16 OUTSIDE DRY CYCLE
67 ;FOLD LIN P6 Vel=2 m/s CPDAT0 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
68 $BWDSTART=FALSE
69 LDAT_ACT=LCPDAT0
70 FDAT_ACT=FP6
71 BAS(#CP_PARAMS,2)
72 LIN XP6
73 ;ENDFOLD
74
75 ;OUTSIDE DRY CYCLE ONLY (SIGNAL $IN[4] di004UseDryCycle OFF): WELD POSITION P16 AND CENTERLINE_WELD()
76 ;NOTE: $IN[4] IS RE-READ AT EVERY STEP; SWITCHED ON MID-CYCLE, THIS NUT IS NOT WELDED AND
77 ; THE PART CAN STILL REACH THE GOOD DROP (F05)
78 IF NOT di004UseDryCycle THEN
79 ;FOLD LIN P16 Vel=2 m/s CPDAT15 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
80 $BWDSTART=FALSE
81 LDAT_ACT=LCPDAT15
82 FDAT_ACT=FP16
83 BAS(#CP_PARAMS,2)
84 LIN XP16
85 ;ENDFOLD
86 ;ROBOT AT THE WELD POSITION (WAIT SEC 0 STOPS THE ADVANCE RUN): GUN MONITORING OFF, THE GUN
87 ;CLOSES ON PURPOSE IN CENTERLINE_WELD
88 WAIT SEC 0
89 INTERRUPT OFF 20
90 INTERRUPT OFF 21
91 INTERRUPT OFF 22
92 ;NUT WELD AT THE PEDESTAL (CENTERLINE_WELD: CLAMP, GUN, INTENSIFY, BOSCH WELD, GUN OPEN)
93 CENTERLINE_WELD()
94 ;GUN MONITORING ON WHILE LEAVING, THEN THE GUN OPEN IS CONFIRMED AGAIN (GUN_OPEN_CHECK): A GUN THAT
95 ;LEFT THE OPEN RANGE AFTER THE CHECK INSIDE CENTERLINE_WELD WOULD NOT TRIGGER THE INTERRUPTS (F01)
96 INTERRUPT ON 20
97 INTERRUPT ON 21
98 INTERRUPT ON 22
99 GUN_OPEN_CHECK()
100 ENDIF
101 ;LEAVE THE PEDESTAL - P06, P05, P0 (THE DRY CYCLE CONTINUES HERE)
102 ;FOLD LIN P06 CONT Vel=2 m/s CPDAT6 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
103 $BWDSTART=FALSE
104 LDAT_ACT=LCPDAT6
105 FDAT_ACT=FP06
106 BAS(#CP_PARAMS,2)
107 LIN XP06 C_DIS C_DIS

```

```

108 ;ENDFOLD
109 ;ROBOT OUT OF THE PEDESTAL (WAIT SEC 0 STOPS THE ADVANCE RUN, NO APPROXIMATION HERE): GUN
110 ;MONITORING OFF
111 WAIT SEC 0
112 INTERRUPT OFF 20
113 INTERRUPT OFF 21
114 INTERRUPT OFF 22
115 ;FOLD LIN P05 Vel=2 m/s CPDAT14 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
116 $BWDSTART=FALSE
117 LDAT_ACT=LCPDAT14
118 FDAT_ACT=FP05
119 BAS(#CP_PARAMS,2)
120 LIN XP05
121 ;ENDFOLD
122 ;NEXT NUT - TRIGGER ON THE MOTION TO P0 (DISTANCE = 1: FIRES WHEN THE MOTION ENDS, PRIO -1):
123 ;Request_next_nut() CLEARS NEXT_NUT_READY AND STARTS THE PRELOAD OF NUT 3 (NUT_START=TRUE)
124 ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP OF P0 CANNOT DELETE IT; IT ACTS ON
125 ; THE NEXT MOTION. A BLOCK SELECTION ONTO THE MOTION FOLD SKIPS IT (SELECT THE TRIGGER LINE),
126 ; AND SO DOES A PROGRAM RESET: NUT 3 THEN WAITS FOR EVER FOR NEXT_NUT_READY (F14, F38)
127 TRIGGER WHEN DISTANCE = 1 DELAY = 0 DO Request_next_nut() PRIO= -1
128 ;FOLD PTP P0 Vel=100 % PDAT26 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
129 $BWDSTART=FALSE
130 PDAT_ACT=PPDAT26
131 FDAT_ACT=FP0
132 BAS(#PTP_PARAMS,100)
133 PTP XP0
134 ;ENDFOLD
135
136
137 END

```

KRC/R1/Program/StyleApps/Options/style1app1opt2B.src

```

1 &ACCESS RV01
2 &REL 246
3 &COMMENT 03-10-R1 PNW1 NUT 2 ST2
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 &PARAM DISKPATH = KRC:\R1\Program\StyleApps\Options
7 DEF style1app1opt2B( )
8   ;FOLD INI
9       ;FOLD BASISTECH INI
10         GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11         INTERRUPT ON 3
12         BAS (#INITMOV,0 )
13     ;ENDFOLD (BASISTECH INI)
14     ;FOLD NutWeld
15     ;ENDFOLD (NutWeld)
16     ;FOLD USER INI
17         ;Make your modifications here
18
19     ;ENDFOLD (USER INI)
20 ;ENDFOLD (INI)
21 ;*****
22 ; Gestamp Standards
23 ; Line - BMW G65 (cell 03)
24 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
25 ; Application - WELD NUT 2 (PN2) AT THE CENTERLINE PEDESTAL - PART FROM PICKUP 2 (B)

```

```

26 ;*****
27 ; CALLED BY Style10pt1. APPROACH THE PEDESTAL (BASE 3 "NUT WELDER"), WAIT FOR THE NUT PRELOAD
28 ; (NEXT_NUT_READY), MOVE THE PART TO THE WELD POSITION P16, WELD WITH CENTERLINE_WELD (NOT IN DRY
29 ; CYCLE), LEAVE THE PEDESTAL.
30 ;WARNING: BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) -
31 ; NOT CLAMP OPEN OR QFP RETURNED (F01)
32 ;NOTE: THE PEDESTAL MOVES MIX BASE MODE AND EXTERNAL-TCP MODE (extTCP); THE WELD POSITION
33 ; P16 IS TAUGHT IN BASE MODE (F22)
34 ;APPROACH THE PEDESTAL - P0, P5
35 ;FOLD PTP P0 CONT Vel=100 % PDAT0 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
36 $BWDSTART=FALSE
37 PDAT_ACT=PPDAT0
38 FDAT_ACT=FP0
39 BAS(#PTP_PARAMS,100)
40 PTP XP0 C_DIS
41 ;ENDFOLD
42 ;FOLD LIN P5 Vel=2 m/s CPDAT11 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
43 $BWDSTART=FALSE
44 LDAT_ACT=LCPDAT11
45 FDAT_ACT=FP5
46 BAS(#CP_PARAMS,2)
47 LIN XP5
48 ;ENDFOLD
49 ;GUN MONITORING - INTERRUPTS 20-22 CALL GUN_OPEN_LOST (BRAKE F, QUIT MESSAGE) WHEN THE GUN LEAVES THE
50 ;OPEN RANGE (CL_GunStrokePositionLPT 137000..142000) OR THE SUBMIT STOPS WHILE THE ROBOT MOVES INTO
51 ;OR OUT OF THE PEDESTAL. EDGE TRIGGERED: GUN_OPEN_CHECK CONFIRMS THE GUN OPEN AFTER EVERY ON (F01)
52 ;WARNING: THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN
53 ; GUN_OPEN_CHECK.dat (F01)
54 INTERRUPT DECL 20 WHEN CL_GunStrokePositionLPT<137000 DO GUN_OPEN_LOST()
55 INTERRUPT DECL 21 WHEN CL_GunStrokePositionLPT>142000 DO GUN_OPEN_LOST()
56 INTERRUPT DECL 22 WHEN $PRO_STATE0<>#P_ACTIVE DO GUN_OPEN_LOST()
57 ;WAIT NUT PRELOADED - NEXT_NUT_READY (SET WHEN THE PRELOAD SEQUENCE ENDS: TIMED, NO NUT SENSOR, F13)
58 ;NOTE: NO TIMEOUT, NO MESSAGE (F15). IN DRY CYCLE NOTHING SETS IT: THE CYCLE HANGS HERE (F04)
59 WAIT FOR NEXT_NUT_READY==TRUE
60 ;GUN MONITORING ON FIRST (THE INTERRUPTS FIRE ONLY ON A CHANGE), THEN THE INTERLOCK: GUN OPEN,
61 ;SUBMIT RUNNING, WATER OK (GUN_OPEN_CHECK); THE MONITORING STAYS ON WHILE THE ROBOT MOVES IN (F01)
62 INTERRUPT ON 20
63 INTERRUPT ON 21
64 INTERRUPT ON 22
65 GUN_OPEN_CHECK()
66 ;MOVE IN - P6 (2 M/S), THEN THE WELD POSITION P16 OUTSIDE DRY CYCLE
67 ;FOLD LIN P6 Vel=2 m/s CPDAT0 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
68 $BWDSTART=FALSE
69 LDAT_ACT=LCPDAT0
70 FDAT_ACT=FP6
71 BAS(#CP_PARAMS,2)
72 LIN XP6
73 ;ENDFOLD
74
75 ;OUTSIDE DRY CYCLE ONLY (SIGNAL $IN[4] di004UseDryCycle OFF): WELD POSITION P16 AND CENTERLINE_WELD()
76 ;NOTE: $IN[4] IS RE-READ AT EVERY STEP; SWITCHED ON MID-CYCLE, THIS NUT IS NOT WELDED AND
77 ; THE PART CAN STILL REACH THE GOOD DROP (F05)
78 IF NOT di004UseDryCycle THEN
79 ;FOLD LIN P16 Vel=2 m/s CPDAT15 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
80 $BWDSTART=FALSE
81 LDAT_ACT=LCPDAT15
82 FDAT_ACT=FP16

```

```

83 BAS(#CP_PARAMS,2)
84 LIN XP16
85 ;ENDFOLD
86 ;ROBOT AT THE WELD POSITION (WAIT SEC 0 STOPS THE ADVANCE RUN): GUN MONITORING OFF, THE GUN
87 ;CLOSES ON PURPOSE IN CENTERLINE_WELD
88 WAIT SEC 0
89 INTERRUPT OFF 20
90 INTERRUPT OFF 21
91 INTERRUPT OFF 22
92 ;NUT WELD AT THE PEDESTAL (CENTERLINE_WELD: CLAMP, GUN, INTENSIFY, BOSCH WELD, GUN OPEN)
93 CENTERLINE_WELD()
94 ;GUN MONITORING ON WHILE LEAVING, THEN THE GUN OPEN IS CONFIRMED AGAIN (GUN_OPEN_CHECK): A GUN THAT
95 ;LEFT THE OPEN RANGE AFTER THE CHECK INSIDE CENTERLINE_WELD WOULD NOT TRIGGER THE INTERRUPTS (F01)
96 INTERRUPT ON 20
97 INTERRUPT ON 21
98 INTERRUPT ON 22
99 GUN_OPEN_CHECK()
100 ENDIF
101 ;LEAVE THE PEDESTAL - P06, P05, P0 (THE DRY CYCLE CONTINUES HERE)
102 ;FOLD LIN P06 Vel=2 m/s CPDAT6 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
103 $BWDSTART=FALSE
104 LDAT_ACT=LCPDAT6
105 FDAT_ACT=FP06
106 BAS(#CP_PARAMS,2)
107 LIN XP06 C_DIS C_DIS
108 ;ENDFOLD
109 ;ROBOT OUT OF THE PEDESTAL (WAIT SEC 0 STOPS THE ADVANCE RUN, NO APPROXIMATION HERE): GUN
110 ;MONITORING OFF
111 WAIT SEC 0
112 INTERRUPT OFF 20
113 INTERRUPT OFF 21
114 INTERRUPT OFF 22
115 ;FOLD LIN P05 Vel=2 m/s CPDAT14 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
116 $BWDSTART=FALSE
117 LDAT_ACT=LCPDAT14
118 FDAT_ACT=FP05
119 BAS(#CP_PARAMS,2)
120 LIN XP05
121 ;ENDFOLD
122 ;NEXT NUT - TRIGGER ON THE MOTION TO P0 (DISTANCE = 1: FIRES WHEN THE MOTION ENDS, PRIO -1):
123 ;Request_next_nut() CLEARS NEXT_NUT_READY AND STARTS THE PRELOAD OF NUT 3 (NUT_START=TRUE)
124 ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP OF P0 CANNOT DELETE IT; IT ACTS ON
125 ; THE NEXT MOTION. A BLOCK SELECTION ONTO THE MOTION FOLD SKIPS IT (SELECT THE TRIGGER LINE),
126 ; AND SO DOES A PROGRAM RESET: NUT 3 THEN WAITS FOR EVER FOR NEXT_NUT_READY (F14, F38)
127 TRIGGER WHEN DISTANCE = 1 DELAY = 0 DO Request_next_nut() PRIO= -1
128 ;FOLD PTP P0 Vel=100 % PDAT26 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
129 $BWDSTART=FALSE
130 PDAT_ACT=PPDAT26
131 FDAT_ACT=FP0
132 BAS(#PTP_PARAMS,100)
133 PTP XP0
134 ;ENDFOLD
135
136
137 END

```

KRC/R1/Program/StyleApps/Options/style1app1opt3A.src

```

1 &ACCESS RV01
2 &REL 227
3 &COMMENT 03-10-R1 PNW1 NUT 3 ST1
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 &PARAM DISKPATH = KRC:\R1\Program\StyleApps\Options
7 DEF styleapplopt3A( )
8     ;FOLD INI
9     ;FOLD BASISTECH INI
10     GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11     INTERRUPT ON 3
12     BAS (#INITMOV,0 )
13     ;ENDFOLD (BASISTECH INI)
14     ;FOLD NutWeld
15     ;ENDFOLD (NutWeld)
16     ;FOLD USER INI
17     ;Make your modifications here
18
19     ;ENDFOLD (USER INI)
20     ;ENDFOLD (INI)
21 ;*****
22 ; Gestamp Standards
23 ; Line - BMW G65 (cell 03)
24 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
25 ; Application - WELD NUT 3 (PN3) AT THE CENTERLINE PEDESTAL - PART FROM PICKUP 1 (A)
26 ;*****
27 ; CALLED BY Style10pt1. APPROACH THE PEDESTAL (BASE 3 "NUT WELDER"), WAIT FOR THE NUT PRELOAD
28 ; (NEXT_NUT_READY), MOVE THE PART TO THE WELD POSITION P16, WELD WITH CENTERLINE_WELD (NOT IN DRY
29 ; CYCLE), LEAVE THE PEDESTAL.
30 ;WARNING: BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) -
31 ; NOT CLAMP OPEN OR QFP RETURNED (F01)
32 ;NOTE: THE PEDESTAL MOVES MIX BASE MODE AND EXTERNAL-TCP MODE (extTCP); THE WELD POSITION
33 ; P16 IS TAUGHT IN BASE MODE (F22)
34
35 ;APPROACH THE PEDESTAL - P20
36 ;FOLD LIN P20 CONT Vel=2 m/s CPDAT6 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
37 $BWDSTART=FALSE
38 LDAT_ACT=LCPDAT6
39 FDAT_ACT=FP20
40 BAS(#CP_PARAMS,2)
41 LIN XP20 C_DIS C_DIS
42 ;ENDFOLD
43 ;GUN MONITORING - INTERRUPTS 20-22 CALL GUN_OPEN_LOST (BRAKE F, QUIT MESSAGE) WHEN THE GUN LEAVES THE
44 ;OPEN RANGE (CL_GunStrokePositionLPT 137000..142000) OR THE SUBMIT STOPS WHILE THE ROBOT MOVES INTO
45 ;OR OUT OF THE PEDESTAL. EDGE TRIGGERED: GUN_OPEN_CHECK CONFIRMS THE GUN OPEN AFTER EVERY ON (F01)
46 ;WARNING: THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN
47 ; GUN_OPEN_CHECK.dat (F01)
48 INTERRUPT DECL 20 WHEN CL_GunStrokePositionLPT<137000 DO GUN_OPEN_LOST()
49 INTERRUPT DECL 21 WHEN CL_GunStrokePositionLPT>142000 DO GUN_OPEN_LOST()
50 INTERRUPT DECL 22 WHEN $PRO_STATE0<>#P_ACTIVE DO GUN_OPEN_LOST()
51 ;WAIT NUT PRELOADED - NEXT_NUT_READY (SET WHEN THE PRELOAD SEQUENCE ENDS: TIMED, NO NUT SENSOR, F13)
52 ;NOTE: NO TIMEOUT, NO MESSAGE (F15). IN DRY CYCLE NOTHING SETS IT: THE CYCLE HANGS HERE (F04)
53 WAIT FOR NEXT_NUT_READY==TRUE
54 ;GUN MONITORING ON FIRST (THE INTERRUPTS FIRE ONLY ON A CHANGE), THEN THE INTERLOCK: GUN OPEN,
55 ;SUBMIT RUNNING, WATER OK (GUN_OPEN_CHECK); THE MONITORING STAYS ON WHILE THE ROBOT MOVES IN (F01)
56 INTERRUPT ON 20
57 INTERRUPT ON 21

```

```

58 INTERRUPT ON 22
59 GUN_OPEN_CHECK()
60 ;MOVE IN - P13 (2 M/S), THEN THE WELD POSITION P16 OUTSIDE DRY CYCLE
61 ;FOLD LIN P13 CONT Vel=2 m/s CPDAT4 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
62 $BWDSTART=FALSE
63 LDAT_ACT=LCPDAT4
64 FDAT_ACT=FP13
65 BAS(#CP_PARAMS,2)
66 LIN XP13 C_DIS C_DIS
67 ;ENDFOLD
68
69 ;OUTSIDE DRY CYCLE ONLY (SIGNAL $IN[4] di004UseDryCycle OFF): WELD POSITION P16 AND CENTERLINE_WELD()
70 ;NOTE: $IN[4] IS RE-READ AT EVERY STEP; SWITCHED ON MID-CYCLE, THIS NUT IS NOT WELDED AND
71 ; THE PART CAN STILL REACH THE GOOD DROP (F05)
72 IF NOT di004UseDryCycle THEN
73 ;FOLD LIN P16 Vel=2 m/s CPDAT11 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
74 $BWDSTART=FALSE
75 LDAT_ACT=LCPDAT11
76 FDAT_ACT=FP16
77 BAS(#CP_PARAMS,2)
78 LIN XP16
79 ;ENDFOLD
80 ;ROBOT AT THE WELD POSITION (WAIT SEC 0 STOPS THE ADVANCE RUN): GUN MONITORING OFF, THE GUN
81 ;CLOSES ON PURPOSE IN CENTERLINE_WELD
82 WAIT SEC 0
83 INTERRUPT OFF 20
84 INTERRUPT OFF 21
85 INTERRUPT OFF 22
86 ;NUT WELD AT THE PEDESTAL (CENTERLINE_WELD: CLAMP, GUN, INTENSIFY, BOSCH WELD, GUN OPEN)
87 CENTERLINE_WELD()
88 ;GUN MONITORING ON WHILE LEAVING, THEN THE GUN OPEN IS CONFIRMED AGAIN (GUN_OPEN_CHECK): A GUN THAT
89 ;LEFT THE OPEN RANGE AFTER THE CHECK INSIDE CENTERLINE_WELD WOULD NOT TRIGGER THE INTERRUPTS (F01)
90 INTERRUPT ON 20
91 INTERRUPT ON 21
92 INTERRUPT ON 22
93 GUN_OPEN_CHECK()
94 ENDIF
95 ;LEAVE THE PEDESTAL - P17, P20, P7 (60 %) (THE DRY CYCLE CONTINUES HERE)
96 ;FOLD LIN P17 Vel=2 m/s CPDAT12 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
97 $BWDSTART=FALSE
98 LDAT_ACT=LCPDAT12
99 FDAT_ACT=FP17
100 BAS(#CP_PARAMS,2)
101 LIN XP17
102 ;ENDFOLD
103 ;ROBOT OUT OF THE PEDESTAL (WAIT SEC 0 STOPS THE ADVANCE RUN, NO APPROXIMATION HERE): GUN
104 ;MONITORING OFF
105 WAIT SEC 0
106 INTERRUPT OFF 20
107 INTERRUPT OFF 21
108 INTERRUPT OFF 22
109 ;FOLD LIN P20 Vel=2 m/s CPDAT10 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
110 $BWDSTART=FALSE
111 LDAT_ACT=LCPDAT10
112 FDAT_ACT=FP20
113 BAS(#CP_PARAMS,2)
114 LIN XP20

```

```

115 ;ENDFOLD
116 ;LAST NUT OF THE PART: NO NEXT NUT IS REQUESTED HERE (THE NEXT PICK STARTS THE PRELOAD)
117 ;FOLD PTP P7 Vel=60 % PDAT25 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
118 $BWDSTART=FALSE
119 PDAT_ACT=PPDAT25
120 FDAT_ACT=FP7
121 BAS(#PTP_PARAMS,60)
122 PTP XP7
123 ;ENDFOLD
124
125
126 END

```

KRC/R1/Program/StyleApps/Options/style1app1opt3B.src

```

1 &ACCESS RV01
2 &REL 227
3 &COMMENT 03-10-R1 PNW1 NUT 3 ST2
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 &PARAM DISKPATH = KRC:\R1\Program\StyleApps\Options
7 DEF style1app1opt3B( )
8     ;FOLD INI
9     ;FOLD BASISTECH INI
10     GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11     INTERRUPT ON 3
12     BAS (#INITMOV,0 )
13     ;ENDFOLD (BASISTECH INI)
14     ;FOLD NutWeld
15     ;ENDFOLD (NutWeld)
16     ;FOLD USER INI
17     ;Make your modifications here
18
19     ;ENDFOLD (USER INI)
20     ;ENDFOLD (INI)
21 ;*****
22 ; Gestamp Standards
23 ; Line - BMW G65 (cell 03)
24 ; Robot 03-10-R1 (BMW-03-10-R1-PNW1)
25 ; Application - WELD NUT 3 (PN3) AT THE CENTERLINE PEDESTAL - PART FROM PICKUP 2 (B)
26 ;*****
27 ; CALLED BY Style1opt1. APPROACH THE PEDESTAL (BASE 3 "NUT WELDER"), WAIT FOR THE NUT PRELOAD
28 ; (NEXT_NUT_READY), MOVE THE PART TO THE WELD POSITION P16, WELD WITH CENTERLINE_WELD (NOT IN DRY
29 ; CYCLE), LEAVE THE PEDESTAL.
30 ;WARNING: BEFORE THE ROBOT MOVES IN, GUN OPEN, SUBMIT AND WATER ARE CHECKED (GUN_OPEN_CHECK) -
31 ; NOT CLAMP OPEN OR QFP RETURNED (F01)
32 ;NOTE: THE PEDESTAL MOVES MIX BASE MODE AND EXTERNAL-TCP MODE (extTCP); THE WELD POSITION
33 ; P16 IS TAUGHT IN BASE MODE (F22)
34
35 ;APPROACH THE PEDESTAL - P20
36 ;FOLD LIN P20 CONT Vel=2 m/s CPDAT6 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
37 $BWDSTART=FALSE
38 LDAT_ACT=LCPDAT6
39 FDAT_ACT=FP20
40 BAS(#CP_PARAMS,2)
41 LIN XP20 C_DIS C_DIS
42 ;ENDFOLD
43 ;GUN MONITORING - INTERRUPTS 20-22 CALL GUN_OPEN_LOST (BRAKE F, QUIT MESSAGE) WHEN THE GUN LEAVES THE

```

```

44 ;OPEN RANGE (CL_GunStrokePositionLPT 137000..142000) OR THE SUBMIT STOPS WHILE THE ROBOT MOVES INTO
45 ;OR OUT OF THE PEDESTAL. EDGE TRIGGERED: GUN_OPEN_CHECK CONFIRMS THE GUN OPEN AFTER EVERY ON (F01)
46 ;WARNING: THE RANGE IS WRITTEN HERE AS LITERALS; KEEP IT EQUAL TO nGunOpenMin/nGunOpenMax IN
47 ; GUN_OPEN_CHECK.dat (F01)
48 INTERRUPT DECL 20 WHEN CL_GunStrokePositionLPT<137000 DO GUN_OPEN_LOST()
49 INTERRUPT DECL 21 WHEN CL_GunStrokePositionLPT>142000 DO GUN_OPEN_LOST()
50 INTERRUPT DECL 22 WHEN $PRO_STATE0<=>#P_ACTIVE DO GUN_OPEN_LOST()
51 ;WAIT NUT PRELOADED - NEXT_NUT_READY (SET WHEN THE PRELOAD SEQUENCE ENDS: TIMED, NO NUT SENSOR, F13)
52 ;NOTE: NO TIMEOUT, NO MESSAGE (F15). IN DRY CYCLE NOTHING SETS IT: THE CYCLE HANGS HERE (F04)
53 WAIT FOR NEXT_NUT_READY==TRUE
54 ;GUN MONITORING ON FIRST (THE INTERRUPTS FIRE ONLY ON A CHANGE), THEN THE INTERLOCK: GUN OPEN,
55 ;SUBMIT RUNNING, WATER OK (GUN_OPEN_CHECK); THE MONITORING STAYS ON WHILE THE ROBOT MOVES IN (F01)
56 INTERRUPT ON 20
57 INTERRUPT ON 21
58 INTERRUPT ON 22
59 GUN_OPEN_CHECK()
60 ;MOVE IN - P13 (2 M/S), THEN THE WELD POSITION P16 OUTSIDE DRY CYCLE
61 ;FOLD LIN P13 CONT Vel=2 m/s CPDAT4 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
62 $BWDSTART=FALSE
63 LDAT_ACT=LCPDAT4
64 FDAT_ACT=FP13
65 BAS(#CP_PARAMS,2)
66 LIN XP13 C_DIS C_DIS
67 ;ENDFOLD
68
69 ;OUTSIDE DRY CYCLE ONLY (SIGNAL $IN[4] di004UseDryCycle OFF): WELD POSITION P16 AND CENTERLINE_WELD()
70 ;NOTE: $IN[4] IS RE-READ AT EVERY STEP; SWITCHED ON MID-CYCLE, THIS NUT IS NOT WELDED AND
71 ; THE PART CAN STILL REACH THE GOOD DROP (F05)
72 IF NOT di004UseDryCycle THEN
73 ;FOLD LIN P16 Vel=2 m/s CPDAT11 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
74 $BWDSTART=FALSE
75 LDAT_ACT=LCPDAT11
76 FDAT_ACT=FP16
77 BAS(#CP_PARAMS,2)
78 LIN XP16
79 ;ENDFOLD
80 ;ROBOT AT THE WELD POSITION (WAIT SEC 0 STOPS THE ADVANCE RUN): GUN MONITORING OFF, THE GUN
81 ;CLOSES ON PURPOSE IN CENTERLINE_WELD
82 WAIT SEC 0
83 INTERRUPT OFF 20
84 INTERRUPT OFF 21
85 INTERRUPT OFF 22
86 ;NUT WELD AT THE PEDESTAL (CENTERLINE_WELD: CLAMP, GUN, INTENSIFY, BOSCH WELD, GUN OPEN)
87 CENTERLINE_WELD()
88 ;GUN MONITORING ON WHILE LEAVING, THEN THE GUN OPEN IS CONFIRMED AGAIN (GUN_OPEN_CHECK): A GUN THAT
89 ;LEFT THE OPEN RANGE AFTER THE CHECK INSIDE CENTERLINE_WELD WOULD NOT TRIGGER THE INTERRUPTS (F01)
90 INTERRUPT ON 20
91 INTERRUPT ON 21
92 INTERRUPT ON 22
93 GUN_OPEN_CHECK()
94 ENDIF
95 ;LEAVE THE PEDESTAL - P17, P20, P7 (60 %) (THE DRY CYCLE CONTINUES HERE)
96 ;FOLD LIN P17 Vel=2 m/s CPDAT12 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
97 $BWDSTART=FALSE
98 LDAT_ACT=LCPDAT12
99 FDAT_ACT=FP17
100 BAS(#CP_PARAMS,2)

```

```

101 LIN XP17
102 ;ENDFOLD
103 ;ROBOT OUT OF THE PEDESTAL (WAIT SEC 0 STOPS THE ADVANCE RUN, NO APPROXIMATION HERE): GUN
104 ;MONITORING OFF
105 WAIT SEC 0
106 INTERRUPT OFF 20
107 INTERRUPT OFF 21
108 INTERRUPT OFF 22
109 ;FOLD LIN P20 Vel=2 m/s CPDAT10 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
110 $BWDSTART=FALSE
111 LDAT_ACT=LCPDAT10
112 FDAT_ACT=FP20
113 BAS(#CP_PARAMS,2)
114 LIN XP20
115 ;ENDFOLD
116 ;LAST NUT OF THE PART: NO NEXT NUT IS REQUESTED HERE (THE NEXT PICK STARTS THE PRELOAD)
117 ;FOLD PTP P7 Vel=60 % PDAT25 Tool[1]:TOOL 1 Base[3]:NUT WELDER extTCP ;%{...}
118 $BWDSTART=FALSE
119 PDAT_ACT=PPDAT25
120 FDAT_ACT=FP7
121 BAS(#PTP_PARAMS,60)
122 PTP XP7
123 ;ENDFOLD
124
125
126 END

```

KRC/R1/Program/StyleApps/Options/style1app2opt1.src

```

1 &ACCESS RV01
2 &REL 19
3 &COMMENT 03-10-R1 NUT CHECK SENSOR
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 &PARAM DISKPATH = KRC:\R1\Program\StyleApps\Options
7 DEF style1app2opt1( )
8 ;FOLD INI
9 ;FOLD BASISTECH INI
10 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11 INTERRUPT ON 3
12 BAS (#INITMOV,0 )
13 ;ENDFOLD (BASISTECH INI)
14 ;FOLD NutWeld
15 ;ENDFOLD (NutWeld)
16 ;FOLD USER INI
17 ;Make your modifications here
18
19 ;ENDFOLD (USER INI)
20 ;ENDFOLD (INI)
21 ;*****
22 ; Gestamp Standards
23 ; Line - BMW G65 (cell 03)
24 ; Robot 03-10-R1 (BMW-03-10-R1-MH1)
25 ; Application - NUT PRESENCE CHECK OF THE 3 NUTS (SENSOR)
26 ;*****
27 ; CALLED BY Style10pt1 AND Style10pt10AutoRR. PRESENTS EACH WELDED NUT TO THE NUT-PRESENT
28 ; SENSOR $IN[227] di227NutPresent1 (P28, P32, P36) AND SETS PARTPRESENT1..3; THEN DECIDES:
29 ; OPTIONS 1, 2, 7, 8 -> bscrapGE4, OPTION 12 -> RED RABBIT RESULT ($OUT[141] / $OUT[142]).

```

```

30 ;NOTE: ANY OTHER OPTION HAS NO CASE: NOTHING IS DECIDED AND bscrapGE4 KEEPS ITS LAST VALUE (F02)
31 ;NOTE: APPLICATION 1 IS REQUESTED HERE AND RELEASED BY TRIGGER ON THE FIRST MOTION AWAY FROM THE
32 ; CAMERA (Style1Drop10pt1 P5, RejectGE4 P23, style1drop10pt2AutoRR P28)
33
34 ;APPLICATION 1 REQUEST - AC_ApplicationCheck(1): WAIT SIGNAL $IN[70] di070ApplicationMachine1 ON
35 ;(WAITING MESSAGE, NO TIMEOUT), THEN SIGNALS $OUT[70] do070ApplicationClear1 OFF,
36 ;$OUT[24] do024ApplClrTooling OFF
37 ;FOLD Application[1] Request
38 ;FOLD ;%{...}
39 ;Params IlfProvider=ac_zoneilf; AC_CmdZones=Application; AC_CmdParam=1; AC_ZoneRepo=False; AC_Blending=False
40 ;ENDFOLD
41 AC_ApplicationCheck (1)
42 ;ENDFOLD
43
44 ;APPROACH THE NUT CHECK SENSOR - P22
45 ;FOLD PTP P22 Vel=100 % PDAT16 Tool[1]:T00L 1 Base[0] ;%{...}
46 $BWDSTART=FALSE
47 PDAT_ACT=PPDAT16
48 FDAT_ACT=FP22
49 BAS(#PTP_PARAMS,100)
50 PTP XP22
51 ;ENDFOLD
52
53 ;NUT 1 AT THE SENSOR - LIN P28 (EXACT STOP)
54 ;FOLD LIN P28 Vel=2 m/s CPDAT15 Tool[1]:T00L 1 Base[0] ;%{...}
55 $BWDSTART=FALSE
56 LDAT_ACT=LCPDAT15
57 FDAT_ACT=FP28
58 BAS(#CP_PARAMS,2)
59 LIN XP28
60 ;ENDFOLD
61 ;FOLD NUT CHECK1
62 ;NUT 1 CHECK - OUTSIDE DRY CYCLE (SIGNAL $IN[4] di004UseDryCycle OFF), AFTER 0.2 S AT THE POINT
63 ;NOTE: THE WAIT SEC ALSO STOPS THE ADVANCE RUN: THE SENSOR IS READ WITH THE ROBOT STANDING
64 ; AT THE POINT, F28. A MISSING NUT WRITES FALSE (F03)
65 IF NOT di004UseDryCycle THEN
66 WAIT SEC 0.2
67 ;PARTPRESENT1 = SIGNAL $IN[227] di227NutPresent1 (TRUE OR FALSE)
68 PARTPRESENT1=di227NutPresent1
69 ENDIF
70 ;ENDFOLD
71 ;FOLD PTP P31 Vel=100 % PDAT35 Tool[1]:T00L 1 Base[0] ;%{...}
72 $BWDSTART=FALSE
73 PDAT_ACT=PPDAT35
74 FDAT_ACT=FP31
75 BAS(#PTP_PARAMS,100)
76 PTP XP31
77 ;ENDFOLD
78
79 ;NUT 2 AT THE SENSOR - LIN P32 (EXACT STOP)
80 ;FOLD LIN P32 Vel=2 m/s CPDAT16 Tool[1]:T00L 1 Base[0] ;%{...}
81 $BWDSTART=FALSE
82 LDAT_ACT=LCPDAT16
83 FDAT_ACT=FP32
84 BAS(#CP_PARAMS,2)
85 LIN XP32
86 ;ENDFOLD

```

```

87 ;FOLD NUT CHECK2
88 ;NUT 2 CHECK - OUTSIDE DRY CYCLE (SIGNAL $IN[4] di004UseDryCycle OFF), AFTER 0.2 S AT THE POINT
89 ;NOTE: THE WAIT SEC ALSO STOPS THE ADVANCE RUN: THE SENSOR IS READ WITH THE ROBOT STANDING
90 ; AT THE POINT, F28. A MISSING NUT WRITES FALSE (F03)
91 IF NOT di004UseDryCycle THEN
92     WAIT SEC 0.2
93     ;PARTPRESENT2 = SIGNAL $IN[227] di227NutPresent1 (TRUE OR FALSE)
94     PARTPRESENT2=di227NutPresent1
95 ENDIF
96 ;ENDFOLD
97 ;FOLD PTP P38 Vel=100 % PDAT36 Tool[1]:T00L 1 Base[0] ;%{...}
98 $BWDSTART=FALSE
99 PDAT_ACT=PPDAT36
100 FDAT_ACT=FP38
101 BAS(#PTP_PARAMS,100)
102 PTP XP38
103 ;ENDFOLD
104
105 ;NUT 3 AT THE SENSOR - LIN P36 (EXACT STOP)
106 ;FOLD LIN P36 Vel=2 m/s CPDAT17 Tool[1]:T00L 1 Base[0] ;%{...}
107 $BWDSTART=FALSE
108 LDAT_ACT=LCPDAT17
109 FDAT_ACT=FP36
110 BAS(#CP_PARAMS,2)
111 LIN XP36
112 ;ENDFOLD
113 ;FOLD NUT CHECK3
114 ;NUT 3 CHECK - OUTSIDE DRY CYCLE (SIGNAL $IN[4] di004UseDryCycle OFF), AFTER 0.2 S AT THE POINT
115 ;NOTE: THE WAIT SEC ALSO STOPS THE ADVANCE RUN: THE SENSOR IS READ WITH THE ROBOT STANDING
116 ; AT THE POINT, F28. A MISSING NUT WRITES FALSE (F03)
117 IF NOT di004UseDryCycle THEN
118     WAIT SEC 0.2
119     ;PARTPRESENT3 = SIGNAL $IN[227] di227NutPresent1 (TRUE OR FALSE)
120     PARTPRESENT3=di227NutPresent1
121 ENDIF
122 ;ENDFOLD
123 ;FOLD PTP P37 Vel=100 % PDAT37 Tool[1]:T00L 1 Base[0] ;%{...}
124 $BWDSTART=FALSE
125 PDAT_ACT=PPDAT37
126 FDAT_ACT=FP37
127 BAS(#PTP_PARAMS,100)
128 PTP XP37
129 ;ENDFOLD
130 ;DECISION - OUTSIDE DRY CYCLE, AFTER 0.2 S, BY OPTION nOption:
131 ;PRODUCTION OPTIONS 1, 2, 7, 8: ALL 3 NUTS PRESENT -> bscrapGE4 = FALSE (GOOD), ELSE TRUE (SCRAP) (F02)
132 ;RED RABBIT OPTION 12: ALL 3 PRESENT -> SIGNAL $OUT[141] do141RedRabbitFailed ON;
133 ;     NONE PRESENT -> SIGNAL $OUT[142] do142RedRabbitPassed ON
134 ;CHECK: THE CAMERA CHECK ALSO SETS $OUT[141]/$OUT[142]: BOTH CAN BE ON TOGETHER, AND THE RED RABBIT
135 ;CHECK: DROP CLEARS THEM BEFORE THE END OF THE CYCLE (F19)
136 ;FOLD NUT CHECK
137 ;OUTSIDE DRY CYCLE ONLY - SIGNAL $IN[4] di004UseDryCycle OFF
138 IF NOT di004UseDryCycle THEN
139     WAIT SEC 0.2
140     SWITCH nOption
141         CASE 1,2,7,8
142             IF (PARTPRESENT1==TRUE ) AND (PARTPRESENT2==TRUE) AND (PARTPRESENT3==TRUE) THEN
143                 bscrapGE4 = FALSE

```

```

144         ELSE
145             bscrapGE4=TRUE
146         ENDIF
147     CASE 12
148         IF (PARTPRESENT1==TRUE ) AND (PARTPRESENT2==TRUE) AND (PARTPRESENT3==TRUE) THEN
149             do141RedRabbitFailed=TRUE
150         ENDIF
151
152     IF (PARTPRESENT1==FALSE ) AND (PARTPRESENT2==FALSE) AND (PARTPRESENT3==FALSE) THEN
153
154         do142RedRabbitPassed=TRUE
155     ENDIF
156     ENDSWITCH
157
158 ENDIF
159
160
161 ;ENDFOLD
162
163
164 END

```

KRC/R1/Program/StyleApps/Options/style1app2opt2.src

```

1 &ACCESS RV01
2 &REL 16
3 &COMMENT 03-10-R1 NUT CHECK CAMERA
4 &PARAM EDITMASK = *
5 &PARAM DISKPATH = KRC:\R1\Program\StyleApps\Options
6 DEF Style1App2Opt2( )
7     ;FOLD INI
8     ;FOLD BASISTECH INI
9     GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
10    INTERRUPT ON 3
11    BAS (#INITMOV,0 )
12    ;ENDFOLD (BASISTECH INI)
13    ;FOLD USER INI
14    ;Make your modifications here
15
16    ;ENDFOLD (USER INI)
17    ;ENDFOLD (INI)
18    ;*****
19    ; Gestamp Standards
20    ; Line - BMW G65 (cell 03)
21    ; Robot 03-10-R1 (BMW-03-10-R1-MH1)
22    ; Application - NUT CHECK WITH THE CAMERA
23    ;*****
24    ; CALLED BY Style1Opt1 AND Style1Opt10AutoRR AFTER THE SENSOR CHECK (style1app2opt1).
25    ; PRESENTS THE PART TO THE CAMERA (P3), ASKS THE PLC FOR THE INSPECTION ($OUT[80]) AND SETS
26    ; bscrapGE4 FROM THE RESULT. APPLICATION 1 IS RELEASED BY THE NEXT PROGRAM, AWAY FROM THE CAMERA.
27    ;NOTE: THE CAMERA RESULT IS EVALUATED BY OPTION (PRODUCTION 1, 2, 7, 8; RED RABBIT 12), ALSO IN
28    ; DRY CYCLE (F02, F05, F19)
29    ;APPROACH THE CAMERA - P6 (TAUGHT IN BASE 3 "NUT WELDER")
30    ;FOLD PTP P6 Vel=100 % PDAT10 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
31    $BWDSTART=FALSE
32    PDAT_ACT=PPDAT10
33    FDAT_ACT=FP6
34    BAS (#PTP_PARAMS,100)

```

```

35 PTP XP6
36 ;ENDFOLD
37
38 ;PART AT THE CAMERA - LIN P3 (1 M/S, EXACT STOP)
39 ;FOLD LIN P3 Vel=1 m/s CPDAT0 Tool[1]:TOOL 1 Base[0] ;%{...}
40 $BWDSTART=FALSE
41 LDAT_ACT=LCPDAT0
42 FDAT_ACT=FP3
43 BAS(#CP_PARAMS,1)
44 LIN XP3
45 ;ENDFOLD
46
47 ;CAMERA INSPECTION REQUEST - SIGNAL $OUT[80] do080RepositionTooling1 ON
48 ;NOTE: NOTHING IN THE PROGRAM SWITCHES $OUT[80] OFF AGAIN (ONLY MaintainSystem AT THE NEXT CYCLE
49 ; START): THE PLC MAY SEE A STANDING REQUEST WHILE THE ROBOT LEAVES (F15)
50 do080RepositionTooling1=TRUE
51
52 ;WAIT INSPECTION DONE - SIGNAL $IN[80] di080ToolRepositioned1 ON (NO TIMEOUT, F15)
53     WAIT FOR di080ToolRepositioned1
54
55 ;AFTER 0.2 S, BY OPTION nOption (F02, F19):
56 ;PRODUCTION OPTIONS 1, 2, 7, 8: GOOD (bscrapGE4 = FALSE) ONLY WITH $IN[100] di100CameraJudgmentOK ON
57 ;AND $IN[101] di101CameraJudgmentNG OFF; NG OR NO VALID RESULT -> bscrapGE4 = TRUE (SCRAP)
58 ;RED RABBIT OPTION 12: CAMERA OK -> $OUT[141] do141RedRabbitFailed ON; CAMERA NG ->
59 ;$OUT[142] do142RedRabbitPassed ON
60     WAIT SEC 0.2
61     SWITCH nOption
62         CASE 1,2,7,8
63             ;GOOD - SIGNALS $IN[100] di100CameraJudgmentOK ON, $IN[101] di101CameraJudgmentNG OFF
64             IF (di100CameraJudgmentOK==TRUE) AND (di101CameraJudgmentNG==FALSE) THEN
65                 bscrapGE4 = FALSE
66             ELSE
67                 bscrapGE4 = TRUE
68             ENDIF
69         CASE 12
70             ;RED RABBIT FAILED - $IN[100] di100CameraJudgmentOK ON, $IN[101] di101CameraJudgmentNG OFF:
71             ;SIGNAL $OUT[141] do141RedRabbitFailed ON
72             IF (di100CameraJudgmentOK==TRUE) AND (di101CameraJudgmentNG==FALSE) THEN
73                 do141RedRabbitFailed=TRUE
74             ENDIF
75             ;RED RABBIT PASSED - $IN[100] di100CameraJudgmentOK OFF, $IN[101] di101CameraJudgmentNG ON:
76             ;SIGNAL $OUT[142] do142RedRabbitPassed ON
77             IF (di100CameraJudgmentOK==FALSE) AND (di101CameraJudgmentNG==TRUE) THEN
78                 do142RedRabbitPassed=TRUE
79             ENDIF
80     ENDSWITCH
81
82
83 ;NOTE: APPLICATION 1 IS NOT RELEASED HERE: THE ROBOT IS STILL AT THE CAMERA. THE NEXT PROGRAM
84 ; (Style1Drop1Opt1, RejectGE4 OR style1drop1opt2AutoRR) SETS $OUT[70] do070ApplicationClear1 ON
85 ; BY TRIGGER ON ITS FIRST MOTION AWAY FROM THE CAMERA (INDEPENDENT REVIEW 2026-10-05, F48, F20)
86 END
87

```

KRC/R1/Program/StyleDrops/Options/style1drop1opt1.src

```

1 &ACCESS RV01
2 &REL 222

```

```

3 &COMMENT 03-10-R1 DROP CONVEYOR
4 &PARAM EDITMASK = *
5 &PARAM DISKPATH = KRC:\R1\Program\StyleDrops\Options
6 DEF Style1Drop10pt1( )
7     ;FOLD INI
8     ;FOLD BASISTECH INI
9     GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
10    INTERRUPT ON 3
11    BAS (#INITMOV,0 )
12    ;ENDFOLD (BASISTECH INI)
13    ;FOLD USER INI
14    ;Make your modifications here
15
16    ;ENDFOLD (USER INI)
17    ;ENDFOLD (INI)
18    ;*****
19    ; Gestamp Standards
20    ; Line - BMW G65 (cell 03)
21    ; Robot 03-10-R1 (BMW-03-10-R1-MH1)
22    ; Application - DROP THE GOOD PART AT DROPOFF 1 (CONVEYOR)
23    ;*****
24    ; CALLED BY Style10pt1 WHEN BOTH NUT CHECKS PASSED. PLACES THE PART AT AtDrop3, OPENS
25    ; GRIPPER 1, WAITS FOR THE GRIPPER EMPTY, LEAVES AND RETURNS HOME.
26    ;NOTE: THE PART IN THE GRIPPER IS NOT CONFIRMED BEFORE OPENING (F40)
27    ;DROPOFF 1 REQUEST - AC_DropOffCheck(1): WAIT SIGNAL $IN[75] di075DropOffMachine1 ON
28    ;(WAITING MESSAGE, NO TIMEOUT), THEN SIGNALS $OUT[75] do075DropOffClear1 OFF,
29    ;$OUT[25] do025DropClearTooling OFF
30    ;WARNING: HIDDEN AC_CmdParam=2 - CONFIRMING THIS FORM REGENERATES AC_DropOffCheck(2) (F21)
31    ;FOLD DropOff[1] Request
32    ;FOLD    ;%{...}
33    ;Params IfProvider=ac_zoneilf; AC_CmdZones=DropOff; AC_CmdParam=2; AC_ZoneRepo=False; AC_Blending=False
34    ;ENDFOLD
35    AC_DropOffCheck (1)
36    ;ENDFOLD
37
38    ;APPROACH DROPOFF 1 - P5, P1, AtDrop3 (DROP POSITION)
39    ;APPLICATION 1 CLEAR WHEN THE ROBOT REACHES P5, ABOUT 1.5 M FROM THE CAMERA AND THE NUT CHECK -
40    ;SIGNAL $OUT[70] do070ApplicationClear1 ON (WAS AC_Application(1,TRUE) AT THE CAMERA POINT, F48, F20)
41    ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP OF P5 CANNOT DELETE IT
42    TRIGGER WHEN DISTANCE=1 DELAY=0 DO do070ApplicationClear1=TRUE
43    ;FOLD PTP P5 CONT Vel=100 % PDAT10 Tool[1]:T00L 1 Base[0] ;%{...}
44    $BWDSTART=FALSE
45    PDAT_ACT=PPDAT10
46    FDAT_ACT=FP5
47    BAS(#PTP_PARAMS,100)
48    PTP XP5 C_DIS
49    ;ENDFOLD
50    ;FOLD PTP P1 Vel=100 % PDAT1 Tool[1]:T00L 1 Base[0] ;%{...}
51    $BWDSTART=FALSE
52    PDAT_ACT=PPDAT1
53    FDAT_ACT=FP1
54    BAS(#PTP_PARAMS,100)
55    PTP XP1
56    ;ENDFOLD
57    ;FOLD LIN AtDrop3 Vel=2 m/s CPDAT1 Tool[1]:T00L 1 Base[0] ;%{...}
58    $BWDSTART=FALSE
59    LDAT_ACT=LCPDAT1

```

```

60 FDAT_ACT=FatDrop3
61 BAS(#CP_PARAMS,2)
62 LIN XAtDrop3
63 ;ENDFOLD
64
65 ;GRIPPER 1 OPEN - SIGNALS $OUT[249] do249Group1Home ON, $OUT[250] do250Group1Work OFF, WAIT 0.2 S,
66 ;THEN CHECK $IN[257] di257Cylinder1Home ON, $IN[258] di258Cylinder1Work OFF (STRATEGY 1)
67 ;NOTE: THE HIDDEN FORM PARAMETERS NOW MATCH THE CALL (WERE setgripper=4;setstate=2, F31)
68 ;FOLD Gripper SET [1]Gripper1 State=[1]OPEN Wait 0.2[s] Check with Strategy 1 ;%{...}
69 ;FOLD Parameters Parameters ;%{...}
70 ;Params IlfProvider=GripperTech.GripperSet;setgripper=1;setstate=1;setcont=nocont;waittime=0.2;errorstrategy=1
71 ;ENDFOLD
72 GRPg_SetStateAndCheck(1, 1, 0.2, 1)
73 ;ENDFOLD
74 ;WAIT GRIPPER EMPTY - SIGNALS $IN[253] di253PartPresent1 OFF, $IN[254] di254PartPresent2 OFF (NO TIMEOUT)
75 ;FOLD WAIT FOR ( NOT IN 253 'di253PartPresent1' ) AND ( NOT IN 254 'di254PartPresent2' ) ;%{...}
76 WAIT FOR ( NOT $IN[253] ) AND ( NOT $IN[254] )
77 ;ENDFOLD
78 ;PART NOT IN GRIPPER - SIGNAL $OUT[18] do018Part1InGripper OFF
79 ;FOLD OUT 18 'do018PartInGripper1' State=FALSE ;%{...}
80 $OUT[18]=FALSE
81 ;ENDFOLD
82
83 ;LEAVE DROPOFF 1 - P2, P7, P3
84 ;FOLD PTP P2 CONT Vel=100 % PDAT7 Tool[1]:TOOL 1 Base[0] ;%{...}
85 $BWDSTART=FALSE
86 PDAT_ACT=PPDAT7
87 FDAT_ACT=FP2
88 BAS(#PTP_PARAMS,100)
89 PTP XP2 C_DIS
90 ;ENDFOLD
91
92 ;FOLD PTP P7 CONT Vel=100 % PDAT13 Tool[1]:TOOL 1 Base[0] ;%{...}
93 $BWDSTART=FALSE
94 PDAT_ACT=PPDAT13
95 FDAT_ACT=FP7
96 BAS(#PTP_PARAMS,100)
97 PTP XP7 C_DIS
98 ;ENDFOLD
99 ;FOLD PTP P3 Vel=100 % PDAT2 Tool[1]:TOOL 1 Base[0] ;%{...}
100 $BWDSTART=FALSE
101 PDAT_ACT=PPDAT2
102 FDAT_ACT=FP3
103 BAS(#PTP_PARAMS,100)
104 PTP XP3
105 ;ENDFOLD
106
107 ;DROPOFF 1 CLEAR - AC_DropOff(1,TRUE): SIGNAL $OUT[75] do075DropOffClear1 ON
108 ;WARNING: HIDDEN AC_CmdParam=2 - CONFIRMING THIS FORM REGENERATES AC_DropOff(2,TRUE) (F21)
109 ;FOLD DropOff[1] Reset
110 ;FOLD ;%{...}
111 ;Params IlfProvider=ac_zoneilf; AC_CmdZones=DropOff; AC_CmdParam=2; AC_ZoneRepo=True; AC_Blending=False
112 ;ENDFOLD
113 AC_DropOff (1,True)
114 ;ENDFOLD
115
116 ;PTP HOME (XHOME)

```

```

117 ;FOLD PTP HOME Vel=100 % PDAT11 ;%{...}
118 $BWDSTART=FALSE
119 PDAT_ACT=PPDAT11
120 FDAT_ACT=FHOME
121 BAS(#PTP_PARAMS,100)
122 $H_POS=XHOME
123 PTP XHOME
124 ;ENDFOLD
125
126 END
127

```

KRC/R1/Program/StyleDrops/Options/style1drop1opt2AutoRR.src

```

1 &ACCESS RV01
2 &REL 269
3 &COMMENT 03-10-R1 RED RABBIT DROP
4 &PARAM EDITMASK = *
5 &PARAM DISKPATH = KRC:\R1\Program\StyleDrops\Options
6 DEF style1drop1opt2AutoRR( )
7     ;FOLD INI
8     ;FOLD BASISTECH INI
9     GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
10    INTERRUPT ON 3
11    BAS (#INITMOV,0 )
12    ;ENDFOLD (BASISTECH INI)
13    ;FOLD USER INI
14    ;Make your modifications here
15
16    ;ENDFOLD (USER INI)
17    ;ENDFOLD (INI)
18    ;*****
19    ; Gestamp Standards
20    ; Line - BMW G65 (cell 03)
21    ; Robot 03-10-R1 (BMW-03-10-R1-MH1)
22    ; Application - RED RABBIT DROP AT DROPOFF 2
23    ;*****
24    ; CALLED BY Style1opt10AutoRR (OPTION 12). TAKES THE RED RABBIT PART TO AtPick3 OF THIS MODULE
25    ; AND RELEASES IT, THEN CLEARS THE RED RABBIT OUTPUTS AND THE NUT FLAGS AND RETURNS HOME.
26    ;WARNING: THE RELEASE COMMAND USES GRIPPER 3. grp_data.dat MAPS GRIPPER 3 TO THE SAME I/O AS
27    ; GRIPPER 1 ($OUT[249]/[250], $IN[257]/[258]) AND GripperTech DOES NOT CHECK THE ACTIVE FLAG,
28    ; SO GRIPPER 1 OPENS. IT WORKS ONLY BECAUSE OF THAT DUPLICATE MAPPING (F19, F31)
29    ;NOTE: THE ROBOT MOVES AWAY (LIN P23) BEFORE THE GRIPPER-EMPTY WAIT (F40)
30    ;DROPOFF 2 REQUEST - AC_DropOffCheck(2): WAIT SIGNAL $IN[76] di076DropOffMachine2 ON
31    ;(WAITING MESSAGE, NO TIMEOUT), THEN SIGNALS $OUT[76] do076DropOffClear2 OFF,
32    ;$OUT[25] do025DropClearTooling OFF
33    ;FOLD DropOff[2] Request
34    ;FOLD    ;%{...}
35    ;Params IfProvider=ac_zoneilf; AC_CmdZones=DropOff; AC_CmdParam=2; AC_ZoneRepo=False; AC_Blending=False
36    ;ENDFOLD
37    AC_DropOffCheck (2)
38    ;ENDFOLD
39
40    ;GRIPPER 1 CLOSE (HAND-WRITTEN OUTPUTS, NO CHECK) - SIGNALS $OUT[249] do249Group1Home OFF,
41    ;$OUT[250] do250Group1Work ON (CONTINUE: SET IN THE ADVANCE RUN)
42    ;FOLD OUT 249 'do249Group1Home' State=FALSE CONT ;%{...}
43    CONTINUE
44    $OUT[249]=FALSE

```

```

45 ;ENDFOLD
46 ;FOLD OUT 250 'do250Group1Work' State=TRUE CONT ;%{...}
47 CONTINUE
48 $OUT[250]=TRUE
49 ;ENDFOLD
50
51 ;APPROACH THE RED RABBIT DROP - P27, P28, P32, P33, P34, P35, P36, P37, AtPick3
52 ;FOLD PTP P27 Vel=100 % PDAT25 Tool[1]:T00L 1 Base[0] ;%{...}
53 $BWDSTART=FALSE
54 PDAT_ACT=PPDAT25
55 FDAT_ACT=FP27
56 BAS(#PTP_PARAMS,100)
57 PTP XP27
58 ;ENDFOLD
59 ;APPLICATION 1 CLEAR WHEN THE ROBOT REACHES P28, ABOUT 0.6 M FROM THE CAMERA AND THE NUT CHECK -
60 ;SIGNAL $OUT[70] do070ApplicationClear1 ON (WAS AC_Application(1,TRUE) AT THE CAMERA POINT, F48, F20)
61 ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP OF P28 CANNOT DELETE IT
62 TRIGGER WHEN DISTANCE=1 DELAY=0 DO do070ApplicationClear1=TRUE
63 ;FOLD PTP P28 Vel=100 % PDAT26 Tool[1]:T00L 1 Base[0] ;%{...}
64 $BWDSTART=FALSE
65 PDAT_ACT=PPDAT26
66 FDAT_ACT=FP28
67 BAS(#PTP_PARAMS,100)
68 PTP XP28
69 ;ENDFOLD
70
71
72 ;FOLD PTP P32 CONT Vel=100 % PDAT30 Tool[1]:T00L 1 Base[0] ;%{...}
73 $BWDSTART=FALSE
74 PDAT_ACT=PPDAT30
75 FDAT_ACT=FP32
76 BAS(#PTP_PARAMS,100)
77 PTP XP32 C_DIS
78 ;ENDFOLD
79 ;FOLD PTP P33 Vel=100 % PDAT31 Tool[1]:T00L 1 Base[0] ;%{...}
80 $BWDSTART=FALSE
81 PDAT_ACT=PPDAT31
82 FDAT_ACT=FP33
83 BAS(#PTP_PARAMS,100)
84 PTP XP33
85 ;ENDFOLD
86
87
88 ;FOLD PTP P34 Vel=100 % PDAT32 Tool[1]:T00L 1 Base[0] ;%{...}
89 $BWDSTART=FALSE
90 PDAT_ACT=PPDAT32
91 FDAT_ACT=FP34
92 BAS(#PTP_PARAMS,100)
93 PTP XP34
94 ;ENDFOLD
95 ;FOLD LIN P35 CONT Vel=2 m/s CPDAT5 Tool[1]:T00L 1 Base[0] ;%{...}
96 $BWDSTART=FALSE
97 LDAT_ACT=LCPDAT5
98 FDAT_ACT=FP35
99 BAS(#CP_PARAMS,2)
100 LIN XP35 C_DIS C_DIS
101 ;ENDFOLD

```

```

102 ;FOLD PTP P36 CONT Vel=100 % PDAT33 Tool[1]:T00L 1 Base[0] ;%{...}
103 $BWDSTART=FALSE
104 PDAT_ACT=PPDAT33
105 FDAT_ACT=FP36
106 BAS(#PTP_PARAMS,100)
107 PTP XP36 C_DIS
108 ;ENDFOLD
109 ;FOLD PTP P37 CONT Vel=100 % PDAT34 Tool[1]:T00L 1 Base[0] ;%{...}
110 $BWDSTART=FALSE
111 PDAT_ACT=PPDAT34
112 FDAT_ACT=FP37
113 BAS(#PTP_PARAMS,100)
114 PTP XP37 C_DIS
115 ;ENDFOLD
116 ;FOLD LIN AtPick3 Vel=2 m/s CPDAT6 Tool[1]:T00L 1 Base[0] ;%{...}
117 $BWDSTART=FALSE
118 LDAT_ACT=LCPDAT6
119 FDAT_ACT=FAtpick3
120 BAS(#CP_PARAMS,2)
121 LIN XAtPick3
122 ;ENDFOLD
123 ;GRIPPER 3 OPEN = GRIPPER 1 OPEN - SIGNALS $OUT[249] do249Group1Home ON, $OUT[250] do250Group1Work OFF
124 ;WARNING: GRIPPER 3 SHARES GRIPPER 1'S I/O IN grp_data.dat; IF GRIPPER 3 IS EVER RECONFIGURED
125 ; THIS RELEASE STOPS WORKING (F19, F31)
126 ;FOLD Gripper SET [3]Gripper3 State=[1]OPEN Wait 0.2[s] Check with Strategy 1 ;%{...}
127 ;FOLD Parameters Parameters ;%{...}
128 ;Params IlfProvider=GripperTech.GripperSet;setgrripper=3;setstate=1;setcont=nocont;waittime=0.2;errorstrategy=1
129 ;ENDFOLD
130 GRPg_SetStateAndCheck(3, 1, 0.2, 1)
131 ;ENDFOLD
132 ;CHECK GRIPPER 1 OPEN - SIGNALS $IN[257] di257Cylinder1Home ON, $IN[258] di258Cylinder1Work OFF
133 ;(STRATEGY 1: DIALOG IN T1/T2, STOP IN AUT/EXT)
134 ;FOLD Gripper CHECK [1]Gripper 1 State=[1]OPEN with Strategy 1 ;%{...}
135 ;FOLD Parameters Parameters ;%{...}
136 ;Params IlfProvider=GripperTech.GripperCheck;setgrripper=1;setstate=1;setcont=nocont;checkstrategy=1
137 ;ENDFOLD
138 GRPg_Check(1, 1, FALSE, 1)
139 ;ENDFOLD
140 ;LEAVE THE DROP - P23 (BEFORE THE GRIPPER-EMPTY WAIT, F40)
141 ;FOLD LIN P23 Vel=2 m/s CPDAT2 Tool[1]:T00L 1 Base[0] ;%{...}
142 $BWDSTART=FALSE
143 LDAT_ACT=LCPDAT2
144 FDAT_ACT=FP23
145 BAS(#CP_PARAMS,2)
146 LIN XP23
147 ;ENDFOLD
148 ;WAIT GRIPPER EMPTY - SIGNALS $IN[253] di253PartPresent1 OFF, $IN[254] di254PartPresent2 OFF (NO TIMEOUT)
149 ;FOLD WAIT FOR ( NOT IN 253 'di253PartPresent1' ) AND ( NOT IN 254 'di254PartPresent2' ) ;%{...}
150 WAIT FOR ( NOT $IN[253] ) AND ( NOT $IN[254] )
151 ;ENDFOLD
152
153 ;LIN P24
154 ;FOLD LIN P24 Vel=2 m/s CPDAT4 Tool[1]:T00L 1 Base[0] ;%{...}
155 $BWDSTART=FALSE
156 LDAT_ACT=LCPDAT4
157 FDAT_ACT=FP24
158 BAS(#CP_PARAMS,2)

```

```

159 LIN XP24
160 ;ENDFOLD
161 ;PART NOT IN GRIPPER - SIGNAL $OUT[18] do018Part1InGripper OFF
162 ;FOLD OUT 18 'do018PartInGripper1' State=FALSE ;%{...}
163 $OUT[18]=FALSE
164 ;ENDFOLD
165 ;LEAVE - P21, P25
166 ;FOLD LIN P21 CONT Vel=2 m/s CPDAT7 Tool[1]:T00L 1 Base[0] ;%{...}
167 $BWDSTART=FALSE
168 LDAT_ACT=LCPDAT7
169 FDAT_ACT=FP21
170 BAS(#CP_PARAMS,2)
171 LIN XP21 C_DIS C_DIS
172 ;ENDFOLD
173 ;FOLD PTP P25 Vel=100 % PDAT23 Tool[1]:T00L 1 Base[0] ;%{...}
174 $BWDSTART=FALSE
175 PDAT_ACT=PPDAT23
176 FDAT_ACT=FP25
177 BAS(#PTP_PARAMS,100)
178 PTP XP25
179 ;ENDFOLD
180
181 ;DROP0FF 2 CLEAR - AC_DropOff(2,TRUE): SIGNAL $OUT[76] do076DropOffClear2 ON
182 ;FOLD DropOff[2] Reset
183 ;FOLD ;%{...}
184 ;Params IfProvider=ac_zoneilf; AC_CmdZones=DropOff; AC_CmdParam=2; AC_ZoneRepo=True; AC_Blending=False
185 ;ENDFOLD
186 AC_DropOff (2,True)
187 ;ENDFOLD
188 ;RED RABBIT OUTPUTS OFF - SIGNALS $OUT[141] do141RedRabbitFailed OFF, $OUT[142] do142RedRabbitPassed OFF -
189 ;AND PARTPRESENT1..3 = FALSE
190 ;NOTE: THESE RESETS USED TO SIT INSIDE THE INLINE FORM ABOVE, WHERE CONFIRMING THE FORM WOULD HAVE
191 ; DELETED THEM (F32). THE RESULT IS CLEARED BEFORE THE PLC CAN READ IT AT THE END OF THE CYCLE (F19)
192 do141RedRabbitFailed=FALSE
193 do142RedRabbitPassed=FALSE
194 PARTPRESENT1=FALSE
195 PARTPRESENT2=FALSE
196 PARTPRESENT3=FALSE
197
198
199 ;PTP HOME (XHOME)
200 ;FOLD PTP HOME Vel=100 % PDAT11 ;%{...}
201 $BWDSTART=FALSE
202 PDAT_ACT=PPDAT11
203 FDAT_ACT=FHOME
204 BAS(#PTP_PARAMS,100)
205 $H_POS=XHOME
206 PTP XHOME
207 ;ENDFOLD
208
209 END
210

```

KRC/R1/Program/StylePicks/Options/style1pick1opt1.src

```

1 &ACCESS RV01
2 &REL 52
3 &COMMENT 03-10-R1 PICK ST1

```

```

4 &PARAM EDITMASK = *
5 &PARAM DISKPATH = KRC:\R1\Program\StylePicks\Options
6 DEF stylepicklopt1( )
7     ;FOLD INI
8     ;FOLD BASISTECH INI
9     GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
10    INTERRUPT ON 3
11    BAS (#INITMOV,0 )
12    ;ENDFOLD (BASISTECH INI)
13    ;FOLD USER INI
14    ;Make your modifications here
15
16    ;ENDFOLD (USER INI)
17    ;ENDFOLD (INI)
18    ;*****
19    ; Gestamp Standards
20    ; Line - BMW G65 (cell 03)
21    ; Robot 03-10-R1 (BMW-03-10-R1-MH1)
22    ; Application - PICK THE PART AT PICKUP 1 (STATION 1) WITH EOAT 1
23    ;*****
24    ;NOTE: CALLED BY Style10pt1 WHEN $IN[65] di065PickupMachine1 ON, $IN[66] di066PickupMachine2 OFF
25    ;NOTE: THE WAIT FOR STATEMENTS BELOW HAVE NO TIMEOUT AND NO MESSAGE (F15)
26    ;PICKUP 1 REQUEST - AC_pickUPCheck(1): WAIT SIGNAL $IN[65] di065PickupMachine1 ON,
27    ;THEN SIGNALS $OUT[65] do065PickupClear1 OFF, $OUT[23] do023PickClearTooling OFF
28    ;FOLD PickUp[1] Request
29    ;FOLD    ;%{...}
30    ;Params IlfProvider=ac_zoneilf; AC_CmdZones=PickUp; AC_CmdParam=1; AC_ZoneRepo=False; AC_Blending=False
31    ;ENDFOLD
32    AC_pickUPCheck (1)
33    ;ENDFOLD
34    ;WAIT GRIPPER EMPTY BEFORE ANY GRIPPER COMMAND - SIGNALS $IN[253] di253PartPresent1 OFF,
35    ;$IN[254] di254PartPresent2 OFF (GESTAMP REVIEW POINT 14)
36    ;FOLD WAIT FOR ( NOT IN 253 'di253PartPresent1' ) AND ( NOT IN 254 'di254PartPresent2' ) ;%{...}
37    WAIT FOR ( NOT $IN[253] ) AND ( NOT $IN[254])
38    ;ENDFOLD
39    ;GRIPPER 1 OPEN - SIGNALS $OUT[249] do249Group1Home ON, $OUT[250] do250Group1Work OFF, WAIT 0.2 S,
40    ;THEN CHECK $IN[257] di257Cylinder1Home ON, $IN[258] di258Cylinder1Work OFF (STRATEGY 1)
41    ;NOTE: THE HIDDEN FORM PARAMETERS NOW MATCH THE CALL (WERE setgripper=4;setstate=2, F31)
42    ;FOLD Gripper SET [1]Gripper1 State=[1]OPEN Wait 0.2[s] Check with Strategy 1 ;%{...}
43    ;FOLD Parameters Parameters    ;%{...}
44    ;Params IlfProvider=GripperTech.GripperSet;setgripper=1;setstate=1;setcont=nocont;waittime=0.2;errorstrategy=1
45    ;ENDFOLD
46    GRPg_SetStateAndCheck(1, 1, 0.2, 1)
47    ;ENDFOLD
48
49    ;APPROACH PICKUP - P12
50    ;THEN P10, P20, P30AbovePick (ABOVE THE PART), AtPick3 (PICK POSITION)
51    ;NUT 1 - TRIGGER ON THE MOTION TO P12 (DISTANCE = 0: FIRES WHEN THE MOTION STARTS): NUT_START=TRUE
52    ;STARTS THE NUT PRELOAD (PRELOAD, CALLED FROM sps.sub) FOR THE FIRST NUT
53    ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP CANNOT DELETE IT; IT ACTS ON
54    ;NOTE: THE NEXT MOTION. A BLOCK SELECTION ONTO THE MOTION FOLD SKIPS IT (SELECT THE TRIGGER
55    ;NOTE: LINE): NO NUT 1 PRELOAD IS REQUESTED (F14)
56    ;NOTE: NUT_START IS SET ALSO IN DRY CYCLE AND WHEN STEPPING IN T1/T2 (F04, F11)
57    TRIGGER WHEN DISTANCE = 0 DELAY = 0 DO NUT_START=TRUE
58    ;FOLD PTP P12 Vel=100 % PDAT15 Tool[1]:TOOL 1 Base[0] ;%{...}
59    $BWDSTART=FALSE
60    PDAT_ACT=PPDAT15

```

```

61 FDAT_ACT=FP12
62 BAS(#PTP_PARAMS,100)
63 PTP XP12
64 ;ENDFOLD
65 ;FOLD LIN P10 CONT Vel=2 m/s CPDAT5 Tool[1]:TOOL 1 Base[0] ;%{...}
66 $BWDSTART=FALSE
67 LDAT_ACT=LCPDAT5
68 FDAT_ACT=FP10
69 BAS(#CP_PARAMS,2)
70 LIN XP10 C_DIS C_DIS
71 ;ENDFOLD
72 ;FOLD PTP P20 CONT Vel=100 % PDAT2 Tool[1]:TOOL 1 Base[0] ;%{...}
73 $BWDSTART=FALSE
74 PDAT_ACT=PPDAT2
75 FDAT_ACT=FP20
76 BAS(#PTP_PARAMS,100)
77 PTP XP20 C_DIS
78 ;ENDFOLD
79 ;FOLD PTP P30AbovePick CONT Vel=100 % PDAT4 Tool[1]:TOOL 1 Base[0] ;%{...}
80 $BWDSTART=FALSE
81 PDAT_ACT=PPDAT4
82 FDAT_ACT=FP30AbovePick
83 BAS(#PTP_PARAMS,100)
84 PTP XP30AbovePick C_DIS
85 ;ENDFOLD
86 ;NOTE: THE FOLD TITLE SAYS AtPick1; THE FORM DATA AND THE CODE MOVE TO AtPick3
87 ;FOLD LIN AtPick1 Vel=2 m/s CPDAT1 Tool[1]:TOOL 1 Base[0] ;%{...}
88 $BWDSTART=FALSE
89 LDAT_ACT=LCPDAT1
90 FDAT_ACT=FAAtPick3
91 BAS(#CP_PARAMS,2)
92 LIN XAtPick3
93 ;ENDFOLD
94
95 ;GRIPPER 1 CLOSE - SIGNALS $OUT[249] do249Group1Home OFF, $OUT[250] do250Group1Work ON, WAIT 0.2 S,
96 ;THEN CHECK $IN[257] di257Cylinder1Home OFF, $IN[258] di258Cylinder1Work ON (STRATEGY 1)
97 ;NOTE: THE HIDDEN FORM PARAMETERS NOW MATCH THE CALL (WERE setgripper=4;setstate=2, F31)
98 ;FOLD Gripper SET [1]Gripper1 State=[2]CLOSE Wait 0.2[s] Check with Strategy 1 ;%{...}
99 ;FOLD Parameters Parameters ;%{...}
100 ;Params IlfProvider=GripperTech.GripperSet;setgripper=1;setstate=2;setcont=nocont;waittime=0.2;errorstrategy=1
101 ;ENDFOLD
102 GRPg_SetStateAndCheck(1, 2, 0.2, 1)
103 ;ENDFOLD
104
105 ;OUTSIDE DRY CYCLE ONLY (SIGNAL $IN[4] di004UseDryCycle OFF): PART-PRESENT WAIT AND
106 $OUT[18] do018Part1InGripper
107 ;NOTE: $IN[4] IS RE-READ HERE AND AT EVERY LATER STEP OF THE CYCLE, NOT LATCHED (F05)
108 IF NOT di004UseDryCycle THEN
109 ;WAIT PART IN GRIPPER - SIGNALS $IN[253] di253PartPresent1 ON, $IN[254] di254PartPresent2 ON
110 ;FOLD WAIT FOR ( IN 253 'di253PartPresent1' ) AND ( IN 254 'di254PartPresent2' ) ;%{...}
111 WAIT FOR ( $IN[253] ) AND ( $IN[254] )
112 ;ENDFOLD
113 ;PART IN GRIPPER - SIGNAL $OUT[18] do018Part1InGripper ON
114 ;NOTE: THE FORM TEXT do018PartInGripper1 IS NOT A DECLARED NAME (DECLARED: do018Part1InGripper).
115 ;NOTE: $OUT[18] IS SET HERE AND RESET AT THE DROP - IT DOES NOT FOLLOW THE SENSORS (GESTAMP POINT 9)
116 ;FOLD OUT 18 'do018PartInGripper1' State=TRUE ;%{...}
117 $OUT[18]=TRUE

```

```

118 ;ENDFOLD
119 ENDIF
120
121 ;LEAVE PICKUP 1 - P40 (ABOVE AtPick3), P9, P8
122 ;FOLD LIN P40 Vel=2 m/s CPDAT2 Tool[1]:T00L 1 Base[0] ;%{...}
123 $BWDSTART=FALSE
124 LDAT_ACT=LCPDAT2
125 FDAT_ACT=FP40
126 BAS(#CP_PARAMS,2)
127 LIN XP40
128 ;ENDFOLD
129 ;FOLD PTP P9 Vel=100 % PDAT13 Tool[1]:T00L 1 Base[0] ;%{...}
130 $BWDSTART=FALSE
131 PDAT_ACT=PPDAT13
132 FDAT_ACT=FP9
133 BAS(#PTP_PARAMS,100)
134 PTP XP9
135 ;ENDFOLD
136 ;FOLD PTP P8 Vel=100 % PDAT12 Tool[1]:T00L 1 Base[0] ;%{...}
137 $BWDSTART=FALSE
138 PDAT_ACT=PPDAT12
139 FDAT_ACT=FP8
140 BAS(#PTP_PARAMS,100)
141 PTP XP8
142 ;ENDFOLD
143
144 ;PICKUP 1 CLEAR - AC_pickUP(1,TRUE): SIGNAL $OUT[65] do065PickupClear1 ON
145 ;FOLD PickUp[1] Reset
146 ;FOLD ;%{...}
147 ;Params IlfProvider=ac_zoneilf; AC_CmdZones=PickUp; AC_CmdParam=1; AC_ZoneRepo=True; AC_Blending=False
148 ;ENDFOLD
149 AC_pickUP (1,True)
150 ;ENDFOLD
151 END

```

KRC/R1/Program/StylePicks/Options/style1pick1opt1AutoRR.src

```

1 &ACCESS RV01
2 &REL 61
3 &COMMENT 03-10-R1 RED RABBIT PICK
4 &PARAM EDITMASK = *
5 &PARAM DISKPATH = KRC:\R1\Program\StylePicks\Options
6 DEF style1pick1opt1AutoRR( )
7 ;FOLD INI
8 ;FOLD BASISTECH INI
9 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
10 INTERRUPT ON 3
11 BAS (#INITMOV,0 )
12 ;ENDFOLD (BASISTECH INI)
13 ;FOLD USER INI
14 ;Make your modifications here
15
16 ;ENDFOLD (USER INI)
17 ;ENDFOLD (INI)
18 ;*****
19 ; Gestamp Standards
20 ; Line - BMW G65 (cell 03)
21 ; Robot 03-10-R1 (BMW-03-10-R1-MH1)

```

```

22 ; Application - RED RABBIT PICK AT PICKUP 1 (STATION 1) WITH EOAT 1
23 ;*****
24 ;NOTE: CALLED BY Style10pt10AutoRR (OPTION 12, RED RABBIT); SAME PATH AS style1pick1opt1 PLUS P16
25 ;NOTE: IT STARTS A NUT PRELOAD (NUT_START AT P12) ALTHOUGH NO WELDING FOLLOWS (F14)
26 ;NOTE: THE WAIT FOR STATEMENTS BELOW HAVE NO TIMEOUT AND NO MESSAGE (F15)
27 ;PICKUP 1 REQUEST - AC_pickUPCheck(1): WAIT SIGNAL $IN[65] di065PickupMachine1 ON,
28 ;THEN SIGNALS $OUT[65] do065PickupClear1 OFF, $OUT[23] do023PickClearTooling OFF
29 ;FOLD PickUp[1] Request
30 ;FOLD ;%{...}
31 ;Params IlfProvider=ac_zoneilf; AC_CmdZones=PickUp; AC_CmdParam=1; AC_ZoneRepo=False; AC_Blending=False
32 ;ENDFOLD
33 AC_pickUPCheck (1)
34 ;ENDFOLD
35 ;WAIT GRIPPER EMPTY BEFORE ANY GRIPPER COMMAND - SIGNALS $IN[253] di253PartPresent1 OFF,
36 ;$IN[254] di254PartPresent2 OFF (GESTAMP REVIEW POINT 14)
37 ;FOLD WAIT FOR ( NOT IN 253 'di253PartPresent1' ) AND ( NOT IN 254 'di254PartPresent2' ) ;%{...}
38 WAIT FOR ( NOT $IN[253] ) AND ( NOT $IN[254] )
39 ;ENDFOLD
40 ;GRIPPER 1 OPEN - SIGNALS $OUT[249] do249Group1Home ON, $OUT[250] do250Group1Work OFF, WAIT 0.2 S,
41 ;THEN CHECK $IN[257] di257Cylinder1Home ON, $IN[258] di258Cylinder1Work OFF (STRATEGY 1)
42 ;NOTE: THE HIDDEN FORM PARAMETERS NOW MATCH THE CALL (WERE setgripper=4;setstate=2, F31)
43 ;FOLD Gripper SET [1]Gripper1 State=[1]OPEN Wait 0.2[s] Check with Strategy 1 ;%{...}
44 ;FOLD Parameters Parameters ;%{...}
45 ;Params IlfProvider=GripperTech.GripperSet;setgripper=1;setstate=1;setcont=nocont;waittime=0.2;errorstrategy=1
46 ;ENDFOLD
47 GRPg_SetStateAndCheck(1, 1, 0.2, 1)
48 ;ENDFOLD
49
50 ;APPROACH PICKUP - P12
51 ;THEN P10, P20, P30AbovePick (ABOVE THE PART), AtPick3 (PICK POSITION)
52 ;NUT 1 - TRIGGER ON THE MOTION TO P12 (DISTANCE = 0: FIRES WHEN THE MOTION STARTS): NUT_START=TRUE
53 ;STARTS THE NUT PRELOAD (PRELOAD, CALLED FROM sps.sub) FOR THE FIRST NUT
54 ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP CANNOT DELETE IT; IT ACTS ON
55 ;NOTE: THE NEXT MOTION. A BLOCK SELECTION ONTO THE MOTION FOLD SKIPS IT (SELECT THE TRIGGER
56 ;NOTE: LINE): NO NUT 1 PRELOAD IS REQUESTED (F14)
57 ;NOTE: NUT_START IS SET ALSO IN DRY CYCLE AND WHEN STEPPING IN T1/T2 (F04, F11)
58 TRIGGER WHEN DISTANCE = 0 DELAY = 0 DO NUT_START=TRUE
59 ;FOLD PTP P12 Vel=100 % PDAT15 Tool[1]:TOOL 1 Base[0] ;%{...}
60 $BWDSTART=FALSE
61 PDAT_ACT=PPDAT15
62 FDAT_ACT=FP12
63 BAS(#PTP_PARAMS,100)
64 PTP XP12
65 ;ENDFOLD
66 ;FOLD LIN P10 CONT Vel=2 m/s CPDAT5 Tool[1]:TOOL 1 Base[0] ;%{...}
67 $BWDSTART=FALSE
68 LDAT_ACT=LCPDAT5
69 FDAT_ACT=FP10
70 BAS(#CP_PARAMS,2)
71 LIN XP10 C_DIS C_DIS
72 ;ENDFOLD
73 ;FOLD PTP P20 CONT Vel=100 % PDAT2 Tool[1]:TOOL 1 Base[0] ;%{...}
74 $BWDSTART=FALSE
75 PDAT_ACT=PPDAT2
76 FDAT_ACT=FP20
77 BAS(#PTP_PARAMS,100)
78 PTP XP20 C_DIS

```

```

79 ;ENDFOLD
80 ;FOLD PTP P30AbovePick CONT Vel=100 % PDAT4 Tool[1]:TOOL 1 Base[0] ;%{...}
81 $BWDSTART=FALSE
82 PDAT_ACT=PPDAT4
83 FDAT_ACT=FP30AbovePick
84 BAS(#PTP_PARAMS,100)
85 PTP XP30AbovePick C_DIS
86 ;ENDFOLD
87 ;NOTE: THE FOLD TITLE SAYS AtPick1; THE FORM DATA AND THE CODE MOVE TO AtPick3
88 ;FOLD LIN AtPick1 Vel=2 m/s CPDAT1 Tool[1]:TOOL 1 Base[0] ;%{...}
89 $BWDSTART=FALSE
90 LDAT_ACT=LCPDAT1
91 FDAT_ACT=FAAtPick3
92 BAS(#CP_PARAMS,2)
93 LIN XAtPick3
94 ;ENDFOLD
95
96 ;GRIPPER 1 CLOSE - SIGNALS $OUT[249] do249Group1Home OFF, $OUT[250] do250Group1Work ON, WAIT 0.2 S,
97 ;THEN CHECK $IN[257] di257Cylinder1Home OFF, $IN[258] di258Cylinder1Work ON (STRATEGY 1)
98 ;NOTE: THE HIDDEN FORM PARAMETERS NOW MATCH THE CALL (WERE setgripper=4;setstate=2, F31)
99 ;FOLD Gripper SET [1]Gripper1 State=[2]CLOSE Wait 0.2[s] Check with Strategy 1 ;%{...}
100 ;FOLD Parameters Parameters ;%{...}
101 ;Params IlfProvider=GripperTech.GripperSet;setgripper=1;setstate=2;setcont=nocont;waittime=0.2;errorstrategy=1
102 ;ENDFOLD
103 GRPg_SetStateAndCheck(1, 2, 0.2, 1)
104 ;ENDFOLD
105
106 ;OUTSIDE DRY CYCLE ONLY (SIGNAL $IN[4] di004UseDryCycle OFF): PART-PRESENT WAIT AND
107 $OUT[18] do018Part1InGripper
108 ;NOTE: $IN[4] IS RE-READ HERE AND AT EVERY LATER STEP OF THE CYCLE, NOT LATCHED (F05)
109 IF NOT di004UseDryCycle THEN
110 ;WAIT PART IN GRIPPER - SIGNALS $IN[253] di253PartPresent1 ON, $IN[254] di254PartPresent2 ON
111 ;FOLD WAIT FOR ( IN 253 'di253PartPresent1' ) AND ( IN 254 'di254PartPresent2' ) ;%{...}
112 WAIT FOR ( $IN[253] ) AND ( $IN[254] )
113 ;ENDFOLD
114 ;PART IN GRIPPER - SIGNAL $OUT[18] do018Part1InGripper ON
115 ;NOTE: THE FORM TEXT do018PartInGripper1 IS NOT A DECLARED NAME (DECLARED: do018Part1InGripper).
116 ;NOTE: $OUT[18] IS SET HERE AND RESET AT THE DROP - IT DOES NOT FOLLOW THE SENSORS (GESTAMP POINT 9)
117 ;FOLD OUT 18 'do018PartInGripper1' State=TRUE ;%{...}
118 $OUT[18]=TRUE
119 ;ENDFOLD
120 ENDIF
121
122 ;LEAVE PICKUP 1 - P40 (ABOVE AtPick3), P9, P8, P16 (60 %)
123 ;FOLD LIN P40 Vel=2 m/s CPDAT2 Tool[1]:TOOL 1 Base[0] ;%{...}
124 $BWDSTART=FALSE
125 LDAT_ACT=LCPDAT2
126 FDAT_ACT=FP40
127 BAS(#CP_PARAMS,2)
128 LIN XP40
129 ;ENDFOLD
130 ;FOLD PTP P9 Vel=100 % PDAT13 Tool[1]:TOOL 1 Base[0] ;%{...}
131 $BWDSTART=FALSE
132 PDAT_ACT=PPDAT13
133 FDAT_ACT=FP9
134 BAS(#PTP_PARAMS,100)
135 PTP XP9

```

```

136 ;ENDFOLD
137 ;FOLD PTP P8 Vel=100 % PDAT12 Tool[1]:T00L 1 Base[0] ;%{...}
138 $BWDSTART=FALSE
139 PDAT_ACT=PPDAT12
140 FDAT_ACT=FP8
141 BAS(#PTP_PARAMS,100)
142 PTP XP8
143 ;ENDFOLD
144
145 ;FOLD PTP P16 Vel=60 % PDAT18 Tool[1]:T00L 1 Base[0] ;%{...}
146 $BWDSTART=FALSE
147 PDAT_ACT=PPDAT18
148 FDAT_ACT=FP16
149 BAS(#PTP_PARAMS,60)
150 PTP XP16
151 ;ENDFOLD
152 ;PICKUP 1 CLEAR - AC_pickUP(1,TRUE): SIGNAL $OUT[65] do065PickupClear1 ON
153 ;FOLD PickUp[1] Reset
154 ;FOLD ;%{...}
155 ;Params IlfProvider=ac_zoneilf; AC_CmdZones=PickUp; AC_CmdParam=1; AC_ZoneRepo=True; AC_Blending=False
156 ;ENDFOLD
157 AC_pickUP (1,True)
158 ;ENDFOLD
159 END

```

KRC/R1/Program/StylePicks/Options/style1pick1opt2.src

```

1 &ACCESS RV01
2 &REL 41
3 &COMMENT 03-10-R1 PICK ST2
4 &PARAM EDITMASK = *
5 &PARAM DISKPATH = KRC:\R1\Program\StylePicks\Options
6 DEF style1pick1opt2( )
7 ;FOLD INI
8 ;FOLD BASISTECH INI
9 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
10 INTERRUPT ON 3
11 BAS (#INITMOV,0 )
12 ;ENDFOLD (BASISTECH INI)
13 ;FOLD USER INI
14 ;Make your modifications here
15
16 ;ENDFOLD (USER INI)
17 ;ENDFOLD (INI)
18 ;*****
19 ; Gestamp Standards
20 ; Line - BMW G65 (cell 03)
21 ; Robot 03-10-R1 (BMW-03-10-R1-MH1)
22 ; Application - PICK THE PART AT PICKUP 2 (STATION 2) WITH EOAT 1
23 ;*****
24 ;NOTE: CALLED BY Style1opt1 WHEN $IN[66] di066PickupMachine2 ON, $IN[65] di065PickupMachine1 OFF
25 ;NOTE: THE WAIT FOR STATEMENTS BELOW HAVE NO TIMEOUT AND NO MESSAGE (F15)
26 ;PICKUP 2 REQUEST - AC_pickUPCheck(2): WAIT SIGNAL $IN[66] di066PickupMachine2 ON,
27 ;THEN SIGNALS $OUT[66] do066PickupClear2 OFF, $OUT[23] do023PickClearTooling OFF
28 ;WARNING: THE HIDDEN FORM PARAMETER IS AC_CmdParam=1 - OPENING AND CONFIRMING THIS FORM
29 ; REGENERATES AC_pickUPCheck(1) (PICKUP 1). DO NOT TOUCH UP THIS FOLD (F21)
30 ;FOLD PickUp[2] Request
31 ;FOLD ;%{...}

```

```

32      ;Params IlfProvider=ac_zoneilf; AC_CmdZones=PickUp; AC_CmdParam=1; AC_ZoneRepo=False; AC_Blending=False
33      ;ENDFOLD
34      AC_pickUPCheck (2)
35      ;ENDFOLD
36 ;WAIT GRIPPER EMPTY BEFORE ANY GRIPPER COMMAND - SIGNALS $IN[253] di253PartPresent1 OFF,
37 ;$IN[254] di254PartPresent2 OFF (GESTAMP REVIEW POINT 14)
38 ;FOLD WAIT FOR ( NOT IN 253 'di253PartPresent1' ) AND ( NOT IN 254 'di254PartPresent2' ) ;%{...}
39 WAIT FOR ( NOT $IN[253] ) AND ( NOT $IN[254])
40 ;ENDFOLD
41 ;GRIPPER 1 OPEN - SIGNALS $OUT[249] do249Group1Home ON, $OUT[250] do250Group1Work OFF, WAIT 0.2 S,
42 ;THEN CHECK $IN[257] di257Cylinder1Home ON, $IN[258] di258Cylinder1Work OFF (STRATEGY 1)
43 ;NOTE: THE HIDDEN FORM PARAMETERS NOW MATCH THE CALL (WERE setgripper=4;setstate=2, F31)
44 ;FOLD Gripper SET [1]Gripper1 State=[1]OPEN Wait 0.2[s] Check with Strategy 1 ;%{...}
45      ;FOLD Parameters Parameters ;%{...}
46      ;Params IlfProvider=GripperTech.GripperSet;setgripper=1;setstate=1;setcont=nocont;waittime=0.2;errorstrategy=1
47      ;ENDFOLD
48      GRPg_SetStateAndCheck(1, 1, 0.2, 1)
49      ;ENDFOLD
50
51 ;APPROACH PICKUP - P10
52 ;THEN P20, P30AbovePick (ABOVE THE PART), AtPick3 (PICK POSITION)
53 ;NUT 1 - TRIGGER ON THE MOTION TO P10 (DISTANCE = 0: FIRES WHEN THE MOTION STARTS): NUT_START=TRUE
54 ;STARTS THE NUT PRELOAD (PRELOAD, CALLED FROM sps.sub) FOR THE FIRST NUT
55 ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP CANNOT DELETE IT; IT ACTS ON
56 ;NOTE: THE NEXT MOTION. A BLOCK SELECTION ONTO THE MOTION FOLD SKIPS IT (SELECT THE TRIGGER
57 ;NOTE: LINE): NO NUT 1 PRELOAD IS REQUESTED (F14)
58 ;NOTE: NUT_START IS SET ALSO IN DRY CYCLE AND WHEN STEPPING IN T1/T2 (F04, F11)
59 TRIGGER WHEN DISTANCE = 0 DELAY = 0 DO NUT_START=TRUE
60 ;FOLD PTP P10 CONT Vel=100 % PDAT1 Tool[1]:TOOL 1 Base[0] ;%{...}
61      $BWDSTART=FALSE
62      PDAT_ACT=PPDAT1
63      FDAT_ACT=FP10
64      BAS(#PTP_PARAMS,100)
65      PTP XP10 C_DIS
66      ;ENDFOLD
67 ;FOLD PTP P20 CONT Vel=100 % PDAT2 Tool[1]:TOOL 1 Base[0] ;%{...}
68      $BWDSTART=FALSE
69      PDAT_ACT=PPDAT2
70      FDAT_ACT=FP20
71      BAS(#PTP_PARAMS,100)
72      PTP XP20 C_DIS
73      ;ENDFOLD
74 ;FOLD PTP P30AbovePick CONT Vel=100 % PDAT4 Tool[1]:TOOL 1 Base[0] ;%{...}
75      $BWDSTART=FALSE
76      PDAT_ACT=PPDAT4
77      FDAT_ACT=FP30AbovePick
78      BAS(#PTP_PARAMS,100)
79      PTP XP30AbovePick C_DIS
80      ;ENDFOLD
81 ;NOTE: THE FOLD TITLE SAYS AtPick1; THE FORM DATA AND THE CODE MOVE TO AtPick3 (OF THIS MODULE)
82 ;FOLD LIN AtPick1 Vel=2 m/s CPDAT1 Tool[1]:TOOL 1 Base[0] ;%{...}
83      $BWDSTART=FALSE
84      LDAT_ACT=LCPDAT1
85      FDAT_ACT=FAtpick3
86      BAS(#CP_PARAMS,2)
87      LIN XAtPick3
88      ;ENDFOLD

```

89

```
90 ;GRIPPER 1 CLOSE - SIGNALS $OUT[249] do249Group1Home OFF, $OUT[250] do250Group1Work ON, WAIT 0.2 S,
91 ;THEN CHECK $IN[257] di257Cylinder1Home OFF, $IN[258] di258Cylinder1Work ON (STRATEGY 1)
92 ;NOTE: THE HIDDEN FORM PARAMETERS NOW MATCH THE CALL (WERE setgripper=4;setstate=2, F31)
93 ;FOLD Gripper SET [1]Gripper1 State=[2]CLOSE Wait 0.2[s] Check with Strategy 1 ;%{...}
94 ;FOLD Parameters Parameters ;%{...}
95 ;Params IlfProvider=GripperTech.GripperSet;setgripper=1;setstate=2;setcont=nocont;waittime=0.2;errorstrategy=1
96 ;ENDFOLD
97 GRPg_SetStateAndCheck(1, 2, 0.2, 1)
98 ;ENDFOLD
99
100 ;OUTSIDE DRY CYCLE ONLY (SIGNAL $IN[4] di004UseDryCycle OFF): PART-PRESENT WAIT AND
101 ;$OUT[18] do018Part1InGripper
102 ;NOTE: $IN[4] IS RE-READ HERE AND AT EVERY LATER STEP OF THE CYCLE, NOT LATCHED (F05)
103 IF NOT di004UseDryCycle THEN
104 ;WAIT PART IN GRIPPER - SIGNALS $IN[253] di253PartPresent1 ON, $IN[254] di254PartPresent2 ON
105 ;FOLD WAIT FOR ( IN 253 'di253PartPresent1' ) AND ( IN 254 'di254PartPresent2' ) ;%{...}
106 WAIT FOR ( $IN[253] ) AND ( $IN[254] )
107 ;ENDFOLD
108 ;PART IN GRIPPER - SIGNAL $OUT[18] do018Part1InGripper ON
109 ;NOTE: THE FORM TEXT do018PartInGripper1 IS NOT A DECLARED NAME (DECLARED: do018Part1InGripper).
110 ;NOTE: $OUT[18] IS SET HERE AND RESET AT THE DROP - IT DOES NOT FOLLOW THE SENSORS (GESTAMP POINT 9)
111 ;FOLD OUT 18 'do018PartInGripper1' State=TRUE ;%{...}
112 $OUT[18]=TRUE
113 ;ENDFOLD
114 ENDIF
115
116 ;LEAVE PICKUP 2 - P40 (ABOVE AtPick3), P8
117
118 ;FOLD LIN P40 Vel=2 m/s CPDAT2 Tool[1]:T00L 1 Base[0] ;%{...}
119 $BWDSTART=FALSE
120 LDAT_ACT=LCPDAT2
121 FDAT_ACT=FP40
122 BAS(#CP_PARAMS,2)
123 LIN XP40
124 ;ENDFOLD
125 ;FOLD PTP P8 Vel=100 % PDAT12 Tool[1]:T00L 1 Base[0] ;%{...}
126 $BWDSTART=FALSE
127 PDAT_ACT=PPDAT12
128 FDAT_ACT=FP8
129 BAS(#PTP_PARAMS,100)
130 PTP XP8
131 ;ENDFOLD
132 ;PICKUP 2 CLEAR - AC_pickUP(2,TRUE): SIGNAL $OUT[66] do066PickupClear2 ON
133 ;WARNING: HIDDEN AC_CmdParam=1 - CONFIRMING THIS FORM REGENERATES AC_pickUP(1,TRUE) (F21)
134 ;FOLD Pickup[2] Reset
135 ;FOLD ;%{...}
136 ;Params IlfProvider=ac_zoneilf; AC_CmdZones=PickUp; AC_CmdParam=1; AC_ZoneRepo=True; AC_Blending=False
137 ;ENDFOLD
138 AC_pickUP (2,True)
139 ;ENDFOLD
140 END
```

KRC/R1/Program/Styles/Options/Style1Opt10AutoRR.src

```
1 &ACCESS RV01
2 &REL 3
3 &COMMENT 03-10-R1 RED RABBIT CYCLE
```

```

4 DEF Style10pt10AutoRR( )
5 ;*****
6 ; Gestamp Standards
7 ; Line - BMW G65 (cell 03)
8 ; Robot 03-10-R1 (BMW-03-10-R1)
9 ; Application - RED RABBIT TEST CYCLE (OPTION 12)
10 ;*****
11 ;WARNING: THE NUT CHECK AND THE CAMERA CHECK EACH SET $OUT[141] do141RedRabbitFailed OR $OUT[142]
12 ; do142RedRabbitPassed FOR OPTION 12 - BOTH CAN BE ON TOGETHER, AND THE DROP CLEARS THEM BEFORE
13 ; THE END OF THE CYCLE. THE DROP RELEASES THE PART WITH GRIPPER 3, WHICH SHARES GRIPPER 1'S I/O
14 ; (F19, F31)
15
16 ;PICK THE RED RABBIT TEST PART AT PICKUP 1 (STATION 1)
17 Style1Pick10pt1AutoRR()
18 ;NUT SENSOR CHECK OF THE 3 NUTS - SIGNAL $IN[227] di227NutPresent1 (PRODUCTION PROGRAM)
19 style1app2opt1()
20 ;NUT CAMERA CHECK - SIGNALS $IN[100] di100CameraJudgmentOK, $IN[101] di101CameraJudgmentNG
21 ;(PRODUCTION PROGRAM: OPTION 12 SETS THE RED RABBIT RESULT); THE RED RABBIT DROP RELEASES
22 ;APPLICATION 1 AT P28
23 style1app2opt2()
24 ;DROP THE PART AT DROPOFF 2 (style1drop1opt2AutoRR), THEN SIGNALS
25 ;$OUT[141] do141RedRabbitFailed OFF, $OUT[142] do142RedRabbitPassed OFF, PARTPRESENT1..3 = FALSE
26 style1drop1opt2AutoRR()
27
28 END

```

KRC/R1/Program/Styles/Options/style10pt1.src

```

1 &ACCESS RV01
2 &REL 40
3 &COMMENT 03-10-R1 PRODUCTION CYCLE
4 DEF Style10pt1 ( )
5 ;*****
6 ; Gestamp Standards
7 ; Line - BMW G65 (cell 03)
8 ; Robot 03-10-R1 (BMW-03-10-R1)
9 ; Application - STYLE 1 PRODUCTION CYCLE (OPTIONS 1/2/7/8)
10 ;*****
11 DECL INT nPickStation
12 ;NOTE: EXACTLY ONE OF $IN[65] di065PickupMachine1 / $IN[66] di066PickupMachine2 MUST BE ON.
13 ;NOTE: WITH BOTH ON OR BOTH OFF NOTHING RUNS AND THE CYCLE ENDS AS IF COMPLETE (EMPTY CYCLE) (F06)
14 ;NOTE: bscrapGE4 IS NOT RESET AT THE START OF THE CYCLE (F03); IT IS WRITTEN BY THE OPTION 1
15 ;NOTE: NUT SENSOR RESULT, THE CAMERA RESULT AND RejectGE4 (FALSE)
16 ;NOTE: ONE PATH PER CYCLE (F30): A REJECT ENDS THE CYCLE (NO CAMERA, NO DROP), A STATION 1
17 ;NOTE: CYCLE NEVER RUNS STATION 2, AND EVERY PATH ENDS AT THE CLEAR OF PARTPRESENT1..3
18
19 ;PICK STATION, READ ONCE AT THE START OF THE CYCLE: 1 OR 2, 0 - NO PICK, EMPTY CYCLE (F06)
20 nPickStation=0
21 ;STATION 1 - SIGNALS $IN[65] di065PickupMachine1 ON AND $IN[66] di066PickupMachine2 OFF
22 IF di065PickupMachine1 AND NOT di066PickupMachine2 THEN
23     nPickStation=1
24 ELSE
25     ;STATION 2 - SIGNALS $IN[66] di066PickupMachine2 ON AND $IN[65] di065PickupMachine1 OFF
26     IF di066PickupMachine2 AND NOT di065PickupMachine1 THEN
27         nPickStation=2
28     ENDIF
29 ENDIF
30

```

```

31 IF nPickStation<>0 THEN
32   ;PICK THE PART AT PICKUP 1 (STATION 1) OR PICKUP 2 (STATION 2)
33   IF nPickStation==1 THEN
34     style1pick1opt1()
35   ELSE
36     style1pick1opt2()
37   ENDIF
38   ;WELD NUTS 1-3 (PN1-PN3) AT THE CENTERLINE PEDESTAL; THE WELD POSITIONS ARE TAUGHT PER PICK
39   ;STATION, BECAUSE THE PART SITS DIFFERENTLY IN THE GRIPPER: A = PICKUP 1, B = PICKUP 2
40   IF nPickStation==1 THEN
41     style1app1opt1A()
42     Style1App10pt2A()
43     Style1App10pt3A()
44   ELSE
45     style1app1opt1B()
46     Style1App10pt2B()
47     Style1App10pt3B()
48   ENDIF
49   ;NUT PRESENCE CHECK OF THE 3 NUTS - SIGNAL $IN[227] di227NutPresent1;
50   ;SETS bscrapGE4 FOR OPTIONS 1, 2, 7 AND 8 (F02)
51   style1app2opt1()
52   IF bScrapGE4==TRUE THEN
53     ;SCRAP: REJECT THE PART (RejectGE4) - END OF THE CYCLE
54     ;NOTE: RejectGE4 RELEASES APPLICATION 1 WHEN IT LEAVES THE INSPECTION AREA (P23)
55     RejectGE4()
56   ELSE
57     ;NUT CAMERA CHECK - SIGNALS $IN[100] di100CameraJudgmentOK, $IN[101] di101CameraJudgmentNG;
58     ;SETS bscrapGE4 (OPTIONS 1, 2, 7, 8); APPLICATION 1 IS RELEASED BY THE NEXT PROGRAM
59     style1app2opt2()
60     IF bScrapGE4==TRUE THEN
61       ;SCRAP: REJECT THE PART (RejectGE4) - END OF THE CYCLE
62       RejectGE4()
63     ELSE
64       ;GOOD PART: DROP OFF AT DROPOFF 1 (Style1Drop10pt1)
65       Style1Drop10pt1()
66     ENDIF
67   ENDIF
68 ENDIF
69
70 ;END OF THE CYCLE - CLEAR THE NUT PRESENCE FLAGS PARTPRESENT1..3 (bscrapGE4 IS NOT CHANGED HERE)
71 PARTPRESENT1=FALSE
72 PARTPRESENT2=FALSE
73 PARTPRESENT3=FALSE
74
75 END

```

KRC/R1/Program/Styles/style_1.src

```

1 &ACCESS RV01
2 &REL 101
3 &COMMENT 03-10-R1 STYLE 1
4 DEF Style_1( )
5   ;FOLD INI ;%{...}
6   ;FOLD BASISTECH INI
7   GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
8   INTERRUPT ON 3
9   BAS (#INITMOV,0 )
10  ;ENDFOLD (BASISTECH INI)

```

```

11      ;FOLD USER INI
12      ;Make your modifications here
13
14      ;ENDFOLD (USER INI)
15      ;ENDFOLD (INI)
16      ;*****
17      ; Gestamp Standards
18      ; Line - BMW G65 (cell 03)
19      ; Robot 03-10-R1 (BMW-03-10-R1)
20      ; Application - STYLE 1: SELECT THE OPTION PROGRAM
21      ;*****
22      ;SELECT THE OPTION PROGRAM BY OPTION NUMBER nOption (gdiOption $IN[38..45])
23      SWITCH nOption
24      CASE 1
25          ;OPTION 1: PRODUCTION CYCLE (PICK, WELD 3 NUTS, NUT CHECKS, DROP OR REJECT)
26          Style10pt1 ( )
27      CASE 2
28          ;OPTION 2: SAME PRODUCTION CYCLE (Style10pt1)
29          ;NOTE: NUT SENSOR AND CAMERA EVALUATE OPTIONS 1, 2, 7 AND 8 THE SAME WAY (F02)
30          Style10pt1 ( )
31      CASE 7
32          ;OPTION 7: SAME PRODUCTION CYCLE (Style10pt1)
33          Style10pt1 ( )
34      CASE 8
35          ;OPTION 8: SAME PRODUCTION CYCLE (Style10pt1)
36          Style10pt1 ( )
37      CASE 12
38          ;OPTION 12: RED RABBIT TEST CYCLE (Style10pt10AutoRR)
39          ;NOTE: THE NUT CHECK AND THE CAMERA CHECK EVALUATE THE RED RABBIT FOR OPTION 12 (F19)
40          Style10pt10AutoRR ( )
41      DEFAULT
42          ;NOTE: ANY OTHER OPTION: WAIT FOR FALSE - THE ROBOT STOPS HERE IN CYCLE FOR EVER,
43          ;NOTE: WITH NO MESSAGE AND NO FAULT TO THE PLC (F06, C21)
44          WAIT FOR FALSE
45      ENDSWITCH
46
47
48
49 END
50

```

KRC/R1/Program/Utilities/RejectGE4.src

```

1  &ACCESS RV02
2  &REL 41
3  &COMMENT 03-10-R1 REJECT
4  &PARAM DISKPATH = KRC:\R1\Program\Utilities
5  DEF RejectGE4( )
6  ;FOLD INI ;%{...}
7  ;FOLD BASISTECH INI
8  GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
9  INTERRUPT ON 3
10 BAS (#INITMOV,0 )
11 ;ENDFOLD (BASISTECH INI)
12 ;FOLD USER INI
13 ;Make your modifications here
14
15 ;ENDFOLD (USER INI)

```

```

16 ;ENDFOLD (INI)
17 ;*****
18 ; Gestamp Standards
19 ; Line - BMW G65 (cell 03)
20 ; Robot 03-10-R1 (BMW-03-10-R1-MH1)
21 ; Application - REJECT THE PART (SCRAP) AFTER A FAILED NUT CHECK
22 ;*****
23 ; CALLED BY Style10pt1 WHEN bscrapGE4 IS TRUE AFTER THE SENSOR OR THE CAMERA CHECK.
24 ; SETS THE REJECT OUTPUT, PLACES THE PART AT THE REJECT LOCATION (P23), OPENS GRIPPER 1, GOES
25 ; HOME, CLEARS THE REJECT OUTPUT AND THE SCRAP AND NUT FLAGS.
26 ;NOTE: NO AutomationCore DROP-OFF REQUEST: NOTHING INTERLOCKS THE REJECT LOCATION WITH THE
27 ; PLC (F20). APPLICATION 1 IS RELEASED BY TRIGGER AT P23, ALSO AFTER A SENSOR SCRAP
28 ;PART REJECTED - SIGNAL $OUT[106] do106PartRejected ON
29 ;FOLD OUT 106 'do106PartRejeted' State=TRUE ;%{...}
30 $OUT[106]=TRUE
31 ;ENDFOLD
32 ;MOVE TO THE REJECT LOCATION - P10, P23
33 ;FOLD PTP P10 Vel=100 % L Tool[1]:TOOL 1 Base[0] ;%{...}
34 $BWDSTART=FALSE
35 PDAT_ACT=PL
36 FDAT_ACT=FP10
37 BAS(#PTP_PARAMS,100)
38 PTP XP10
39 ;ENDFOLD
40
41 ;APPLICATION 1 CLEAR WHEN THE ROBOT REACHES P23, ABOUT 1.25 M FROM THE CAMERA AND THE NUT CHECK -
42 ;SIGNAL $OUT[70] do070ApplicationClear1 ON (WAS AC_Application(1,TRUE) AT THE CAMERA POINT, F48, F20)
43 ;NOTE: THE TRIGGER STANDS BEFORE THE INLINE FORM, SO A TOUCH UP OF P23 CANNOT DELETE IT
44 TRIGGER WHEN DISTANCE=1 DELAY=0 DO do070ApplicationClear1=TRUE
45 ;FOLD LIN P23 Vel=2 m/s CPDAT0 Tool[1]:TOOL 1 Base[0] ;%{...}
46 $BWDSTART=FALSE
47 LDAT_ACT=LCPDAT0
48 FDAT_ACT=FP23
49 BAS(#CP_PARAMS,2)
50 LIN XP23
51 ;ENDFOLD
52
53 ;GRIPPER 1 OPEN - SIGNALS $OUT[249] do249Group1Home ON, $OUT[250] do250Group1Work OFF, WAIT 0.2 S,
54 ;THEN CHECK $IN[257] di257Cylinder1Home ON, $IN[258] di258Cylinder1Work OFF (STRATEGY 1)
55 ;NOTE: THE PART IN THE GRIPPER ($IN[253]/$IN[254]) IS NOT CONFIRMED BEFORE OPENING (F40)
56 ;FOLD Gripper SET [1]Gripper 1 State=[1]OPEN Wait 0.2[s] Check with Strategy 1 ;%{...}
57 ;FOLD Parameters Parameters ;%{...}
58 ;Params IlfProvider=GripperTech.GripperSet;setgrripper=1;setstate=1;setcont=nocont;waittime=0.2;errorstrategy=1
59 ;ENDFOLD
60 GRPg_SetStateAndCheck(1, 1, 0.2, 1)
61 ;ENDFOLD
62 ;WAIT GRIPPER EMPTY - SIGNALS $IN[253] di253PartPresent1 OFF, $IN[254] di254PartPresent2 OFF (NO TIMEOUT)
63 ;FOLD WAIT FOR ( NOT IN 253 'di253PartPresent1' ) AND ( NOT IN 254 'di254PartPresent2' ) ;%{...}
64 WAIT FOR ( NOT $IN[253] ) AND ( NOT $IN[254] )
65 ;ENDFOLD
66
67 ;LEAVE THE REJECT LOCATION - P24 (TAUGHT IN BASE 3 "NUT WELDER")
68 ;FOLD PTP P24 Vel=100 % PDAT29 Tool[1]:TOOL 1 Base[3]:NUT WELDER ;%{...}
69 $BWDSTART=FALSE
70 PDAT_ACT=PPDAT29
71 FDAT_ACT=FP24
72 BAS(#PTP_PARAMS,100)

```

```

73 PTP XP24
74 ;ENDFOLD
75 ;PART NOT IN GRIPPER - SIGNAL $OUT[18] do018PartInGripper OFF
76 ;FOLD OUT 18 'do018PartInGripper1' State=FALSE ;{%...}
77 $OUT[18]=FALSE
78 ;ENDFOLD
79
80 ;RETURN HOME - P27, P25, HOME
81 ;FOLD PTP P27 CONT Vel=100 % PDAT32 Tool[1]:TOOL 1 Base[0] ;{%...}
82 $BWDSTART=FALSE
83 PDAT_ACT=PPDAT32
84 FDAT_ACT=FP27
85 BAS(#PTP_PARAMS,100)
86 PTP XP27 C_DIS
87 ;ENDFOLD
88 ;FOLD PTP P25 CONT Vel=100 % PDAT30 Tool[1]:TOOL 1 Base[0] ;{%...}
89 $BWDSTART=FALSE
90 PDAT_ACT=PPDAT30
91 FDAT_ACT=FP25
92 BAS(#PTP_PARAMS,100)
93 PTP XP25 C_DIS
94 ;ENDFOLD
95
96 ;FOLD PTP HOME Vel=100 % DEFAULT ;{%...}
97 $BWDSTART=FALSE
98 PDAT_ACT=PDEFAULT
99 FDAT_ACT=FHOME
100 BAS(#PTP_PARAMS,100)
101 $H_POS=XHOME
102 PTP XHOME
103 ;ENDFOLD
104
105 ;REJECT DONE - SIGNAL $OUT[106] do106PartRejected OFF
106 ;NOTE: THIS FORM USED TO CARRY THE HIDDEN VALUE 5:TRUE (RE-OPENING IT WOULD HAVE SWITCHED
107 ;NOTE: $OUT[106] ON) AND ENCLOSED THE FLAG RESETS. THE CLEANUP SET THE FORM VALUE TO 5:FALSE AND
108 ;NOTE: ENDED THE FOLD AFTER $OUT[106]=FALSE, SO THE RESETS BELOW ARE NOT PART OF THE FORM (F20)
109 ;FOLD OUT 106 'do106PartRejected' State=FALSE ;{%...}
110 $OUT[106]=FALSE
111 ;ENDFOLD
112 ;CLEAR THE SCRAP AND NUT PRESENCE FLAGS: bScrapCantrail, bscrapGE4, PARTPRESENT1..3 = FALSE
113 ;NOTE: bScrapCantrail IS A LEFTOVER FLAG FROM ANOTHER PROJECT - WRITTEN HERE, NEVER READ
114 bScrapCantrail=FALSE
115 bscrapGE4=FALSE
116 PARTPRESENT1=FALSE
117 PARTPRESENT2=FALSE
118 PARTPRESENT3=FALSE
119
120
121
122
123 END
124

```

KRC/R1/Program/Utilities/closeandcheckallclamps.src

```

1 &ACCESS RV01
2 &REL 3
3 &COMMENT 03-10-R1 CLOSE GRIPPER 1

```

```

4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 DEF CloseAndCheckAllClamps( )
7 ;FOLD INI
8 ;FOLD BASISTECH INI
9 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
10 INTERRUPT ON 3
11 BAS (#INITMOV,0 )
12 ;ENDFOLD (BASISTECH INI)
13 ;FOLD NutWeld
14 ;ENDFOLD (NutWeld)
15 ;FOLD USER INI
16 ;Make your modifications here
17
18 ;ENDFOLD (USER INI)
19 ;ENDFOLD (INI)
20 ;*****
21 ; Gestamp Standards
22 ; Line - BMW G65 (cell 03)
23 ; Robot 03-10-R1 (BMW-03-10-R1-MH1)
24 ; Application - CLOSE AND CHECK THE EOAT 1 GRIPPER (MASTERING REFERENCE)
25 ;*****
26 ;CALLED BY MASREFSTARTG1 (masref_user) BEFORE THE MASTERING REFERENCE PATH
27 ;NOTE: ONLY GRIPPER 1 IS ACTIVE IN grp_data.dat ($OUT[249]/[250], $IN[257]/[258]). THE SET AND
28 ; CHECK FOLDS OF GRIPPERS 2-4 WERE ALREADY DISABLED (CALLS COMMENTED OUT, MARKED SL) AND WERE
29 ; REMOVED BY THE CLEANUP: THIS ROUTINE CLOSES AND CHECKS GRIPPER 1 ONLY (F45)
30
31 ;GRIPPER 1 CLOSE, THEN CHECK GRIPPER 1 CLOSED
32
33 ;FOLD Gripper SET [1]Gripper1 State=[2]CLOSE Wait 0.2[s] Check with Strategy 1 ;%{...}
34 ;FOLD Parameters Parameters ;%{...}
35 ;Params IfProvider=GripperTech.GripperSet;setgrripper=1;setstate=2;setcont=nocont;waittime=0.2;errorstrategy=1
36 ;ENDFOLD
37 GRPg_SetStateAndCheck(1, 2, 0.2, 1)
38 ;ENDFOLD
39 ;FOLD Gripper CHECK [1]Gripper1 State=[2]CLOSE with Strategy 1 ;%{...}
40 ;FOLD Parameters Parameters ;%{...}
41 ;Params IfProvider=GripperTech.GripperCheck;setgrripper=1;setstate=2;setcont=nocont;checkstrategy=1
42 ;ENDFOLD
43 GRPg_Check(1, 2, FALSE, 1)
44 ;ENDFOLD
45
46 END
47

```

KRC/R1/Program/Utilities/gunelectrodechange.src

```

1 &ACCESS RV01
2 &REL 12
3 &COMMENT 03-10-R1 ELECTRODE CHANGE
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 DEF GunElectrodeChange( )
7 ;FOLD DECL
8 DECL B00L bMsgRelease
9 DECL B00L bCapsChanged
10 ;ENDFOLD
11 ;FOLD INI

```

```

12      ;FOLD BASISTECH INI
13      GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
14      INTERRUPT ON 3
15      BAS (#INITMOV,0 )
16      ;ENDFOLD (BASISTECH INI)
17      ;FOLD NutWeld
18      ;ENDFOLD (NutWeld)
19      ;FOLD USER INI
20      ;Make your modifications here
21      bMsgRelease=FALSE
22      bCapsChanged=FALSE
23      bElectrChanged=FALSE
24      ;ENDFOLD (USER INI)
25      ;ENDFOLD (INI)
26      ;*****
27      ; Gestamp Standards
28      ; Line - BMW G65 (cell 03)
29      ; Robot 03-10-R1 (BMW-03-10-R1)
30      ; Application - ELECTRODE CHANGE FOR NUTWELD GUNS 1-3 (CALLED BY CELL, NEVER TRIGGERED)
31      ;*****
32      ; NUTWELD TEMPLATE ROUTINE (NutWeld_Vorgabe): PTP HOME; THEN, FOR EACH NUTWELD GUN 1-3 WHOSE
33      ; ELECTRODE-CHANGE OUTPUT IS ON: WATER OFF, DIALOG "ARE CAPS CHANGED" UNTIL THE PLC CONFIRMS
34      ; THE CAP CHANGE, CAP-CHANGE-COMPLETE HANDSHAKE AND STEPPER RESET, WATER ON; FINALLY PTP HOME
35      ;NOTE: THE TRIGGER $OUT[111] do111Gun1ElectrodeChange IS WRITTEN BY AutomationCore_Bkg FROM
36      ; dipw1_EndofStepper; THE TASK RUNS SINCE 2026-10-05, SO CELL CALLS THIS PROGRAM AT THE STEPPER END
37      ;CHECK: NEVER RUN BEFORE - TEST THE WHOLE SEQUENCE AND THE PLC HANDSHAKE (F43, F49)
38      ;NOTE: THIS STATION HAS ONE NUT WELDER (PW1 = CENTERLINE PEDESTAL); THE GUN 2 / GUN 3 OUTPUTS
39      ; WRITTEN HERE ($OUT[523], $OUT[533], $OUT[571], $OUT[581]) ARE NOT MAPPED TO ANY BUS BIT
40      ;NOTE: sps.sub SWITCHES $OUT[475] dopw1_StartWater BACK ON IN THE NEXT SUBMIT CYCLE OUTSIDE
41      ; DRY CYCLE, SO THE GUN 1 WATER-OFF STEP HAS NO LASTING EFFECT (F09)
42      ;NOTE: $OUT[111] SHARES ITS PLC ADAPTER BIT WITH $OUT[930] do930MasterRefInProcess (F18)
43      ;NOTE: STEPPER RESET IS A 0.1 S PULSE WITH NO CONFIRMATION AND THE CAP-CHANGE HANDSHAKE
44      ; DOES NOT CLOSE ON LEVELS (F43, GESTAMP REVIEW POINT 6)
45      ;*****
46
47      ;PTP HOME (XHOME)
48      ;FOLD PTP HOME Vel= 100 % DEFAULT ;%{...}
49      $BWDSTART = FALSE
50      PDAT_ACT=PDEFAULT
51      FDAT_ACT=FHOME
52      BAS (#PTP_PARAMS,100 )
53      $H_POS=XHOME
54      PTP XHOME
55      ;ENDFOLD
56
57      ;GUN 1 ELECTRODE CHANGE - ONLY WHEN SIGNAL $OUT[111] do111Gun1ElectrodeChange IS ON
58      IF do111Gun1ElectrodeChange THEN
59          ;FOLD GUN 1 WATER OFF
60          ;WATER OFF - SIGNAL $OUT[475] dopw1_StartWater OFF
61          dopw1_StartWater=FALSE
62          ;ENDFOLD
63          ;FOLD GUN 1 WAIT FOR CAP CHANGE CONFIRMATION
64          ;REPEATS THE DIALOG UNTIL SIGNAL $IN[92] di092CapsIareChanged ON IS SEEN WHILE THE DIALOG
65          ;IS OPEN - THE KEYS YES / NO ALONE DO NOT END THE LOOP
66          REPEAT
67          REPEAT
68          ;READY FOR CAP CHANGE - SIGNAL $OUT[91] do091RobotReadyCapChange ON, THEN DIALOG

```

```

69      ;AutomationCore_Msg[26] (AC_AreCapsChgd) WITH THE KEYS DIAGNOSTIC / YES / NO
70      do091RobotReadyCapChange=TRUE
71      nAutoCore_Handle = Set_KrLDlg (AutomationCore_Msg[26], AutoCore_MsgPar[], AC_SK_YesNoNav[], AutoCore_MsgOpt_Stop)
72      IF (nAutoCore_Handle > 0) THEN
73          WHILE (Exists_KrLDlg(nAutoCore_Handle, nAutoCore_Key))
74              WAIT SEC 0.024
75      ;PLC CONFIRMS CAPS CHANGED - SIGNAL $IN[92] di092CapsIareChanged ON:
76      ;CLEAR THE DIALOG (Clear_KRLMsg); THE ANSWER IS THEN TAKEN AS YES (KEY 2)
77      IF (di092CapsIareChanged==TRUE) THEN
78          bCapsChanged=TRUE
79          bElectrchanged=TRUE
80          bAutoCore_Clear=Clear_KRLMsg(nAutoCore_Handle)
81      ENDIF
82      ENDWHILE
83      IF (bCapsChanged==TRUE) THEN
84          nAutoCore_Key=2
85      ENDIF
86      SWITCH nAutoCore_Key
87          CASE 1 ; Diagnostic
88              nAutoCore_Handle = Set_Krlmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_Stop)
89              WHILE (Exists_KrlMsg(nAutoCore_Handle))
90                  WAIT SEC 0.024
91              ENDWHILE
92              bMsgRelease = FALSE
93          CASE 2 ; Yes
94              ;ONLY AFTER THE PLC CONFIRMATION: CAP CHANGE COMPLETE -
95              ;SIGNAL $OUT[92] do092RobotCapChg1Cmplt ON, THEN PULSE
96              ;SIGNAL $OUT[485] dopw1_StepperReset ON FOR 0.1 S
97              IF (bElectrchanged == TRUE) THEN
98                  bMsgRelease = TRUE
99                  bCapsChanged=TRUE
100                 do092RobotCapChg1Cmplt=TRUE
101                 PULSE(dopw1_StepperReset,TRUE,0.1)
102             ENDIF
103             CASE 3 ; No
104                 bMsgRelease = FALSE
105                 bCapsChanged=FALSE
106             ENDSWITCH
107         ENDIF
108         UNTIL bElectrChanged
109         UNTIL (bMsgRelease==TRUE)
110     ;READY FOR CAP CHANGE OFF - SIGNAL $OUT[91] do091RobotReadyCapChange OFF
111     do091RobotReadyCapChange=FALSE
112 ;ENDFOLD (GUN 1 WAIT FOR CAP CHANGE CONFIRMATION)
113 ;FOLD GUN 1 RESET CAP CHANGE COMPLETE
114 ;CAP CHANGE COMPLETE OFF - SIGNAL $OUT[92] do092RobotCapChg1Cmplt OFF, ONLY IF
115 ;$OUT[111] do111Gun1ElectrodeChange IS ALREADY OFF - OTHERWISE IT STAYS ON
116 IF (bCapsChanged AND NOT do111Gun1ElectrodeChange) THEN
117     do092RobotCapChg1Cmplt=False
118 ENDIF
119 bElectrChanged=FALSE
120 ;ENDFOLD
121 ;FOLD GUN 1 WATER ON
122 ;WATER ON - SIGNAL $OUT[475] dopw1_StartWater ON
123 dopw1_StartWater=TRUE
124 ;ENDFOLD (GUN 1 WATER ON)
125 ENDIF

```

126

127 ;GUN 2 ELECTRODE CHANGE - ONLY WHEN SIGNAL \$OUT[112] do112Gun2ElectrodeChange IS ON

128 IF do112Gun2ElectrodeChange THEN

129 ;FOLD GUN 2 WATER OFF

130 ;WATER OFF - SIGNAL \$OUT[523] dopw2_StartWater OFF (NOT MAPPED)

131 dopw2_StartWater=FALSE

132 ;ENDFOLD

133 ;FOLD GUN 2 WAIT FOR CAP CHANGE CONFIRMATION

134 ;REPEATS THE DIALOG UNTIL SIGNAL \$IN[93] di093Caps2areChanged ON IS SEEN WHILE THE DIALOG

135 ;IS OPEN - THE KEYS YES / NO ALONE DO NOT END THE LOOP

136 REPEAT

137 REPEAT

138 ;READY FOR CAP CHANGE - SIGNAL \$OUT[91] do091RobotReadyCapChange ON, THEN DIALOG

139 ;AutomationCore_Msg[26] (AC_AreCapsChgd) WITH THE KEYS DIAGNOSTIC / YES / NO

140 do091RobotReadyCapChange=TRUE

141 nAutoCore_Handle = Set_KrLdlg (AutomationCore_Msg[26], AutoCore_MsgPar[], AC_SK_YesNoNav[], AutoCore_MsgOpt_Stop)

142 IF (nAutoCore_Handle > 0) THEN

143 WHILE (Exists_KrLdlg(nAutoCore_Handle, nAutoCore_Key))

144 WAIT SEC 0.024

145 ;PLC CONFIRMS CAPS CHANGED - SIGNAL \$IN[93] di093Caps2areChanged ON:

146 ;CLEAR THE DIALOG (Clear_KRLMsg); THE ANSWER IS THEN TAKEN AS YES (KEY 2)

147 IF (di093Caps2areChanged==TRUE) THEN

148 bCapsChanged=TRUE

149 bElectrchanged=TRUE

150 bAutoCore_Clear=Clear_KRLMsg(nAutoCore_Handle)

151 ENDIF

152 ENDWHILE

153 IF (bCapsChanged==TRUE) THEN

154 nAutoCore_Key=2

155 ENDIF

156 SWITCH nAutoCore_Key

157 CASE 1 ; Diagnostic

158 nAutoCore_Handle = Set_Krlmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_Stop)

159 WHILE (Exists_KrLMsg(nAutoCore_Handle))

160 WAIT SEC 0.024

161 ENDWHILE

162 bMsgRelease = FALSE

163 CASE 2 ; Yes

164 ;ONLY AFTER THE PLC CONFIRMATION: CAP CHANGE COMPLETE -

165 ;SIGNAL \$OUT[93] do093RobotCapChg2Cmplt ON, THEN PULSE

166 ;SIGNAL \$OUT[533] dopw2_StepperReset ON FOR 0.1 S (NOT MAPPED)

167 IF (bElectrchanged == TRUE) THEN

168 bMsgRelease = TRUE

169 bCapsChanged=TRUE

170 do093RobotCapChg2Cmplt=TRUE

171 PULSE(dopw2_StepperReset,TRUE,0.1)

172 ENDIF

173 CASE 3 ; No

174 bMsgRelease = FALSE

175 bCapsChanged=FALSE

176 ENDSWITCH

177 ENDIF

178 UNTIL bElectrChanged

179 UNTIL (bMsgRelease==TRUE)

180 ;READY FOR CAP CHANGE OFF - SIGNAL \$OUT[91] do091RobotReadyCapChange OFF

181 do091RobotReadyCapChange=FALSE

182 ;ENDFOLD (GUN 2 WAIT FOR CAP CHANGE CONFIRMATION)

```

183      ;FOLD GUN 2 RESET CAP CHANGE COMPLETE
184      ;CAP CHANGE COMPLETE OFF - SIGNAL $OUT[93] do093RobotCapChg2Cmplt OFF, ONLY IF
185      ;$OUT[112] do112Gun2ElectrodeChange IS ALREADY OFF - OTHERWISE IT STAYS ON
186      IF (bCapsChanged AND NOT do112Gun2ElectrodeChange) THEN
187          do093RobotCapChg2Cmplt=False
188      ENDIF
189      bElectrChanged=False
190      ;ENDFOLD
191      ;FOLD GUN 2 WATER ON
192      ;WATER ON - SIGNAL $OUT[523] dopw2_StartWater ON (NOT MAPPED)
193      dopw2_StartWater=True
194      ;ENDFOLD (GUN 2 WATER ON)
195  ENDIF
196
197      ;GUN 3 ELECTRODE CHANGE - ONLY WHEN SIGNAL $OUT[113] do113Gun3ElectrodeChange IS ON
198      IF do113Gun3ElectrodeChange THEN
199          ;FOLD GUN 3 WATER OFF
200          ;WATER OFF - SIGNAL $OUT[571] dopw3_StartWater OFF (NOT MAPPED)
201          dopw3_StartWater=False
202          ;ENDFOLD
203          ;FOLD GUN 3 WAIT FOR CAP CHANGE CONFIRMATION
204          ;REPEATS THE DIALOG UNTIL SIGNAL $IN[94] di094Caps3areChanged ON IS SEEN WHILE THE DIALOG
205          ;IS OPEN - THE KEYS YES / NO ALONE DO NOT END THE LOOP
206          REPEAT
207          REPEAT
208          ;READY FOR CAP CHANGE - SIGNAL $OUT[91] do091RobotReadyCapChange ON, THEN DIALOG
209          ;AutomationCore_Msg[26] (AC_AreCapsChgd) WITH THE KEYS DIAGNOSTIC / YES / NO
210          do091RobotReadyCapChange=True
211          nAutoCore_Handle = Set_KrLDlg (AutomationCore_Msg[26], AutoCore_MsgPar[], AC_SK_YesNoNav[], AutoCore_MsgOpt_Stop)
212          IF (nAutoCore_Handle > 0) THEN
213              WHILE (Exists_KrLDlg(nAutoCore_Handle, nAutoCore_Key))
214                  WAIT SEC 0.024
215              ;PLC CONFIRMS CAPS CHANGED - SIGNAL $IN[94] di094Caps3areChanged ON:
216              ;CLEAR THE DIALOG (Clear_KRLMsg); THE ANSWER IS THEN TAKEN AS YES (KEY 2)
217              IF (di094Caps3areChanged==TRUE) THEN
218                  bCapsChanged=True
219                  bElectrchanged=True
220                  bAutoCore_Clear=Clear_KRLMsg(nAutoCore_Handle)
221              ENDIF
222              ENDWHILE
223              IF (bCapsChanged==TRUE) THEN
224                  nAutoCore_Key=2
225              ENDIF
226              SWITCH nAutoCore_Key
227              CASE 1 ; Diagnostic
228                  nAutoCore_Handle = Set_KrLmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_Stop)
229                  WHILE (Exists_KrLMsg(nAutoCore_Handle))
230                      WAIT SEC 0.024
231                  ENDWHILE
232                  bMsgRelease = FALSE
233              CASE 2 ; Yes
234                  ;ONLY AFTER THE PLC CONFIRMATION: CAP CHANGE COMPLETE -
235                  ;SIGNAL $OUT[95] do095RobotCapChg3Cmplt ON, THEN PULSE
236                  ;SIGNAL $OUT[581] dopw3_StepperReset ON FOR 0.1 S (NOT MAPPED)
237                  IF (bElectrchanged == TRUE) THEN
238                      bMsgRelease = TRUE
239                      bCapsChanged=True

```

```

240             do095RobotCapChg3Cmplt=TRUE
241             PULSE(dopw3_StepperReset,TRUE,0.1)
242             ENDIF
243             CASE 3 ; No
244                 bMsgRelease = FALSE
245                 bCapsChanged=FALSE
246             ENDSWITCH
247             ENDIF
248             UNTIL bElectrChanged
249             UNTIL (bMsgRelease==TRUE)
250             ;READY FOR CAP CHANGE OFF - SIGNAL $OUT[91] do091RobotReadyCapChange OFF
251             do091RobotReadyCapChange=FALSE
252             ;ENDFOLD (GUN 3 WAIT FOR CAP CHANGE CONFIRMATION)
253             ;FOLD GUN 3 RESET CAP CHANGE COMPLETE
254             ;CAP CHANGE COMPLETE OFF - SIGNAL $OUT[95] do095RobotCapChg3Cmplt OFF, ONLY IF
255             ;$OUT[113] do113Gun3ElectrodeChange IS ALREADY OFF - OTHERWISE IT STAYS ON
256             IF (bCapsChanged AND NOT do113Gun3ElectrodeChange) THEN
257                 do095RobotCapChg3Cmplt=False
258             ENDIF
259             bElectrChanged=FALSE
260             ;ENDFOLD
261             ;FOLD GUN 3 WATER ON
262             ;WATER ON - SIGNAL $OUT[571] dopw3_StartWater ON (NOT MAPPED)
263             dopw3_StartWater=TRUE
264             ;ENDFOLD (GUN 3 WATER ON)
265             ENDIF
266
267             ;PTP HOME (XHOME)
268             ;FOLD PTP HOME Vel= 100 % DEFAULT ;%{...}
269             $BWDSTART = FALSE
270             PDAT_ACT=PDEFAULT
271             FDAT_ACT=FHOME
272             BAS (#PTP_PARAMS,100 )
273             $H_POS=XHOME
274             PTP XHOME
275             ;ENDFOLD
276             bElectrChanged=FALSE
277             END

```

KRC/R1/Program/Utilities/hometopounce.src

```

1  &ACCESS RVP3
2  &REL 1
3  &COMMENT 03-10-R1 NOT CALLED POUNCE
4  &PARAM DISKPATH = Program/Utilities
5  DEF HomeToPounce ( )
6  ;FOLD INI ;%{...}
7  ;FOLD BASISTECH INI
8      GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
9      INTERRUPT ON 3
10     BAS (#INITMOV,0 )
11 ;ENDFOLD (BASISTECH INI)
12 ;FOLD USER INI
13 ;Make your modifications here
14
15 ;ENDFOLD (USER INI)
16 ;ENDFOLD (INI)
17 ;*****

```

```

18 ; Gestamp Standards
19 ; Line - BMW G65 (cell 03)
20 ; Robot 03-10-R1 (BMW-03-10-R1)
21 ; Application - NOT CALLED - AUTOMATIONCORE POUNCE SKELETON
22 ;*****
23 ; AUTOMATIONCORE STANDARD PATH HOME TO POUNCE: PTP HOME, PTP POUNCE (GLOBAL XPOUNCE).
24 ; NO PROGRAM CALLS IT - IT IS ONLY DECLARED (EXT) IN automationcoredata.dat (C12)
25 ;NOTE: NO INTERFERENCE ZONE IS REQUESTED (C11, C13)
26 ;*****
27
28 ;PTP HOME (XHOME), THEN PTP POUNCE (GLOBAL XPOUNCE)
29 ;FOLD PTP HOME Vel= 100 % DEFAULT ;%{...}
30 $BWDSTART = FALSE
31 PDAT_ACT=PDEFAULT
32 FDAT_ACT=FHOME
33 BAS (#PTP_PARAMS,100 )
34 $H_POS=XHOME
35 PTP XHOME
36 ;ENDFOLD
37
38 ;FOLD PTP POUNCE Vel= 100 % DEFAULT ;%{...}
39 $BWDSTART = FALSE
40 PDAT_ACT=PPOUNCE
41 FDAT_ACT=FPOUNCE
42 BAS (#PTP_PARAMS,100 )
43 PTP XPOUNCE
44 ;ENDFOLD
45
46 END
47

```

KRC/R1/Program/Utilities/hometorepair.src

```

1 &ACCESS RVP14
2 &REL 3
3 &COMMENT 03-10-R1 HOME TO REPAIR
4 DEF HomeToRepair ( )
5 ;FOLD INI ;%{...}
6 ;FOLD BASISTECH INI
7 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
8 INTERRUPT ON 3
9 BAS (#INITMOV,0 )
10 ;ENDFOLD (BASISTECH INI)
11 ;FOLD USER INI
12 ;Make your modifications here
13
14 ;ENDFOLD (USER INI)
15 ;ENDFOLD (INI)
16 ;*****
17 ; Gestamp Standards
18 ; Line - BMW G65 (cell 03)
19 ; Robot 03-10-R1 (BMW-03-10-R1)
20 ; Application - SERVICE PATH HOME TO REPAIR
21 ;*****
22 ; CALLED BY CELL (AFTER HomeCheck) WHEN THE PLC SETS $IN[9] di009GoToRepair:
23 ; PTP HOME, PTP REPAIR (GLOBAL XREPAIR). CELL THEN WAITS FOR $IN[8] di008ReturnFromRepair
24 ; AND CALLS RepairToHome
25 ;NOTE: NO INTERFERENCE ZONE IS REQUESTED ON THE WAY TO REPAIR (C11, C13)

```

```

26 ;*****
27
28 ;PTP HOME (XHOME), THEN PTP REPAIR (GLOBAL XREPAIR)
29 ;FOLD PTP HOME Vel= 100 % DEFAULT ;%{...}
30 $BWDSTART = FALSE
31 PDAT_ACT=PDEFAULT
32 FDAT_ACT=FHOME
33 BAS (#PTP_PARAMS,100 )
34 $H_POS=XHOME
35 PTP XHOME
36 ;ENDFOLD
37
38 ;FOLD PTP REPAIR Vel= 100 % DEFAULT ;%{...}
39 $BWDSTART = FALSE
40 PDAT_ACT=PREPAIR
41 FDAT_ACT=FREPAIR
42 BAS (#PTP_PARAMS,100 )
43 PTP XREPAIR
44 ;ENDFOLD
45
46 END
47

```

KRC/R1/Program/Utilities/openandcheckallclamps.src

```

1 &ACCESS RV01
2 &REL 4
3 &COMMENT 03-10-R1 OPEN GRIPPER 1
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\TP\Nutweld\Template\NutWeld_Vorgabe
6 &PARAM DISKPATH = Program/Utilities
7 DEF OpenAndCheckAllClamps( )
8 ;FOLD INI
9 ;FOLD BASISTECH INI
10 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11 INTERRUPT ON 3
12 BAS (#INITMOV,0 )
13 ;ENDFOLD (BASISTECH INI)
14 ;FOLD NutWeld
15 ;ENDFOLD (NutWeld)
16 ;FOLD USER INI
17 ;Make your modifications here
18
19 ;ENDFOLD (USER INI)
20 ;ENDFOLD (INI)
21 ;*****
22 ; Gestamp Standards
23 ; Line - BMW G65 (cell 03)
24 ; Robot 03-10-R1 (BMW-03-10-R1-MH1)
25 ; Application - OPEN AND CHECK THE EOAT 1 GRIPPER (MASTERING REFERENCE)
26 ;*****
27 ;CALLED BY MASREFBACKG1 (masref_user) AT THE END OF THE MASTERING REFERENCE PATH
28 ;NOTE: ONLY GRIPPER 1 IS ACTIVE IN grp_data.dat ($OUT[249]/[250], $IN[257]/[258]). THE SET AND
29 ; CHECK FOLDS OF GRIPPERS 2-4 WERE ALREADY DISABLED (CALLS COMMENTED OUT, MARKED SL) AND WERE
30 ; REMOVED BY THE CLEANUP: THIS ROUTINE OPENS AND CHECKS GRIPPER 1 ONLY (F45)
31
32 ;GRIPPER 1 OPEN, THEN CHECK GRIPPER 1 OPEN
33

```

```

34 ;FOLD Gripper SET [1]Gripper1 State=[1]OPEN Wait 0.2[s] Check with Strategy 1 ;%{...}
35 ;FOLD Parameters Parameters ;%{...}
36 ;Params IlfProvider=GripperTech.GripperSet;setgrripper=1;setstate=1;setcont=nocont;waittime=0.2;errorstrategy=1
37 ;ENDFOLD
38 GRPg_SetStateAndCheck(1, 1, 0.2, 1)
39 ;ENDFOLD
40 ;FOLD Gripper CHECK [1]Gripper1 State=[1]OPEN with Strategy 1 ;%{...}
41 ;FOLD Parameters Parameters ;%{...}
42 ;Params IlfProvider=GripperTech.GripperCheck;setgrripper=1;setstate=1;setcont=nocont;checkstrategy=1
43 ;ENDFOLD
44 GRPg_Check(1, 1, FALSE, 1)
45 ;ENDFOLD
46
47 END

```

KRC/R1/Program/Utilities/pouncetohome.src

```

1 &ACCESS RVP4
2 &REL 1
3 &COMMENT 03-10-R1 POUNCE TO HOME
4 &PARAM DISKPATH = Program/Utilities
5 DEF PounceToHome ( )
6 ;FOLD INI ;%{...}
7 ;FOLD BASISTECH INI
8 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
9 INTERRUPT ON 3
10 BAS (#INITMOV,0 )
11 ;ENDFOLD (BASISTECH INI)
12 ;FOLD USER INI
13 ;Make your modifications here
14
15 ;ENDFOLD (USER INI)
16 ;ENDFOLD (INI)
17 ;*****
18 ; Gestamp Standards
19 ; Line - BMW G65 (cell 03)
20 ; Robot 03-10-R1 (BMW-03-10-R1)
21 ; Application - AUTOMATIONCORE PATH POUNCE TO HOME
22 ;*****
23 ; CALLED BY AutomationCore HomeCheck WHEN THE ROBOT IS NOT HOME, IN EXTERNAL MODE AND
24 ; $OUT[7] do007RobotAtPounce IS ON - THAT BIT IS WRITTEN ONLY BY THE DISABLED BACKGROUND
25 ; TASK (F12), SO IN PRACTICE HomeCheck ASKS THE OPERATOR INSTEAD
26 ;*****
27
28 ;PTP POUNCE (GLOBAL XPOUNCE), THEN PTP HOME (XHOME)
29 ;FOLD PTP POUNCE Vel= 100 % DEFAULT ;%{...}
30 $BWDSTART = FALSE
31 PDAT_ACT=PPOUNCE
32 FDAT_ACT=FPOUNCE
33 BAS (#PTP_PARAMS,100 )
34 PTP XPOUNCE
35 ;ENDFOLD
36
37 ;FOLD PTP HOME Vel= 100 % DEFAULT ;%{...}
38 $BWDSTART = FALSE
39 PDAT_ACT=PDEFAULT
40 FDAT_ACT=FHOME
41 BAS (#PTP_PARAMS,100 )

```

```

42 $H_POS=XHOME
43 PTP XHOME
44 ;ENDFOLD
45
46 END
47

```

KRC/R1/Program/Utilities/repairtohome.src

```

1 &ACCESS RVP15
2 &REL 3
3 &COMMENT 03-10-R1 REPAIR TO HOME
4 DEF RepairToHome ( )
5 ;FOLD INI ;%{...}
6 ;FOLD BASISTECH INI
7 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
8 INTERRUPT ON 3
9 BAS (#INITMOV,0 )
10 ;ENDFOLD (BASISTECH INI)
11 ;FOLD USER INI
12 ;Make your modifications here
13
14 ;ENDFOLD (USER INI)
15 ;ENDFOLD (INI)
16 ;*****
17 ; Gestamp Standards
18 ; Line - BMW G65 (cell 03)
19 ; Robot 03-10-R1 (BMW-03-10-R1)
20 ; Application - SERVICE PATH REPAIR TO HOME
21 ;*****
22 ; CALLED BY CELL AFTER $IN[8] di008ReturnFromRepair: PTP REPAIR (GLOBAL XREPAIR), PTP HOME
23 ;*****
24
25 ;PTP REPAIR (GLOBAL XREPAIR), THEN PTP HOME (XHOME)
26 ;FOLD PTP REPAIR Vel= 100 % DEFAULT ;%{...}
27 $BWDSTART = FALSE
28 PDAT_ACT=PREPAIR
29 FDAT_ACT=FREPAIR
30 BAS (#PTP_PARAMS,100 )
31 PTP XREPAIR
32 ;ENDFOLD
33
34 ;FOLD PTP HOME Vel= 100 % DEFAULT ;%{...}
35 $BWDSTART = FALSE
36 PDAT_ACT=PDEFAULT
37 FDAT_ACT=FHOME
38 BAS (#PTP_PARAMS,100 )
39 $H_POS=XHOME
40 PTP XHOME
41 ;ENDFOLD
42
43 END
44

```

KRC/R1/Program/masref_user.src

```

1 &ACCESS RV01
2 &REL 171
3 &COMMENT 03-10-R1 MASTERING REF

```

```

4 &PARAM DISKPATH = KRC:\R1\Program
5 DEF MasRef_USER()
6 ;*****
7 ; Gestamp Standards
8 ; Line - BMW G65 (cell 03)
9 ; Robot 03-10-R1 (BMW-03-10-R1)
10 ; Application - MASTERING REFERENCE USER PATHS (KUKA MASREF USER HOOKS)
11 ;*****
12 ; KUKA MASTERING-REFERENCE USER HOOKS. masref_main (CALLED BY CELL WHEN THE PLC SETS
13 ; $IN[22] di022MstrRefRequest) CALLS MASREFSTARTG1 (PATH FROM HOME TO THE REFERENCE SWITCH)
14 ; AND MASREFBACKG1 (PATH BACK TO HOME). ONLY GROUP 1 IS TESTED: THE G2/G3 HOOKS ARE EMPTY
15 ; KUKA TEMPLATES, KEPT BECAUSE masref_main REFERENCES THEM. MasRef_USER ITSELF IS EMPTY.
16 ;*****
17 END
18
19 GLOBAL DEF MASREFSTARTG1()
20 ;PATH FROM HOME TO THE MASTERING REFERENCE SWITCH (GROUP 1) - CALLED BY masref_main
21 ;FOLD Teach path and position for reference group 1
22 ;ENDFOLD
23 ;MASTERING REFERENCE RUNNING - SIGNAL $OUT[930] do930MasterRefInProgress ON
24 ;NOTE: $OUT[930] SHARES ITS PLC ADAPTER BIT WITH $OUT[111] do111Gun1ElectrodeChange (F18)
25 do930MasterRefInProgress=TRUE
26
27 ;CLOSE GRIPPER 1 AND CHECK IT (CloseAndCheckAllClamps) BEFORE THE REFERENCE PATH
28 CloseAndCheckAllClamps ( )
29
30 ;PTP HomePos, P24, P23, P7 (APPROXIMATED), LIN P10, LIN P9, LIN TO THE REFERENCE SWITCH, WAIT 1 S
31
32 ;FOLD PTP HomePos Vel=80 % PDAT4 Tool[1]:T00L 1 Base[0] ;%{...}
33 $BWDSTART=FALSE
34 PDAT_ACT=PPDAT4
35 FDAT_ACT=FHomePos
36 BAS(#PTP_PARAMS,80)
37 PTP XHomePos
38 ;ENDFOLD
39 ;FOLD PTP P24 Vel=100 % PDAT16 Tool[1]:T00L 1 Base[0] ;%{...}
40 $BWDSTART=FALSE
41 PDAT_ACT=PPDAT16
42 FDAT_ACT=FP24
43 BAS(#PTP_PARAMS,100)
44 PTP XP24
45 ;ENDFOLD
46 ;FOLD PTP P23 Vel=100 % PDAT15 Tool[1]:T00L 1 Base[0] ;%{...}
47 $BWDSTART=FALSE
48 PDAT_ACT=PPDAT15
49 FDAT_ACT=FP23
50 BAS(#PTP_PARAMS,100)
51 PTP XP23
52 ;ENDFOLD
53 ;FOLD PTP P7 CONT Vel=80 % PDAT5 Tool[1]:T00L 1 Base[0] ;%{...}
54 $BWDSTART=FALSE
55 PDAT_ACT=PPDAT5
56 FDAT_ACT=FP7
57 BAS(#PTP_PARAMS,80)
58 PTP XP7 C_DIS
59 ;ENDFOLD
60 ;FOLD LIN P10 CONT Vel=1.2 m/s CPDAT8 Tool[1]:T00L 1 Base[0] ;%{...}

```

```

61 $BWDSTART=FALSE
62 LDAT_ACT=LCPDAT8
63 FDAT_ACT=FP10
64 BAS(#CP_PARAMS,1.2)
65 LIN XP10 C_DIS C_DIS
66 ;ENDFOLD
67 ;FOLD LIN P9 Vel=0.6 m/s CPDAT7 Tool[1]:T00L 1 Base[0] ;%{...}
68 $BWDSTART=FALSE
69 LDAT_ACT=LCPDAT7
70 FDAT_ACT=FP9
71 BAS(#CP_PARAMS,0.6)
72 LIN XP9
73 ;ENDFOLD
74 ;FOLD LIN RefSwitch Vel=0.2 m/s CPDAT6 Tool[1]:T00L 1 Base[0] ;%{...}
75 $BWDSTART=FALSE
76 LDAT_ACT=LCPDAT6
77 FDAT_ACT=FRefSwitch
78 BAS(#CP_PARAMS,0.2)
79 LIN XRefSwitch
80 ;ENDFOLD
81 ;FOLD WAIT Time=1 sec ;%{...}
82 WAIT SEC 1
83 ;ENDFOLD
84
85 END
86
87 GLOBAL DEF MASREFBACKG1()
88 ;PATH BACK FROM THE REFERENCE SWITCH TO HomePos (GROUP 1) - CALLED BY masref_main
89 ;FOLD Teach path back for reference group 1
90 ;ENDFOLD
91
92 ;LIN P29, LIN P28 (APPROXIMATED), PTP P27 (APPROXIMATED), PTP P26, PTP P25, PTP HomePos
93 ;FOLD LIN P29 Vel=0.6 m/s CPDAT15 Tool[1]:T00L 1 Base[0] ;%{...}
94 $BWDSTART=FALSE
95 LDAT_ACT=LCPDAT15
96 FDAT_ACT=FP29
97 BAS(#CP_PARAMS,0.6)
98 LIN XP29
99 ;ENDFOLD
100 ;FOLD LIN P28 CONT Vel=1.2 m/s CPDAT14 Tool[1]:T00L 1 Base[0] ;%{...}
101 $BWDSTART=FALSE
102 LDAT_ACT=LCPDAT14
103 FDAT_ACT=FP28
104 BAS(#CP_PARAMS,1.2)
105 LIN XP28 C_DIS C_DIS
106 ;ENDFOLD
107 ;FOLD PTP P27 CONT Vel=80 % PDAT18 Tool[1]:T00L 1 Base[0] ;%{...}
108 $BWDSTART=FALSE
109 PDAT_ACT=PPDAT18
110 FDAT_ACT=FP27
111 BAS(#PTP_PARAMS,80)
112 PTP XP27 C_DIS
113 ;ENDFOLD
114 ;FOLD PTP P26 Vel=100 % PDAT17 Tool[1]:T00L 1 Base[0] ;%{...}
115 $BWDSTART=FALSE
116 PDAT_ACT=PPDAT17
117 FDAT_ACT=FP26

```

```

118 BAS(#PTP_PARAMS,100)
119 PTP XP26
120 ;ENDFOLD
121 ;FOLD PTP P25 Vel=100 % PDAT0 Tool[1]:T00L 1 Base[0] ;%{...}
122 $BWDSTART=FALSE
123 PDAT_ACT=PPDAT0
124 FDAT_ACT=FP25
125 BAS(#PTP_PARAMS,100)
126 PTP XP25
127 ;ENDFOLD
128 ;FOLD PTP HomePos Vel=80 % PDAT19 Tool[1]:T00L 1 Base[0] ;%{...}
129 $BWDSTART=FALSE
130 PDAT_ACT=PPDAT19
131 FDAT_ACT=FHomePos
132 BAS(#PTP_PARAMS,80)
133 PTP XHomePos
134 ;ENDFOLD
135
136 ;OPEN GRIPPER 1 AND CHECK IT (OpenAndCheckAllClamps)
137 OpenAndCheckAllClamps ( )
138 ;MASTERING REFERENCE RUNNING - SIGNAL $OUT[930] do930MasterRefInProcess ON
139 ;CHECK: THE FLAG IS SET ON AGAIN AT THE END INSTEAD OF OFF (THE OLD do828 LINE WAS FALSE); NOTHING
140 ;CHECK: EVER SWITCHES $OUT[930] OFF. AutomationCore REFUSES REQUEST TO ENTER WHILE IT IS ON (F35)
141 do930MasterRefInProcess=TRUE
142 ;FOLD WAIT Time=1 sec ;%{...}
143 WAIT SEC 1
144 ;ENDFOLD
145 END
146
147 GLOBAL DEF MASREFSTARTG2()
148 ;FOLD Teach path and position for reference group 2
149 ;ENDFOLD
150
151 END
152
153 GLOBAL DEF MASREFBACKG2()
154 ;FOLD Teach path back for reference group 2
155 ;ENDFOLD
156
157 END
158
159 GLOBAL DEF MASREFSTARTG3()
160 ;FOLD Teach path and position for reference group 3
161 ;ENDFOLD
162
163 END
164
165 GLOBAL DEF MASREFBACKG3()
166 ;FOLD Teach path back for reference group 3
167 ;ENDFOLD
168
169 END
170

```

KRC/R1/Program/tm_useraction.src

```

1 &ACCESS R
2 &COMMENT 03-10-R1 COLLISION STOP

```

```

3 DEF TM_USERACTION( )
4 ;*****
5 ; Gestamp Standards
6 ; Line - BMW G65 (cell 03)
7 ; Robot 03-10-R1 (BMW-03-10-R1)
8 ; Application - COLLISION DETECTION REACTION
9 ;*****
10 ;NOTE: CALLED BY THE KUKA TORQUE MONITORING (tm_bib.src, AFTER BRAKE F) ON A DETECTED COLLISION.
11 ;NOTE: IT ONLY HALTS - NO MESSAGE OF ITS OWN AND NO FAULT TO THE PLC (C21).
12 ;NOTE: COLLISION DETECTION IS NOT COMMISSIONED ON THIS ROBOT (F39, C09)
13
14 ;STOP THE ROBOT PROGRAM
15 HALT
16
17 END

```

KRC/R1/System/sps.sub

```

1 &ACCESS RVP
2 &REL 1
3 &COMMENT 03-10-R1 SUBMIT SPS
4 &PARAM DISKPATH = System
5 DEF SPS ( )
6 ;FOLD DECLARATIONS
7 ;FOLD BASISTECH DECL
8 ;Automatic External
9 DECL STATE_T STAT
10 DECL MODUS_T MODE
11 ;ENDFOLD (BASISTECH DECL)
12 ;FOLD USER DECL
13 ; Please insert user defined declarations
14
15 ;ENDFOLD (USER DECL)
16 ;ENDFOLD (DECLARATIONS)
17 ;FOLD INI
18 ;FOLD AUTOEXT INIT
19 INTERRUPT DECL 91 WHEN $PRO_STATE1==#P_FREE DO RESET_OUT ( )
20 INTERRUPT ON 91
21 $LOOP_MSG[]=" "
22 MODE=#SYNC
23 $H_POS=$H_POS
24 ;Automatic External: IN EXTERNAL AUTOMATIC MODE SELECT THE MAIN PROGRAM /R1/CELL()
25 IF $MODE_OP==#EX THEN
26 CWRITE($CMD,STAT,MODE,"RUN /R1/CELL()")
27 ENDIF
28 ;ENDFOLD (AUTOEXT INIT)
29 ;FOLD BACKUPMANAGER PLC INIT
30 BM_ENABLED = FALSE
31 BM_OUTPUTVALUE = 0
32 ;ENDFOLD (BACKUPMANAGER PLC INIT)
33 ;FOLD TQM_INIT
34 TorqueDefinitions()
35 ;ENDFOLD (TQM_INIT)
36
37
38
39 ;FOLD USER INIT
40 ; Please insert user defined initialization commands

```

```

41 ;AutomationCore_BKG IS OFF AGAIN (F49): CLEAR THE OUTPUTS IT MAY HAVE LEFT ON WHILE IT RAN -
42 ;SIGNALS $OUT[145] doCriticalWZ OFF (AS BEFORE: THE ROBOT STOPS ON A REQUEST TO ENTER),
43 ;$OUT[111] do111Gun1ElectrodeChange OFF (CELL WOULD OTHERWISE CALL GunElectrodeChange)
44 doCriticalWZ=FALSE
45 do111Gun1ElectrodeChange=FALSE
46 ;THE OTHER PLC STATUS OUTPUTS ONLY THAT TASK WRITES, OFF AS IN THE PROGRAM AS RECEIVED
47 ;(INDEPENDENT REVIEW 2026-10-05, F49)
48 do006RobotBatteryAlarm=FALSE ;SIGNAL $OUT[6] do006RobotBatteryAlarm OFF
49 do007RobotAtPounce=FALSE ;SIGNAL $OUT[7] do007RobotAtPounce OFF
50 do009RobotAtRepair=FALSE ;SIGNAL $OUT[9] do009RobotAtRepair OFF
51 do010RobotAtTipDress=FALSE ;SIGNAL $OUT[10] do010RobotAtTipDress OFF
52 do013WaterIsOn=FALSE ;SIGNAL $OUT[13] do013WaterIsOn OFF
53 do021BrakeTestReqd=FALSE ;SIGNAL $OUT[21] do021BrakeTestReqd OFF
54 do022MstrRefRequired=FALSE ;SIGNAL $OUT[22] do022MstrRefRequired OFF
55 do046WeldMode=FALSE ;SIGNAL $OUT[46] do046WeldMode OFF
56 do048RPWCtrlrEnetOK=FALSE ;SIGNAL $OUT[48] do048RPWCtrlrEnetOK OFF
57 do050TDVlvManifold=FALSE ;SIGNAL $OUT[50] do050TDVlvManifold OFF
58 do051RbtAtMstrRefSwitch=FALSE ;SIGNAL $OUT[51] do051RbtAtMstrRefSwitch OFF
59 do052EOATVlvMnflEnetOK=FALSE ;SIGNAL $OUT[52] do052EOATVlvMnflEnetOK OFF
60 do053MigReamerEnetOK=FALSE ;SIGNAL $OUT[53] do053MigReamerEnetOK OFF
61 do054SRWTipDressIOEnetOK=FALSE ;SIGNAL $OUT[54] do054SRWTipDressIOEnetOK OFF
62 do055EndEffectorIOEnetOK=FALSE ;SIGNAL $OUT[55] do055EndEffectorIOEnetOK OFF
63 do056WeldCntlEnetOK=FALSE ;SIGNAL $OUT[56] do056WeldCntlEnetOK OFF
64 do057RPWSlrDarc1EnetOK=FALSE ;SIGNAL $OUT[57] do057RPWSlrDarc1EnetOK OFF
65 do058PinScribeCntlrIOOK=FALSE ;SIGNAL $OUT[58] do058PinScribeCntlrIOOK OFF
66 do059RPWSlrDarc2EnetOK=FALSE ;SIGNAL $OUT[59] do059RPWSlrDarc2EnetOK OFF
67 do060VlvManifoldEnetOK=FALSE ;SIGNAL $OUT[60] do060VlvManifoldEnetOK OFF
68 do094AirOK=FALSE ;SIGNAL $OUT[94] do094AirOK OFF
69 gdoAxis1Posn=0 ;SIGNAL $OUT[97..105] gdoAxis1Posn OFF
70 do107Hopper1LowLevel=FALSE ;SIGNAL $OUT[107] do107Hopper1LowLevel OFF
71 do118Nut1Ready=FALSE ;SIGNAL $OUT[118] do118Nut1Ready OFF
72 do123ToolIDBit0=FALSE ;SIGNAL $OUT[123] do123ToolIDBit0 OFF
73 do124ToolIDBit1=FALSE ;SIGNAL $OUT[124] do124ToolIDBit1 OFF
74 do134Nut1Fault=FALSE ;SIGNAL $OUT[134] do134Nut1Fault OFF
75 do146SpeedNot100=FALSE ;SIGNAL $OUT[146] do146SpeedNot100 OFF
76 do464WaterBeacon=FALSE ;SIGNAL $OUT[464] do464WaterBeacon OFF

```

```

77
78 ;ENDFOLD (USER INIT)
79 ;ENDFOLD (INI)

```

```

80 ;*****
81 ; Gestamp Standards
82 ; Line - BMW G65 (cell 03)
83 ; Robot 03-10-R1 (BMW-03-10-R1)
84 ; Application - SUBMIT INTERPRETER (BACKGROUND LOGIC, CENTERLINE SENSORS, NUT PRELOAD)
85 ;*****
86
87 LOOP
88   WAIT FOR NOT($POWER_FAIL)
89   TORQUE_MONITORING()
90   ;FOLD BACKUPMANAGER PLC
91   IF BM_ENABLED THEN
92     BM_OUTPUTSIGNAL = BM_OUTPUTVALUE
93   ENDIF
94   ;ENDFOLD (BACKUPMANAGER PLC)
95 ;START AUTOGENERATED -- DO NOT CHANGE -- GripperSpotTech sps SpsSub.add InsertGripperTechGlobals
96   ;FOLD GRIPPERTech LOOP
97   GRPg_ChkSetStatePLC()

```

```

98 ;ENDFOLD (GRIPPERTECH LOOP)
99 ;END AUTOGENERATED -- DO NOT CHANGE -- (GripperSpotTech sps SpsSub.add InsertGripperTechGlobals)
100 ;NOTE: THE CALL NutWeld_BKG() WAS COMMENTED OUT IN THE ARCHIVE, THE FOLD BELOW IS EMPTY: NO HOPPER
101 ;NOTE: LEVEL LIGHTS $OUT[477]/$OUT[478],
102 ;NOTE: NO NUTWELD STATUS KEYS AND NO NUTWELD PROCESS RESET ON PROGRAM CANCEL (F12, F29)
103 ;FOLD NutWeld PLC
104 ;ENDFOLD (NutWeld PLC)
105 ;NOTE: AutomationCore_BKG() IS OFF, AS IN THE ARCHIVE AS RECEIVED: THE FOLD BELOW IS EMPTY AND ITS
106 ;NOTE: PLC STATUS BITS STAY STATIC (F12). THE PROGRAMMER SWITCHED IT ON ON 2026-10-05; IT IS OFF
107 ;NOTE: AGAIN UNTIL GESTAMP DECIDES AND ITS PROBLEMS ARE FIXED: REQUEST TO ENTER WITH THE STUCK
108 ;NOTE: $OUT[930], TWO WRITERS ON $OUT[481]/$OUT[482], UNTESTED ELECTRODE CHANGE, AC_MODE (F49)
109 ;FOLD Automation Core PLC
110 ;ENDFOLD (Automation Core PLC)
111
112 ;FOLD Centerline Pedestal
113 ;NOTE: INTEGRATOR CODE FOR THE CENTERLINE PEDESTAL NUT WELDER, RUN IN EVERY SUBMIT CYCLE
114
115
116 ;GUN STROKE POSITION (BALLUFF BTL6 LPT) - SIGNALS $IN[567..590] CL_LPT_BYTE1..3 INTO
117 ;CL_GunStrokePositionLPT, BYTE ORDER HIGH TO LOW: BYTE1, BYTE2, BYTE3 (BYTE0 IS NOT USED);
118 ;CENTERLINE_WELD CHECKS IT (GUN ON THE NUT 69000..73000); GUN OPEN 137000..142000 IS CHECKED BY
119 ;GUN_OPEN_CHECK AND INTERRUPTS 20-22 OF THE WELD APPS (F01)
120 ;CHECK: R20 BUILDS THE SAME VALUE FROM ALL FOUR BYTES IN ANOTHER ORDER - CONFIRM THE BTL6
121 ;CHECK: OUTPUT FORMAT ON THIS PEDESTAL (F23)
122 CL_GunStrokePositionLPT=(CL_LPT_BYTE1*65536)+(CL_LPT_BYTE2*256)+CL_LPT_BYTE3
123
124 ;UPPER PIN POSITION - SIGNALS $IN[591..606] CL_PIN_BYTE0..1 INTO CL_WeldPinPosition
125 ;NOTE: NO PROGRAM READS CL_WeldPinPosition - THE PIN POSITIONS ARE NOT CHECKED (F16)
126 CL_WeldPinPosition=(CL_PIN_BYTE0*256)+CL_PIN_BYTE1
127
128 ;WATER FLOW SV0500 - SIGNALS $IN[3000..3015] SV0500_BYTE0..1 INTO SV0500_FLOW
129 SV0500_FLOW = ( SV0500_BYTE0*256)+SV0500_BYTE1
130
131 ;SV0500 = PARKER 5097335 VORTEX FLOW SENSOR, IO-LINK ON TBEN C7: BITS 15..2 = FLOW IN 0.1 L/MIN,
132 ;BIT 1 = OUT2, BIT 0 = OUT1. nWaterFlow 250 = 25.0 L/MIN
133 ;WATER OK (dipw1_WaterOk, READ BY GUN_OPEN_CHECK) AT OR ABOVE nWaterFlowMin, NOT OK BELOW
134 ;nWaterFlowMin - nWaterFlowHyst; LIMITS IN GUN_OPEN_CHECK.dat (F09)
135 ;CHECK: nWaterFlowMin 150 (15.0 L/MIN) TO BE CONFIRMED ON THIS ROBOT (F09)
136 nWaterFlow = SV0500_FLOW / 4
137 IF nWaterFlow >= nWaterFlowMin THEN
138     dipw1_WaterOk = TRUE
139 ELSE
140     IF nWaterFlow < (nWaterFlowMin - nWaterFlowHyst) THEN
141         dipw1_WaterOk = FALSE
142     ENDIF
143 ENDIF
144 ;NOTE: CL_WaterOk IS KEPT FOR COMPATIBILITY; NO PROGRAM READS IT
145 CL_WaterOk = dipw1_WaterOk
146
147 ;KL50L2 BEACON - SIGNALS $OUT[3008] CL_BeaconBit3008 (BYTE 589 BIT 0),
148 ;$OUT[3024] CL_BeaconBit3024 (BYTE 591 BIT 0), $OUT[3001] CL_BeaconBit3001 (NOT MAPPED ON THE BUS)
149 ;NOTE: NONE OF THE BITS IS EVER SWITCHED OFF EXCEPT $OUT[3024]. $OUT[478]/$OUT[477] ARE WRITTEN
150 ;NOTE: ONLY BY NutWeld_Bkg, WHICH IS DISABLED: THIS BLOCK NEVER SWITCHES THE BEACON (F29)
151 ;RED: WHEN SIGNAL $OUT[478] dopw1_LowLevel ON - SIGNALS $OUT[3008] CL_BeaconBit3008 ON,
152 ;$OUT[3024] CL_BeaconBit3024 ON
153 IF dopw1_LowLevel THEN
154     CL_BeaconBit3008 = TRUE

```

```

155     CL_BeaconBit3024 = TRUE
156 ENDIF
157 ;GREEN: WHEN SIGNAL $OUT[477] dopw1_Level0k ON - SIGNALS $OUT[3001] CL_BeaconBit3001 ON,
158 ;$OUT[3024] CL_BeaconBit3024 OFF
159 ;CHECK: $OUT[3001] IS NOT MAPPED; R20 USES $OUT[3000] AND $OUT[3016] FOR THE SAME BEACON (F29)
160 IF dopw1_Level0k THEN
161     CL_BeaconBit3001 = TRUE
162     CL_BeaconBit3024 = FALSE
163 ENDIF
164
165
166 ;ENDFOLD (Centerline Pedestal)
167
168 ;COOLING WATER VALVE - SIGNAL $OUT[475] dopw1_StartWater FOLLOWS SIGNAL $IN[13] di013WaterEnable
169 ;(PLC WATER ENABLE), IN EVERY SUBMIT CYCLE (F09)
170 ;NOTE: IT STILL OVERRIDES THE WATER OFF OF CENTERLINE_HOME WHILE THE PLC ENABLES THE WATER
171 dopw1_StartWater=di013WaterEnable
172
173 ;NUT PRELOAD SEQUENCE (Program/Centerline/PRELOAD.src): SWITCHES THE PEDESTAL VALVES
174 ;$OUT[471..505] AND SETS NUT_READY; STARTED BY NUT_START (MOTION TRIGGERS IN PICK/WELD APPS)
175 ;WARNING: RUNS IN EVERY OPERATING MODE (ALSO T1/T2, NO ENABLING SWITCH) AND WRITES THE SAME
176 ;WARNING: VALVES AS THE ROBOT PROGRAMS, WITH NO OWNERSHIP BETWEEN THEM (F11)
177 PRELOAD()
178
179 ;FOLD USER PLC
180 ;Make your modifications here
181
182 ;ENDFOLD (USER PLC)
183 ENDOLOOP
184 ;FOLD ;%{...}
185 ;FOLD
186 END
187 ;ENDFOLD
188
189
190 ;NOTE: RESET_OUT RUNS WHEN THE ROBOT PROGRAM IS CANCELLED ($PRO_STATE1 = #P_FREE) AND ONLY RESETS
191 ;NOTE: THE PGNO HANDSHAKE OUTPUTS - THE CENTERLINE PEDESTAL IS NOT PUT IN A SAFE STATE (F10)
192 DEF RESET_OUT ( )
193     INT N
194     MsgLoop(" ")
195     IF REFLECT_PROG_NR == 1 THEN
196         FOR N = 0 TO PGNO_LENGTH - 1
197             $OUT[PGNO_FBIT_REFL + N] = FALSE
198         ENDFOR
199     ENDIF
200     IF (PGNO_REQ>0) THEN
201         $OUT[PGNO_REQ]=FALSE
202     ELSE
203         IF (PGNO_REQ<0) THEN
204             $OUT[-PGNO_REQ]=TRUE
205         ENDIF
206     ENDIF
207 END
208
209 ;FOLD USER SUBROUTINE
210 ; Integrate your user defined subroutines
211

```

```

212 ;ENDFOLD (USER SUBROUTINE)
213 ;ENDFOLD
214

```

KRC/R1/System/tm_bib.src

```

1 &ACCESS R7
2 &REL 953
3 &COMMENT Collision Detection Library
4 &PARAM DISKPATH = KRC:\R1\System
5 DEF TM_BIB()
6 ;*****
7 ;* Date 24.02.2015      *
8 ;* Only >=KSS V8.3      *
9 ;* Changes also at SPS.SUB *
10 ;*****
11 END
12
13 GLOBAL DEF TQM_INI()
14 ;FOLD TQM_INI
15 CHECK_SUBMIT_RUNS()      ;Submit #P_ACTIVE
16 CHECK_VARSTATES()      ;Variables: bTQM_FIRST_AFTER_BCO,bTQM_RINGMEM_INIT_C,bTQM_OV_ONL,bTQM_UP_INIT
17
18 IF bTQM_UP_INIT THEN
19
20 GLOBAL INTERRUPT DECL iTQM_INTERRUPT WHEN $CYCFLAG[iTQM_CYCFLAG] == TRUE DO TQM_INTER_STOP ()      ;ISR 18 (Default)
21 GLOBAL INTERRUPT DECL iTQM_INTERRUPT_OVPRO WHEN $OV_PRO <> iTQM_LAST_OVPRO DO TQM_CHANGE_OFFSETS () ;ISR 19 (Default)
22
23 FHOME.TQ_STATE = FALSE
24 iCD_Step=0
25 iTQM_ZEIGER = 1
26 iTQM_ZEIGER_C = 1
27 iTQM_DELAY = $FILTER/2 + 36 ;[ms]
28 iTQM_Multiplier = 60 ;[ms]
29 iTQM_LAST_OVPRO = $OV_PRO
30 bTQM_RINGMEM_INIT = TRUE
31 bTQM_UP_INIT = FALSE
32 bTQM_CYC = FALSE
33 bTQM_Restart = FALSE
34 bTQM_STOP = FALSE
35 SET_TQM_ACT(200)
36
37 INTERRUPT ON iTQM_INTERRUPT_OVPRO
38 $CYCFLAG[iTQM_CYCFLAG] = FALSE
39 $CYCFLAG[iTQM_CYCFLAG]=((( $TORQ_DIFF[1]>TQM_ACT.T11)OR($TORQ_DIFF[2]>TQM_ACT.T12)OR($TORQ_DIFF[3]>TQM_ACT.T13)OR($TORQ_DIFF
[4]>T
QM_ACT.T14)OR($TORQ_DIFF[5]>TQM_ACT.T15)OR($TORQ_DIFF[6]>TQM_ACT.T16)OR($TORQ_DIFF2[1]>TQM_ACT.T21)OR($TORQ_DIFF2[2]>TQM_ACT.T2
2)OR(
$torq_diff2[3]>TQM_ACT.T23)OR($TORQ_DIFF2[4]>TQM_ACT.T24)OR($TORQ_DIFF2[5]>TQM_ACT.T25)OR($TORQ_DIFF2[6]>TQM_ACT.T26)) AND bTQM
_CYC
AND bTQM_ACTIVE AND($MODE_OP<>#T1))
40 ENDF
41 TQM_OverDriveFlag=FALSE
42 ;ENDFOLD ;(TQM_INI)
43 END ;(TQM_INI)
44 ;*****
45 ;* Interrupt service routine *
46 ;* Change of override >|5| % *
47 ;*****
48 GLOBAL DEF TQM_CHANGE_OFFSETS()

```

```

49 DECL REAL OV_FACTOR
50
51 IF GetSysState("STOPBITS", iSTOP_TYPE, rPAR) THEN
52     IF (iSTOP_TYPE <> 0) THEN
53         RETURN
54     ENDIF
55 ENDIF
56
57 INTERRUPT OFF iTQM_INTERRUPT_OVPRO
58
59 IF CHECK_IVAR(TQM_TEMP.OVM) <> #INITIALIZED THEN
60     TQM_DEACT_TORQUE1(1.25)
61     TQM_DEACT_TORQUE2(1.5)
62     iTQM_LAST_OVPRO = $OV_PRO
63     INTERRUPT ON iTQM_INTERRUPT_OVPRO
64     RETURN
65 ELSE
66     OV_FACTOR = ABS($OV_PRO-TQM_TEMP.OVM) * rTQM_OVM ;rTQM_OVM=0.15
67     IF OV_FACTOR < 1.5 THEN
68         OV_FACTOR = 1.5
69     ENDIF
70 ENDIF
71
72 ;Change of override recognized at change > |5|
73 IF (TQM_TEMP.OVM > 0) AND (TQM_TEMP.ID <> 0) then
74     IF $OV_PRO > (TQM_TEMP.OVM + 5) THEN
75
76         TQM_DEACT_TORQUE1(1.25)
77         TQM_DEACT_TORQUE2(OV_FACTOR)
78         bTQM_OV_ONL = TRUE
79     ELSE
80         IF $OV_PRO < (TQM_TEMP.OVM - 5) THEN
81
82             bTQM_OV_ONL = TRUE
83             TQM_DEACT_TORQUE1(1.25)
84             TQM_DEACT_TORQUE2(0.51*OV_FACTOR)
85         ENDIF
86     ENDIF
87 ENDIF
88
89 iTQM_LAST_OVPRO = $OV_PRO
90 INTERRUPT ON iTQM_INTERRUPT_OVPRO
91
92 END ;(TQM_CHANGE_OFFSETS)
93 ;*****
94 ;* Call by Submit loop *
95 ;*****
96 ;FOLD TQM Submit actions
97 ;*****
98 GLOBAL DEF TorqueDefinitions()
99 DECL INT i
100
101 FOR i = 1 TO 8
102     TQM_DATA_C[i]=TM0
103 ENDFOR
104 ;Definitions in SPS_SUB_INIT
105 iTQM_ZEIGER=1

```

```

106  iTQM_ZEIGER_C=1
107
108 END ; (TorqueDefinitions)
109 ;*****
110 ;* Submit Task          *
111 ;*****
112 GLOBAL DEF TORQUE_MONITORING()
113 DECL STATE_T STAT
114 DECL MODUS_T MODE
115
116 IF $ADAP_ACC == #NONE THEN
117
118     bTQM_KCPSTATUS = FALSE
119     bTQM_ACTIVE = FALSE
120     RETURN
121 ENDIF
122
123 IF bTQM_CANCEL_PROG == TRUE THEN
124     CWRITE($CMD, STAT, MODE, "CANCEL 1")
125 ENDIF
126
127 IF $PRO_STATE1 == #P_FREE THEN
128     bTQM_CANCEL_PROG = FALSE
129 ENDIF
130
131 IF ($PRO_STATE1 == #P_FREE) OR ($PRO_STATE1 == #P_RESET) THEN
132     bTQM_UP_INIT=TRUE
133     bTQM_CYC = FALSE ;clear additional
134     TQM_TEMP.ID = 0
135 ENDIF
136
137 IF bTQM_START_PROG == TRUE THEN
138     IF $PRO_STATE1 == #P_FREE THEN
139         bTQM_START_PROG = FALSE
140     ENDIF
141 ENDIF
142
143 IF GetSysState("STOPBITS", iSTOP_TYPE, rPAR) THEN
144     IF (iSTOP_TYPE <> 0) THEN
145         bTQM_STOP = TRUE ;Checked by Submit
146     ENDIF
147 ENDIF
148
149 IF NOT ($PRO_MODE == #G0) OR ($MODE_OP == #T1) THEN
150
151     bTQM_CYC = FALSE
152     bTQM_PMODE = FALSE
153 ENDIF
154
155 IF bTQM_ACTIVE THEN
156     IF GetSysState("STOPBITS", iSTOP_TYPE, rPAR) THEN
157         IF (iSTOP_TYPE <> 0) THEN
158
159             bTQM_CYC = FALSE
160             iCD_Step = 1
161
162             IF (($PRO_STATE1==#P_ACTIVE)AND($MOVE_BC0==FALSE)) THEN

```

```

163         bTQM_Restart = TRUE
164         TQM_IPO_TIMER(iTQM_TI_RESTART, iTQM_TMR_FACTOR) ;iTQM_TMR_FACTOR * 60 * 13 ms ==> 780 ms
165         TQM_IPO_TIMER(iTQM_TI_UPDATE, iTQM_TMR_FACTOR)
166         iCD_Step = 2
167     ENDIF
168
169 ELSE
170     IF (($PRO_STATE1==#P_ACTIVE)AND($MOVE_BC0==FALSE)) THEN
171         SWITCH iCD_Step ;Collision detect step
172         CASE 1
173             bTQM_Restart = TRUE
174             TQM_IPO_TIMER(iTQM_TI_RESTART, iTQM_TMR_FACTOR)
175             TQM_IPO_TIMER(iTQM_TI_UPDATE, iTQM_TMR_FACTOR) ;Timer[8] as default
176             iCD_Step = 2
177         CASE 2
178             IF ($TIMER[iTQM_TI_RESTART]>0) THEN
179                 iCD_Step = 0
180                 TORQ_DIFF_RESET()
181                 $TIMER_STOP[iTQM_TI_RESTART] = TRUE ;Timer[7] as default
182                 IF CHECK_IVAR(TQM_TEMP.ID) == #INITIALIZED THEN
183                     IF (TQM_TEMP.ID <> 0) THEN
184                         bTQM_CYC = TRUE ;ID <> 0 && CHECK_IVAR(TQM_TEMP.ID) == #INITIALIZED && $TIMER[iTQM_TI_RESTART]>0
185                     ELSE
186                         bTQM_CYC = FALSE ;Together with identifier of none CD motion disables $CYCFLAG control
187                     ENDIF
188                 ELSE
189                     bTQM_CYC = FALSE
190                 ENDIF
191             ENDIF
192         DEFAULT
193         ENDSWITCH
194     ENDIF
195 ENDIF
196 ENDIF
197 ENDIF
198
199 IF ($MODE_OP == #EX) THEN
200     IF (sTQM_SPSACTIVE) THEN
201         sTQM_SPSSTATUS = TRUE
202         IF (bTQM_KCPSTATUS) THEN
203             bTQM_ACTIVE = TRUE
204         ELSE
205             bTQM_ACTIVE = FALSE
206         ENDIF
207     ELSE
208         sTQM_SPSSTATUS = FALSE
209         bTQM_ACTIVE = FALSE
210     ENDIF
211 ELSE
212     IF (NOT bTQM_KCPSTATUS) THEN
213         bTQM_ACTIVE = FALSE
214     ELSE
215         bTQM_ACTIVE = TRUE
216     ENDIF
217 ENDIF
218 END ;(TORQUE_MONITORING)
219 ;ENDFOLD ;(TQM Submit actions)

```

```

220 ;*****
221 ;* Advance part of anti collision
222 ;* at CP and PTP motions
223 ;* or executed with TM0 at none collision detect
224 ;* subroutine in advance or at Init_Mov()
225 ;*****
226 GLOBAL DEF TQM_FIFO_READ (TQM_DATAS :IN)
227
228 DECL TQM_TQDAT_T TQM_DATAS
229 DECL INT iTQM_REAR_ZEIGER
230
231
232
233
234 TQM_DATA_C[iTQM_ZEIGER] = TQM_DATAS ;in advance
235
236 CONTINUE
237 IF ($MOVE_BCO==FALSE) THEN
238 ;Case for InitMov() or motions without TQM: If not values kept to old one
239 IF TQM_DATAS.ID == 0 THEN
240
241
242 IF NOT bTQM_RINGMEM_INIT THEN
243 iTQM_REAR_ZEIGER = iTQM_ZEIGER - 1
244
245 IF iTQM_REAR_ZEIGER == 0 THEN
246 iTQM_REAR_ZEIGER = 8
247 ENDIF
248
249 IF CHECK_IVAR(TQM_DATA_C[iTQM_REAR_ZEIGER].ID) == #DECLARED THEN
250 TQM_DATA_C[iTQM_REAR_ZEIGER] = TQM_DATAS
251 ENDIF
252
253 IF TQM_DATA_C[iTQM_REAR_ZEIGER].ID == 0 THEN
254
255 iTQM_ZEIGER = iTQM_ZEIGER - 1
256
257 IF iTQM_ZEIGER == 0 THEN
258 iTQM_ZEIGER = 8
259 ENDIF
260
261 ENDIF
262 ENDIF
263 ENDIF
264 ENDIF
265
266 CONTINUE
267 IF ($MOVE_BCO==FALSE) THEN
268 iTQM_ZEIGER = iTQM_ZEIGER + 1
269
270 IF iTQM_ZEIGER == 9 THEN
271 iTQM_ZEIGER = 1
272 ENDIF
273
274 IF $PRO_MODE == #MSTEP THEN
275 iTQM_ZEIGER_C=iTQM_ZEIGER
276 IF TQM_DATAS.ID == 0 THEN

```

```

277         TQM_TEMP.ID=0
278     ENDIF
279 ENDIF
280
281     bTQM_RINGMEM_INIT = FALSE
282 ELSE
283     bTQM_FIRST_AFTER_BCO = TRUE
284     bTQM_TRIGGER_ON = TRUE
285     bTQM_RINGMEM_INIT = TRUE
286     bTQM_RINGMEM_INIT_C = TRUE
287     iTQM_ZEIGER = 1
288     iTQM_ZEIGER_C = 1
289     TQM_TEMP.ID = 0
290 ENDIF
291
292 END ;TQM_FIFO_READ
293 ;*****
294 ;* CP TQM Call only at CP motions but only with CD property
295 ;* ==> No read when motion without CD
296 ;*****
297 GLOBAL DEF TQM_SWAP_VALUES(TQM_VALUES :OUT)
298 DECL TQM_TQDAT_T TQM_VALUES
299 ;FOLD CP Implementation
300
301
302
303
304
305
306 ;ENDFOLD
307 INTERRUPT OFF iTQM_INTERRUPT
308 bTQM_CYC = FALSE
309
310
311 IF (($VW_PSPS_STOP_B_AND 2) > 0) THEN
312
313     RETURN
314 ENDIF
315
316 IF bTQM_FIRST_AFTER_BCO THEN
317     bTQM_FIRST_AFTER_BCO = FALSE
318
319
320 ENDIF
321
322 IF bTQM_RESTART AND ($TIMER[iTQM_INTERRUPT] >= 0) THEN
323     IF $PRO_STATE1 == #P_ACTIVE THEN
324         IF ($TIMER[iTQM_TI_UPDATE]>=0) THEN
325             $TIMER_STOP[iTQM_TI_UPDATE]=TRUE
326
327             bTQM_RESTART=FALSE
328         ENDIF
329     ENDIF
330 ENDIF
331
332
333 IF ((bTQM_STOP==FALSE) AND ($STOPMESS==FALSE)) THEN

```

```

334     UPDATE_DATAS(TQM_VALUES)
335 ENDIF
336 bTQM_COLLISION = FALSE
337
338
339
340 IF bTQM_RINGMEM_INIT_C THEN ;TRUE WHEN $MOVE_BCO==TRUE
341     IF TQM_DATA_C[iTQM_ZEIGER_C].ID == 0 THEN
342
343         iTQM_ZEIGER_C = iTQM_ZEIGER_C + 1
344         IF iTQM_ZEIGER_C == 9 THEN
345             iTQM_ZEIGER_C = 1
346         ENDIF
347     ENDIF
348     bTQM_RINGMEM_INIT_C = FALSE
349 ENDIF
350
351 iTQM_ZEIGER_C = iTQM_ZEIGER_C + 1
352
353 IF iTQM_ZEIGER_C == 9 THEN
354     iTQM_ZEIGER_C = 1
355 ENDIF
356
357 IF iTQM_ZEIGER == iTQM_ZEIGER_C THEN
358     bTQM_ACTIVE = FALSE
359     bTQM_TRIGGER_ENABLE = TRUE
360     bTQM_TRIGGER_ON = TRUE
361     TQM_TEMP.ID = 0
362 ELSE
363
364     IF TQM_DATA_C[iTQM_ZEIGER_C].ID == 0 THEN
365
366         bTQM_TRIGGER_ENABLE = TRUE
367
368         bTQM_TRIGGER_ON = TRUE
369         iTQM_ZEIGER_C = iTQM_ZEIGER_C + 1
370
371         IF iTQM_ZEIGER_C == 9 THEN
372             iTQM_ZEIGER_C = 1
373         ENDIF
374         TQM_TEMP.ID = 0
375     ENDIF
376 ENDIF
377
378
379 IF IS_MSTEP_MOVE() THEN
380     bTQM_TRIGGER_ON = TRUE
381 ENDIF
382
383 bTQM_CYC = FALSE
384
385
386
387 IF (TQM_OverDriveFlag==TRUE) THEN
388
389     TQM_TEMP = TQM_TempSet2
390     TORQ_DIFF_RESET() ;TorqDiff(s) auf 0

```

```

391
392 IF bTQM_TRIGGER_ENABLE THEN
393     bTQM_TRIGGER_ENABLE=FALSE
394     IF bTQM_TRIGGER_ON THEN
395
396         IF IS_MSTEP_MOVE() THEN
397
398             bTQM_TRIGGER_ON = TRUE
399         ELSE
400             bTQM_TRIGGER_ON = FALSE
401         ENDIF
402     ENDIF
403 ELSE
404     bTQM_TRIGGER_ENABLE = TRUE
405 ENDIF
406
407
408 IF (bTQM_ACTIVE) AND (NOT IS_MSTEP_MOVE() AND (bTQM_STOP==FALSE) AND ($STOPMESS==FALSE)) THEN
409     MOT_FACTOR(TQM_VALUES)
410     FILL_TQM_ACT(TQM_VALUES)
411 ENDIF
412
413 IF (NOT bTQM_ACTIVE OR (TQM_TEMP.ID == 0)) THEN
414     SET_TQM_ACT(200)
415 ENDIF
416
417
418 IF ((TQM_TEMP.ID <> 0) AND (NOT IS_MSTEP_MOVE())AND (TQM_TEMP.OVM<>0)) THEN
419     CALL_INTERRUPT_ON(iTQM_INTERRUPT)
420 ELSE
421
422     INTERRUPT OFF iTQM_INTERRUPT
423 ENDIF
424
425 ELSE
426
427 IF NOT bTQM_TRIGGER_ON AND ($PRO_MODE==#G0) THEN
428     ELSE
429         IF (($VW_PSPS_STOP B_AND 2) > 0) THEN
430             TQM_TEMP.ID = 0
431         ENDIF
432     ENDIF
433
434     TQM_TEMP.ID = 0
435
436 ENDIF ;(TQM_OverDriveFlag==TRUE)
437
438 END ;(TQM_SWAP_VALUES)
439 ;*****
440 ;Call via TQM_MON_OFF at spline Trigger
441 ;*****
442 DEF TQM_SWAP_VALUES_S(TQM_VALUES :OUT)
443 ;FOLD Spline implementation
444 ;TRIGGER WHEN PATH=0 ONSTART DELAY=0 DO TQM_MON_ON(TM4) PRI0=-1
445 ;TRIGGER WHEN PATH=0 DELAY=-12 DO TQM_MON_OFF(TM4) PRI0=-1
446 ;SPL XP#
447 ;ENDFOLD

```

```

448 DECL TQM_TQDAT_T TQM_VALUES
449
450 IF (NOT bTQM_COLLISION AND NOT bTQM_FIRST_AFTER_BCO AND NOT bTQM_RESTART AND bTQM_PMODE) THEN
451
452
453
454 IF bTQM_OV_ONL THEN
455     bTQM_OV_ONL = FALSE
456 ELSE
457     TQM_VALUES.OVM = $OV_PRO
458 ENDIF
459
460 SCALE_TO_STANDARD(TQM_VALUES.TMF,rTQM_REDTMF)
461
462 ENDIF
463
464 STORE_MOT_TEMP(TQM_VALUES)
465
466 bTQM_FIRST_AFTER_BCO = FALSE
467
468 IF bTQM_RESTART AND ($TIMER[iTQM_INTERRUPT] >= 0) THEN
469     IF $PRO_STATE1 == #P_ACTIVE THEN
470         IF ($TIMER[iTQM_TI_UPDATE]>=0) THEN
471             $TIMER_STOP[iTQM_TI_UPDATE]=TRUE
472
473             bTQM_RESTART = FALSE
474         ENDIF
475     ENDIF
476 ENDIF
477
478
479 UPDATE_TMVALUE(TQM_VALUES);spline
480 bTQM_COLLISION = FALSE
481 TQM_TEMP.ID = 0
482 bTQM_CYC = FALSE
483
484 END ;(TQM_SWAP_VALUES_S)
485 ;*****
486 ;First Subroutine call at spline motion
487 ;Trigger call
488 ;*****
489 GLOBAL DEF TQM_MON_ON( TQM_VALUES :OUT)
490 DECL TQM_TQDAT_T TQM_VALUES
491
492
493 TQM_TEMP =TQM_VALUES
494
495
496 IF (($MOVE_BCO==FALSE) AND ($TIMER[iTQM_TI_RESTART] >= 0)) THEN
497
498     IF $PRO_STATE1 == #P_ACTIVE THEN
499         bTQM_CYC = TRUE
500     ENDIF
501 ENDIF
502
503 IF ($PRO_MODE == #G0) AND ($MODE_OP <> #T1) THEN
504

```

```

505     bTQM_PMODE = TRUE
506 ENDIF
507
508 IF NOT bTQM_ACTIVE OR $MOVE_BCO THEN
509     RETURN
510 ENDIF
511
512 TORQ_DIFF_RESET() ;TorqDiff(s) to 0
513 IF ((bTQM_ACTIVE==TRUE) AND (NOT IS_MSTEP_MOVE())) AND (bTQM_STOP==FALSE) AND ($STOPMESS==FALSE)) THEN
514     MOT_FACTOR(TQM_VALUES)
515     FILL_TQM_ACT(TQM_VALUES);spline
516 ENDIF
517
518
519
520 IF ((TQM_TEMP.ID <> 0) AND (NOT IS_MSTEP_MOVE())AND (TQM_TEMP.OVM<>0)) THEN
521
522     CALL_INTERRUPT_ON(iTQM_INTERRUPT)
523 ELSE
524
525     INTERRUPT OFF iTQM_INTERRUPT
526 ENDIF
527
528 END ;(TQM_MON_ON => Spline)
529 ;*****
530 ;* Spline subroutine call at end of spline pos
531 ;* Includes call of TQM_SWAP_VALUES_S
532 ;*****
533 GLOBAL DEF TQM_MON_OFF(TQM_VALUES :OUT)
534 DECL TQM_TQDAT_T TQM_VALUES
535
536 bTQM_CYC = FALSE
537
538 IF GetSysState("STOPBITS", iSTOP_TYPE, rPAR) THEN
539     IF (iSTOP_TYPE == 0) THEN
540         TQM_SWAP_VALUES_S(TQM_VALUES)
541     ENDIF
542 ENDIF
543
544 INTERRUPT OFF iTQM_INTERRUPT
545 SET_TQM_ACT(200)
546
547 END ;(TQM_MON_OFF => Spline)
548 ;*****
549 ;* Start Trigger standard motion      *
550 ;*****
551 GLOBAL DEF TQM_START_MON(TQM_VALUES:OUT)
552 DECL TQM_TQDAT_T TQM_VALUES
553 DECL REAL rTQ2
554
555 ;Save actual one
556 TQM_History=TQM_VALUES
557
558 IF (NOT IS_MSTEP_MOVE()) AND ($TIMER[iTQM_TI_RESTART] >= 0) THEN
559
560     bTQM_CYC = TRUE
561 ENDIF

```

```

562
563 IF bTQM_STOP THEN
564     bTQM_STOP = FALSE
565 ENDIF
566
567 IF ($PRO_MODE == #G0) AND ($MODE_OP <> #T1) THEN
568
569     bTQM_PMODE = TRUE
570 ELSE
571     bTQM_PMODE = FALSE
572 ENDIF
573
574 ;Actual TM# zu Beginn der Bewegung
575 IF iTQM_TRIGGERID <> TQM_VALUES.ID THEN
576
577     TQM_OverDriveFlag=TRUE
578
579     IF ((bTQM_ACTIVE==TRUE) AND (NOT IS_MSTEP_MOVE()))AND (bTQM_STOP==FALSE) AND ($STOPMESS==FALSE)) THEN
580         MOT_FACTOR(TQM_VALUES) ;IN
581         FILL_TQM_ACT(TQM_VALUES) ;(standard) :OUT
582     ENDIF
583
584
585     TQM_TempSet2=TQM_VALUES
586     RETURN
587 ELSE
588     ;correct CD set recognized => under CD control and clear workaround
589     TQM_OverDriveFlag=FALSE
590     TQM_TEMP = TQM_VALUES
591 ENDIF
592
593 TORQ_DIFF_RESET() ;TorqDiff(s) auf 0
594
595 IF bTQM_TRIGGER_ENABLE THEN
596
597     bTQM_TRIGGER_ENABLE=FALSE
598
599     IF bTQM_TRIGGER_ON THEN
600
601         IF IS_MSTEP_MOVE() THEN
602
603             bTQM_TRIGGER_ON = TRUE
604         ELSE
605             bTQM_TRIGGER_ON = FALSE
606         ENDIF
607     ENDIF
608
609 ELSE
610     bTQM_TRIGGER_ENABLE = TRUE
611 ENDIF
612
613 IF ((bTQM_ACTIVE==TRUE) AND (NOT IS_MSTEP_MOVE())) AND (bTQM_STOP==FALSE) AND ($STOPMESS==FALSE)) THEN
614     MOT_FACTOR(TQM_VALUES) ;IN
615     FILL_TQM_ACT(TQM_VALUES) ;OUT
616 ENDIF
617
618 IF (NOT bTQM_ACTIVE OR (TQM_TEMP.ID == 0)) THEN

```

```

619     SET_TQM_ACT(200)
620 ENDIF
621
622 IF ((TQM_TEMP.ID <> 0) AND (NOT IS_MSTEP_MOVE())AND (TQM_TEMP.OVM<>0)) THEN
623     CALL_INTERRUPT_ON(iTQM_INTERRUPT)
624 ELSE
625
626     INTERRUPT OFF iTQM_INTERRUPT
627 ENDIF
628
629 END ;(TQM_START_MON)
630 ;*****
631 DEF FILL_TQM_ACT(TQM_VALUES:OUT)
632 DECL REAL rTQ2
633 DECL TQM_TQDAT_T TQM_VALUES
634 DECL INT Max_INT
635 DECL INT iTemp
636
637 IF TQM_VALUES.OVM < $OV_PRO THEN
638     TQM_VALUES.TMF = TQM_VALUES.TMF + (ABS($OV_PRO-TQM_VALUES.OVM) * rTQM_TMF)
639     IF TQM_VALUES.TMF <= 1.0 THEN
640         TQM_VALUES.TMF = 1.0
641     ELSE
642
643         IF (TQM_VALUES.TMF > 1) THEN
644             TQM_VALUES.TMF = 1.25
645         ENDIF
646     ENDIF
647 ELSE
648     IF TQM_VALUES.OVM > $OV_PRO THEN
649         IF TQM_VALUES.TMF < 1.5 THEN
650             TQM_VALUES.TMF = 1.5
651         ENDIF
652     ELSE
653         ;OVM == $OV_PRO
654         IF ((bTQM_Stop==TRUE) OR ($STOPMESS==TRUE)) THEN
655
656             IF GetSysState("STOPBITS", iSTOP_TYPE, rPAR) THEN
657                 IF (iSTOP_TYPE <> 0) THEN
658                     IF ((iSTOP_TYPE B_AND 8)==8) THEN
659                         TQM_VALUES.TMF = 1.25
660                     ELSE
661                         TQM_VALUES.TMF = 1.125
662                     ENDIF
663                 ENDIF
664             ENDIF
665         ELSE
666
667             TQM_VALUES.TMF = 1.0
668
669             IF GetSysState("STOPBITS", iSTOP_TYPE, rPAR) THEN
670                 IF (iSTOP_TYPE <> 0) THEN
671                     IF ((iSTOP_TYPE B_AND 8)==8) THEN
672                         TQM_VALUES.TMF = 1.25
673                     ELSE
674                         TQM_VALUES.TMF = 1.125
675                     ENDIF

```

```

676         ELSE
677             TQM_VALUES.TMF = 1.0
678         ENDIF
679     ENDIF
680 ENDIF
681
682 ENDIF
683 ENDIF
684
685
686
687 IF (TQM_VALUES.TMF <> 1.0) THEN
688
689     rTQ2 = 1.25
690 ELSE
691     rTQ2 = 1.0
692 ENDIF
693
694 ;Limitations
695 Max_INT=2147483646 ;INT: 2^31 -1 => 2147483646
696 ;A1
697 IF ((TQM_VALUES.T11 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[1])) < Max_INT) THEN
698     TQM_ACT.T11 = TQM_VALUES.T11 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[1])
699 ELSE
700
701 ENDIF
702
703 IF (((TQM_VALUES.T21 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[1]) < Max_INT) THEN
704     TQM_ACT.T21 = (TQM_VALUES.T21 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[1]
705 ELSE
706
707 ENDIF
708
709 ;A2
710 IF ((TQM_VALUES.T12 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[2])) < Max_INT) THEN
711     TQM_ACT.T12 = TQM_VALUES.T12 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[2])
712 ELSE
713
714 ENDIF
715 IF (((TQM_VALUES.T22 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[2]) < Max_INT) THEN
716     TQM_ACT.T22 = (TQM_VALUES.T22 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[2]
717 ELSE
718
719 ENDIF
720
721 ;A3
722 IF ((TQM_VALUES.T13 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[3])) < Max_INT) THEN
723     TQM_ACT.T13 = TQM_VALUES.T13 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[3])
724 ELSE
725
726 ENDIF
727 IF (((TQM_VALUES.T23 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[3]) < Max_INT) THEN
728     TQM_ACT.T23 = (TQM_VALUES.T23 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[3]
729 ELSE
730
731 ENDIF
732

```

```

733 ;A4
734 IF ((TQM_VALUES.T14 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[4])) < Max_INT) THEN
735     TQM_ACT.T14 = TQM_VALUES.T14 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[4])
736 ELSE
737
738 ENDIF
739 IF (((TQM_VALUES.T24 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[4]) < Max_INT) THEN
740     TQM_ACT.T24 = (TQM_VALUES.T24 + TQM_VALUES.02*rTQM_T2H*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[4]
741 ELSE
742
743 ENDIF
744
745 ;A5
746 IF ((TQM_VALUES.T15 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[5])) < Max_INT) THEN
747     TQM_ACT.T15 = TQM_VALUES.T15 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[5])
748 ELSE
749
750 ENDIF
751 IF (((TQM_VALUES.T25 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[5]) < Max_INT) THEN
752     TQM_ACT.T25 = (TQM_VALUES.T25 + TQM_VALUES.02*rTQM_T2H*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[5]
753 ELSE
754
755 ENDIF
756
757 ;A6
758 IF ((TQM_VALUES.T16 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[6])) < Max_INT) THEN
759     TQM_ACT.T16 = TQM_VALUES.T16 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[6])
760 ELSE
761
762 ENDIF
763 IF (((TQM_VALUES.T26 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[6]) < Max_INT) THEN
764     TQM_ACT.T26 = (TQM_VALUES.T26 + TQM_VALUES.02*rTQM_T2H*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[6]
765 ELSE
766
767 ENDIF
768
769 TQM_ACT.01 = TQM_VALUES.01
770 TQM_ACT.02 = TQM_VALUES.02
771 TQM_ACT.ID = TQM_VALUES.ID
772
773 ;Update des OVM for diagnosis
774 TQM_ACT.OVM=TQM_VALUES.OVM
775
776 END ;(FILL_TQM_ACT)
777 ;-----
778 DEF FILL_TQM_ACT2(TQM_Values:IN,rTQ2:IN)
779 DECL REAL rTQ2 ;==> r_TQ2
780 DECL TQM_TQDAT_T TQM_Values
781 DECL INT Max_INT
782
783
784 Max_INT=2147483646 ;INT: 2'31 -1 => 2147483646
785 ;A1
786 IF ((TQM_VALUES.T11 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[1])) < Max_INT) THEN
787     TQM_ACT.T11 = TQM_VALUES.T11 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[1])
788 ELSE
789

```

```

790  ENDIF
791  ;INT: 2'31 -1 => 2147483646 * 1.0 (8.2)
792  IF (((TQM_VALUES.T21 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[1]) < Max_INT) THEN
793    TQM_ACT.T21 = (TQM_VALUES.T21 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[1]
794  ELSE
795
796  ENDIF
797
798  ;A2
799  IF ((TQM_VALUES.T12 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[2])) < Max_INT) THEN
800    TQM_ACT.T12 = TQM_VALUES.T12 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[2])
801  ELSE
802
803  ENDIF
804  IF (((TQM_VALUES.T22 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[2]) < Max_INT) THEN
805    TQM_ACT.T22 = (TQM_VALUES.T22 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[2]
806  ELSE
807
808  ENDIF
809
810  ;A3
811  IF ((TQM_VALUES.T13 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[3])) < Max_INT) THEN
812    TQM_ACT.T13 = TQM_VALUES.T13 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[3])
813  ELSE
814
815  ENDIF
816  IF (((TQM_VALUES.T23 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[3]) < Max_INT) THEN
817    TQM_ACT.T23 = (TQM_VALUES.T23 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[3]
818  ELSE
819
820  ENDIF
821
822  ;A4
823  IF ((TQM_VALUES.T14 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[4])) < Max_INT) THEN
824    TQM_ACT.T14 = TQM_VALUES.T14 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[4])
825  ELSE
826
827  ENDIF
828  IF (((TQM_VALUES.T24 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[4]) < Max_INT) THEN
829    TQM_ACT.T24 = (TQM_VALUES.T24 + TQM_VALUES.02*rTQM_T2H*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[4]
830  ELSE
831
832  ENDIF
833
834  ;A5
835  IF ((TQM_VALUES.T15 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[5])) < Max_INT) THEN
836    TQM_ACT.T15 = TQM_VALUES.T15 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[5])
837  ELSE
838
839  ENDIF
840  IF (((TQM_VALUES.T25 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[5]) < Max_INT) THEN
841    TQM_ACT.T25 = (TQM_VALUES.T25 + TQM_VALUES.02*rTQM_T2H*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[5]
842  ELSE
843
844  ENDIF
845
846  ;A6

```

```

847 IF ((TQM_VALUES.T16 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[6])) < Max_INT) THEN
848     TQM_ACT.T16 = TQM_VALUES.T16 + (TQM_VALUES.01*TQM_VALUES.TMF*rTQM_MOT[6])
849 ELSE
850
851 ENDIF
852 IF (((TQM_VALUES.T26 + TQM_VALUES.02*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[6]) < Max_INT) THEN
853     TQM_ACT.T26 = (TQM_VALUES.T26 + TQM_VALUES.02*rTQM_T2H*ScalingImpact)*TQM_VALUES.TMF*rTQ2*rTQM_MOT[6]
854 ELSE
855
856 ENDIF
857
858 TQM_ACT.01 = TQM_Values.01
859 TQM_ACT.02 = TQM_Values.02
860 TQM_ACT.ID = TQM_Values.ID
861
862 ;Update des OVM for diagnosis
863 TQM_ACT.OVM=TQM_VALUES.OVM
864
865 END ; (FILL_TQM_ACT2)
866 ;-----
867
868 ;-----
869 GLOBAL DEF TQM_INTER_STOP()
870
871 DECL INT iTORQUES
872 DECL INT iIMPACTS
873 DECL BOOL bRESULT
874 DECL STATE_T STATE
875 DECL CHAR cTQM_TORQPARAM[20]
876 DECL CHAR cTQM_IMPPARAM[20]
877
878 DECL INT Delta_TORQ_DIFF ;prepared for debug
879 DECL INT Delta_IMPACTS_DIFF ;prepared for debug
880 DECL INT i
881
882 iTORQUES = 0
883 iIMPACTS = 0
884 bRESULT = STRCLEAR(cTQM_TORQPARAM[])
885 bRESULT = STRCLEAR(cTQM_IMPPARAM[])
886
887 ;For diagnostic
888 ;Detected TQ_Set
889 TQM_Compare=TQM_ACT
890
891 ;Cause of event
892 TQM_ISR_Event.ID=TQM_ACT.ID
893 TQM_ISR_Event.OVM=TQM_ACT.OVM
894 TQM_ISR_Event.TMF=TQM_ACT.TMF
895 TQM_ISR_Event.T11=$TORQ_DIFF[1]
896 TQM_ISR_Event.T12=$TORQ_DIFF[2]
897 TQM_ISR_Event.T13=$TORQ_DIFF[3]
898 TQM_ISR_Event.T14=$TORQ_DIFF[4]
899 TQM_ISR_Event.T15=$TORQ_DIFF[5]
900 TQM_ISR_Event.T16=$TORQ_DIFF[6]
901 TQM_ISR_Event.T21=$TORQ_DIFF2[1]
902 TQM_ISR_Event.T22=$TORQ_DIFF2[2]
903 TQM_ISR_Event.T23=$TORQ_DIFF2[3]

```

```

904 TQM_ISR_Event.T24=$TORQ_DIFF2[4]
905 TQM_ISR_Event.T25=$TORQ_DIFF2[5]
906 TQM_ISR_Event.T26=$TORQ_DIFF2[6]
907
908 IF ($STOPMESS==FALSE) THEN
909     ServiceStopFlag=bTQM_STOP
910 ELSE
911     ServiceStopFlag=FALSE
912 ENDIF
913
914 FOR i = 1 TO 6
915     DeltaTorque[i]=0
916     DeltaImpact[i]=0
917 ENDFOR
918
919 IF bTQM_CYC THEN
920
921     INTERRUPT OFF iTQM_INTERRUPT
922     bTQM_COLLISION = TRUE
923
924     IF ($TORQ_DIFF[1] > TQM_ACT.T11) THEN
925         iTORQUES = STRING_BUILDER(cTQM_TORQPARAM[], "A1")
926         iTORQUES = 1
927         Delta_TORQ_DIFF=$TORQ_DIFF[1]-TQM_ACT.T11
928         DeltaTorque[1]=Delta_TORQ_DIFF
929     ENDIF
930
931     IF ($TORQ_DIFF2[1] > TQM_ACT.T21) THEN
932         iIMPACTS = STRING_BUILDER(cTQM_IMPPARAM[], "A1")
933         iIMPACTS = 1
934         Delta_IMPACTS_DIFF = $TORQ_DIFF2[1] - TQM_ACT.T21
935         DeltaImpact[1]=Delta_IMPACTS_DIFF
936     ENDIF
937
938     IF ($TORQ_DIFF[2] > TQM_ACT.T12) THEN
939         iTORQUES = STRING_BUILDER(cTQM_TORQPARAM[], "A2")
940         iTORQUES = 2
941         Delta_TORQ_DIFF=$TORQ_DIFF[2]-TQM_ACT.T12
942         DeltaTorque[2]=Delta_TORQ_DIFF
943     ENDIF
944
945     IF ($TORQ_DIFF2[2] > TQM_ACT.T22) THEN
946         iIMPACTS = STRING_BUILDER(cTQM_IMPPARAM[], "A2")
947         iIMPACTS = 2
948         Delta_IMPACTS_DIFF = $TORQ_DIFF2[2] - TQM_ACT.T22
949         DeltaImpact[2]=Delta_IMPACTS_DIFF
950     ENDIF
951
952     IF ($TORQ_DIFF[3] > TQM_ACT.T13) THEN
953         iTORQUES = STRING_BUILDER(cTQM_TORQPARAM[], "A3")
954         iTORQUES = 3
955         Delta_TORQ_DIFF=$TORQ_DIFF[3]-TQM_ACT.T13
956         DeltaTorque[3]=Delta_TORQ_DIFF
957     ENDIF
958
959     IF ($TORQ_DIFF2[3] > TQM_ACT.T23) THEN
960         iIMPACTS = STRING_BUILDER(cTQM_IMPPARAM[], "A3")
961         iIMPACTS = 3

```

```

961     Delta_IMPACTS_DIFF = $TORQ_DIFF2[3] - TQM_ACT.T23
962     DeltaImpact[3]=Delta_IMPACTS_DIFF
963 ENDIF
964
965 IF ($TORQ_DIFF[4] > TQM_ACT.T14) THEN
966     iTORQUES = STRING_BUILDER(cTQM_TORQPARAM[], "A4")
967     iTORQUES = 4
968     Delta_TORQ_DIFF=$TORQ_DIFF[4]-TQM_ACT.T14
969     DeltaTorque[4]=Delta_TORQ_DIFF
970 ENDIF
971 IF ($TORQ_DIFF2[4] > TQM_ACT.T24) THEN
972     iIMPACTS = STRING_BUILDER(cTQM_IMPPARAM[], "A4")
973     iIMPACTS = 4
974     Delta_IMPACTS_DIFF = $TORQ_DIFF2[4] - TQM_ACT.T24
975     DeltaImpact[4]=Delta_IMPACTS_DIFF
976 ENDIF
977
978 IF ($TORQ_DIFF[5] > TQM_ACT.T15) THEN
979     iTORQUES = STRING_BUILDER(cTQM_TORQPARAM[], "A5")
980     iTORQUES = 5
981     Delta_TORQ_DIFF=$TORQ_DIFF[5]-TQM_ACT.T15
982     DeltaTorque[5]=Delta_TORQ_DIFF
983 ENDIF
984 IF ($TORQ_DIFF2[5] > TQM_ACT.T25) THEN
985     iIMPACTS = STRING_BUILDER(cTQM_IMPPARAM[], "A5")
986     iIMPACTS = 5
987     Delta_IMPACTS_DIFF = $TORQ_DIFF2[5] - TQM_ACT.T25
988     DeltaImpact[5]=Delta_IMPACTS_DIFF
989 ENDIF
990
991 IF ($TORQ_DIFF[6] > TQM_ACT.T16) THEN
992     iTORQUES = STRING_BUILDER(cTQM_TORQPARAM[], "A6")
993     iTORQUES = 6
994     Delta_TORQ_DIFF=$TORQ_DIFF[6]-TQM_ACT.T16
995     DeltaTorque[6]=Delta_TORQ_DIFF
996 ENDIF
997 IF ($TORQ_DIFF2[6] > TQM_ACT.T26) THEN
998     iIMPACTS = STRING_BUILDER(cTQM_IMPPARAM[], "A6")
999     iIMPACTS = 6
1000     Delta_IMPACTS_DIFF = $TORQ_DIFF2[6] - TQM_ACT.T26
1001     DeltaImpact[6]=Delta_IMPACTS_DIFF
1002 ENDIF
1003
1004 IF bTQM_CYC THEN
1005
1006     IF (iTORQUES > 0) THEN
1007
1008         IF ($0V_PRO == TQM_TEMP.OVM) THEN
1009             MSG_TQM{MODUL[] "TQM", NR 10, MSG_TXT[] "TorqueDetected"}, cTQM_TORQPARAM[])
1010         ENDIF
1011     ENDIF
1012
1013     IF (iIMPACTS > 0) THEN
1014         IF ($0V_PRO == TQM_TEMP.OVM) THEN
1015             MSG_TQM{MODUL[] "TQM", NR 11, MSG_TXT[] "ImpactDetected"}, cTQM_IMPPARAM[])
1016         ENDIF
1017     ENDIF

```

```

1018
1019     IF ($OV_PRO == TQM_TEMP.OVM) THEN
1020         IF ($PRO_MODE==#GO) THEN
1021             BRAKE F
1022             TM_USERACTION() ;==> HALT
1023         ELSE
1024             ;No action at other modes
1025         ENDIF
1026     ELSE
1027         ;At reference run no action
1028     ENDIF
1029 ELSE
1030     bTQM_COLLISION=FALSE
1031 ENDIF
1032
1033 ENDIF ;(bTQM_CYC)
1034 INTERRUPT ON iTQM_INTERRUPT
1035 bTQM_COLLISION=FALSE
1036 END ;(TQM_INTER_STOP)
1037 ;*****
1038 DEF TORQ_DIFF_RESET()
1039 DECL INT I
1040
1041     FOR I = 1 TO 6
1042         $TORQ_DIFF[I]=0
1043         $TORQ_DIFF2[I]=0
1044     ENDFOR
1045 END
1046 ;*****
1047 DEF STORE_MOT_TEMP(TQM_VALUES:OUT)
1048 DECL TQM_TQDAT_T TQM_VALUES
1049
1050     TQM_VALUES.K1 = $MOT_TEMP[1]
1051     TQM_VALUES.K2 = $MOT_TEMP[2]
1052     TQM_VALUES.K3 = $MOT_TEMP[3]
1053     TQM_VALUES.K4 = $MOT_TEMP[4]
1054     TQM_VALUES.K5 = $MOT_TEMP[5]
1055     TQM_VALUES.K6 = $MOT_TEMP[6]
1056 END
1057 ;*****
1058 ;* Sets values to motor temperatures datas *
1059 ;*****
1060 DEF MOT_FACTOR(TQM_VALUES:IN)
1061 DECL TQM_TQDAT_T TQM_VALUES
1062
1063     IF $MOT_TEMP[1] < TQM_VALUES.K1 - 5 THEN
1064         rTQM_MOT[1] = rTQM_F_MOT
1065     ELSE
1066         SCALE_TO_STANDARD(rTQM_MOT[1],0.8)
1067     ENDIF
1068
1069     IF $MOT_TEMP[2] < TQM_VALUES.K2 - 5 THEN
1070         rTQM_MOT[2] = rTQM_F_MOT
1071     ELSE
1072         SCALE_TO_STANDARD(rTQM_MOT[2],0.8)
1073     ENDIF
1074

```

```

1075 IF $MOT_TEMP[3] < TQM_VALUES.K3 - 5 THEN
1076     rTQM_MOT[3] = rTQM_F_MOT
1077 ELSE
1078     SCALE_TO_STANDARD(rTQM_MOT[3],0.8)
1079 ENDIF
1080
1081 IF $MOT_TEMP[4] < TQM_VALUES.K4 - 5 THEN
1082     rTQM_MOT[4] = rTQM_F_MOT
1083 ELSE
1084     SCALE_TO_STANDARD(rTQM_MOT[4],0.8)
1085 ENDIF
1086
1087 IF $MOT_TEMP[5] < TQM_VALUES.K5 - 5 THEN
1088     rTQM_MOT[5] = rTQM_F_MOT
1089 ELSE
1090     SCALE_TO_STANDARD(rTQM_MOT[5],0.8)
1091 ENDIF
1092
1093 IF $MOT_TEMP[6] < TQM_VALUES.K6 - 5 THEN
1094     rTQM_MOT[6] = rTQM_F_MOT
1095 ELSE
1096     SCALE_TO_STANDARD(rTQM_MOT[6],0.8)
1097 ENDIF
1098 END ;(MOT_FACTOR)
1099 ;*****
1100 DEF SCALE_TO_STANDARD(VALUE:OUT,FACTOR:IN)
1101 DECL REAL VALUE
1102 DECL REAL FACTOR
1103
1104 IF VALUE > 1.0 THEN
1105     IF NOT IS_WARMUP() THEN
1106         VALUE = VALUE * FACTOR
1107         IF VALUE < 1.0 THEN
1108             VALUE = 1.0
1109         ENDIF
1110     ENDIF
1111 ENDIF
1112
1113 END ;(SCALE_TO_STANDARD)
1114 ;*****
1115 DEF SWRITER(STR_A[:OUT, NUM:IN)
1116 DECL INT nRET, OFFSET, NUM
1117 DECL CHAR STR_A[]
1118 DECL CHAR STR_B[8]
1119 DECL STATE_T STATE
1120
1121 OFFSET = 0;STRLEN(STRING[])+1
1122 SWRITE(STR_B[], STATE, OFFSET, ":%s%d", "TM",NUM)
1123 nRET = STRADD(STR_A[],STR_B[])
1124 END ;(SWRITER)
1125 ;*****
1126 ;* Return value always 1
1127 ;*****
1128 DEFFCT INT STRING_BUILDER(STRING[:OUT, AX_NAME[:IN)
1129 DECL INT nRET
1130 DECL CHAR STRING[]
1131 DECL CHAR AX_NAME[]

```

```

1132
1133   nRET = STRADD(STRING[],AX_NAME[])
1134   RETURN (1)
1135 ENDFCT ;(STRING_BUILDER)
1136 ;*****
1137 ;* Deactivates TQM 1 *
1138 ;*****
1139 DEF TQM_DEACT_TORQUE1(FACT: IN)
1140 DECL REAL FACT
1141
1142   IF TQM_ACT.T11 < 200 THEN
1143     TQM_ACT.T11 = FACT*TQM_ACT.T11
1144   ELSE
1145     TQM_ACT.T11 = 200
1146   ENDIF
1147   IF TQM_ACT.T12 < 200 THEN
1148     TQM_ACT.T12 = FACT*TQM_ACT.T12
1149   ELSE
1150     TQM_ACT.T12 = 200
1151   ENDIF
1152   IF TQM_ACT.T13 < 200 THEN
1153     TQM_ACT.T13 = FACT*TQM_ACT.T13
1154   ELSE
1155     TQM_ACT.T13 = 200
1156   ENDIF
1157   IF TQM_ACT.T14 < 200 THEN
1158     TQM_ACT.T14 = FACT*TQM_ACT.T14
1159   ELSE
1160     TQM_ACT.T14 = 200
1161   ENDIF
1162   IF TQM_ACT.T15 < 200 THEN
1163     TQM_ACT.T15 = FACT*TQM_ACT.T15
1164   ELSE
1165     TQM_ACT.T15 = 200
1166   ENDIF
1167   IF TQM_ACT.T16 < 200 THEN
1168     TQM_ACT.T16 = FACT*TQM_ACT.T16
1169   ELSE
1170     TQM_ACT.T16 = 200
1171   ENDIF
1172
1173 END ;(TQM_DEACT_TORQUE1)
1174 ;*****
1175 ;* Deactivates TQM 2 *
1176 ;*****
1177 DEF TQM_DEACT_TORQUE2(FACT: IN)
1178 DECL REAL FACT
1179
1180   IF (FACT*TQM_ACT.T21 < 1000) THEN
1181     TQM_ACT.T21 = FACT*TQM_ACT.T21
1182   ELSE
1183     TQM_ACT.T21 = TooLarge ;100000
1184   ENDIF
1185   IF (FACT*TQM_ACT.T22 < 1000) THEN
1186     TQM_ACT.T22 = FACT*TQM_ACT.T22
1187   ELSE
1188     TQM_ACT.T22 = TooLarge

```

```

1189     ENDIF
1190     IF (FACT*TQM_ACT.T23 < 1000) THEN
1191         TQM_ACT.T23 = FACT*TQM_ACT.T23
1192     ELSE
1193         TQM_ACT.T23 = TooLarge
1194     ENDIF
1195     IF (FACT*TQM_ACT.T24 < 1000) THEN
1196         TQM_ACT.T24 = FACT*TQM_ACT.T24
1197     ELSE
1198         TQM_ACT.T24 = TooLarge
1199     ENDIF
1200     IF (FACT*TQM_ACT.T25 < 1000) THEN
1201         TQM_ACT.T25 = FACT*TQM_ACT.T25
1202     ELSE
1203         TQM_ACT.T25 = TooLarge
1204     ENDIF
1205     IF (FACT*TQM_ACT.T26 < 1000) THEN
1206         TQM_ACT.T26 = FACT*TQM_ACT.T26
1207     ELSE
1208         TQM_ACT.T26 = TooLarge
1209     ENDIF
1210
1211 END ;(TQM_DEACT_TORQUE2)
1212 ;*****
1213 ;* Set Actual TQM values *
1214 ;*****
1215 DEF SET_TQM_ACT(IVAL: IN)
1216 DECL INT IVAL
1217
1218 TQM_ACT.T11 = IVAL ;Default 200
1219 TQM_ACT.T12 = IVAL
1220 TQM_ACT.T13 = IVAL
1221 TQM_ACT.T14 = IVAL
1222 TQM_ACT.T15 = IVAL
1223 TQM_ACT.T16 = IVAL
1224
1225 TQM_ACT.T21 = IVAL*5 ;Default 200*5
1226 TQM_ACT.T22 = IVAL*5
1227 TQM_ACT.T23 = IVAL*5
1228 TQM_ACT.T24 = IVAL*5
1229 TQM_ACT.T25 = IVAL*5
1230 TQM_ACT.T26 = IVAL*5
1231
1232 END ;(SET_TQM_ACT)
1233 ;*****
1234 DEF UPDATE_DATAS(TQM_VALUES:OUT)
1235 DECL TQM_TQDAT_T TQM_VALUES
1236 DECL CHAR xTQM_TORQPARAM[20]
1237 DECL BOOL bResult
1238
1239 ;no collision no block coincidence no restart no stop no #G0 && no #T1
1240 bRESULT = STRCLEAR(xTQM_TORQPARAM[])
1241 xTQM_TORQPARAM[1]="F"
1242 IF (NOT bTQM_COLLISION AND NOT bTQM_FIRST_AFTER_BCO AND NOT bTQM_RESTART AND NOT bTQM_STOP AND bTQM_PMODE) THEN
1243
1244     IF NOT GetSysState("STOPBITS", iSTOP_TYPE, rPAR) THEN
1245         ENDF

```

```

1246
1247 IF (iSTOP_TYPE == 0) THEN
1248
1249
1250     UPDATE_TMVALUE(TQM_VALUES)
1251     IF bTQM_OV_ONL THEN
1252         bTQM_OV_ONL = FALSE
1253     ELSE
1254         TQM_VALUES.OVM = $OV_PRO
1255     ENDIF
1256
1257     IF CHECK_RVAR(TQM_VALUES.TMF) <> #INITIALIZED THEN
1258         TQM_VALUES.TMF = 1.0
1259     ENDIF
1260
1261     SCALE_TO_STANDARD(TQM_VALUES.TMF,rTQM_REDTMF)
1262 ENDIF
1263 ELSE
1264
1265     IF NOT GetSysState("STOPBITS", iSTOP_TYPE, rPAR) THEN
1266     ENDIF
1267
1268     IF (iSTOP_TYPE <> 0) THEN
1269     ENDIF
1270     IF ((bTQM_RESTART==TRUE)AND(iSTOP_TYPE == 0)) THEN
1271         UPDATE_TMVALUE(TQM_VALUES) ;after delay
1272
1273         IF bTQM_OV_ONL THEN
1274             bTQM_OV_ONL = FALSE
1275         ELSE
1276             TQM_VALUES.OVM = $OV_PRO
1277         ENDIF
1278
1279         IF CHECK_RVAR(TQM_VALUES.TMF) <> #INITIALIZED THEN
1280             TQM_VALUES.TMF = 1.0
1281         ENDIF
1282
1283         SCALE_TO_STANDARD(TQM_VALUES.TMF,rTQM_REDTMF)
1284     ELSE
1285
1286         IF (iSTOP_TYPE == 0) THEN
1287
1288
1289         ENDIF
1290
1291     ENDIF
1292
1293 ENDIF
1294
1295 IF bTQM_STOP THEN
1296     bTQM_STOP = FALSE
1297     TQM_VALUES.T21 = $TORQ_DIFF2[1] * 1.2
1298     TQM_VALUES.T22 = $TORQ_DIFF2[2] * 1.2
1299     TQM_VALUES.T23 = $TORQ_DIFF2[3] * 1.2
1300     TQM_VALUES.T24 = $TORQ_DIFF2[4] * 1.2
1301     TQM_VALUES.T25 = $TORQ_DIFF2[5] * 1.2
1302     TQM_VALUES.T26 = $TORQ_DIFF2[6] * 1.2

```

```

1303   ENDIF
1304
1305   STORE_MOT_TEMP(TQM_VALUES)
1306
1307 END ;(UPDATE_DATAS)
1308 ;*****
1309 ;* Update of values also at spline
1310 ;*****
1311 DEF UPDATE_TMVALUE(TQM_VALUES:OUT)
1312 DECL TQM_TQDAT_T TQM_VALUES
1313
1314   TQM_VALUES.T11=$TORQ_DIFF[1]
1315   TQM_VALUES.T12=$TORQ_DIFF[2]
1316   TQM_VALUES.T13=$TORQ_DIFF[3]
1317   TQM_VALUES.T14=$TORQ_DIFF[4]
1318   TQM_VALUES.T15=$TORQ_DIFF[5]
1319   TQM_VALUES.T16=$TORQ_DIFF[6]
1320
1321
1322   IF (( $OV_PRO == TQM_VALUES.OVM) AND (TQM_VALUES.ID<>0)AND(TQM_VALUES.OVM<>0)) THEN
1323
1324     ;FOLD Force dynamical as average
1325     IF (TQM_VALUES.T21 < $TORQ_DIFF2[1]) THEN
1326       TQM_VALUES.T21 = $TORQ_DIFF2[1]
1327     ELSE
1328       TQM_VALUES.T21 = (TQM_VALUES.T21 + $TORQ_DIFF2[1])/2
1329     ENDIF
1330
1331     IF (TQM_VALUES.T22 < $TORQ_DIFF2[2]) THEN
1332       TQM_VALUES.T22 = $TORQ_DIFF2[2]
1333     ELSE
1334       TQM_VALUES.T22 = (TQM_VALUES.T22 + $TORQ_DIFF2[2])/2
1335     ENDIF
1336
1337     IF (TQM_VALUES.T23 < $TORQ_DIFF2[3]) THEN
1338       TQM_VALUES.T23 = $TORQ_DIFF2[3]
1339     ELSE
1340       TQM_VALUES.T23 = (TQM_VALUES.T23 + $TORQ_DIFF2[3])/2
1341     ENDIF
1342
1343     IF (TQM_VALUES.T24 < $TORQ_DIFF2[4]) THEN
1344       TQM_VALUES.T24 = $TORQ_DIFF2[4]
1345     ELSE
1346       TQM_VALUES.T24 = (TQM_VALUES.T24 + $TORQ_DIFF2[4])/2
1347     ENDIF
1348
1349     IF (TQM_VALUES.T25 < $TORQ_DIFF2[5]) THEN
1350       TQM_VALUES.T25 = $TORQ_DIFF2[5]
1351     ELSE
1352       TQM_VALUES.T25 = (TQM_VALUES.T25 + $TORQ_DIFF2[5])/2
1353     ENDIF
1354
1355     IF (TQM_VALUES.T26 < $TORQ_DIFF2[6]) THEN
1356       TQM_VALUES.T26 = $TORQ_DIFF2[6]
1357     ELSE
1358       TQM_VALUES.T26 = (TQM_VALUES.T26 + $TORQ_DIFF2[6])/2
1359     ENDIF

```

```

1360      ;ENDFOLD
1361  ELSE
1362      ;Override changed
1363      TQM_VALUES.T11 = $TORQ_DIFF[1]
1364      TQM_VALUES.T12 = $TORQ_DIFF[2]
1365      TQM_VALUES.T13 = $TORQ_DIFF[3]
1366      TQM_VALUES.T14 = $TORQ_DIFF[4]
1367      TQM_VALUES.T15 = $TORQ_DIFF[5]
1368      TQM_VALUES.T16 = $TORQ_DIFF[6]
1369
1370      TQM_VALUES.T21 = $TORQ_DIFF2[1]
1371      TQM_VALUES.T22 = $TORQ_DIFF2[2]
1372      TQM_VALUES.T23 = $TORQ_DIFF2[3]
1373      TQM_VALUES.T24 = $TORQ_DIFF2[4]
1374      TQM_VALUES.T25 = $TORQ_DIFF2[5]
1375      TQM_VALUES.T26 = $TORQ_DIFF2[6]
1376  ENDIF
1377
1378  END ;(UPDATE_TMVALUE)
1379  ,*****
1380  ;FOLD <Help procedures>
1381  ,*****
1382  ;* Load and start timer *
1383  ,*****
1384  DEF TQM_IPO_TIMER(INDEX: IN, VALUE: IN)
1385  DECL INT INDEX, VALUE
1386
1387      $TIMER_STOP[INDEX] = TRUE
1388      $TIMER[INDEX] = (-VALUE * iTQM_MULTIPLIER) ;Multiplier =60
1389      $TIMER_STOP[INDEX] = FALSE
1390  END ;(TQM_IPO_TIMER)
1391  ,*****
1392  GLOBAL DEFFCT INT TQM_DLY()
1393
1394      RETURN ($FILTER/2 + 48) ;before 36
1395  ENDFCT
1396  ,*****
1397  ;* Avoids runtime error at          *
1398  ;* activating of undeclared interrupt *
1399  ,*****
1400  DEF CALL_INTERRUPT_ON(iNUM:IN)
1401
1402  DECL INT iNUM
1403
1404      ON_ERROR_PROCEED
1405      INTERRUPT ON iNUM
1406
1407      SWITCH($ERR.NUMBER)
1408      CASE 0
1409          RETURN
1410      CASE 1430
1411
1412          IF (iNUM == iTQM_INTERRUPT) THEN
1413              GLOBAL INTERRUPT DECL iNUM WHEN $CYCFLAG[iTQM_CYCFLAG] == TRUE DO TQM_INTER_STOP ()
1414          ELSE
1415              GLOBAL INTERRUPT DECL iNUM WHEN $OV_PRO <> iTQM_LAST_OVPRO DO TQM_CHANGE_OFFSETS ()
1416          ENDIF

```

```

1417     INTERRUPT ON iNUM
1418     ERR_CLEAR($ERR)
1419     DEFAULT
1420     ERR_RAISE($ERR)
1421     ENDSWITCH
1422
1423 END ;(CALL_INTERRUPT_ON)
1424 ;*****
1425 ;* Avoids runtime with VarState Check *
1426 ;*****
1427 DEFFCT VAR_STATE CHECK_IVAR(iPARAM:OUT)
1428 DECL INT iPARAM, iCheck
1429
1430 ON_ERROR_PROCEED
1431 iCHECK = iPARAM
1432
1433 SWITCH($ERR.NUMBER)
1434     CASE 0
1435         RETURN #INITIALIZED
1436     CASE 1422
1437
1438         ERR_CLEAR($ERR)
1439     DEFAULT
1440         ERR_RAISE($ERR)
1441     ENDSWITCH
1442
1443     RETURN (#DECLARED)
1444 ENDFCT ;(VAR_STATE CHECK_IVAR)
1445 ;*****
1446 DEFFCT VAR_STATE CHECK_RVAR(rPARAM:OUT)
1447 DECL REAL rPARAM, rCheck
1448
1449 ON_ERROR_PROCEED
1450 rCHECK = rPARAM
1451
1452 SWITCH($ERR.NUMBER)
1453     CASE 0
1454         RETURN #INITIALIZED
1455     CASE 1422
1456
1457         ERR_CLEAR($ERR)
1458     DEFAULT
1459         ERR_RAISE($ERR)
1460     ENDSWITCH
1461
1462     RETURN #DECLARED
1463 ENDFCT ;(VAR_STATE CHECK_RVAR)
1464 ;*****
1465 DEFFCT VAR_STATE CHECK_SET_IVAR(iPARAM:OUT, iDEFAULT:IN)
1466 DECL INT iPARAM, iDEFAULT, iCHECK
1467
1468 ON_ERROR_PROCEED
1469 iCHECK = iPARAM
1470
1471 SWITCH($ERR.NUMBER)
1472     CASE 0
1473         RETURN #INITIALIZED

```

```

1474     CASE 1422
1475
1476         iParam = iDEFAULT
1477         ERR_CLEAR($ERR)
1478     DEFAULT
1479         ERR_RAISE($ERR)
1480 ENDSWITCH
1481
1482 RETURN (#DECLARED)
1483 ENDFCT ;(VAR_STATE CHECK_SET_IVAR)
1484 ;*****
1485 DEFFCT VAR_STATE CHECK_SET_RVAR(rPARAM:OUT, rDEFAULT:IN)
1486 DECL REAL rPARAM, rDEFAULT, rCHECK
1487
1488 ON_ERROR_PROCEED
1489 rCHECK = rPARAM
1490 SWITCH($ERR.NUMBER)
1491     CASE 0
1492         RETURN #INITIALIZED
1493     CASE 1422
1494
1495         rPARAM = rDEFAULT
1496         ERR_CLEAR($ERR)
1497     DEFAULT
1498         ERR_RAISE($ERR)
1499 ENDSWITCH
1500
1501 RETURN #DECLARED
1502 ENDFCT ;(VAR_STATE CHECK_SET_RVAR)
1503 ;*****
1504 DEFFCT VAR_STATE CHECK_SET_BVAR(bPARAM:OUT, bDEFAULT:IN)
1505 DECL BOOL bPARAM, bDEFAULT, bCHECK
1506
1507 ON_ERROR_PROCEED
1508 bCHECK = bPARAM
1509 SWITCH($ERR.NUMBER)
1510     CASE 0
1511         RETURN #INITIALIZED
1512     CASE 1422
1513
1514         bParam = bDEFAULT
1515         ERR_CLEAR($ERR)
1516     DEFAULT
1517         ERR_RAISE($ERR)
1518 ENDSWITCH
1519
1520 RETURN (#DECLARED)
1521 ENDFCT ;(VAR_STATE CHECK_SET_BVAR)
1522 ;*****
1523 ;* Varstate check of variables *
1524 ;* bTQM_FIRST_AFTER_BCO,bTQM_RINGMEM_INIT_C *
1525 ;* bTQM_OV_ONL and bTQM_UP_INIT *
1526 ;*****
1527 DEF CHECK_VARSTATES()
1528 DECL VAR_STATE eRET
1529
1530 eRET = CHECK_SET_BVAR(bTQM_FIRST_AFTER_BCO, FALSE)

```

```

1531 eRET = CHECK_SET_BVAR(bTQM_RINGMEM_INIT_C, FALSE)
1532 eRET = CHECK_SET_BVAR(bTQM_OV_ONL, FALSE)
1533 eRET = CHECK_SET_BVAR(bTQM_UP_INIT, TRUE) ;Requested TQM_INI()
1534
1535 END ;(CheckVarStates)
1536 ;*****
1537 DEFFCT BOOL IS_MSTEP_MOVE()
1538
1539 IF ($MODE_OP<>#EX) AND (($VW_PSPS_STOP B_AND 4) > 0) THEN
1540
1541     RETURN (TRUE)
1542 ELSE
1543     RETURN (FALSE)
1544 ENDIF
1545
1546 ENDFCT ;(IS_MSTEP_MOVE)
1547 ;*****
1548 ;* Warmup Check *
1549 ;*****
1550 DEFFCT BOOL IS_WARMUP()
1551
1552 IF ISMESSAGESET(375) THEN
1553     RETURN TRUE
1554 ELSE
1555     RETURN FALSE
1556 ENDIF
1557
1558 ENDFCT
1559 ;*****
1560 ;* Own TQM Notify message routine *
1561 ;*****
1562 DEF MSG_TQM(MSG:IN, PARAM[]:OUT)
1563 DECL KRLMSG_T MSG
1564 DECL KRLMSGPAR_T PAR[1]
1565 DECL KRLMSGOPT_T OPT
1566 DECL BOOL bRESULT
1567 DECL CHAR PARAM[]
1568 DECL INT nHandle
1569
1570
1571 PAR[1].PAR_TXT[]=" "
1572 bRESULT= STRCOPY(PAR[1].PAR_TXT[],PARAM[])
1573 PAR[1].PAR_TYPE=#Value
1574 OPT={VL_STOP TRUE, CLEAR_P_RESET TRUE, LOG_TO_DB TRUE}
1575 nHandle=SET_KRLMSG(#Notify,MSG,PAR[],OPT)
1576
1577 END ;(MSG_TQM)
1578 ;*****
1579 ;* Check of running submit *
1580 ;*****
1581 DEF CHECK_SUBMIT_RUNS()
1582 DECL KRLMSG_T MSG
1583 DECL KRLMSGPAR_T PAR[1]
1584 DECL KRLMSGOPT_T OPT
1585 DECL INT nHandle
1586
1587 IF NOT bTQM_SubICheck THEN

```

```

1588     RETURN
1589 ENDIF
1590
1591 MSG={MODUL[] "TQM", NR 12, MSG_TXT[] "SubmitNotActive"}
1592 OPT={VL_STOP TRUE, CLEAR_P_RESET TRUE, LOG_TO_DB FALSE}
1593
1594 WHILE $PRO_STATE0 <> #P_ACTIVE
1595     nHandle=SET_KRLMSG(#State,MSG,PAR[],OPT)
1596     IF nHANDLE > 0 THEN
1597         WHILE $PRO_STATE0 <> #P_ACTIVE
1598             WAIT SEC 0.06
1599         ENDWHILE
1600         WAIT FOR CLEAR_KRLMSG(nHandle)
1601     ELSE
1602         HALT ; R1 => (TQM_INI)
1603     ENDIF
1604 ENDWHILE
1605 END ;(CHECK_SUBMIT_RUNS)
1606

```

KRC/R1/TP/AutomationCore/automationcoreroutines.src

```

1 &ACCESS R03
2 &REL 11
3 DEF AutomationCoreRoutines( )
4
5 END
6
7 GLOBAL DEF AutomationCore_Bkg( )
8     ;FOLD Details
9     ;FOLD Declaration
10     DECL INT nAC_MsgQty
11     DECL MSGBUF_T gAC_MsgLog[100]
12     DECL INT nAC_MsgCtr
13     DECL BOOL bTDVlvMan
14     DECL BOOL bSRWTD
15     DECL BOOL bEOATIn
16     DECL BOOL bWeldCtrl1
17     DECL BOOL bRPWSealrDArc1
18     DECL BOOL bPSE
19     DECL BOOL bMigRmr
20     DECL BOOL bEOATVlvMan
21     DECL BOOL bVlvMan
22     DECL BOOL bRPWWeldCtrl
23     DECL BOOL bRPWSealrDArc2
24     DECL BOOL bVlvMan2
25     DECL BOOL bWeldCtrl2
26     ;ENDFOLD (Declaration)
27     ;FOLD Implementation
28     ; GE4-002 cell: only nut welder 1 (pw1). Welder 2/3 signals removed 2026-10-05
29     IF (di004UseDryCycle==TRUE) THEN
30         dopw1_WeldContactEnable=FALSE
31         dopw1_WeldOnExternal=FALSE
32     ELSE
33         dopw1_WeldContactEnable=TRUE
34         dopw1_WeldOnExternal=TRUE
35     ENDIF
36     ; TOOL ID TO PLC

```

```

37      ; see go123ToolID
38      do123ToolIDBit0 = di823ToolIDBit0
39      do124ToolIDBit1 = di825ToolIDBit1
40      AC_Request_to_enter( )
41      ;FOLD Mainline Auto Select
42      IF (($PRO_STATE1==#P_FREE) AND ($MODE_OP==#EX) AND (bACMainLine==FALSE)) THEN
43          bACMainLine=TRUE
44          CWRITE($CMD,AC_STAT,AC_MODE,"RUN /R1/CELL ()")
45      ELSE
46          IF ($MODE_OP==#T1) OR ($MODE_OP==#T2) THEN
47              bACMainLine=FALSE
48          ENDIF
49      ENDIF
50      IF (($MODE_OP==#EX) AND (bACMainLineReset==TRUE)) THEN
51          IF (($PRO_STATE1==#P_RESET) OR ($PRO_STATE1==#P_ACTIVE) OR ($PRO_STATE1==#P_STOP) OR ($PRO_STATE1==#P_END)) THEN
52              CWRITE($CMD,AC_STAT,AC_MODE,"STOP 1")
53              WAIT SEC 0.100
54              CWRITE($CMD,AC_STAT,AC_MODE,"CANCEL 1")
55              bACMainLineReset=FALSE
56              bACMainLine=FALSE
57          ENDIF
58      ENDIF
59      ;ENDFOLD (Program Reset/Cancel)
60      ;FOLD Program Reset/Cancel
61      IF ((($PRO_STATE1==#P_FREE) OR ($PRO_STATE1==#P_RESET)) AND (bACProcessReset==FALSE)) THEN
62          bACProcessReset=TRUE
63          AC_ProcessReset( )
64      ELSE
65          IF (($PRO_STATE1<>#P_FREE) AND ($PRO_STATE1<>#P_RESET)) THEN
66              bACProcessReset=FALSE
67          ENDIF
68      ENDIF
69      ;ENDFOLD (Program Reset/Cancel)
70      ;FOLD Automation Core Cyclical Monitoring
71      ;FOLD Battery Monitoring
72      SWITCH $ACCU_STATE
73          CASE #CHARGE_OK
74              do006RobotBatteryAlarm=FALSE
75          CASE #CHARGE_OK_LOW
76              do006RobotBatteryAlarm=TRUE
77          CASE #CHARGE_UNKNOWN
78              do006RobotBatteryAlarm=TRUE
79          CASE #CHARGE_TEST_NOK
80              do006RobotBatteryAlarm=TRUE
81          CASE #CHARGE_NOK
82              do006RobotBatteryAlarm=TRUE
83          CASE #CHARGE_OFF
84              do006RobotBatteryAlarm=TRUE
85      END SWITCH
86      ;ENDFOLD (Battery Monitoring)
87      do046WeldMode = (NOT di004UseDryCycle)
88      do021BrakeTestReqd = (($BRAKETEST_MONTIME==TRUE) OR ($BRAKETEST_REQ_EX==TRUE) OR ($BRAKETEST_REQ_INT==TRUE))
89      do022MstrRefRequired = ($MasteringTest_Req_Int==TRUE)
90      gdoAxis1Posn=$AXIS_ACT.A1 + nA10ffset
91      do094AirOK= dipw1_AirOK ; Air pressure switch ($IN[3132]) to PLC - only nut welder 1
92      do013WaterIsOn = dipw1_WaterOk ; Water flow OK (SV0500, calculated in sps.sub) to PLC - only nut welder 1
93      do107Hopper1LowLevel= NOT dipw1_HopperLowLevel

```

```

94      do111Gun1ElectrodeChange= dipw1_EndofStepper
95      do118Nut1Ready= (dipw1_Ready AND NOT dipw1_Fault)
96      do134Nut1Fault= (dipw1_Fault OR NOT dipw1_Ready)
97
98      ; do475StartWater ($OUT[475]) follows di013WaterEnable ($IN[13]) - set in sps.sub
99
100     do464WaterBeacon = (di472WaterTempOK AND di474WaterOk)
101
102     ; sends to PLC when the robot isn't in 100%
103     IF (($MODE_OP==#EX) AND NOT ($OV_PRO==100)) THEN
104         do146SpeedNot100=TRUE
105     ELSE
106         do146SpeedNot100=FALSE
107     ENDIF
108
109     ;FOLD Positional Monitoring
110     IF (VARSTATE("$POS_ACT")==#INITIALIZED) THEN
111         ON_ERROR_PROCEED
112         PosCheck = $POS_ACT
113         IF ($ERR.NUMBER<>1419) THEN
114             do007RobotAtPounce=AC_CheckSphWZ(xPounce,$Pounce_POS_TOL)
115             do009RobotAtRepair=AC_CheckSphWZ(xRepair,$Repair_POS_TOL)
116             do010RobotAtTipDress=AC_CheckSphWZ(xTipDress,$TipDress_POS_TOL)
117             do051RbtAtMstrRefSwitch=AC_CheckSphWZ(xRefSwitch,$RefSwitch_POS_TOL)
118             IF ((($WORKSPACE[1].X<>0.0) OR ($WORKSPACE[1].Y<>0.0) OR ($WORKSPACE[1].Z<>00)) AND ($WORKSPACE[1].MODE==#
OFF))
THEN
119                 $WORKSPACE[1].MODE=#OUTSIDE
120             ENDIF
121             IF ((($WORKSPACE[2].X<>0.0) OR ($WORKSPACE[2].Y<>0.0) OR ($WORKSPACE[2].Z<>00)) AND ($WORKSPACE[2].MODE==#
OFF))
THEN
122                 $WORKSPACE[2].MODE=#OUTSIDE
123             ENDIF
124         ELSE
125             IF ($WORKSPACE[1].MODE<>#OFF) THEN
126                 $WORKSPACE[1].MODE=#OFF
127             ENDIF
128             IF ($WORKSPACE[2].MODE<>#OFF) THEN
129                 $WORKSPACE[2].MODE=#OFF
130             ENDIF
131         ENDIF
132     ENDIF
133     ;ENDFOLD (Positional Monitoring)
134     ;ENDFOLD (Automation Core Cyclical Monitoring)
135     ;FOLD Bus Status IO
136     ;FOLD Initialize boolean variables
137     bTDVlvMan = FALSE
138     bSRWTD = FALSE
139     bEOATIn = FALSE
140     bWeldCtrl1 = FALSE
141     bRPWSealrDArc1 = FALSE
142     bPSE = FALSE
143     bMigRmr = FALSE
144     bEOATVlvMan = FALSE
145     bVlvMan = FALSE
146     bRPWweldCtrl = FALSE
147     bRPWSealrDArc2 = FALSE
148     bVlvMan2 = FALSE

```

```

149         bWeldCtrl2 = FALSE
150     ;ENDFOLD (Initialize boolean variables)
151     ;FOLD Check IO Status
152     IF ((IsMessageSet(10043)==TRUE) OR (IsMessageSet(10045)==TRUE)) THEN
153         nAC_MsgQty = GET_MSGBUFFER(gAC_MsgLog[])
154         IF (nAC_MsgQty > 0) THEN
155             FOR nAC_MsgCtr = 1 TO nAC_MsgQty
156                 IF ((gAC_MsgLog[nAC_MsgCtr].NR==10043) OR (gAC_MsgLog[nAC_MsgCtr].NR==10045)) THEN
157                     bRPWweldCtrl = bRPWweldCtrl OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "RPW_WELD_CONTROLLER", #NOT_
CASE_
SENS)
158                     bTDVlvMan = bTDVlvMan OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "TD_VLV_MANIFOLD", #NOT_CASE_SENS)
159                     bEOATVlvMan = bEOATVlvMan OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "EOAT_VLV_MANIFOLD", #NOT_CASE
SENS
)
160                     bMigRmr = bMigRmr OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "MIG_REAMER", #NOT_CASE_SENS)
161                     bSRWTD = bSRWTD OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "SRW_TD", #NOT_CASE_SENS)
162                     bEOATIn = bEOATIn OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "EOAT_INPUTS", #NOT_CASE_SENS)
163                     bWeldCtrl1 = bWeldCtrl1 OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "WELD_CONTROLLER", #NOT_CASE_SEN
S)
164                     bRPWSealrDarc1 = bRPWSealrDarc1 OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "RPW_SEALER_DRAWN_ARC",
#NOT_
CASE_SENS)
165                     bPSE = bPSE OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "PSE", #NOT_CASE_SENS)
166                     bRPWSealrDarc2 = bRPWSealrDarc2 OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "RPW_SEALER_DRAWN_ARC_2
", #NO
T_CASE_SENS)
167                     bVlvMan = bVlvMan OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "VLV_MANIFOLD", #NOT_CASE_SENS)
168                     bVlvMan2 = bVlvMan2 OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "VLV_MANIFOLD_2", #NOT_CASE_SENS)
169                     bWeldCtrl2 = bWeldCtrl2 OR STRCOMP(gAC_MsgLog[nAC_MsgCtr].PAR_TXT1[], "WELD_CONTROLLER_2", #NOT_CASE_S
ENS)
170                 ENDIF
171             ENDFOR
172         ENDIF
173     ENDIF
174 ;ENDFOLD (Check IO Status)
175 ;FOLD Set IO Bits
176     do048RPWCtrlrEnetOK = ((bRPWweldCtrl==FALSE) AND (bRPWweldCtrl_Enable==TRUE))
177     do050TDVlvManifold = ((bTDVlvMan==FALSE) AND (bTDVlvMan_Enable==TRUE))
178     do052EOATVlvMnflDEnetOK =true; ((bEOATVlvMan==FALSE) AND (bEOATVlvMan_Enable==TRUE))
179     do053MigReamerEnetOK = ((bMigRmr==FALSE) AND (bMigRmr_Enable==TRUE))
180     do054SRWTipDressIOEnetOK = ((bSRWTD==FALSE) AND (bSRWTD_Enable==TRUE))
181     do055EndEffectorIOEnetOK = ((bEOATIn==FALSE) AND (bEOATIn_Enable==TRUE))
182     do056WeldCntl1EnetOK = ((bWeldCtrl1==FALSE) AND (bWeldCtrl1_Enable==TRUE))
183     do057RPWSlrDarc1EnetOK = ((bRPWSealrDarc1==FALSE) AND (bRPWSealrDarc1_Enable==TRUE))
184     do058PinScribeCntlrIOOK = ((bPSE==FALSE) AND (bPSE_Enable==TRUE))
185     do059RPWSlrDarc2EnetOK = ((bRPWSealrDarc2==FALSE) AND (bRPWSealrDarc2_Enable==TRUE))
186     do060VlvManifoldEnetOK = ((bVlvMan==FALSE) AND (bVlvMan_Enable==TRUE))
187 ;ENDFOLD (Set IO Bits)
188 ;ENDFOLD (Bus Status IO)
189 ;FOLD Wait Message Reset
190     IF ((Exists_KrlMsg(nWaitMsg_Handle)==TRUE) AND ($PRO_STATE1<>P_ACTIVE) AND (bAutoCoreSubmit_Clear==FALSE)) THEN
191         bAutoCoreSubmit_Clear = Clear_KRLMsg(nWaitMsg_Handle)
192     ENDIF
193 ;ENDFOLD (Wait Message Reset)
194 ;ENDFOLD (Implementation)
195 ;ENDFOLD (Details)
196 END ; (AutomationCore_Bkg)
197
198 GLOBAL DEF InitializeSystem( )
199     ;FOLD Details
200     ;FOLD Declaration

```

```

201     DECL CHAR DMY[3]
202     DMY[]="---"
203 ;ENDFOLD (Declaration)
204 ;FOLD Implementation
205 ;FOLD BASISTECH INI
206     GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
207     INTERRUPT ON 3
208     BAS (#INITMOV,0 )
209 ;ENDFOLD (BASISTECH INI)
210 ;FOLD HomeCheck
211     HomeCheck()
212     Stop_at_End_of_Cycle = FALSE
213 ;ENDFOLD (HomeCheck)
214 ;FOLD Process Reset
215     AC_ProcessReset( )
216 ;ENDFOLD (Process Reset)
217 ;ENDFOLD (Implementation)
218 ;ENDFOLD (Details)
219 END ; (InitializeSystem)
220
221 GLOBAL DEF MaintainSystem( )
222 ;FOLD Details
223 ;FOLD Declaration
224 ;ENDFOLD (Declaration)
225 ;FOLD Implementation
226     bOption1=FALSE
227     bOption2=FALSE
228     bOption3=FALSE
229     bOption4=FALSE
230     bOption5=FALSE
231     bOption6=FALSE
232     bOption7=FALSE
233     bOption8=FALSE
234     nStyle=0
235     nOption=0
236     nPickUp=0
237     nApps=0
238     nDropOff=0
239     gdoStyleAckn=0
240     gdoPickupClear=31
241     gdoApplicationClear=31
242     gdoDropOffClear=31
243     gdoRepositionTooling=0
244     gdoRobotClearZone=127
245     do023PickClearTooling=TRUE
246     do024ApplClrTooling=TRUE
247     do025DropClearTooling=TRUE
248     do004ProcessFault=FALSE
249     do005RobotInCycle = FALSE
250     do015WorkComplete=FALSE
251     do047QualityFault = FALSE
252     AC_Process_Count(#INIT)
253     do091RobotReadyCapChange=FALSE
254     gdoOptionAckn=0
255     gdoProcessSync=0
256     do141RedRabbitFailed=FALSE
257     do142RedRabbitPassed=FALSE

```

```

258
259      ; add to stop robot in Maintain system
260      Stop_at_End_of_Cycle = Stop_at_End_of_Cycle OR di147SE0C      ;Operator or PLC request stop at end of cycle
261      WHILE Stop_at_End_of_Cycle
262          $LOOP_MSG[] = "
                "
263          IF di147SE0C THEN
264              $LOOP_MSG[] = " Stop_at_End_of_Cycle is selected by PLC ( Robot Input 147, di147SE0C )
                "
265          ELSE
266              $LOOP_MSG[] = " Stop_at_End_of_Cycle is selected by User
                "
267          ENDIF
268          WAIT SEC 1
269      ENDWHILE
270      $LOOP_MSG[] = "
                "
271
272      ;ENDFOLD (Implementation)
273      ;ENDFOLD (Details)
274  END ; (MaintainSystem)
275
276  GLOBAL DEF HomeCheck( )
277      ;FOLD Details
278      ;FOLD Declaration
279      DECL BOOL bMsgRelease
280      GLOBAL INTERRUPT DECL IR_HOME WHEN $T1==FALSE DO IR_HOME_T1 ( )
281      ;ENDFOLD (Declaration)
282      ;FOLD Implementation
283      bMsgRelease=FALSE
284      WHILE ($IN_HOME==FALSE)
285          IF (($MODE_OP==#EX) AND (do007RobotAtPounce==TRUE)) THEN
286              PounceToHome()
287          ELSE
288              do004ProcessFault=TRUE
289              ;FOLD Dialog "Robot is not home. To Allow Robot To Move Home Select Continue"
290              REPEAT
291                  nAutoCore_Handle = Set_Krldlg (AutomationCore_Msg[3], AutoCore_MsgPar[], AC_SK_ContNav[], AutoCore_MsgOpt_
292  Stop)
293                  IF (nAutoCore_Handle > 0) THEN
294                      WHILE (Exists_Krldlg(nAutoCore_Handle, nAutoCore_Key))
295                          do004ProcessFault=TRUE
296                          WAIT SEC 0.024
297                      ENDWHILE
298                      do004ProcessFault=FALSE
299                      SWITCH nAutoCore_Key
300                          CASE 1 ; Diagnostic
301                              nAutoCore_Handle = Set_Krldlg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
302  top)
303                              WHILE (Exists_Krldlg(nAutoCore_Handle))
304                                  WAIT SEC 0.024
305                              ENDWHILE
306                          CASE 2 ; Continue
307                              REPEAT
308                                  WAIT FOR TRUE
309                                  $OV_PRO=0
310                                  WAIT FOR $T1
311                                  INTERRUPT ON IR_HOME

```

```

310                UNTIL $T1
311                ;FOLD PTP HOME Vel= 100 % DEFAULT ;%{...}
312                $H_POS=XHOME
313                PDAT_ACT=PDEFAULT
314                BAS (#PTP_DAT )
315                FDAT_ACT=FHOME
316                BAS (#FRAMES )
317                BAS (#VEL_PTP,100 )
318                PTP XHOME
319                ;ENDFOLD
320                WAIT FOR TRUE
321                INTERRUPT OFF IR_HOME
322                do004ProcessFault=FALSE
323                bMsgRelease = TRUE
324                ENDSWITCH
325                ENDF
326                UNTIL (bMsgRelease==TRUE)
327                bMsgRelease=FALSE
328                ;ENDFOLD (Dialog)
329                ENDF
330                ENDWHILE
331                ;ENDFOLD (Implementation)
332                ;ENDFOLD (Details)
333 END ; (HomeCheck)
334
335 GLOBAL DEF IR_HOME_T1( )
336     ;FOLD Details
337     ;FOLD Declaration
338     ;ENDFOLD (Declaration)
339     ;FOLD Implementation
340     INTERRUPT OFF IR_HOME
341     $OV_PRO=0
342     WAIT FOR $T1
343     WAIT FOR TRUE
344     INTERRUPT ON IR_HOME
345     ;ENDFOLD (Implementation)
346     ;ENDFOLD (Details)
347 END ; (IR_HOME_T1)
348
349 GLOBAL DEF GetOptions( )
350     ;FOLD Details
351     ;FOLD Declaration
352     ;ENDFOLD (Declaration)
353     ;FOLD Implementation
354     bOption1=((nOption B_AND 1) > 0)
355     do0380ptionAckn1=bOption1
356     bOption2=((nOption B_AND 2) > 0)
357     do0390ptionAckn2=bOption2
358     bOption3=((nOption B_AND 4) > 0)
359     do0400ptionAckn3=bOption3
360     bOption4=((nOption B_AND 8) > 0)
361     do0410ptionAckn4=bOption4
362     bOption5=((nOption B_AND 16) > 0)
363     do0420ptionAckn5=bOption5
364     bOption6=((nOption B_AND 32) > 0)
365     do0430ptionAckn6=bOption6
366     bOption7=((nOption B_AND 64) > 0)

```

```

367         do0440optionAckn7=b0option7
368         b0option8=((n0option_B_AND 128) > 0)
369         do0450optionAckn8=b0option8
370     ;ENDFOLD (Implementation)
371 ;ENDFOLD (Details)
372 END ; (GetOptions)
373
374 GLOBAL DEF EndOfCycle( )
375 ;FOLD Details
376 ;FOLD Declaration
377 ;ENDFOLD (Declaration)
378 INT S,0
379 ;FOLD Implementation
380 ;AC_Process_Count(#CHECK)
381 ;=====SL=====do015WorkComplete=TRUE
382 ;WHILE (di015WorkCompleteAckn==FALSE)
383     ;FOLD Status Waiting for Work Complete Ackn.
384     ;IF (Exists_KrlMsg(nWorkComplete_Handle)==FALSE) THEN
385         ;nWorkComplete_Handle=Set_Krlmsg (#STATE, AutomationCore_Msg[4], AutoCore_MsgPar[], AutoCore_MsgOpt_NoStop)
386     ;ENDIF
387 ;ENDFOLD (Status)
388 ;ENDWHILE
389 ;=====SL=====do005RobotInCycle = FALSE
390 bAutoCore_Clear=Clear_KRLMsg(nWorkComplete_Handle)
391 ;ENDFOLD (Implementation)
392 ;ENDFOLD (Details)
393 END ; (EndOfCycle)
394
395 GLOBAL DEF CriticalWZ(bState :IN )
396 ;FOLD Details
397 ;FOLD Declaration
398 DECL BOOL bState
399 ;ENDFOLD (Declaration)
400 ;FOLD Implementation
401 TRIGGER WHEN DISTANCE=0 DELAY=0 DO doCriticalWZ=bState
402 ;ENDFOLD (Implementation)
403 ;ENDFOLD (Details)
404 END ; (CriticalWZ)
405
406 GLOBAL DEF AC_ProcessSync(nProcess :IN )
407 ;FOLD Details
408 ;FOLD Declaration
409 DECL INT nProcess
410 ;ENDFOLD (Declaration)
411 ;FOLD Implementation
412 gdoProcessSync = AC_POW(2,(nProcess-1))
413 WHILE (gdoProcessSync <> gdiProcessSync)
414     ;FOLD Dialog "Waiting for Process %1 Synchronization(DI-%2).".
415     AutoCore_MsgPar[1]={par_type #value, par_int 0}
416     AutoCore_MsgPar[1].par_int=nProcess
417     AutoCore_MsgPar[2]={par_type #value, par_int 0}
418     AutoCore_MsgPar[2].par_int=nProcess + 60
419     nProcRequest_Handle = Set_Krlmsg (#STATE, AutomationCore_Msg[18], AutoCore_MsgPar[], AutoCore_MsgOpt_NoStop)
420     IF (nProcRequest_Handle > 0) THEN
421         WHILE (Exists_KrlMsg(nProcRequest_Handle))
422             WAIT SEC 0.012
423         IF (gdoProcessSync == gdiProcessSync) THEN

```

```

424             bAutoCore_Clear=Clear_KRLMsg(nProcRequest_Handle)
425         ENDIF
426     ENDWHILE
427     ENDIF
428     ;ENDFOLD (Dialog)
429     ENDWHILE
430     gdoProcessSync=0
431     ;ENDFOLD (Implementation)
432     ;ENDFOLD (Details)
433 END ; (AC_ProcessSync)
434
435 GLOBAL DEF AC_Application(nApplication :IN, bState :IN )
436     ;FOLD Details
437     ;FOLD Declaration
438     DECL INT nApplication
439     DECL BOOL bState
440     DECL BOOL bMsgRelease
441     ;ENDFOLD (Declaration)
442     ;FOLD Implementation
443     bMsgRelease=FALSE
444     SWITCH nApplication
445         CASE 1
446             do070ApplicationClear1 = bState
447         CASE 2
448             do071ApplicationClear2 = bState
449         CASE 3
450             do072ApplicationClear3 = bState
451         CASE 4
452             do073ApplicationClear4 = bState
453         CASE 5
454             do074ApplicationClear5 = bState
455         DEFAULT
456             do004ProcessFault=TRUE
457             ;FOLD Dialog "Application %1 is invalid. Reprogram with application 1 - 5."
458             REPEAT
459                 AutoCore_MsgPar[1]={par_type #value, par_int 0}
460                 AutoCore_MsgPar[1].par_int=nApplication
461                 nAutoCore_Handle = Set_KrDlg (AutomationCore_Msg[8], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_MsgOpt_Sto
p)
462             IF (nAutoCore_Handle > 0) THEN
463                 WHILE (Exists_KrDlg(nAutoCore_Handle, nAutoCore_Key))
464                     do004ProcessFault=TRUE
465                     IF ($ERR.Int_Prio>0) THEN
466                         BRAKE
467                     ENDIF
468                     WAIT SEC 0.024
469                 ENDWHILE
470                 do004ProcessFault=FALSE
471                 SWITCH nAutoCore_Key
472                     CASE 1 ; Diagnostic
473                         do004ProcessFault=FALSE
474                         nAutoCore_Handle = Set_Krlmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
top)
475                         WHILE (Exists_KrlMsg(nAutoCore_Handle))
476                             WAIT SEC 0.024
477                         ENDWHILE
478                     ENDSWITCH
479                 ENDIF
480             UNTIL (bMsgRelease==TRUE)

```

```

481             bMsgRelease=FALSE
482         ;ENDFOLD (Dialog)
483     ENDSWITCH
484     ;ENDFOLD (Implementation)
485 ;ENDFOLD (Details)
486 END ; (AC_Application)
487
488 GLOBAL DEF AC_ApplicationCheck(nApplication :IN )
489     ;FOLD Details
490     ;FOLD Declaration
491     DECL INT nApplication
492     ;ENDFOLD (Declaration)
493     ;FOLD Implementation
494     SWITCH nApplication
495     CASE 1
496         ;FOLD Dialog "Waiting for Application %1 permission to enter (DI-%2).".
497         AutoCore_MsgPar[1]={par_type #value, par_int 0}
498         AutoCore_MsgPar[1].par_int=nApplication
499         AutoCore_MsgPar[2]={par_type #value, par_int 0}
500         AutoCore_MsgPar[2].par_int=nApplication + 69
501         SetWaitingMessage(AutomationCore_Msg[17], di070ApplicationMachine1, TRUE )
502     ;ENDFOLD (Dialog)
503     do070ApplicationClear1 = FALSE
504     do024ApplClrTooling = FALSE
505     CASE 2
506         ;FOLD Dialog "Waiting for Application %1 permission to enter (DI-%2).".
507         AutoCore_MsgPar[1]={par_type #value, par_int 0}
508         AutoCore_MsgPar[1].par_int=nApplication
509         AutoCore_MsgPar[2]={par_type #value, par_int 0}
510         AutoCore_MsgPar[2].par_int=nApplication + 69
511         SetWaitingMessage(AutomationCore_Msg[17], di071ApplicationMachine2, TRUE )
512     ;ENDFOLD (Dialog)
513     do071ApplicationClear2 = FALSE
514     do024ApplClrTooling = FALSE
515     CASE 3
516         ;FOLD Dialog "Waiting for Application %1 permission to enter (DI-%2).".
517         AutoCore_MsgPar[1]={par_type #value, par_int 0}
518         AutoCore_MsgPar[1].par_int=nApplication
519         AutoCore_MsgPar[2]={par_type #value, par_int 0}
520         AutoCore_MsgPar[2].par_int=nApplication + 69
521         SetWaitingMessage(AutomationCore_Msg[17], di072ApplicationMachine3, TRUE )
522     ;ENDFOLD (Dialog)
523     di072ApplicationMachine3 = FALSE
524     do024ApplClrTooling = FALSE
525     CASE 4
526         ;FOLD Dialog "Waiting for Application %1 permission to enter (DI-%2).".
527         AutoCore_MsgPar[1]={par_type #value, par_int 0}
528         AutoCore_MsgPar[1].par_int=nApplication
529         AutoCore_MsgPar[2]={par_type #value, par_int 0}
530         AutoCore_MsgPar[2].par_int=nApplication + 69
531         SetWaitingMessage(AutomationCore_Msg[17], di073ApplicationMachine4, TRUE )
532     ;ENDFOLD (Dialog)
533     di073ApplicationMachine4 = FALSE
534     do024ApplClrTooling = FALSE
535     CASE 5
536         ;FOLD Dialog "Waiting for Application %1 permission to enter (DI-%2).".
537         AutoCore_MsgPar[1]={par_type #value, par_int 0}

```

```

538         AutoCore_MsgPar[1].par_int=nApplication
539         AutoCore_MsgPar[2]={par_type #value, par_int 0}
540         AutoCore_MsgPar[2].par_int=nApplication + 69
541         SetWaitingMessage(AutomationCore_Msg[17], di074ApplicationMachine5, TRUE )
542     ;ENDFOLD (Dialog)
543     di074ApplicationMachine5 = FALSE
544     do024ApplClrTooling = FALSE
545     DEFAULT
546         do004ProcessFault=TRUE
547     ;FOLD Dialog "Application %1 is invalid. Reprogram with application 1 - 5."
548     LOOP
549         AutoCore_MsgPar[1]={par_type #value, par_int 0}
550         AutoCore_MsgPar[1].par_int=nApplication
551         nAutoCore_Handle = Set_KrLDlg (AutomationCore_Msg[8], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_MsgOpt_Sto
p)
552     IF (nAutoCore_Handle > 0) THEN
553         WHILE (Exists_KrLDlg(nAutoCore_Handle, nAutoCore_Key))
554             do004ProcessFault=TRUE
555             IF ($ERR.Int_Prio>0) THEN
556                 BRAKE
557             ENDIF
558             WAIT SEC 0.024
559         ENDWHILE
560         do004ProcessFault=FALSE
561         SWITCH nAutoCore_Key
562             CASE 1 ; Diagnostic
563                 do004ProcessFault=FALSE
564                 nAutoCore_Handle = Set_KrLmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
top)
565                 WHILE (Exists_KrLmsg(nAutoCore_Handle))
566                     WAIT SEC 0.024
567                 ENDWHILE
568             ENDSWITCH
569         ENDIF
570     ENDLLOOP
571     ;ENDFOLD (Dialog)
572     ENDSWITCH
573     ;ENDFOLD (Implementation)
574     ;ENDFOLD (Details)
575 END ; (AC_ApplicationCheck)
576
577 GLOBAL DEF AC_PickUp(nPickUp :IN, bState :IN )
578     ;FOLD Details
579     ;FOLD Declaration
580     DECL INT nPickUp
581     DECL BOOL bState
582     DECL BOOL bMsgRelease
583     ;ENDFOLD (Declaration)
584     ;FOLD Implementation
585     bMsgRelease=FALSE
586     SWITCH nPickUp
587         CASE 1
588             do065PickupClear1 = bState
589         CASE 2
590             do066PickupClear2 = bState
591         CASE 3
592             do067PickupClear3 = bState
593         CASE 4
594             do068PickupClear4 = bState

```

```

595     CASE 5
596         do069PickupClear5 = bState
597     DEFAULT
598         do004ProcessFault=TRUE
599         ;FOLD Dialog "PickUp %1 is invalid. Reprogram with Pickup 1 - 5."
600         REPEAT
601             AutoCore_MsgPar[1]={par_type #value, par_int 0}
602             AutoCore_MsgPar[1].par_int=nPickUp
603             nAutoCore_Handle = Set_KrDlG (AutomationCore_Msg[9], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_MsgOpt_Sto
p)
604             IF (nAutoCore_Handle > 0) THEN
605                 WHILE (Exists_KrDlG(nAutoCore_Handle, nAutoCore_Key))
606                     do004ProcessFault=TRUE
607                     IF ($ERR.Int_Prio>0) THEN
608                         BRAKE
609                     ENDIF
610                     WAIT SEC 0.024
611                 ENDWHILE
612                 do004ProcessFault=FALSE
613                 SWITCH nAutoCore_Key
614                     CASE 1 ; Diagnostic
615                         do004ProcessFault=FALSE
616                         nAutoCore_Handle = Set_KrLmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
top)
617                         WHILE (Exists_KrLmsg(nAutoCore_Handle))
618                             WAIT SEC 0.024
619                         ENDWHILE
620                     ENDSWITCH
621                 ENDIF
622                 UNTIL (bMsgRelease==TRUE)
623                 bMsgRelease=FALSE
624             ;ENDFOLD (Dialog)
625         ENDSWITCH
626     ;ENDFOLD (Implementation)
627 ;ENDFOLD (Details)
628 END ; (AC_PickUp)
629
630 GLOBAL DEF AC_PickUpCheck(nPickUp :IN )
631 ;FOLD Details
632 ;FOLD Declaration
633     DECL INT nPickUp
634 ;ENDFOLD (Declaration)
635 ;FOLD Implementation
636     SWITCH nPickUp
637     CASE 1
638         ;FOLD Dialog "Waiting for Pickup %1 permission to enter (DI-%2).".
639         AutoCore_MsgPar[1]={par_type #value, par_int 0}
640         AutoCore_MsgPar[1].par_int=nPickUp
641         AutoCore_MsgPar[2]={par_type #value, par_int 0}
642         AutoCore_MsgPar[2].par_int=nPickUp + 64
643         SetWaitingMessage(AutomationCore_Msg[15], di065PickupMachine1, TRUE )
644     ;ENDFOLD (Dialog)
645     do065PickupClear1=FALSE
646     do023PickClearTooling=FALSE
647     CASE 2
648         ;FOLD Dialog "Waiting for Pickup %1 permission to enter (DI-%2).".
649         AutoCore_MsgPar[1]={par_type #value, par_int 0}
650         AutoCore_MsgPar[1].par_int=nPickUp
651         AutoCore_MsgPar[2]={par_type #value, par_int 0}

```

```

652         AutoCore_MsgPar[2].par_int=nPickUp + 64
653         SetWaitingMessage(AutomationCore_Msg[15], di066PickupMachine2, TRUE )
654     ;ENDFOLD (Dialog)
655     do066PickupClear2=FALSE
656     do023PickClearTooling=FALSE
657 CASE 3
658     ;FOLD Dialog "Waiting for Pickup %1 permission to enter (DI-%2).".
659     AutoCore_MsgPar[1]={par_type #value, par_int 0}
660     AutoCore_MsgPar[1].par_int=nPickUp
661     AutoCore_MsgPar[2]={par_type #value, par_int 0}
662     AutoCore_MsgPar[2].par_int=nPickUp + 64
663     SetWaitingMessage(AutomationCore_Msg[15], di067PickupMachine3, TRUE )
664     ;ENDFOLD (Dialog)
665     do067PickupClear3=FALSE
666     do023PickClearTooling=FALSE
667 CASE 4
668     ;FOLD Dialog "Waiting for Pickup %1 permission to enter (DI-%2).".
669     AutoCore_MsgPar[1]={par_type #value, par_int 0}
670     AutoCore_MsgPar[1].par_int=nPickUp
671     AutoCore_MsgPar[2]={par_type #value, par_int 0}
672     AutoCore_MsgPar[2].par_int=nPickUp + 64
673     SetWaitingMessage(AutomationCore_Msg[15], di068PickupMachine4, TRUE )
674     ;ENDFOLD (Dialog)
675     do068PickupClear4=FALSE
676     do023PickClearTooling=FALSE
677 CASE 5
678     ;FOLD Dialog "Waiting for Pickup %1 permission to enter (DI-%2).".
679     AutoCore_MsgPar[1]={par_type #value, par_int 0}
680     AutoCore_MsgPar[1].par_int=nPickUp
681     AutoCore_MsgPar[2]={par_type #value, par_int 0}
682     AutoCore_MsgPar[2].par_int=nPickUp + 64
683     SetWaitingMessage(AutomationCore_Msg[15], di069PickupMachine5, TRUE )
684     ;ENDFOLD (Dialog)
685     do069PickupClear5=FALSE
686     do023PickClearTooling=FALSE
687 DEFAULT
688     do004ProcessFault=TRUE
689     ;FOLD Dialog "PickUp %1 is invalid. Reprogram with Pickup 1 - 5."
690     LOOP
691         AutoCore_MsgPar[1]={par_type #value, par_int 0}
692         AutoCore_MsgPar[1].par_int=nPickUp
693         nAutoCore_Handle = Set_KrDlG (AutomationCore_Msg[9], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_MsgOpt_Sto
p)
694         IF (nAutoCore_Handle > 0) THEN
695             WHILE (Exists_KrDlG(nAutoCore_Handle, nAutoCore_Key))
696                 do004ProcessFault=TRUE
697                 IF ($ERR.Int_Prio>0) THEN
698                     BRAKE
699                 ENDIF
700                 WAIT SEC 0.024
701             ENDWHILE
702             do004ProcessFault=FALSE
703             SWITCH nAutoCore_Key
704                 CASE 1 ; Diagnostic
705                     do004ProcessFault=FALSE
706                     nAutoCore_Handle = Set_KrLmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
top)
707                     WHILE (Exists_KrLmsg(nAutoCore_Handle))
708                         WAIT SEC 0.024

```

```

709             ENDWHILE
710         ENDSWITCH
711     ENDIF
712     ENDLLOOP
713     ;ENDFOLD (Dialog)
714     ENDSWITCH
715     ;ENDFOLD (Implementation)
716     ;ENDFOLD (Details)
717 END ; (AC_PickUpCheck)
718
719 GLOBAL DEF AC_DropOff(nDropOff :IN, bState :IN )
720     ;FOLD Details
721     ;FOLD Declaration
722     DECL INT nDropOff
723     DECL BOOL bState
724     DECL BOOL bMsgRelease
725     ;ENDFOLD (Declaration)
726     ;FOLD Implementation
727     bMsgRelease=FALSE
728     SWITCH nDropOff
729     CASE 1
730         do075DropOffClear1 = bState
731     CASE 2
732         do076DropOffClear2 = bState
733     CASE 3
734         do077DropOffClear3 = bState
735     CASE 4
736         do078DropOffClear4 = bState
737     CASE 5
738         do079DropOffClear5 = bState
739     DEFAULT
740         do004ProcessFault=TRUE
741         ;FOLD Dialog "DropOff %1 is invalid. Reprogram with DropOff 1 - 5."
742         REPEAT
743             AutoCore_MsgPar[1]={par_type #value, par_int 0}
744             AutoCore_MsgPar[1].par_int=nDropOff
745             nAutoCore_Handle = Set_KrDlg (AutomationCore_Msg[10], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_MsgOpt_St
746 op)             IF (nAutoCore_Handle > 0) THEN
747                 WHILE (Exists_KrDlg(nAutoCore_Handle, nAutoCore_Key))
748                     do004ProcessFault=TRUE
749                     IF ($ERR.Int_Prio>0) THEN
750                         BRAKE
751                     ENDIF
752                     WAIT SEC 0.024
753                 ENDWHILE
754                 do004ProcessFault=FALSE
755                 SWITCH nAutoCore_Key
756                 CASE 1 ; Diagnostic
757                     do004ProcessFault=FALSE
758                     nAutoCore_Handle = Set_KrLmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
759 top)                 WHILE (Exists_KrLmsg(nAutoCore_Handle))
760                     WAIT SEC 0.024
761                 ENDWHILE
762             ENDSWITCH
763         ENDIF
764         UNTIL (bMsgRelease==TRUE)
765         bMsgRelease=FALSE

```

```

766             ;ENDFOLD (Dialog)
767         ENDSWITCH
768     ;ENDFOLD (Implementation)
769 ;ENDFOLD (Details)
770 END ; (AC_DropOff)
771
772 GLOBAL DEF AC_DropOffCheck(nDropOff :IN )
773 ;FOLD Details
774 ;FOLD Declaration
775     DECL INT nDropOff
776 ;ENDFOLD (Declaration)
777 ;FOLD Implementation
778     SWITCH nDropOff
779     CASE 1
780         ;FOLD Dialog "Waiting for DropOff %1 permission to enter (DI-%2).".
781         AutoCore_MsgPar[1]={par_type #value, par_int 0}
782         AutoCore_MsgPar[1].par_int=nDropOff
783         AutoCore_MsgPar[2]={par_type #value, par_int 0}
784         AutoCore_MsgPar[2].par_int=nDropOff + 74
785         SetWaitingMessage(AutomationCore_Msg[16], di075DropOffMachine1, TRUE )
786     ;ENDFOLD (Dialog)
787     do075DropOffClear1 = FALSE
788     do025DropClearTooling = FALSE
789     CASE 2
790         ;FOLD Dialog "Waiting for DropOff %1 permission to enter (DI-%2).".
791         AutoCore_MsgPar[1]={par_type #value, par_int 0}
792         AutoCore_MsgPar[1].par_int=nDropOff
793         AutoCore_MsgPar[2]={par_type #value, par_int 0}
794         AutoCore_MsgPar[2].par_int=nDropOff + 74
795         SetWaitingMessage(AutomationCore_Msg[16], di076DropOffMachine2, TRUE )
796     ;ENDFOLD (Dialog)
797     do076DropOffClear2 = FALSE
798     do025DropClearTooling = FALSE
799     CASE 3
800         ;FOLD Dialog "Waiting for DropOff %1 permission to enter (DI-%2).".
801         AutoCore_MsgPar[1]={par_type #value, par_int 0}
802         AutoCore_MsgPar[1].par_int=nDropOff
803         AutoCore_MsgPar[2]={par_type #value, par_int 0}
804         AutoCore_MsgPar[2].par_int=nDropOff + 74
805         SetWaitingMessage(AutomationCore_Msg[16], di077DropOffMachine3, TRUE )
806     ;ENDFOLD (Dialog)
807     do077DropOffClear3 = FALSE
808     do025DropClearTooling = FALSE
809     CASE 4
810         ;FOLD Dialog "Waiting for DropOff %1 permission to enter (DI-%2).".
811         AutoCore_MsgPar[1]={par_type #value, par_int 0}
812         AutoCore_MsgPar[1].par_int=nDropOff
813         AutoCore_MsgPar[2]={par_type #value, par_int 0}
814         AutoCore_MsgPar[2].par_int=nDropOff + 74
815         SetWaitingMessage(AutomationCore_Msg[16], di078DropOffMachine4, TRUE )
816     ;ENDFOLD (Dialog)
817     do078DropOffClear4 = FALSE
818     do025DropClearTooling = FALSE
819     CASE 5
820         ;FOLD Dialog "Waiting for DropOff %1 permission to enter (DI-%2).".
821         AutoCore_MsgPar[1]={par_type #value, par_int 0}
822         AutoCore_MsgPar[1].par_int=nDropOff

```

```

823         AutoCore_MsgPar[2]={par_type #value, par_int 0}
824         AutoCore_MsgPar[2].par_int=nDropOff + 74
825         SetWaitingMessage(AutomationCore_Msg[16], di079DropOffMachine5, TRUE )
826     ;ENDFOLD (Dialog)
827     do079DropOffClear5 = TRUE
828     do025DropClearTooling = FALSE
829     DEFAULT
830         do004ProcessFault=TRUE
831     ;FOLD Dialog "DropOff %1 is invalid. Reprogram with DropOff 1 - 5."
832     LOOP
833         AutoCore_MsgPar[1]={par_type #value, par_int 0}
834         AutoCore_MsgPar[1].par_int=nDropOff
835         nAutoCore_Handle = Set_KrDlg (AutomationCore_Msg[10], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_MsgOpt_St
op)
836         IF (nAutoCore_Handle > 0) THEN
837             WHILE (Exists_KrDlg(nAutoCore_Handle, nAutoCore_Key))
838                 do004ProcessFault=TRUE
839                 IF ($ERR.Int_Prio>0) THEN
840                     BRAKE
841                 ENDIF
842                 WAIT SEC 0.024
843             ENDWHILE
844             do004ProcessFault=FALSE
845             SWITCH nAutoCore_Key
846                 CASE 1 ; Diagnostic
847                     do004ProcessFault=FALSE
848                     nAutoCore_Handle = Set_Krlmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
top)
849                     WHILE (Exists_KrlMsg(nAutoCore_Handle))
850                         WAIT SEC 0.024
851                     ENDWHILE
852                 ENDSWITCH
853             ENDIF
854         ENLOOP
855     ;ENDFOLD (Dialog)
856     ENDSWITCH
857
858     ;ENDFOLD (Implementation)
859     ;ENDFOLD (Details)
860 END ; (AC_DropOffCheck)
861
862 GLOBAL DEF AC_ZoneLogic(nZone :IN, bState :IN )
863     ;FOLD Details
864     ;FOLD Declaration
865     DECL INT nZone
866     DECL BOOL bState
867     DECL BOOL bMsgRelease
868     ;ENDFOLD (Declaration)
869     ;FOLD Implementation
870     bMsgRelease=FALSE
871     SWITCH nZone
872         CASE 1
873             do026RobotClearZone1 = bState
874         CASE 2
875             do027RobotClearZone2 = bState
876         CASE 3
877             do028RobotClearZone3 = bState
878         CASE 4
879             do029RobotClearZone4 = bState

```

```

880     CASE 5
881         do030RobotClearZone5 = bState
882     CASE 6
883         do031RobotClearZone6 = bState
884     CASE 7
885         do032RobotClearZone7 = bState
886     DEFAULT
887         do004ProcessFault=TRUE
888         ;FOLD Dialog "Zone %1 is invalid. Reprogram with Zone 1 - 7."
889         REPEAT
890             AutoCore_MsgPar[1]={par_type #value, par_int 0}
891             AutoCore_MsgPar[1].par_int=nZone
892             nAutoCore_Handle = Set_KrDlG (AutomationCore_Msg[11], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_MsgOpt_St
op)
893             IF (nAutoCore_Handle > 0) THEN
894                 WHILE (Exists_KrDlG(nAutoCore_Handle, nAutoCore_Key))
895                     do004ProcessFault=TRUE
896                     IF ($ERR.Int_Prio>0) THEN
897                         BRAKE
898                     ENDIF
899                     WAIT SEC 0.024
900                 ENDWHILE
901                 do004ProcessFault=FALSE
902                 SWITCH nAutoCore_Key
903                     CASE 1 ; Diagnostic
904                         do004ProcessFault=FALSE
905                         nAutoCore_Handle = Set_KrLmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
top)
906                         WHILE (Exists_KrLmsg(nAutoCore_Handle))
907                             WAIT SEC 0.024
908                         ENDWHILE
909                     ENDSWITCH
910                 ENDIF
911                 UNTIL (bMsgRelease==TRUE)
912                 bMsgRelease=FALSE
913             ;ENDFOLD (Dialog)
914         ENDSWITCH
915     ;ENDFOLD (Implementation)
916 ;ENDFOLD (Details)
917 END ; (AC_ZoneEnter)
918
919 GLOBAL DEF AC_ZoneCheck(nZone :IN )
920 ;FOLD Details
921 ;FOLD Declaration
922     DECL INT nZone
923 ;ENDFOLD (Declaration)
924 ;FOLD Implementation
925     SWITCH nZone
926     CASE 1
927         ;FOLD Dialog "Waiting for Zone %1 permission to enter (DI-%2)."
928         AutoCore_MsgPar[1]={par_type #value, par_int 0}
929         AutoCore_MsgPar[1].par_int=nZone
930         AutoCore_MsgPar[2]={par_type #value, par_int 0}
931         AutoCore_MsgPar[2].par_int=nZone + 25
932         SetWaitingMessage(AutomationCore_Msg[12], di026Zone1ClearToEnter, TRUE )
933     ;ENDFOLD (Dialog)
934     CASE 2
935         ;FOLD Dialog "Waiting for Zone %1 permission to enter (DI-%2)."
936         AutoCore_MsgPar[1]={par_type #value, par_int 0}

```

```

937         AutoCore_MsgPar[1].par_int=nZone
938         AutoCore_MsgPar[2]={par_type #value, par_int 0}
939         AutoCore_MsgPar[2].par_int=nZone + 25
940         SetWaitingMessage(AutomationCore_Msg[12], di027Zone2ClearToEnter, TRUE )
941     ;ENDFOLD (Dialog)
942 CASE 3
943     ;FOLD Dialog "Waiting for Zone %1 permission to enter (DI-%2)."
944         AutoCore_MsgPar[1]={par_type #value, par_int 0}
945         AutoCore_MsgPar[1].par_int=nZone
946         AutoCore_MsgPar[2]={par_type #value, par_int 0}
947         AutoCore_MsgPar[2].par_int=nZone + 25
948         SetWaitingMessage(AutomationCore_Msg[12], di028Zone3ClearToEnter, TRUE )
949     ;ENDFOLD (Dialog)
950 CASE 4
951     ;FOLD Dialog "Waiting for Zone %1 permission to enter (DI-%2)."
952         AutoCore_MsgPar[1]={par_type #value, par_int 0}
953         AutoCore_MsgPar[1].par_int=nZone
954         AutoCore_MsgPar[2]={par_type #value, par_int 0}
955         AutoCore_MsgPar[2].par_int=nZone + 25
956         SetWaitingMessage(AutomationCore_Msg[12], di029Zone4ClearToEnter, TRUE )
957     ;ENDFOLD (Dialog)
958 CASE 5
959     ;FOLD Dialog "Waiting for Zone %1 permission to enter (DI-%2)."
960         AutoCore_MsgPar[1]={par_type #value, par_int 0}
961         AutoCore_MsgPar[1].par_int=nZone
962         AutoCore_MsgPar[2]={par_type #value, par_int 0}
963         AutoCore_MsgPar[2].par_int=nZone + 25
964         SetWaitingMessage(AutomationCore_Msg[12], di030Zone5ClearToEnter, TRUE )
965     ;ENDFOLD (Dialog)
966 CASE 6
967     ;FOLD Dialog "Waiting for Zone %1 permission to enter (DI-%2)."
968         AutoCore_MsgPar[1]={par_type #value, par_int 0}
969         AutoCore_MsgPar[1].par_int=nZone
970         AutoCore_MsgPar[2]={par_type #value, par_int 0}
971         AutoCore_MsgPar[2].par_int=nZone + 25
972         SetWaitingMessage(AutomationCore_Msg[12], di031Zone6ClearToEnter, TRUE )
973     ;ENDFOLD (Dialog)
974 CASE 7
975     ;FOLD Dialog "Waiting for Zone %1 permission to enter (DI-%2)."
976         AutoCore_MsgPar[1]={par_type #value, par_int 0}
977         AutoCore_MsgPar[1].par_int=nZone
978         AutoCore_MsgPar[2]={par_type #value, par_int 0}
979         AutoCore_MsgPar[2].par_int=nZone + 25
980         SetWaitingMessage(AutomationCore_Msg[12], di032Zone7ClearToEnter, TRUE )
981     ;ENDFOLD (Dialog)
982 DEFAULT
983     do004ProcessFault=TRUE
984     ;FOLD Dialog "Zone %1 is invalid. Reprogram with Zone 1 - 7."
985     LOOP
986         AutoCore_MsgPar[1]={par_type #value, par_int 0}
987         AutoCore_MsgPar[1].par_int=nZone
988         nAutoCore_Handle = Set_KrDlG (AutomationCore_Msg[11], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_Msg0pt_St
989 op) IF (nAutoCore_Handle > 0) THEN
990         WHILE (Exists_KrDlG(nAutoCore_Handle, nAutoCore_Key))
991             do004ProcessFault=TRUE
992             IF ($ERR.Int_Prio>0) THEN
993                 BRAKE

```

```

994             ENDIF
995             WAIT SEC 0.024
996         ENDWHILE
997         do004ProcessFault=FALSE
998         SWITCH nAutoCore_Key
999             CASE 1 ; Diagnostic
1000                 do004ProcessFault=FALSE
1001                 nAutoCore_Handle = Set_Krlmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
top)
1002                 WHILE (Exists_KrlMsg(nAutoCore_Handle))
1003                     WAIT SEC 0.024
1004                 ENDWHILE
1005             ENDSWITCH
1006         ENDIF
1007         ENDL00P
1008     ;ENDFOLD (Dialog)
1009     ENDSWITCH
1010 ;ENDFOLD (Implementation)
1011 ;ENDFOLD (Details)
1012 END ; (AC_ZoneCheck)
1013
1014 GLOBAL DEF AC_Reposition(nRepo :IN, bState :IN )
1015 ;FOLD Details
1016 ;FOLD Declaration
1017     DECL INT nRepo
1018     DECL BOOL bState
1019     DECL BOOL bMsgRelease
1020 ;ENDFOLD (Declaration)
1021 ;FOLD Implementation
1022     bMsgRelease=FALSE
1023     SWITCH nRepo
1024         CASE 1
1025             do080RepositionTooling1 = bState
1026         CASE 2
1027             do081RepositionTooling2 = bState
1028         CASE 3
1029             do082RepositionTooling3 = bState
1030         CASE 4
1031             do083RepositionTooling4 = bState
1032         CASE 5
1033             do084RepositionTooling5 = bState
1034         DEFAULT
1035             do004ProcessFault=TRUE
1036             ;FOLD Dialog "Reposition %1 is invalid. Reprogram with Reposition 1 - 5."
1037             REPEAT
1038                 AutoCore_MsgPar[1]={par_type #value, par_int 0}
1039                 AutoCore_MsgPar[1].par_int=nRepo
1040                 nAutoCore_Handle = Set_Krldlg (AutomationCore_Msg[13], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_MsgOpt_St
op)
1041             IF (nAutoCore_Handle > 0) THEN
1042                 WHILE (Exists_Krldlg(nAutoCore_Handle, nAutoCore_Key))
1043                     do004ProcessFault=TRUE
1044                     IF ($ERR.Int_Prio>0) THEN
1045                         BRAKE
1046                     ENDIF
1047                     WAIT SEC 0.024
1048                 ENDWHILE
1049                 do004ProcessFault=FALSE
1050                 SWITCH nAutoCore_Key

```

```

1051             CASE 1 ; Diagnostic
1052                 do004ProcessFault=FALSE
1053                 nAutoCore_Handle = Set_KrLmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
top)
1054                 WHILE (Exists_KrLmsg(nAutoCore_Handle))
1055                     WAIT SEC 0.024
1056                 ENDWHILE
1057             ENDSWITCH
1058         ENDIF
1059         UNTIL (bMsgRelease==TRUE)
1060             bMsgRelease=FALSE
1061         ;ENDFOLD (Dialog)
1062     ENDSWITCH
1063 ;ENDFOLD (Implementation)
1064 ;ENDFOLD (Details)
1065 END ; (AC_Reposition)
1066
1067 GLOBAL DEF AC_RepoCheck(nRepo :IN )
1068 ;FOLD Details
1069 ;FOLD Declaration
1070     DECL INT nRepo
1071     DECL BOOL bMsgRelease
1072 ;ENDFOLD (Declaration)
1073 ;FOLD Implementation
1074     bMsgRelease=FALSE
1075     SWITCH nRepo
1076     CASE 1
1077         ;FOLD Dialog "Waiting for Tool %1 repositioned (DI-%2)."  

1078         AutoCore_MsgPar[1]={par_type #value, par_int 0}  

1079         AutoCore_MsgPar[1].par_int=nRepo  

1080         AutoCore_MsgPar[2]={par_type #value, par_int 0}  

1081         AutoCore_MsgPar[2].par_int=nRepo + 79  

1082         SetWaitingMessage(AutomationCore_Msg[14], di080ToolRepositioned1, TRUE )  

1083     ;ENDFOLD (Dialog)
1084     CASE 2
1085         ;FOLD Dialog "Waiting for Tool %1 repositioned (DI-%2)."  

1086         AutoCore_MsgPar[1]={par_type #value, par_int 0}  

1087         AutoCore_MsgPar[1].par_int=nRepo  

1088         AutoCore_MsgPar[2]={par_type #value, par_int 0}  

1089         AutoCore_MsgPar[2].par_int=nRepo + 79  

1090         SetWaitingMessage(AutomationCore_Msg[14], di081ToolRepositioned2, TRUE )  

1091     ;ENDFOLD (Dialog)
1092     CASE 3
1093         ;FOLD Dialog "Waiting for Tool %1 repositioned (DI-%2)."  

1094         AutoCore_MsgPar[1]={par_type #value, par_int 0}  

1095         AutoCore_MsgPar[1].par_int=nRepo  

1096         AutoCore_MsgPar[2]={par_type #value, par_int 0}  

1097         AutoCore_MsgPar[2].par_int=nRepo + 79  

1098         SetWaitingMessage(AutomationCore_Msg[14], di082ToolRepositioned3, TRUE )  

1099     ;ENDFOLD (Dialog)
1100     CASE 4
1101         ;FOLD Dialog "Waiting for Tool %1 repositioned (DI-%2)."  

1102         AutoCore_MsgPar[1]={par_type #value, par_int 0}  

1103         AutoCore_MsgPar[1].par_int=nRepo  

1104         AutoCore_MsgPar[2]={par_type #value, par_int 0}  

1105         AutoCore_MsgPar[2].par_int=nRepo + 79  

1106         SetWaitingMessage(AutomationCore_Msg[14], di083ToolRepositioned4, TRUE )  

1107     ;ENDFOLD (Dialog)

```

```

1108 CASE 5
1109     ;FOLD Dialog "Waiting for Tool %1 repositioned (DI-%2).".
1110     AutoCore_MsgPar[1]={par_type #value, par_int 0}
1111     AutoCore_MsgPar[1].par_int=nRepo
1112     AutoCore_MsgPar[2]={par_type #value, par_int 0}
1113     AutoCore_MsgPar[2].par_int=nRepo + 79
1114     SetWaitingMessage(AutomationCore_Msg[14], di084ToolRepositioned5, TRUE )
1115     ;ENDFOLD (Dialog)
1116     DEFAULT
1117     do004ProcessFault=TRUE
1118     ;FOLD Dialog "Reposition %1 is invalid. Reprogram with Reposition 1 - 5."
1119     REPEAT
1120     AutoCore_MsgPar[1]={par_type #value, par_int 0}
1121     AutoCore_MsgPar[1].par_int=nRepo
1122     nAutoCore_Handle = Set_KrDlG (AutomationCore_Msg[13], AutoCore_MsgPar[], AC_SK_DIAG[], AutoCore_MsgOpt_St
op)
1123     IF (nAutoCore_Handle > 0) THEN
1124     WHILE (Exists_KrDlG(nAutoCore_Handle, nAutoCore_Key))
1125     do004ProcessFault=TRUE
1126     IF ($ERR.Int_Prio>0) THEN
1127     BRAKE
1128     ENDIF
1129     WAIT SEC 0.024
1130     ENDWHILE
1131     do004ProcessFault=FALSE
1132     SWITCH nAutoCore_Key
1133     CASE 1 ; Diagnostic
1134     do004ProcessFault=FALSE
1135     nAutoCore_Handle = Set_KrLmsg (#QUIT, AutomationCore_Msg[2], AutoCore_MsgPar[], AutoCore_MsgOpt_S
top)
1136     WHILE (Exists_KrLmsg(nAutoCore_Handle))
1137     WAIT SEC 0.024
1138     ENDWHILE
1139     ENDSWITCH
1140     ENDIF
1141     UNTIL (bMsgRelease==TRUE)
1142     bMsgRelease=FALSE
1143     ;ENDFOLD (Dialog)
1144     ENDSWITCH
1145     AC_Reposition(nRepo, FALSE) ; Reset Reposition Request
1146     ;ENDFOLD (Implementation)
1147     ;ENDFOLD (Details)
1148 END ; (AC_RepoCheck)
1149
1150 GLOBAL DEF AC_PointArrival( )
1151     ;FOLD Details
1152     ;FOLD Declaration
1153     ;ENDFOLD (Declaration)
1154     ;FOLD Implementation
1155     IF ((doCriticalWZ==FALSE) AND (di006RequestToEnter==TRUE)) THEN
1156     REPEAT
1157     BRAKE
1158     IF ((Exists_KrLmsg(nReqEnter_Handle))==FALSE) THEN
1159     nReqEnter_Handle=Set_KrLmsg (#STATE, AutomationCore_Msg[25], AutoCore_MsgPar[], AutoCore_MsgOpt_NoStop)
1160     ENDIF
1161     HALT
1162     WAIT SEC 0.012
1163     UNTIL (di006RequestToEnter==FALSE)
1164     bAutoCore_Clear=Clear_KrLmsg(nReqEnter_Handle)

```

```

1165         ENDIF
1166         ;ENDFOLD (Implementation)
1167     ;ENDFOLD (Details)
1168 END ; (AC_PointArrival)
1169
1170 DEF AC_ProcessReset( )
1171     ;FOLD Details
1172     ;FOLD Declaration
1173     ;ENDFOLD (Declaration)
1174     ;FOLD Implementation
1175         do004ProcessFault=TRUE
1176         bResetProcessCount=TRUE
1177     ;ENDFOLD (Implementation)
1178     ;ENDFOLD (Details)
1179 END ; (AC_ProcessReset)
1180
1181 GLOBAL DEF StartCycTime( )
1182     ;FOLD Details
1183     ;FOLD Declaration
1184     ;ENDFOLD (Declaration)
1185     ;FOLD Implementation
1186         $TIMER_STOP[nCycleTimer]=TRUE
1187         $TIMER[nCycleTimer]=0
1188         $TIMER_STOP[nCycleTimer]=FALSE
1189     ;ENDFOLD (Implementation)
1190     ;ENDFOLD (Details)
1191 END ; (StartCycTime)
1192
1193 GLOBAL DEF StopCycTime( )
1194     ;FOLD Details
1195     ;FOLD Declaration
1196     ;ENDFOLD (Declaration)
1197     ;FOLD Implementation
1198         $TIMER_STOP[nCycleTimer]=TRUE
1199         nCycleTime=$TIMER[nCycleTimer]/1000
1200     ;ENDFOLD (Implementation)
1201     ;ENDFOLD (Details)
1202 END ; (StopCycTime)
1203
1204 GLOBAL DEF GetCycTime( )
1205     ;FOLD Details
1206     ;FOLD Declaration
1207     ;ENDFOLD (Declaration)
1208     ;FOLD Implementation
1209         StopCycTime( )
1210         IF ((nCycleTime>0) AND (Exists_KrlMsg(nCycTime_Handle)==FALSE)) THEN
1211             AutoCore_MsgPar[1]={par_type #value, par_int 0}
1212             AutoCore_MsgPar[1].par_int=nCycleTime
1213             nCycTime_Handle=Set_Krlmsg (#STATE, AutomationCore_Msg[7], AutoCore_MsgPar[], AutoCore_MsgOpt_NoStop)
1214         ENDIF
1215     ;ENDFOLD (Implementation)
1216     ;ENDFOLD (Details)
1217 END ; (GetCycTime)
1218
1219 DEFFCT REAL AC_POW(rBase:IN, iExponent:IN)
1220
1221 DECL REAL rBase, rResult

```

```

1222     DECL INT iExponent, i
1223
1224     IF iExponent == 0 THEN
1225         rResult = 1.0
1226     ELSE
1227         IF iExponent < 0 THEN
1228             IF rBase == 0 THEN
1229                 ; Ungueltige Eingabe, da Division durch 0
1230                 WAIT FOR FALSE
1231                 ; error, division by 0
1232             ENDIF
1233             iExponent = iExponent * (-1)
1234             rResult = rBase
1235             FOR i = 1 TO (iExponent - 1)
1236                 rResult = rResult * rBase
1237             ENDFOR
1238             rResult = 1 / rResult
1239         ELSE
1240             rResult = rBase
1241             FOR i = 1 TO (iExponent - 1)
1242                 rResult = rResult * rBase
1243             ENDFOR
1244         ENDIF
1245     ENDIF
1246
1247     RETURN rResult
1248
1249 ENDFCT
1250
1251 GLOBAL DEFFCT BOOL AC_CheckSphWZ (pCenter: IN, nRadius: IN )
1252 ;FOLD Details
1253 ;FOLD Declaration
1254     E6POS pCenter
1255     INT nRadius
1256     BOOL bResult
1257     REAL pdx, pdy, pdz
1258 ;ENDFOLD (Declaration)
1259 ;FOLD Implementation
1260     IF ((VARSTATE("pCenter") == #INITIALIZED) AND (VARSTATE("nRadius") == #INITIALIZED))THEN
1261         pdx = PosCheck.x - pCenter.x;    // vector from pt1 to test point.
1262         pdy = PosCheck.y - pCenter.y;
1263         pdz = PosCheck.z - pCenter.z;
1264
1265         bResult = (((pdx*pdx) + (pdy*pdy) + (pdz*pdz)) <= (nRadius*nRadius))
1266         RETURN (bResult)
1267     ELSE
1268         RETURN (FALSE)
1269     ENDIF
1270 ;ENDFOLD (Implementation)
1271 ;ENDFOLD (Details)
1272 ENDFCT ; (AC_CheckSphWZ)
1273
1274 GLOBAL DEF AC_DeleteLine(nMsg :IN, nOpt1 :IN, nOpt2 :IN, nOpt3 :IN )
1275 ;FOLD Details
1276 ;FOLD Declaration
1277     DECL INT nMsg
1278     DECL INT nOpt1

```

```

1279     DECL INT nOpt2
1280     DECL INT nOpt3
1281 ;ENDFOLD (Declaration)
1282 ;FOLD Implementation
1283     LOOP
1284         IF (VARSTATE("nMsg")<=>#INITIALIZED) THEN
1285             nMsg=1
1286         ENDIF
1287         IF (VARSTATE("nOpt1")==#INITIALIZED) THEN
1288             AutoCore_MsgPar[1]={par_type #value, par_int 0}
1289             AutoCore_MsgPar[1].par_int=nOpt1
1290         ENDIF
1291         IF (VARSTATE("nOpt2")==#INITIALIZED) THEN
1292             AutoCore_MsgPar[2]={par_type #value, par_int 0}
1293             AutoCore_MsgPar[2].par_int=nOpt2
1294         ENDIF
1295         IF (VARSTATE("nOpt3")==#INITIALIZED) THEN
1296             AutoCore_MsgPar[3]={par_type #value, par_int 0}
1297             AutoCore_MsgPar[3].par_int=nOpt3
1298         ENDIF
1299         nAutoCore_Handle = Set_Krlmsg (#QUIT, AutomationCore_Msg[nMsg], AutoCore_MsgPar[], AutoCore_MsgOpt_Stop)
1300         WHILE (Exists_Krlmsg(nAutoCore_Handle))
1301             WAIT SEC 0.024
1302         ENDWHILE
1303     ENDLLOOP
1304 ;ENDFOLD (Implementation)
1305 ;ENDFOLD (Details)
1306 END ; (AC_DeleteLine)
1307
1308 GLOBAL DEF AC_Erase( )
1309 ;FOLD Details
1310 ;FOLD Declaration
1311 ;ENDFOLD (Declaration)
1312 ;FOLD Implementation
1313     bAutoCore_Clear=Clear_KRLMsg(-99)
1314 ;ENDFOLD (Implementation)
1315 ;ENDFOLD (Details)
1316 END ; (AC_Erase)
1317
1318 DEF SetWaitingMessage(KrlmsgDisplay :IN, bCondition :OUT, bState :IN )
1319 ;FOLD Details
1320 ;FOLD Declaration
1321     DECL KRLMSG_T KrlmsgDisplay
1322     DECL BOOL bCondition
1323     DECL BOOL bState
1324     DECL BOOL bMsgExt
1325     DECL BOOL bMsgSimulate
1326     DECL BOOL bMsgClear
1327     DECL KRLMSG_T SimulateMsg
1328     DECL INT nMsgLength
1329 ;ENDFOLD (Declaration)
1330 ;FOLD Implementation
1331     bMsgSimulate = FALSE
1332     bAutoCoreSubmit_Clear=FALSE
1333     IF ($MODE_OP==#EX) THEN
1334         nWaitMsg_Handle = Set_Krlmsg (#STATE, KrlmsgDisplay, AutoCore_MsgPar[], AutoCore_MsgOpt_NoStop)
1335         bMsgExt=TRUE

```

```

1336     ELSE
1337         nWaitMsg_Handle = Set_KrLmsg (#WAITING, KrLmsgDisplay, AutoCore_MsgPar[], AutoCore_MsgOpt_NoStop)
1338         bMsgExt=FALSE
1339     ENDIF
1340     WHILE ((bCondition <> bState) AND (bMsgSimulate==FALSE))
1341         IF (nWaitMsg_Handle > 0) THEN
1342             IF ($ERR.Int_Prio>0) THEN
1343                 BRAKE
1344             ENDIF
1345             WAIT SEC 0.012
1346             IF ((Exists_KrLmsg(nWaitMsg_Handle)==FALSE) AND (bAutoCoreSubmit_Clear==FALSE)) THEN
1347                 nWaitMsg_Handle = 0
1348                 bMsgSimulate=TRUE
1349                 bMsgClear = StrClear(SimulateMsg.Msg_Txt[])
1350                 nMsgLength = StrAdd(SimulateMsg.Msg_Txt[], "{")
1351                 nMsgLength = StrAdd(SimulateMsg.Msg_Txt[], KrLmsgDisplay.Msg_Txt[])
1352                 nMsgLength = StrAdd(SimulateMsg.Msg_Txt[], ",")
1353                 nMsgLength = StrAdd(SimulateMsg.Msg_Txt[], " has been simulated.")
1354                 bMsgClear = StrCopy(SimulateMsg.Modul[], KrLmsgDisplay.Modul[])
1355                 SimulateMsg.Nr = 1001
1356                 IF (nMsgLength==0) THEN
1357                     bMsgClear = StrClear(SimulateMsg.Msg_Txt[])
1358                     SimulateMsg.Msg_Txt[] = "Signal wait condition has been simulated"
1359                 ENDIF
1360                 nLogOnlyMsg_Handle = Set_KrLmsg (#STATE, SimulateMsg, AutoCore_MsgPar[], AutoCore_MsgOpt_NoStop)
1361                 IF (Exists_KrLmsg(nLogOnlyMsg_Handle)==TRUE) THEN
1362                     bMsgClear=Clear_KRLMsg(nLogOnlyMsg_Handle)
1363                 ENDIF
1364             ELSE
1365                 IF (($MODE_OP<>#EX) AND ((bMsgExt==TRUE) OR (Exists_KrLmsg(nWaitMsg_Handle)==FALSE))) THEN
1366                     bAutoCoreSubmit_Clear=FALSE
1367                     bMsgClear=Clear_KRLMsg(nWaitMsg_Handle)
1368                     bMsgExt=FALSE
1369                     nWaitMsg_Handle = Set_KrLmsg (#WAITING, KrLmsgDisplay, AutoCore_MsgPar[], AutoCore_MsgOpt_NoLog)
1370                 ENDIF
1371                 IF (($MODE_OP==#EX) AND ((bMsgExt==FALSE) OR (Exists_KrLmsg(nWaitMsg_Handle)==FALSE))) THEN
1372                     bAutoCoreSubmit_Clear=FALSE
1373                     bMsgClear=Clear_KRLMsg(nWaitMsg_Handle)
1374                     bMsgExt=TRUE
1375                     nWaitMsg_Handle = Set_KrLmsg (#STATE, KrLmsgDisplay, AutoCore_MsgPar[], AutoCore_MsgOpt_NoLog)
1376                 ENDIF
1377             ENDIF
1378         ENDIF
1379     ENDWHILE
1380     bMsgSimulate=FALSE
1381     IF (Exists_KrLmsg(nWaitMsg_Handle)==TRUE) THEN
1382         bMsgClear=Clear_KRLMsg(nWaitMsg_Handle)
1383         nWaitMsg_Handle = 0
1384     ENDIF
1385     ;ENDFOLD (Implementation)
1386     ;ENDFOLD (Details)
1387 END ; (SetWaitingMessage)
1388
1389 ;FOLD Check ToolID
1390 GLOBAL DEFFCT BOOL ToolID (iToolNo:IN)
1391
1392     INT iToolNo

```

```

1393     INT maxToolNo
1394     maxToolNo = 2
1395     IF (NOT (VARSTATE("iToolNo")==#INITIALIZED)) THEN
1396         RETURN(FALSE)
1397     ENDIF
1398     IF (NOT ((iToolNo>=0) AND (iToolNo<=maxToolNo))) THEN
1399         RETURN(FALSE)
1400     ENDIF
1401     IF (iToolNo==go123ToolID) THEN
1402         RETURN(TRUE)
1403     ELSE
1404         RETURN(FALSE)
1405     ENDIF
1406
1407     ENDFCT
1408
1409 ;ENDFOLD (Check ToolID)
1410
1411 GLOBAL DEF AC_Request_to_enter( )
1412     ;FOLD Details
1413     ;FOLD Declaration
1414     ;ENDFOLD (Declaration)
1415     ;FOLD Implementation
1416
1417     ; 2026-10-05: no WAIT here - a WAIT in the Submit froze the whole SPS
1418     ; (gun position, nut preload, gripper) while the request was active.
1419     ; Entry stays granted until the request is released (same behavior).
1420     IF di006RequestToEnter THEN
1421         IF NOT bACEntryGranted AND NOT (dipw1_GunAtWork OR do932GripperInProcess OR do931BrakeTestInProcess OR do930MasterR
efInP
rocess) THEN             ;NUT / GRIPPER / BRAKETEST / MASTERREF
1422             bACEntryGranted = TRUE             ; request to enter allowed!
1423         ENDIF
1424     ELSE
1425         bACEntryGranted = FALSE
1426     ENDIF
1427     doCriticalWZ = NOT bACEntryGranted             ; FALSE = entry allowed
1428
1429     IF (doCriticalWZ AND NOT $MOVE_ENABLE) THEN
1430         $LOOP_MSG[] = " Critical Process interrupted by General Motion enable
"
1431     ENDIF
1432     ;ENDFOLD (Implementation)
1433     ;ENDFOLD (Details)
1434 END ; (AC_Request_to_enter)
1435
1436 GLOBAL DEF AC_Process_Count(eProcessCount:IN)
1437     ;FOLD Details
1438     ; INIT: Reset the current Process Counter
1439     ; Countlup: add 1 after a successful weld, clinch or rivet
1440     ; Check: verify all process points are done, otherwise set do047QualityFault
1441     ;FOLD Declaration
1442     DECL MyCountCmd eProcessCount
1443     INT ResetStyle, ResetOption
1444     ;ENDFOLD (Declaration)
1445     ;FOLD Implementation
1446     geProcessCount=eProcessCount
1447     SWITCH eProcessCount

```

```

1448     CASE #INIT
1449         do047QualityFault = FALSE
1450     IF bResetProcessCount THEN
1451         FOR ResetStyle = 1 TO 31
1452             FOR ResetOption = 1 TO 255
1453                 WeldCount[ResetStyle,ResetOption] = 0
1454             ENDFOR
1455         ENDFOR
1456         bResetProcessCount=FALSE
1457     ENDIF
1458     CASE #CountIup
1459         SWITCH nStyle
1460             CASE 1 ; select with nStyle
1461                 SWITCH nOption
1462                     CASE 1,2,3,4,5,6,7,8,9 ; select with nOption
1463                         WeldCount[nStyle,nOption] = WeldCount[nStyle,nOption] + 1
1464                     DEFAULT
1465                         ; wrong option selected
1466                         do047QualityFault = TRUE
1467                 ENDSWITCH
1468             DEFAULT
1469                 ; wrong style selected
1470                 do047QualityFault = TRUE
1471             ENDSWITCH
1472     CASE #Check
1473         WAIT FOR TRUE
1474         IF ((nStyle>0) AND (nOption>0)) THEN
1475             do047QualityFault = (WeldCount[nStyle,nOption]<>WeldNoRequired[nStyle,nOption])
1476         ELSE
1477             ; nStyle or nOption invalid
1478             do047QualityFault =TRUE
1479         ENDIF
1480         IF do047QualityFault THEN
1481             WAIT SEC 5
1482         ENDIF
1483         bResetProcessCount=TRUE
1484     ENDSWITCH
1485     ;ENDFOLD (Implementation)
1486 ;ENDFOLD (Details)
1487 END ; (AC_Process_Count)
1488
1489 GLOBAL DEF AutomationCore_Spare1( )
1490 ;FOLD Details
1491 ;FOLD Declaration
1492 ;ENDFOLD (Declaration)
1493 ;FOLD Implementation
1494 ;ENDFOLD (Implementation)
1495 ;ENDFOLD (Details)
1496 END ; (AutomationCore_Spare1)
1497

```

KRC/R1/TP/BrakeTest/braketestback.src

```

1 &ACCESS RV07
2 &REL 7
3 &COMMENT path after test
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\Roboter\Template\vorgabe

```

```

6 &PARAM DISKPATH = TP/BrakeTest
7 DEF BrakeTestBack( )
8 ;FOLD INI
9 ;FOLD BASISTECH INI
10 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11 INTERRUPT ON 3
12 BAS (#INITMOV,0 )
13 ;ENDFOLD (BASISTECH INI)
14 ;FOLD USER INI
15 ;Make your modifications here
16 ;do829BrakeTestInProgress=FALSE
17 ;do931BrakeTestInProgress=FALSE
18 ;ENDFOLD (USER INI)
19 ;ENDFOLD (INI)
20 ;Teach here the motion to the position after the brake test.
21
22
23 ;FOLD PTP P2 Vel=100 % PDAT2 Tool[1]:TOOL 1 Base[0] ;%{...}
24 $BWDSTART=FALSE
25 PDAT_ACT=PPDAT2
26 FDAT_ACT=FP2
27 BAS(#PTP_PARAMS,100)
28 PTP XP2
29 ;ENDFOLD
30
31
32
33 ;FOLD PTP PHOME Vel=100 % PDAT1 Tool[1]:Tool 1 Base[0] ;%{...}
34 $BWDSTART=FALSE
35 PDAT_ACT=PPDAT1
36 FDAT_ACT=FPHOME
37 BAS(#PTP_PARAMS,100)
38 PTP XPHOME
39 ;ENDFOLD
40 END

```

KRC/R1/TP/BrakeTest/braketestreq.src

```

1 &ACCESS RV07
2 &REL 2
3 &COMMENT BrkTst-main task
4 &PARAM EDITMASK = *
5 &PARAM DISKPATH = KRC:\R1\TP\BrakeTest
6 DEF BRAKETESTREQ()
7 ;FOLD perform brake test
8
9 ;ENDFOLD
10 ;FOLD ;%{...}
11 DECL E6AXIS BtStPos
12 INT Idx, i, Ret, IntArray[5]
13 INT BitfieldAxesInTest
14 REAL RealArray[1]
15
16 if Varstate("$pro_ip.si01.name[]")==#declared then
17   MsgQuit("AcknowledgeToManuallyPerformBrakeTest", brakeTestMdbName[])
18   MD_SETSTATE(brakeTestDevName[], #INITIALIZED)
19 endif
20

```

```

21 BAS (#INITMOV,0 )
22
23 repeat
24   Ret = MD_CMD(brakeTestDevName[], "START_TEST", IntArray[], RealArray[])
25   if ( Ret <> 0 ) then
26     ; Entweder (Ret == +1) - dann liegt keine Testanforderung vor - oder
27     ; (Ret < 0) - dann wurde bereits eine Zustandsmeldung vom Treiber abgesetzt
28     Return
29   endif
30
31   ; Die Achsmasken aus der INI-Datei auslesen
32   BitfieldAxesInTest = IntArray[1]
33
34   BrakeTestStart()
35   BtStPos=$AXIS_ACT
36
37   i=1
38   FOR Idx = 1 TO 12
39     if ( ( BitfieldAxesInTest b_and i) == i ) then
40       if (Idx==5) AND $PAL_MODE AND NOT $DEF_A5LINK AND $DEF_A4FIX then
41         $PAL_MODE=FALSE
42       endif
43       repeat
44         TestAxis(Idx)
45         PTP BtStPos
46         until GetResult()
47         if (Idx==5) AND NOT $PAL_MODE AND NOT $DEF_A5LINK AND $DEF_A4FIX then
48           $PAL_MODE=TRUE
49         endif
50       endif
51       i=i*2
52     ENDFOR
53     WAIT FOR TRUE
54     Ret = MD_CMD(brakeTestDevName[], "END_TEST", IntArray[], RealArray[])
55 until ( Ret == 0 )
56
57 BrakeTestBack()
58 END
59
60
61 DEF TestAxis(AxisNr: IN)
62 INT AxisNr
63 INT Ret, NrFlag
64 INT IntArray[3]
65 REAL RealArray[1]
66 AxNr=AxisNr
67 IntArray[1] = AxisNr
68 Ret = MD_CMD(brakeTestDevName[], "SELECT_TARGET", IntArray[], RealArray[])
69 NrFlag=IntArray[1]
70 BtIrNr=IntArray[3]
71 $FLAG[NrFlag]=FALSE
72 IF(AxNr<7) THEN
73   $VEL_AXIS[AxNr]=IntArray[2]
74 ELSE
75   $VEL_EXTAX[AxNr-6]=IntArray[2]
76 ENDIF
77 INTERRUPT DECL BtIrNr WHEN $FLAG[NrFlag] DO Decelerate()

```

```

78 INTERRUPT ON BtIrNr
79 MoveAxis(AxisNr)
80 END
81
82
83 DEF MoveAxis(AxisNr: IN)
84 INT Ret, AxisNr
85 INT IntArray[5]
86 REAL RealArray[6]
87 E6AXIS AxisRel
88 $ADVANCE=0
89 Ret = MD_CMD(brakeTestDevName[], "GET_ROB_ANGLES", IntArray[], RealArray[])
90 AxisRel.A1=RealArray[1]
91 AxisRel.A2=RealArray[2]
92 AxisRel.A3=RealArray[3]
93 AxisRel.A4=RealArray[4]
94 AxisRel.A5=RealArray[5]
95 AxisRel.A6=RealArray[6]
96 Ret = MD_CMD(brakeTestDevName[], "GET_EXT_ANGLES", IntArray[], RealArray[])
97 AxisRel.E1=RealArray[1]
98 AxisRel.E2=RealArray[2]
99 AxisRel.E3=RealArray[3]
100 AxisRel.E4=RealArray[4]
101 AxisRel.E5=RealArray[5]
102 AxisRel.E6=RealArray[6]
103 PTP $AXIS_ACT
104 Ret = MD_CMD(brakeTestDevName[], "TRIGGER_TEST_RUN", IntArray[], RealArray[])
105 PTP_REL AxisRel
106 WAIT FOR TRUE
107 ; Hier kommt man nur hin, wenn die Bewegung zu Ende gefahren wurde.
108 ; Im Gutfall wird aber ein BRAKE / RESUME ausgeführt und damit diese Funktion verlassen
109 MsgQuit("BT_MAX_AXIS_MOVE", brakeTestMdbName[], AxisNr)
110 END
111
112
113 DEF Decelerate()
114 INT Ret
115 INT IntArray[5]
116 REAL RealArray[6]
117 INTERRUPT OFF BtIrNr
118 BRAKE
119 Ret = MD_CMD(brakeTestDevName[], "FINISH_TEST_RUN", IntArray[], RealArray[])
120 if ret <> 0 then
121   MsgNotify("FINISH_TEST_RUN returns %1 ", brakeTestMdbName[], ret)
122 endif
123 RESUME
124 END
125
126
127 DEFFCT BOOL GetResult()
128 INT Ret
129 INT IntArray[5]
130 REAL RealArray[1]
131 Decl CHAR DialogText[20]
132 DECL CHAR AxNrText[10]
133
134 Ret=MD_CMD(brakeTestDevName[], "GET_RESULT", IntArray[], RealArray[])

```

```

135 SWITCH IntArray[1]
136 CASE 0
137   ; BT_BRAKE_OK
138   Return TRUE
139 CASE 1
140   ; BT_BRAKE_WEAK
141   Return TRUE
142 CASE 2
143   ;BT_BRAKE_NOT_OK
144   DialogText[] = "BT_BRAKE_NOT_OK"
145 CASE 3
146   ; BT_NO BRAKING
147   DialogText[] = "BT_BRAKE_NOT_OK"
148 CASE 4
149   ; BT_MOTION_FAULT
150   DialogText[] = "BT_EVALUATION_FAILED"
151 CASE 5
152   ; BT_INCOMPLETE
153   DialogText[] = "BT_INCOMPLETE"
154 CASE 6
155   ; BT_REPEAT
156   Return FALSE
157 ENDSWITCH
158
159 IF(IntArray[2]<7) THEN
160   AxNrText[]="A "
161   AxNrText[2]=IntArray[2]+48
162 ELSE
163   AxNrText[]="E "
164   AxNrText[2]=IntArray[2]+42
165 ENDIF
166
167 MsgDialog(ret, DialogText[], brakeTestMdbName[], AxNrText[], "TryAgain", "GoToPark",,,,,TRUE)
168
169 if Ret==7 then
170   ; Rückgabewert 7 = Softkey "TryAgain" ganz rechts
171   ; Testlauf wiederholen
172   Return FALSE
173 endif
174
175 ;goto parkposition
176 BrakeTestPark()
177 WAIT FOR TRUE
178
179 Ret = MD_CMD(brakeTestDevName[], "SIGNAL_PARK_POS", IntArray[], RealArray[])
180
181 repeat
182   MsgNotify("ParkPositionReached", brakeTestMdbName[])
183   halt
184 until false
185
186 ENDFCT
187 ;ENDFOLD ;BRAKETEST

```

KRC/R1/TP/BrakeTest/braketeststart.src

```

1 &ACCESS RV07
2 &REL 6

```

```

3 &COMMENT path to test
4 &PARAM EDITMASK = *
5 &PARAM TEMPLATE = C:\KRC\Roboter\Template\vorgabe
6 &PARAM DISKPATH = KRC:\R1\TP\BrakeTest
7 DEF BrakeTestStart( )
8 ;FOLD INI
9 ;FOLD BASISTECH INI
10 GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11 INTERRUPT ON 3
12 BAS (#INITMOV,0 )
13 ;ENDFOLD (BASISTECH INI)
14 ;FOLD USER INI
15 ;Make your modifications here
16 ;do829BrakeTestInProgress=TRUE
17 ;do931BrakeTestInProgress=TRUE
18 ;ENDFOLD (USER INI)
19 ;ENDFOLD (INI)
20 ;FOLD PTP m Vel=100 % PDAT2 Tool[3]:Tool 3 Base[1]:3-30-R1 03-20B ;%{...}
21 $BWDSTART=FALSE
22 PDAT_ACT=PPDAT2
23 FDAT_ACT=Fm
24 BAS(#PTP_PARAMS,100)
25 PTP Xm
26 ;ENDFOLD
27 ;FOLD PTP h Vel=100 % PDAT1 Tool[3]:Tool 3 Base[1]:3-30-R1 03-20B ;%{...}
28 $BWDSTART=FALSE
29 PDAT_ACT=PPDAT1
30 FDAT_ACT=Fh
31 BAS(#PTP_PARAMS,100)
32 PTP Xh
33 ;ENDFOLD
34 END

```

KRC/R1/cell.src

```

1 &ACCESS RVP
2 &REL 20
3 &COMMENT 03-10-R1 AUTO EXTERNAL MAIN
4 DEF CELL ( )
5 ;*****
6 ; Gestamp Standards
7 ; Line - BMW G65 (cell 03)
8 ; Robot 03-10-R1 (BMW-03-10-R1)
9 ; Application - AUTOMATIC EXTERNAL MAIN PROGRAM (AUTOMATIONCORE CELL)
10 ;*****
11 ;FOLD Initialize System
12 ;NOTE: DMY IS A REMNANT OF THE KUKA CELL TEMPLATE (cellVorgabe): WRITTEN HERE, NEVER READ
13 DECL CHAR DMY[3]
14 DMY[]="---"
15 ;AUTOMATIONCORE START-UP (HomeCheck, PROCESS RESET), THEN PTP TO HOME (HOME.src, P1 = XHOME)
16 InitializeSystem()
17 HOME()
18 ;ENDFOLD (Initialize System)
19 LOOP
20 ;FOLD Maintain System
21 ;RESET CYCLE DATA AND THE PLC HANDSHAKE OUTPUTS TO THEIR REST STATE (MaintainSystem), E.G.
22 ;SIGNALS $OUT[5] do005RobotInCycle OFF, $OUT[15] do015WorkComplete OFF,
23 ;$OUT[4] do004ProcessFault OFF, $OUT[141] do141RedRabbitFailed OFF,

```

```

24 ;$OUT[142] do142RedRabbitPassed OFF; THEN WAITS HERE WHILE STOP AT END OF CYCLE IS
25 ;SELECTED (SIGNAL $IN[147] di147SEOC ON, OR SELECTED BY THE USER)
26 MaintainSystem()
27 ;ENDFOLD (Maintain System)
28 ;FOLD Check Style
29 ;READ THE STYLE NUMBER - SIGNALS $IN[33..37] gdiStyle, ECHOED ON $OUT[33..37] gdoStyleAckn
30 nStyle=gdiStyle
31 gdoStyleAckn=nStyle
32 ;READ THE OPTION NUMBER - SIGNALS $IN[38..45] gdiOption, ECHOED ON $OUT[38..45] gdoOptionAckn
33 nOption=gdiOption
34 gdoOptionAckn=nOption
35 ;DECODE nOption INTO bOption1..8 (GetOptions) AND WRITE THEM TO
36 ;SIGNALS $OUT[38..45] do0380ptionAckn1..do0450ptionAckn8
37 GetOptions( )
38 ;SHOW THE LAST CYCLE TIME AS A STATE MESSAGE
39 GetCycTime( )
40 ;FOLD State Message - Waiting for Cycle Start or Style Selection.
41 IF (Exists_KrMsg(nCycleStart_Handle)==FALSE) THEN
42     nCycleStart_Handle = Set_KrMsg (#STATE, AutomationCore_Msg[24], AutoCore_MsgPar[], AutoCore_MsgOpt_NoStop)
43 ENDIF
44 ;ENDFOLD (State Message - Waiting for Cycle Start or Style Selection.)
45 ;ENDFOLD (Check Style)
46 ;CYCLE START - SIGNAL $IN[5] di005InitiateCycle ON, WITH nStyle > 0 AND nOption > 0
47 IF ((di005InitiateCycle==TRUE) AND (nStyle>0) AND (nOption>0)) THEN
48     ;FOLD Start Cycle
49     ;CLEAR AUTOMATIONCORE MESSAGES, THEN HomeCheck: THE ROBOT MUST BE AT HOME
50     ;(IF NOT: PounceToHome WHEN AT POUNCE IN EXT, OTHERWISE AN OPERATOR DIALOG)
51     AC_Erase( )
52     HomeCheck( )
53     ;ROBOT IN CYCLE - SIGNAL $OUT[5] do005RobotInCycle ON
54     do005RobotInCycle = TRUE
55     ;START THE CYCLE TIMER
56     StartCycTime( )
57     ;ENDFOLD (Start Cycle)
58     ;SELECT THE STYLE PROGRAM BY STYLE NUMBER nStyle (gdiStyle); ONLY STYLE 1 EXISTS
59     SWITCH nStyle
60     CASE 1
61         ;STYLE 1 PROGRAM
62         Style_1( )
63     DEFAULT
64     ;NOTE: ANY OTHER STYLE RUNS NOTHING AND THE CYCLE ENDS AS IF COMPLETE (F06)
65     ENDSWITCH
66     ;FOLD Finish Cycle
67     IF ((nStyle>0) AND (nOption>0)) THEN
68         ;END OF CYCLE (EndOfCycle)
69         ;WARNING: THE VENDOR ROUTINE EndOfCycle WAS EDITED ON THIS ROBOT: IT NO LONGER SETS
70         ; $OUT[15] do015WorkComplete NOR WAITS FOR $IN[15] di015WorkCompleteAckn - THE PLC
71         ; NEVER RECEIVES WORK COMPLETE FROM THIS ROBOT (F46)
72         EndOfCycle( )
73         ;STOP THE CYCLE TIMER
74         StopCycTime( )
75     ENDIF
76     ;ENDFOLD (Finish Cycle)
77 ELSE
78     ;NO VALID CYCLE START - SERVICE REQUESTS FROM THE PLC (REPAIR, MASTERING REFERENCE,
79     ;BRAKE TEST, ELECTRODE CHANGE)
80     ;GO TO REPAIR - SIGNAL $IN[9] di009GoToRepair ON

```

```

81      IF (di009GoToRepair==TRUE) THEN
82          HomeCheck( )
83          HomeToRepair( )
84 ;WAIT FOR RETURN FROM REPAIR - SIGNAL $IN[8] di008ReturnFromRepair ON (NO TIMEOUT, F15)
85 ;FOLD WAIT FOR ( IN 8 'di008ReturnFromRepair' ) ;%{...}
86 WAIT FOR ( $IN[8] )
87 ;ENDFOLD
88      RepairToHome( )
89      ENDIF
90      ;MASTERING REFERENCE TEST - SIGNAL $IN[22] di022MstrRefRequest ON
91      IF (di022MstrRefRequest==TRUE) THEN
92          HomeCheck( )
93          masref_main( )
94 ;FOLD WAIT Time=5 sec ;%{...}
95 WAIT SEC 5
96 ;ENDFOLD
97      ENDIF
98      ;BRAKE TEST - SIGNAL $IN[21] di021BrakeTestRequest ON
99      IF (di021BrakeTestRequest==TRUE) THEN
100          HomeCheck( )
101          BrakeTestReq( )
102 ;FOLD WAIT Time=5 sec ;%{...}
103 WAIT SEC 5
104 ;ENDFOLD
105      ENDIF
106      ;ELECTRODE CHANGE - SIGNAL $OUT[111] do111Gun1ElectrodeChange, $OUT[112] do112Gun2ElectrodeChange
107      ;OR $OUT[113] do113Gun3ElectrodeChange ON
108      ;NOTE: ROBOT OUTPUTS WRITTEN ONLY BY THE AutomationCore BACKGROUND TASK: $OUT[111] FOLLOWS
109      ; dipw1_EndofStepper SINCE THE TASK WAS SWITCHED ON (F49); $OUT[112]/$OUT[113] ARE NO LONGER
110      ;CHECK: GunElectrodeChange NOW RUNS AT THE STEPPER END - TEST IT BEFORE PRODUCTION (F43, F49)
111      IF (do111Gun1ElectrodeChange==TRUE) OR (do112Gun2ElectrodeChange==TRUE) OR (do113Gun3ElectrodeChange==TRUE) THEN
112          GunElectrodeChange()
113      ENDIF
114      ENDIF
115      ENDLLOOP
116 END
117

```

KRC/R1/safetest.src

```

1  &ACCESS RVP
2  &REL 2
3  &COMMENT 03-10-R1 TEST LOOP
4  &PARAM EDITMASK = *
5  &PARAM TEMPLATE = C:\KRC\Roboter\Template\vorgabe
6  &PARAM DISKPATH = KRC:\R1
7  DEF safetest( )
8  ;FOLD INI ;%{...}
9      ;FOLD BASISTECH INI
10      GLOBAL INTERRUPT DECL 3 WHEN $STOPMESS==TRUE DO IR_STOPM ( )
11      INTERRUPT ON 3
12      BAS (#INITMOV,0 )
13  ;ENDFOLD (BASISTECH INI)
14  ;FOLD USER INI
15      ;Make your modifications here
16
17  ;ENDFOLD (USER INI)
18 ;ENDFOLD (INI)

```

```

19 ;*****
20 ; Gestamp Standards
21 ; Line - BMW G65 (cell 03)
22 ; Robot 03-10-R1 (BMW-03-10-R1)
23 ; Application - MANUAL TEST ONLY - ENDLESS PTP TO ONE POINT
24 ;*****
25 ; NOT CALLED BY ANY PROGRAM. MOVES PTP TO THE POINT "safetest" (TOOL 1, WORLD) IN AN ENDLESS
26 ; LOOP - PRESUMABLY USED TO TEST A SAFEOPERATION ZONE OR SPEED LIMIT. THE ONLY WAY OUT IS TO
27 ; STOP AND CANCEL THE PROGRAM. KEEP OR DELETE: GESTAMP REVIEW POINT 5 (UNUSED PROGRAMS)
28 ;*****
29
30 loop
31 ;FOLD PTP safetest Vel=100 % PDAT1 Tool[1]:TOOL 1 Base[0] ;%{...}
32 $BWDSTART=FALSE
33 PDAT_ACT=PPDAT1
34 FDAT_ACT=Fsafetest
35 BAS(#PTP_PARAMS,100)
36 PTP Xsafetest
37 ;ENDFOLD
38 endloop
39
40 END
41

```